MicroCluster

Source code, submitted for review.

https://github.com/koridoesstuff/MicroCluster

2026-10-10

12,586 lines across 65 files.

Contents

microcluster/

microcluster/__init__.py 4
microcluster/config.py 5
microcluster/demo.py 10
microcluster/detection.py 13
microcluster/disclosure.py 20
microcluster/engine.py 27
microcluster/intake.py 29
microcluster/models.py 32

simulation/

simulation/__init__.py 36
simulation/agent.py 38
simulation/benchmark.py 40
simulation/config.py 50
simulation/contact.py 57
simulation/detection_demo.py 60
simulation/experiments.py 62
simulation/infection.py 75
simulation/pipeline.py 77
simulation/population.py 83
simulation/realism.py 86
simulation/reporting.py 94
simulation/scale.py 96
simulation/simulator.py 102
simulation/smoke.py 106

api/

api/__init__.py 108
api/app.py 109
api/bands.py 118

adversarial/

adversarial/__init__.py 120
adversarial/__main__.py 121
adversarial/differencing.py 122
adversarial/injection.py 125
adversarial/observer.py 129
adversarial/selftest.py 131
adversarial/wandering.py 134

web/

web/app.js 135
web/gates.js 155
web/index.html 157
web/privacy.html 165
web/style.css 169
web/terms.html 183

tests/

tests/__init__.py 185
tests/fixtures.py 186
tests/test_adversarial.py 188
tests/test_api.py 191
tests/test_bands.py 197
tests/test_benchmark.py 199
tests/test_detection.py 202
tests/test_disclosure.py 210
tests/test_engine.py 217
tests/test_experiments.py 220
tests/test_intake.py 226
tests/test_models.py 229
tests/test_motion_constraints.py 231
tests/test_pipeline.py 233
tests/test_reporting.py 238
tests/test_scale_and_realism.py 240
tests/test_simulation.py 243

scripts/

scripts/__init__.py 254
scripts/precompute_results.py 255
scripts/stress_tests.py 259
scripts/verify_epidemic_curves.py 261

other

render.yaml 263
requirements.txt 264
requirements-runtime.txt 265
Procfile 266
CLAUDE.md 267

microcluster/__init__.py

```
"""MicroCluster: privacy-preserving health-signal detection and disclosure.

Two independent engines:

- ``microcluster.detection``  -- is anything unusual happening?
- ``microcluster.disclosure`` -- what, if anything, may be said, and at
  what scope?

See ``CLAUDE.md`` for the thesis and the decided rules.
"""

from . import config, detection, disclosure, engine, models

__all__ = ["config", "detection", "disclosure", "engine", "models"]
```

microcluster/config.py

```
  1 | """Policy constants and configuration for MicroCluster.
  2 |
  3 | EVERYTHING IN THIS MODULE IS A PROJECT-DEFINED POLICY, NOT A PROVEN LAW.
  4 |
  5 | Each value is:
  6 |   - named (nothing here is inlined into scoring code),
  7 |   - configurable (via ``DetectionConfig`` / ``DisclosureConfig``),
  8 |   - a deliberate choice, explained in the comment beside it and in
  9 |     ``CLAUDE.md``.
 10 |
 11 | None of these is derived from an epidemiological or statistical result.
 12 | ``ceil(sqrt(n))`` is a reasonable *shape* for "evidence should scale with
 13 | the size of the claim" -- not a theorem about this problem.
 14 | """
 15 |
 16 | from __future__ import annotations
 17 |
 18 | from dataclasses import dataclass
 19 |
 20 | # ---------------------------------------------------------------------------
 21 | # Detection window (rule 5)
 22 | # ---------------------------------------------------------------------------
 23 |
 24 | # How recently a report must have been submitted to count as "current
 25 | # activity". Policy choice. Default 72 hours.
 26 | DETECTION_WINDOW_HOURS: int = 72
 27 |
 28 |
 29 | # ---------------------------------------------------------------------------
 30 | # Absolute detector (rule 3, rule 4)
 31 | # ---------------------------------------------------------------------------
 32 |
 33 | # BACKGROUND_RATE: the approximate fraction of a population expected to
 34 | # submit a qualifying symptom report within ONE detection window under
 35 | # ordinary, non-outbreak conditions.
 36 | #
 37 | # THIS IS AN ILLUSTRATIVE FIGURE. IT IS NOT A CLINICAL OR EPIDEMIOLOGICAL
 38 | # ESTIMATE. It exists only to give the absolute detector a reference point
 39 | # to compare against. Before trusting the absolute detector for a real
 40 | # community, replace this with a rate measured from that community's own
 41 | # quiet-period baseline.
 42 | BACKGROUND_RATE: float = 0.02
 43 |
 44 | # The absolute detector fires when the observed within-window report count
 45 | # reaches this multiple of the background expectation
 46 | # (BACKGROUND_RATE * declared campus population). Policy choice.
 47 | ABSOLUTE_EXCESS_RATIO: float = 1.5
 48 |
 49 |
 50 | # ---------------------------------------------------------------------------
 51 | # Relative detector (rule 3)
 52 | # ---------------------------------------------------------------------------
```

```
 53 |
 54 | # The relative detector fires when a location's per-capita within-window
 55 | # report rate reaches this multiple of its contemporaneous peers' pooled
 56 | # per-capita rate. Policy choice.
 57 | RELATIVE_EXCESS_RATIO: float = 2.0
 58 |
 59 | # Additive (Laplace) smoothing applied to BOTH sides of the relative
 60 | # excess ratio:
 61 | #
 62 | #   (reports_here + k) / (n_here + k)   divided by
 63 | #   (reports_control + k) / (n_control + k)
 64 | #
 65 | # Without it, a small quiet peer group drives the control rate to nearly
 66 | # zero and the ratio explodes to hundreds or infinity -- a number that
 67 | # looks precise but is dominated by division noise. k pulls both rates
 68 | # toward a neutral prior; the ratio stays finite and interpretable.
 69 | # Policy choice. Default 1 (one pseudo-report per pseudo-person).
 70 | RELATIVE_SMOOTHING: float = 1.0
 71 |
 72 | # ...and only if that location has at least this many within-window
 73 | # reports. Without a floor, one report in a tiny group produces an
 74 | # enormous ratio and constant false positives. Policy choice.
 75 | MIN_RELATIVE_CLUSTER_REPORTS: int = 3
 76 |
 77 |
 78 | # ---------------------------------------------------------------------------
 79 | # Detection hysteresis
 80 | # ---------------------------------------------------------------------------
 81 | #
 82 | # detection.evaluate() is otherwise a pure, single-instant function: fired
 83 | # is recomputed from scratch each call, from whatever happens to be in the
 84 | # window that instant. Called once a day over a live event, that means an
 85 | # ordinary day-to-day wobble in which reports happen to fall inside a
 86 | # rolling 72h window can flip "fired" off and back on in the middle of one
 87 | # ongoing event -- not because anything really went away, but because the
 88 | # window's edge moved. Hysteresis smooths that out. It does not change what
 89 | # a single evaluation computes; it changes how a SEQUENCE of evaluations
 90 | # (threaded via ``HysteresisState``) is allowed to report "fired".
 91 |
 92 | # Once fired, "fired" is held True for at least this many consecutive
 93 | # evaluations, even on a day where the raw relative/absolute check alone
 94 | # would say no. Policy choice.
 95 | HYSTERESIS_MIN_FIRED_DAYS: int = 3
 96 |
 97 | # ...unless the window's qualifying report count drops to this floor or
 98 | # below, in which case the hold releases immediately regardless of the
 99 | # minimum above -- a count this low means the event is genuinely over, and
100 | # holding "fired" would describe a signal that is no longer there. Policy
101 | # choice.
102 | HYSTERESIS_RELEASE_THRESHOLD: int = 3
103 |
104 |
105 | # ---------------------------------------------------------------------------
106 | # Disclosure -- statistical gate (rule 8)
107 | # ---------------------------------------------------------------------------
```

```
108 |
109 | # statistical threshold = max(STATISTICAL_GATE_FLOOR, ceil(sqrt(n)))
110 | #
111 | # Scales UP with declared population n: a claim about a larger population
112 | # must be backed by more reports. The floor keeps very small scopes from
113 | # clearing the statistical gate on just one or two reports.
114 | STATISTICAL_GATE_FLOOR: int = 5
115 |
116 |
117 | # ---------------------------------------------------------------------------
118 | # Disclosure -- privacy gate (rule 8)
119 | # ---------------------------------------------------------------------------
120 |
121 | # A scope may only be disclosed if its DECLARED population is at least this
122 | # large. Small scopes are where a disclosure can re-identify the person who
123 | # reported. This bound scales DOWN with population -- it only bites on the
124 | # small scopes -- which is the opposite direction to the statistical gate,
125 | # on purpose. Policy choice. Default 20.
126 | MIN_SCOPE_POPULATION: int = 20
127 |
128 | # ...and the qualifying report count must not exceed this fraction of the
129 | # declared population. If half of a declared group is "reporting", naming
130 | # the group tells you a lot about specific people in it. Policy choice.
131 | MAX_REPORT_FRACTION: float = 0.5
132 |
133 |
134 | # ---------------------------------------------------------------------------
135 | # Disclosure -- scope stability across a sequence of disclosures
136 | # ---------------------------------------------------------------------------
137 | #
138 | # disclosure.evaluate() is otherwise a pure, single-instant function too:
139 | # on each call it picks the finest scope passing both gates *that instant*,
140 | # with no memory of what was disclosed last time. Called once a day over a
141 | # live event, that means the named scope can wander from suite to suite as
142 | # the evidence shifts night to night. Each individual disclosure still
143 | # passes both gates on its own, but the SEQUENCE leaks far more than any
144 | # one of them does: an observer who watches it wander learns about every
145 | # group it ever names, not just one. Once a scope has been named, evaluate()
146 | # keeps naming that scope (or a coarser ancestor of it, if the exact scope
147 | # stops qualifying) unless a different scope's qualifying count clearly
148 | # outweighs it.
149 |
150 | # How much a candidate OUTSIDE the currently disclosed scope's own lineage
151 | # (itself or its ancestors) must exceed the current scope's qualifying
152 | # count by before disclosure is allowed to switch to it. Below this margin,
153 | # disclosure stays put -- it never silently hops between sibling scopes on
154 | # a small, possibly noisy, difference in count. Policy choice.
155 | SIBLING_SWITCH_MARGIN: int = 5
156 |
157 |
158 | # ---------------------------------------------------------------------------
159 | # Abuse mitigation (rule 11)
160 | # ---------------------------------------------------------------------------
161 |
162 | # Reports are anonymous. We therefore accept that abuse is possible and
```

```
# apply only light, anonymous mitigation -- never identity-based controls,
# because identity is exactly what the system exists to protect.
#
# These caps are enforced at INTAKE by ``microcluster.intake`` (wired into
# ``engine.analyze`` as its first step). A report carries an opaque,
# ephemeral ``session_id``; that is the ONLY thing the cap keys on. It is
# rate limiting, not authentication: a report with no session_id is not
# rate limited (there is nothing to key on), and an attacker who spreads
# their reports across many sessions defeats a per-session cap outright.
# ``adversarial.injection`` measures exactly what the cap does and does
# not buy.
SESSION_SUBMISSION_CAP: int = 3     # max accepted reports per session_id (its lifetim
e)
DAILY_SUBMISSION_CAP: int = 10      # max accepted reports per session_id per calendar
day


# ---------------------------------------------------------------------------
# Configuration objects
# ---------------------------------------------------------------------------


@dataclass(frozen=True)
class DetectionConfig:
    """Tunable inputs to the detection engine. Defaults come from the
    module-level policy constants above."""

    detection_window_hours: int = DETECTION_WINDOW_HOURS
    background_rate: float = BACKGROUND_RATE
    absolute_excess_ratio: float = ABSOLUTE_EXCESS_RATIO
    relative_excess_ratio: float = RELATIVE_EXCESS_RATIO
    relative_smoothing: float = RELATIVE_SMOOTHING
    min_relative_cluster_reports: int = MIN_RELATIVE_CLUSTER_REPORTS
    hysteresis_min_fired_days: int = HYSTERESIS_MIN_FIRED_DAYS
    hysteresis_release_threshold: int = HYSTERESIS_RELEASE_THRESHOLD


@dataclass(frozen=True)
class DisclosureConfig:
    """Tunable inputs to the disclosure engine. Defaults come from the
    module-level policy constants above."""

    statistical_gate_floor: int = STATISTICAL_GATE_FLOOR
    min_scope_population: int = MIN_SCOPE_POPULATION
    max_report_fraction: float = MAX_REPORT_FRACTION
    sibling_switch_margin: int = SIBLING_SWITCH_MARGIN


@dataclass(frozen=True)
class IntakeConfig:
    """Tunable inputs to the intake submission caps (rule 11). Defaults
    come from the module-level policy constants above. Anonymous rate
    limiting only -- see the SESSION_SUBMISSION_CAP comment."""

    session_submission_cap: int = SESSION_SUBMISSION_CAP
```

```
    daily_submission_cap: int = DAILY_SUBMISSION_CAP


DEFAULT_DETECTION_CONFIG = DetectionConfig()
DEFAULT_DISCLOSURE_CONFIG = DisclosureConfig()
DEFAULT_INTAKE_CONFIG = IntakeConfig()
```

microcluster/demo.py

```
  1 | """Print the six reference scenarios and their evaluation tables.
  2 |
  3 |     python -m microcluster.demo
  4 |
  5 | This is a read-only illustration of the two engines. It builds the same
  6 | fixtures the tests use.
  7 | """
  8 |
  9 | from __future__ import annotations
 10 |
 11 | import sys
 12 | from pathlib import Path
 13 |
 14 | sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
 15 |
 16 | from microcluster.disclosure import DisclosureResult  # noqa: E402
 17 | from microcluster.engine import AnalysisResult, analyze  # noqa: E402
 18 | from microcluster.models import Category  # noqa: E402
 19 | from tests.fixtures import (  # noqa: E402
 20 |     NOW,
 21 |     reports,
 22 |     small_registry_floor18,
 23 |     standard_registry,
 24 | )
 25 |
 26 |
 27 | def _fmt_detection(result: AnalysisResult) -> str:
 28 |     d = result.detection
 29 |     f = d.findings
 30 |     fired = []
 31 |     if d.relative_detector_fired:
 32 |         fired.append("RELATIVE")
 33 |     if d.absolute_detector_fired:
 34 |         fired.append("ABSOLUTE")
 35 |     lines = [
 36 |         f"  detection fired : {d.fired}  ({' + '.join(fired) if fired else 'neither det
ector'})",
 37 |         f"  findings        : relative_excess_ratio = {f.relative_excess_ratio:.2f}"
 38 |         f" (hotspot {d.hotspot_location_id})",
 39 |         f"                    count_within_window   = {f.count_within_window}"
 40 |         f"  (absolute expectation {d.absolute_expected:.1f})",
 41 |         f"                    count_sharing_category= {f.count_sharing_category}"
 42 |         f"  (category {d.dominant_category.value if d.dominant_category else '-'})",
 43 |     ]
 44 |     return "\n".join(lines)
 45 |
 46 |
 47 | def _fmt_table(disc: DisclosureResult) -> str:
 48 |     if not disc.evaluations:
 49 |         return "  (disclosure engine did not run -- nothing detected)"
 50 |     header = (
 51 |         f"  {'scope':<26} {'lvl':<8} {'n':>5} {'q':>4} "
```

```
 52 |         f"{'stat>=':>7} {'stat':>5} {'n>=20':>6} {'frac':>6} {'frac<=':>7} "
 53 |         f"{'priv':>5} {'DISCLOSE':>9}"
 54 |     )
 55 |     rows = [header, "  " + "-" * (len(header) - 2)]
 56 |     for ev in disc.evaluations:
 57 |         label = ev.scope_label if len(ev.scope_label) <= 26 else ev.scope_label[:23] +
"..."
 58 |         rows.append(
 59 |             f"  {label:<26} {ev.scope_level.name:<8} {ev.population:>5} "
 60 |             f"{ev.qualifying_reports:>4} {ev.statistical_threshold:>7} "
 61 |             f"{('PASS' if ev.statistical_pass else 'FAIL'):>5} "
 62 |             f"{('PASS' if ev.min_population_pass else 'FAIL'):>6} "
 63 |             f"{ev.report_fraction:>6.2f} "
 64 |             f"{('PASS' if ev.report_fraction_pass else 'FAIL'):>7} "
 65 |             f"{('PASS' if ev.privacy_pass else 'FAIL'):>5} "
 66 |             f"{('*SELECTED*' if ev.selected else ('eligible' if ev.disclosure_eligible
else 'no')):>9}"
 67 |         )
 68 |     rows.append("")
 69 |     for ev in disc.evaluations:
 70 |         rows.append(f"    {ev.scope_label}: {ev.disclosure_reason}")
 71 |     return "\n".join(rows)
 72 |
 73 |
 74 | def _scenario(title: str, result: AnalysisResult) -> None:
 75 |     print("=" * 78)
 76 |     print(title)
 77 |     print("=" * 78)
 78 |     print(_fmt_detection(result))
 79 |     print()
 80 |     print(_fmt_table(result.disclosure))
 81 |     disclosed = result.disclosure.disclosed()
 82 |     print()
 83 |     if disclosed is None:
 84 |         print("  >> DISCLOSED: nothing")
 85 |     else:
 86 |         print(f"  >> DISCLOSED: {disclosed.scope_label}  (n={disclosed.population}, "
 87 |               f"{disclosed.qualifying_reports} qualifying reports)")
 88 |     print()
 89 |
 90 |
 91 | def main() -> None:
 92 |     reg = standard_registry()
 93 |
 94 |     # 1. Localized cluster -- relative fires, absolute does not.
 95 |     #    All reports at suite level (detection requires one scope level).
 96 |     localized = (
 97 |         reports("S1", 9, category=Category.RESPIRATORY)
 98 |         + reports("S2", 2, category=Category.GENERAL)
 99 |         + reports("S3", 1, category=Category.GASTROINTESTINAL)
100 |         + reports("S4", 1, category=Category.OTHER)
101 |         + reports("S5", 1, category=Category.GENERAL)
102 |         + reports("S3", 4, category=Category.RESPIRATORY, hours_ago=400)  # outside win
dow
103 |     )
```

```
    _scenario("SCENARIO 1  Localized cluster (relative fires, absolute does not)",
              analyze(localized, reg, now=NOW))

    # 2. Campus-wide rise -- absolute fires, relative does not.
    campus_wide = []
    for b in ("B1", "B2", "B3", "B4", "B5"):
        campus_wide += reports(b, 14, category=Category.RESPIRATORY)
    _scenario("SCENARIO 2  Campus-wide rise (absolute fires, relative does not)",
              analyze(campus_wide, reg, now=NOW))

    # 3. Scattered noise -- neither fires. Uniform building level.
    scattered = (
        reports("B1", 3, category=Category.GENERAL)
        + reports("B2", 3, category=Category.RESPIRATORY)
        + reports("B3", 3, category=Category.GASTROINTESTINAL)
        + reports("B4", 3, category=Category.OTHER)
        + reports("B5", 3, category=Category.GENERAL)
    )
    _scenario("SCENARIO 3  Scattered noise (neither detector fires)",
              analyze(scattered, reg, now=NOW))

    # 4. Overclaim refusal -- 7 reports, campus n=2000 -> statistical FAIL.
    overclaim = reports("B1", 7, category=Category.RESPIRATORY)
    _scenario("SCENARIO 4  Overclaim refusal (7 reports vs campus n=2000)",
              analyze(overclaim, reg, now=NOW))

    # 5. Privacy refusal -- 5 reports, floor n=18 -> statistical PASS, privacy FAIL.
    small = small_registry_floor18()
    privacy = reports("F", 5, category=Category.RESPIRATORY)
    _scenario("SCENARIO 5  Privacy refusal (5 reports, floor n=18 -> coarser scope)",
              analyze(privacy, small, now=NOW))

    # 6. Both gates across populations -- see tests/test_disclosure.py for the
    #    full parametric table; here we show the n=40 case passing both.
    from tests.fixtures import single_scope_registry

    across = reports("C", 8, category=Category.RESPIRATORY)
    _scenario("SCENARIO 6  Both gates, n=40 (statistical threshold 7, privacy ok)",
              analyze(across, single_scope_registry(40), now=NOW))

    # 7. Roster refusal -- 15 reports in a declared group of n=25: passes
    #    the statistical gate and the population minimum, refused because
    #    15/25 = 0.60 exceeds MAX_REPORT_FRACTION.
    from tests.fixtures import roster_registry

    roster = reports("G", 15, category=Category.RESPIRATORY)
    _scenario("SCENARIO 7  Roster refusal (15 reports, group n=25 -> fraction 0.60)",
              analyze(roster, roster_registry(), now=NOW))


if __name__ == "__main__":
    main()
```

microcluster/detection.py

```
"""Detection engine: is anything unusual happening?

Two independent detectors (rule 3), either of which can fire:

  * relative -- a location unusual versus its contemporaneous peers
  * absolute -- overall activity unusual versus a configured background
    rate (``config.BACKGROUND_RATE``, rule 4)

The engine reports three plain findings and never combines them into a
composite score (rule 6):

  * ``relative_excess_ratio``
  * ``count_within_window``
  * ``count_sharing_category``

Both detectors are scored on the same subset -- within-window reports in
the *dominant category* -- so that whatever trips a detector is exactly
what the disclosure engine goes on to describe, never a superset of it.

Its other job is to hand the disclosure engine an *aggregate* view --
per-scope qualifying report counts -- so that the disclosure engine never
touches individual records (rule 10).

All reports passed to :func:`evaluate` must resolve to scopes at a single
``ScopeLevel``; see that function's docstring.

A single call to :func:`evaluate` is still a pure, single-instant
computation -- ``raw_fired`` is recomputed from nothing but this call's
reports. ``fired`` (the field disclosure and everything downstream acts
on) additionally applies HYSTERESIS across a *sequence* of calls, threaded
explicitly via ``HysteresisState`` (see that class and
``config.HYSTERESIS_MIN_FIRED_DAYS`` / ``HYSTERESIS_RELEASE_THRESHOLD``).
Passing no state (the default) makes a single call behave exactly as if
hysteresis did not exist.
"""

from __future__ import annotations

from collections import Counter
from dataclasses import dataclass
from datetime import datetime, timedelta, timezone

from .config import DEFAULT_DETECTION_CONFIG, DetectionConfig
from .models import Category, Report, ScopeLevel, ScopeRegistry

__all__ = [
    "DetectionFindings",
    "DetectionResult",
    "HysteresisState",
    "evaluate",
]
```

```
 53 |
 54 | @dataclass(frozen=True)
 55 | class DetectionFindings:
 56 |     """The three findings, reported plainly and separately (rule 6)."""
 57 |
 58 |     relative_excess_ratio: float
 59 |     count_within_window: int
 60 |     count_sharing_category: int
 61 |
 62 |
 63 | @dataclass(frozen=True)
 64 | class HysteresisState:
 65 |     """Carries hysteresis across a SEQUENCE of :func:`evaluate` calls (one
 66 |     per day, say). Immutable: each call returns the next state as part of
 67 |     its :class:`DetectionResult`; the caller threads it into the following
 68 |     call. The default, ``HysteresisState()``, is "not currently active" --
 69 |     passing it (or nothing) makes hysteresis a no-op.
 70 |     """
 71 |
 72 |     active: bool = False
 73 |     consecutive_days: int = 0
 74 |
 75 |
 76 | @dataclass(frozen=True)
 77 | class DetectionResult:
 78 |     fired: bool
 79 |     raw_fired: bool
 80 |     relative_detector_fired: bool
 81 |     absolute_detector_fired: bool
 82 |     findings: DetectionFindings
 83 |     hysteresis_state: HysteresisState
 84 |
 85 |     # Context for the findings and for disclosure.
 86 |     dominant_category: Category | None
 87 |     hotspot_location_id: str | None
 88 |     absolute_expected: float
 89 |     window_start: datetime
 90 |     window_end: datetime
 91 |
 92 |     # Aggregate handed to the disclosure engine (rule 10): for every scope
 93 |     # that contains at least one qualifying report, how many qualifying
 94 |     # reports it contains. "Qualifying" == within the detection window and
 95 |     # sharing the dominant category. Both detectors are scored on the same
 96 |     # dominant-category subset, so whatever fires is what this table (and
 97 |     # therefore the disclosure) describes.
 98 |     qualifying_scope_counts: dict[str, int]
 99 |
100 |
101 | def _now(now: datetime | None) -> datetime:
102 |     return now if now is not None else datetime.now(timezone.utc)
103 |
104 |
105 | def _within_window(
106 |     reports: list[Report], window_start: datetime, window_end: datetime
107 | ) -> list[Report]:
```

```
    return [r for r in reports if window_start <= r.submitted_at <= window_end]


def _dominant_category(window_reports: list[Report]) -> Category | None:
    """The category with the most within-window reports. Ties broken
    deterministically by category name so the result never depends on
    input ordering."""
    if not window_reports:
        return None
    counts = Counter(r.category for r in window_reports)
    top = max(counts.values())
    return sorted((c for c, n in counts.items() if n == top), key=lambda c: c.value)[0]


def _run_relative_detector(
    window_reports: list[Report],
    registry: ScopeRegistry,
    config: DetectionConfig,
    dominant_category: Category | None,
) -> tuple[bool, float, str | None]:
    """Compare each reporting location's per-capita rate against the pooled
    per-capita rate of everyone else in the campus (its contemporaneous
    peers). Returns (fired, excess_ratio_of_hotspot, hotspot_location_id).

    The excess ratio is computed on TOTAL within-window volume at each
    location -- "is this location busier than its peers" is a volume
    question. The minimum-cluster-size gate
    (``config.min_relative_cluster_reports``), however, counts only the
    hotspot's reports **in the dominant category**. A location whose extra
    volume is really three unrelated illnesses therefore does not clear the
    gate: what fires is what the disclosure engine will describe.

    Both rates are additively smoothed by ``config.relative_smoothing`` so
    a quiet peer group cannot drive the ratio to infinity (see
    ``config.RELATIVE_SMOOTHING``).

    Assumes every report resolves to a scope at the SAME hierarchy level
    (``evaluate`` enforces this): peer population is
    ``campus_population - location_population``, which is only correct when
    report locations do not nest inside one another.
    """
    total = len(window_reports)
    campus_population = registry.campus_population()
    if total == 0:
        return (False, 0.0, None)

    per_location = Counter(r.location_id for r in window_reports)
    per_location_dominant: Counter[str] = Counter(
        r.location_id for r in window_reports if r.category == dominant_category
    )
    k = config.relative_smoothing

    best_ratio = 0.0
    hotspot: str | None = None
    hotspot_dominant_count = 0
```

```
    for location_id, loc_count in per_location.items():
        loc_population = registry.get(location_id).population
        peer_population = campus_population - loc_population
        peer_count = total - loc_count
        if peer_population <= 0:
            # This location IS the whole comparison set; there are no peers
            # to be unusual against. Skip it for the relative detector.
            continue

        # Additively smoothed rates on both sides (rule: named constant).
        loc_rate = (loc_count + k) / (loc_population + k)
        peer_rate = (peer_count + k) / (peer_population + k)
        ratio = loc_rate / peer_rate

        if ratio > best_ratio:
            best_ratio = ratio
            hotspot = location_id
            hotspot_dominant_count = per_location_dominant[location_id]

    fired = (
        hotspot is not None
        and best_ratio >= config.relative_excess_ratio
        and hotspot_dominant_count >= config.min_relative_cluster_reports
    )
    return (fired, best_ratio, hotspot)


def _run_absolute_detector(
    window_reports: list[Report],
    registry: ScopeRegistry,
    config: DetectionConfig,
    dominant_category: Category | None,
) -> tuple[bool, float]:
    """Compare the within-window count of DOMINANT-CATEGORY reports against
    the background expectation for the whole declared campus population.
    Returns (fired, expected_count).

    Only dominant-category reports are counted. A spike built from several
    unrelated illnesses therefore cannot fire this detector and then leave
    the disclosure engine describing only one category's slice of it -- the
    thing that fires is the thing that gets described. If campus-wide
    elevation across mixed categories is worth flagging, it should be an
    explicit, separately reported signal, not a side effect of this one.

    ``expected`` is still ``BACKGROUND_RATE * campus_population``, the
    whole-population background rather than a per-category one, which makes
    this a deliberately conservative test. Give ``BACKGROUND_RATE`` a
    per-category value when tuning it against real baseline data.
    """
    expected = config.background_rate * registry.campus_population()
    observed = sum(1 for r in window_reports if r.category == dominant_category)
    fired = observed >= expected * config.absolute_excess_ratio
    return (fired, expected)

```

```
|
| def _qualifying_scope_counts(
|     window_reports: list[Report],
|     registry: ScopeRegistry,
|     dominant_category: Category | None,
| ) -> dict[str, int]:
|     """For each scope containing at least one qualifying report, the count
|     of qualifying reports in its subtree. Qualifying == within window and
|     sharing the dominant category."""
|     counts: Counter[str] = Counter()
|     if dominant_category is None:
|         return {}
|     for report in window_reports:
|         if report.category != dominant_category:
|             continue
|         for scope in registry.chain_to_root(report.location_id):
|             counts[scope.id] += 1
|     return dict(counts)
|
|
| def _apply_hysteresis(
|     raw_fired: bool,
|     count_within_window: int,
|     prior: HysteresisState,
|     config: DetectionConfig,
| ) -> tuple[bool, HysteresisState]:
|     """Turn this instant's raw fired/not-fired into a hysteresis-aware
|     ``fired`` plus the state to pass into the next call.
|
|     Rules (config.HYSTERESIS_MIN_FIRED_DAYS / HYSTERESIS_RELEASE_THRESHOLD):
|
|     * Raw fired -> always fired; the active streak continues (or starts).
|     * Raw not fired, and not currently active -> stays not fired. Nothing
|       to hold.
|     * Raw not fired, but currently active -> HOLD fired True, UNLESS
|       either the window has clearly emptied out (count_within_window at or
|       below the release threshold: release immediately) or the minimum
|       hold duration has already been served (release on schedule).
|     """
|     if raw_fired:
|         streak = prior.consecutive_days + 1 if prior.active else 1
|         return True, HysteresisState(active=True, consecutive_days=streak)
|
|     if not prior.active:
|         return False, HysteresisState(active=False, consecutive_days=0)
|
|     if count_within_window <= config.hysteresis_release_threshold:
|         return False, HysteresisState(active=False, consecutive_days=0)
|
|     if prior.consecutive_days < config.hysteresis_min_fired_days:
|         return True, HysteresisState(
|             active=True, consecutive_days=prior.consecutive_days + 1
|         )
|
|     return False, HysteresisState(active=False, consecutive_days=0)
```

```
|
|
| def evaluate(
|     reports: list[Report],
|     registry: ScopeRegistry,
|     *,
|     now: datetime | None = None,
|     config: DetectionConfig = DEFAULT_DETECTION_CONFIG,
|     hysteresis_state: HysteresisState | None = None,
| ) -> DetectionResult:
|     """Run both detectors and assemble the findings.
|
|     ``now`` defaults to the current UTC time; pass it explicitly for
|     deterministic evaluation.
|
|     **Mixed-level assumption (enforced).** Every report must resolve to a
|     scope at the *same* ``ScopeLevel``. The relative detector derives a
|     location's peer population as ``campus_population -
|     location_population``; that subtraction is only correct when report
|     locations are siblings, not nested (a floor inside a building it also
|     receives reports for would be double-counted). If reports arrive at
|     more than one level this function raises ``ValueError`` -- normalise
|     them to a single level upstream. Handling true nesting is left for a
|     later change.
|
|     ``hysteresis_state`` is the ``HysteresisState`` returned by the
|     PREVIOUS call in a sequence (e.g. yesterday's); omit it (or pass
|     ``None``) for a one-off evaluation, which is equivalent to always
|     starting "not active". The next state to thread into the following
|     call is returned on the result as ``hysteresis_state``.
|     """
|     window_end = _now(now)
|     window_start = window_end - timedelta(hours=config.detection_window_hours)
|
|     report_levels: set[ScopeLevel] = set()
|     for report in reports:
|         # ``registry.get`` fails loudly on an unknown location; we also
|         # collect the scope level of every report here.
|         report_levels.add(registry.get(report.location_id).level)
|     if len(report_levels) > 1:
|         raise ValueError(
|             "detection.evaluate requires every report to be filed at the "
|             "same scope level; got "
|             f"{sorted(level.name for level in report_levels)}. The relative "
|             "detector computes peer population as campus_population minus the "
|             "reporting location's population, which is only correct when "
|             "report locations do not nest. Normalise reports to one level "
|             "before calling."
|         )
|
|     window_reports = _within_window(reports, window_start, window_end)
|
|     dominant = _dominant_category(window_reports)
|     count_within_window = len(window_reports)
|     count_sharing_category = (
```

```
        sum(1 for r in window_reports if r.category == dominant) if dominant else 0
    )

    relative_fired, excess_ratio, hotspot = _run_relative_detector(
        window_reports, registry, config, dominant
    )
    absolute_fired, expected = _run_absolute_detector(
        window_reports, registry, config, dominant
    )

    findings = DetectionFindings(
        relative_excess_ratio=excess_ratio,
        count_within_window=count_within_window,
        count_sharing_category=count_sharing_category,
    )

    raw_fired = relative_fired or absolute_fired
    fired, next_hysteresis_state = _apply_hysteresis(
        raw_fired, count_within_window, hysteresis_state or HysteresisState(), config
    )

    return DetectionResult(
        fired=fired,
        raw_fired=raw_fired,
        relative_detector_fired=relative_fired,
        absolute_detector_fired=absolute_fired,
        findings=findings,
        hysteresis_state=next_hysteresis_state,
        dominant_category=dominant,
        hotspot_location_id=hotspot,
        absolute_expected=expected,
        window_start=window_start,
        window_end=window_end,
        qualifying_scope_counts=_qualifying_scope_counts(
            window_reports, registry, dominant
        ),
    )
```

microcluster/disclosure.py

```
"""Disclosure engine: what, if anything, may be said, and at what scope?

Runs after detection (rule 10). It is handed an *aggregate* table --
per-scope qualifying report counts -- plus the scope registry. It never
sees an individual record.

Two separate gates (rule 8), evaluated independently, never merged into
one number:

  * statistical gate: qualifying >= max(floor, ceil(sqrt(n)))  -- up with n
  * privacy gate:     n >= MIN_SCOPE_POPULATION
                      AND qualifying / n <= MAX_REPORT_FRACTION -- down with n

Both must pass for a scope to be disclosure-eligible. The finest eligible
scope is selected (rule 9). Every candidate scope -- winner and rejected
alike -- gets a complete ``ScopeEvaluation`` record (rule 12), and the
winner is picked *from that table*, never on a separate path.

A single call to :func:`evaluate` is still exactly that: one instant's
gate table. Called in a SEQUENCE (e.g. once a day), it additionally offers
SCOPE STABILITY -- pass the previous call's winning scope id as
``prior_disclosed_scope_id`` and evaluate() prefers to keep disclosing that
scope, or a coarser ancestor of it, rather than hopping to a different
scope on a small night-to-night difference in count (see
``config.SIBLING_SWITCH_MARGIN``). Passing no prior id (the default) makes
a single call behave exactly as if stability did not exist.
"""

from __future__ import annotations

import math
from dataclasses import dataclass

from .config import DEFAULT_DISCLOSURE_CONFIG, DisclosureConfig
from .models import ScopeLevel, ScopeRegistry

__all__ = [
    "ScopeEvaluation",
    "DisclosureResult",
    "statistical_threshold",
    "evaluate",
]


def _ceil_sqrt(n: int) -> int:
    """ceil(sqrt(n)) computed in integers, so perfect squares are exact
    (float sqrt(x*x) can land just under x)."""
    root = math.isqrt(n)
    return root if root * root == n else root + 1


def statistical_threshold(n: int, config: DisclosureConfig = DEFAULT_DISCLOSURE_CONFIG)
```

```
-> int:
 53 |     """The statistical gate threshold for a declared population ``n``:
 54 |
 55 |         max(STATISTICAL_GATE_FLOOR, ceil(sqrt(n)))
 56 |
 57 |     Scales up with n -- a bigger claim needs more evidence (rule 8). This
 58 |     is a policy shape, not a proven law (see ``config.py``).
 59 |     """
 60 |     return max(config.statistical_gate_floor, _ceil_sqrt(n))
 61 |
 62 |
 63 | @dataclass(frozen=True)
 64 | class ScopeEvaluation:
 65 |     """A complete evaluation record for one candidate scope (rule 12).
 66 |
 67 |     A future UI renders these directly and must never recompute a gate.
 68 |     """
 69 |
 70 |     scope_id: str
 71 |     scope_label: str
 72 |     scope_level: ScopeLevel
 73 |     population: int
 74 |     qualifying_reports: int
 75 |
 76 |     # Statistical gate.
 77 |     statistical_threshold: int
 78 |     statistical_pass: bool
 79 |
 80 |     # Privacy gate (two conditions, both required).
 81 |     min_scope_population: int
 82 |     min_population_pass: bool
 83 |     report_fraction: float
 84 |     max_report_fraction: float
 85 |     report_fraction_pass: bool
 86 |     privacy_pass: bool
 87 |
 88 |     # Outcome.
 89 |     disclosure_eligible: bool
 90 |     selected: bool
 91 |     disclosure_reason: str
 92 |
 93 |
 94 | @dataclass(frozen=True)
 95 | class DisclosureResult:
 96 |     # Ordered finest scope first. Includes rejected scopes (rule 12).
 97 |     evaluations: tuple[ScopeEvaluation, ...]
 98 |     disclosed_scope_id: str | None
 99 |
100 |     def disclosed(self) -> ScopeEvaluation | None:
101 |         for ev in self.evaluations:
102 |             if ev.selected:
103 |                 return ev
104 |         return None
105 |
106 |
```

```
def _fineness_key(ev_scope_level: ScopeLevel, population: int, scope_id: str):
    """Sort key: finest first. Deeper level wins; then smaller declared
    population; then scope id for a total, stable order."""
    return (-int(ev_scope_level), population, scope_id)


def _reason(
    *,
    selected: bool,
    eligible: bool,
    statistical_pass: bool,
    min_population_pass: bool,
    report_fraction_pass: bool,
    qualifying: int,
    threshold: int,
    n: int,
    min_pop: int,
    fraction: float,
    max_fraction: float,
    stability_note: str | None = None,
) -> str:
    if selected:
        return stability_note or "Disclosed: finest scope passing both the statistical and privacy gates."
    if eligible:
        return stability_note or "Eligible: passes both gates, but a finer scope was disclosed."

    fails: list[str] = []
    if not statistical_pass:
        fails.append(
            f"statistical gate failed ({qualifying} qualifying reports < "
            f"required {threshold} for n={n})"
        )
    if not min_population_pass:
        fails.append(
            f"population n={n} below minimum privacy scope ({min_pop})"
        )
    if not report_fraction_pass:
        fails.append(
            f"report fraction {fraction:.2f} exceeds maximum {max_fraction:.2f} "
            f"(disclosure would name so large a share of the group that the "
            f"statement is effectively a roster)"
        )
    return "Rejected: " + "; ".join(fails)


def _stable_selection(
    gate_rows: list[dict],
    registry: ScopeRegistry,
    prior_disclosed_scope_id: str | None,
    config: DisclosureConfig,
) -> tuple[str | None, dict[str, str]]:
    """Pick the winner with scope stability applied, and any reason-text
    overrides that result from it.
```

```
160 |
161 |     Returns ``(selected_id, stability_notes)``. When there is no usable
162 |     prior scope this round, ``stability_notes`` is empty and
163 |     ``selected_id`` is exactly the plain finest-eligible pick -- a single
164 |     call with no ``prior_disclosed_scope_id`` is unaffected by any of this.
165 |     """
166 |     by_id = {row["scope"].id: row for row in gate_rows}
167 |     eligible_ids = {sid for sid, row in by_id.items() if row["eligible"]}
168 |     finest_eligible_id = next(
169 |         (row["scope"].id for row in gate_rows if row["eligible"]), None
170 |     )
171 |
172 |     lineage_ids: set[str] = set()
173 |     if prior_disclosed_scope_id is not None and prior_disclosed_scope_id in by_id:
174 |         lineage_ids = {
175 |             scope.id
176 |             for scope in registry.chain_to_root(prior_disclosed_scope_id)
177 |             if scope.id in by_id
178 |         }
179 |     lineage_eligible_ids = [sid for sid in lineage_ids if sid in eligible_ids]
180 |
181 |     if not lineage_eligible_ids:
182 |         # No usable incumbent (or ancestor of one) this round: plain
183 |         # finest-eligible selection, identical to having no stability rule.
184 |         return finest_eligible_id, {}
185 |
186 |     stable_id = min(
187 |         lineage_eligible_ids,
188 |         key=lambda sid: _fineness_key(
189 |             by_id[sid]["scope"].level, by_id[sid]["scope"].population, sid
190 |         ),
191 |     )
192 |     stable_count = by_id[stable_id]["qualifying"]
193 |     margin = config.sibling_switch_margin
194 |
195 |     challengers = sorted(
196 |         (
197 |             row
198 |             for sid, row in by_id.items()
199 |             if row["eligible"] and sid not in lineage_ids and row["qualifying"] > stabl
e_count + margin
200 |         ),
201 |         key=lambda row: _fineness_key(row["scope"].level, row["scope"].population, row
["scope"].id),
202 |     )
203 |
204 |     notes: dict[str, str] = {}
205 |     if challengers:
206 |         winner = challengers[0]
207 |         selected_id = winner["scope"].id
208 |         notes[selected_id] = (
209 |             f"Disclosed: qualifying count ({winner['qualifying']}) exceeded the "
210 |             f"previously disclosed scope's ({stable_count}) by more than the "
211 |             f"switch margin ({margin}); disclosure moved here."
212 |         )
```

```
    else:
        selected_id = stable_id
        if stable_id == prior_disclosed_scope_id:
            notes[selected_id] = (
                "Disclosed: kept disclosing the same scope as before for "
                f"stability; no other scope exceeded it by the switch margin "
                f"({margin})."
            )
        else:
            notes[selected_id] = (
                "Disclosed: the previously disclosed scope no longer "
                "qualifies; fell back to a coarser ancestor of it for "
                "stability rather than switching to a different scope."
            )

    # Annotate scopes that lost specifically to the margin, or to a finer
    # scope within the same kept lineage.
    for sid, row in by_id.items():
        if sid == selected_id or not row["eligible"]:
            continue
        if sid in lineage_ids:
            notes[sid] = (
                "Eligible: passes both gates, but a finer scope in the same "
                "disclosure lineage was selected instead."
            )
        elif row["qualifying"] <= stable_count + margin:
            notes[sid] = (
                f"Eligible: passes both gates, but qualifying count "
                f"({row['qualifying']}) did not exceed the disclosed scope's "
                f"({stable_count}) by the required switch margin ({margin}); "
                f"kept the existing disclosure instead of switching here."
            )
        # else: eligible, outside the lineage, cleared the margin, but a
        # finer challenger won -- falls through to the default "a finer
        # scope was disclosed" reason.

    return selected_id, notes


def evaluate(
    registry: ScopeRegistry,
    qualifying_scope_counts: dict[str, int],
    *,
    config: DisclosureConfig = DEFAULT_DISCLOSURE_CONFIG,
    prior_disclosed_scope_id: str | None = None,
) -> DisclosureResult:
    """Evaluate both gates for every candidate scope and select a winner.

    ``qualifying_scope_counts`` maps a scope id to the number of qualifying
    reports in that scope's subtree (produced by the detection engine).
    Only scopes appearing as keys are candidates.

    Without ``prior_disclosed_scope_id``, the winner is simply the finest
    eligible scope (rule 9). With it, scope stability applies: the winner
    stays within the prior scope's own lineage (itself or a coarser
```

```
    ancestor of it) unless some other scope's qualifying count exceeds it
    by more than ``config.sibling_switch_margin`` -- see the module
    docstring.
    """
    candidates = sorted(
        qualifying_scope_counts.keys(),
        key=lambda sid: _fineness_key(
            registry.get(sid).level, registry.get(sid).population, sid
        ),
    )

    # First pass: gate results, no selection yet.
    gate_rows: list[dict] = []
    for scope_id in candidates:
        scope = registry.get(scope_id)
        n = scope.population
        qualifying = qualifying_scope_counts[scope_id]

        threshold = statistical_threshold(n, config)
        statistical_pass = qualifying >= threshold

        min_population_pass = n >= config.min_scope_population
        fraction = qualifying / n
        report_fraction_pass = fraction <= config.max_report_fraction
        privacy_pass = min_population_pass and report_fraction_pass

        gate_rows.append(
            {
                "scope": scope,
                "qualifying": qualifying,
                "threshold": threshold,
                "statistical_pass": statistical_pass,
                "min_population_pass": min_population_pass,
                "fraction": fraction,
                "report_fraction_pass": report_fraction_pass,
                "privacy_pass": privacy_pass,
                "eligible": statistical_pass and privacy_pass,
            }
        )

    selected_id, stability_notes = _stable_selection(
        gate_rows, registry, prior_disclosed_scope_id, config
    )

    evaluations = tuple(
        ScopeEvaluation(
            scope_id=row["scope"].id,
            scope_label=row["scope"].label,
            scope_level=row["scope"].level,
            population=row["scope"].population,
            qualifying_reports=row["qualifying"],
            statistical_threshold=row["threshold"],
            statistical_pass=row["statistical_pass"],
            min_scope_population=config.min_scope_population,
            min_population_pass=row["min_population_pass"],
```

```
            report_fraction=row["fraction"],
            max_report_fraction=config.max_report_fraction,
            report_fraction_pass=row["report_fraction_pass"],
            privacy_pass=row["privacy_pass"],
            disclosure_eligible=row["eligible"],
            selected=(row["scope"].id == selected_id),
            disclosure_reason=_reason(
                selected=(row["scope"].id == selected_id),
                eligible=row["eligible"],
                statistical_pass=row["statistical_pass"],
                min_population_pass=row["min_population_pass"],
                report_fraction_pass=row["report_fraction_pass"],
                qualifying=row["qualifying"],
                threshold=row["threshold"],
                n=row["scope"].population,
                min_pop=config.min_scope_population,
                fraction=row["fraction"],
                max_fraction=config.max_report_fraction,
                stability_note=stability_notes.get(row["scope"].id),
            ),
        )
        for row in gate_rows
    )

    return DisclosureResult(evaluations=evaluations, disclosed_scope_id=selected_id)
```

microcluster/engine.py

```
"""Thin wiring between the engines.

Order is fixed (rule 10): intake caps first, then detection, then -- only
if detection fired -- disclosure. The disclosure engine receives the
detection engine's aggregate per-scope table, never the reports.
"""

from __future__ import annotations

from dataclasses import dataclass, field

from datetime import datetime

from . import detection, disclosure, intake
from .config import (
    DEFAULT_DETECTION_CONFIG,
    DEFAULT_DISCLOSURE_CONFIG,
    DEFAULT_INTAKE_CONFIG,
    DetectionConfig,
    DisclosureConfig,
    IntakeConfig,
)
from .detection import DetectionResult, HysteresisState
from .disclosure import DisclosureResult
from .intake import IntakeResult, RejectedReport
from .models import Report, ScopeRegistry

__all__ = ["AnalysisResult", "analyze"]


@dataclass(frozen=True)
class AnalysisResult:
    detection: DetectionResult
    disclosure: DisclosureResult
    intake: IntakeResult | None = field(default=None)

    @property
    def disclosed_scope_id(self) -> str | None:
        return self.disclosure.disclosed_scope_id

    @property
    def intake_rejected(self) -> tuple[RejectedReport, ...]:
        return () if self.intake is None else self.intake.rejected


def analyze(
    reports: list[Report],
    registry: ScopeRegistry,
    *,
    now: datetime | None = None,
    detection_config: DetectionConfig = DEFAULT_DETECTION_CONFIG,
    disclosure_config: DisclosureConfig = DEFAULT_DISCLOSURE_CONFIG,
```

```
    intake_config: IntakeConfig | None = DEFAULT_INTAKE_CONFIG,
    hysteresis_state: HysteresisState | None = None,
    prior_disclosed_scope_id: str | None = None,
) -> AnalysisResult:
    """Run intake caps, then detection, then -- if it fired -- disclosure.

    ``intake_config`` applies the per-session submission caps (rule 11)
    before detection sees anything; only ``accepted`` reports go forward,
    and any rejections are on ``result.intake``. Reports with no
    ``session_id`` are never rate-limited, so a stream of anonymous
    simulator reports is unaffected. Pass ``intake_config=None`` to skip
    the caps entirely (``result.intake`` is then ``None``).

    ``hysteresis_state`` and ``prior_disclosed_scope_id`` thread state from
    the PREVIOUS call in a sequence (e.g. yesterday's ``AnalysisResult``);
    omit both for a one-off analysis. The next hysteresis state to pass
    into the following call is on ``result.detection.hysteresis_state``,
    and the scope to pass as next call's ``prior_disclosed_scope_id`` is
    ``result.disclosed_scope_id``.
    """
    intake_result: IntakeResult | None = None
    if intake_config is not None:
        intake_result = intake.enforce_submission_caps(reports, config=intake_config)
        reports = list(intake_result.accepted)

    det = detection.evaluate(
        reports, registry, now=now, config=detection_config, hysteresis_state=hysteresis
_state
    )

    if not det.fired:
        # Nothing detected -> the disclosure engine does not run at all.
        return AnalysisResult(
            det,
            DisclosureResult(evaluations=(), disclosed_scope_id=None),
            intake_result,
        )

    dis = disclosure.evaluate(
        registry,
        det.qualifying_scope_counts,
        config=disclosure_config,
        prior_disclosed_scope_id=prior_disclosed_scope_id,
    )
    return AnalysisResult(det, dis, intake_result)
```

microcluster/intake.py

```
"""Intake submission caps (rule 11).

The system is anonymous by design, so it cannot authenticate a reporter.
The only handle it has on a submission is an opaque, ephemeral
``session_id`` the client sends with each report. This module caps how
many reports one session_id may file. That is the whole mitigation:

  * RATE LIMITING, NOT AUTHENTICATION. A report with ``session_id is
    None`` has no key to rate-limit and is always accepted. The simulator
    generates such reports (it is not modelling an intake client), so the
    normal pipeline is unaffected by this module.
  * A per-session cap is trivially defeated by using more sessions. An
    attacker who wants K fabricated reports past the cap needs
    ``ceil(K / session_submission_cap)`` sessions -- the same K reports,
    from more sources. ``adversarial.injection`` measures this directly.

Enforcement is a pure function over a batch of reports plus an
``IntakeConfig``. It is wired into ``engine.analyze`` as the first step,
so a rejection is a normal engine outcome (``AnalysisResult.intake``),
never a silent drop.
"""

from __future__ import annotations

from dataclasses import dataclass, field

from .config import DEFAULT_INTAKE_CONFIG, IntakeConfig
from .models import Report

__all__ = ["RejectedReport", "IntakeResult", "enforce_submission_caps"]


@dataclass(frozen=True)
class RejectedReport:
    """A report intake refused, with the reason. The report is kept so a
    caller can surface or log it -- it just never reaches detection."""

    report: Report
    reason: str


@dataclass(frozen=True)
class IntakeResult:
    accepted: tuple[Report, ...]
    rejected: tuple[RejectedReport, ...] = field(default=())

    @property
    def n_rejected(self) -> int:
        return len(self.rejected)


def enforce_submission_caps(
```

```
    reports: list[Report],
    *,
    config: IntakeConfig = DEFAULT_INTAKE_CONFIG,
) -> IntakeResult:
    """Apply the per-session and per-session-per-day submission caps.

    Reports are considered in ``submitted_at`` order (stable for ties), so
    which reports of an over-cap session get through is deterministic: the
    earliest ones, up to the cap. A report is accepted only if, counting
    it, that session_id is still within BOTH:

      * ``session_submission_cap`` reports total (the session's lifetime,
        as far as this batch shows), and
      * ``daily_submission_cap`` reports on that report's calendar date.

    ``session_id is None`` -> always accepted (nothing to key on).
    """
    ordered = sorted(
        enumerate(reports), key=lambda pair: (pair[1].submitted_at, pair[0])
    )

    session_total: dict[str, int] = {}
    session_day: dict[tuple[str, object], int] = {}
    verdicts: dict[int, RejectedReport | None] = {}

    for original_index, report in ordered:
        sid = report.session_id
        if sid is None:
            verdicts[original_index] = None
            continue

        day_key = (sid, report.submitted_at.date())
        would_be_total = session_total.get(sid, 0) + 1
        would_be_day = session_day.get(day_key, 0) + 1

        if would_be_total > config.session_submission_cap:
            verdicts[original_index] = RejectedReport(
                report,
                f"session submission cap reached "
                f"({config.session_submission_cap} per session)",
            )
        elif would_be_day > config.daily_submission_cap:
            verdicts[original_index] = RejectedReport(
                report,
                f"daily submission cap reached "
                f"({config.daily_submission_cap} per session per day)",
            )
        else:
            session_total[sid] = would_be_total
            session_day[day_key] = would_be_day
            verdicts[original_index] = None

    accepted: list[Report] = []
    rejected: list[RejectedReport] = []
    for i, report in enumerate(reports):  # original order preserved
```

```
        v = verdicts[i]
        if v is None:
            accepted.append(report)
        else:
            rejected.append(v)

    return IntakeResult(accepted=tuple(accepted), rejected=tuple(rejected))
```

microcluster/models.py

```
"""Data model: the fixed checklist, scopes, and reports.

There is no free text anywhere in this module and no medical concept
finer than a *category* (rule 1, rule 2). A ``Report`` records a specific
symptom only for the reporter's own clarity; nothing in the engines ever
reads ``Report.symptom`` for matching.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from datetime import datetime
from enum import Enum, IntEnum


class Category(str, Enum):
    """The four fixed checklist categories. Clustering happens here and
    nowhere finer."""

    RESPIRATORY = "Respiratory"
    GASTROINTESTINAL = "Gastrointestinal"
    GENERAL = "General"
    OTHER = "Other"


# The fixed checklist: 3-5 named symptoms per category. The symptom text is
# a closed vocabulary chosen at design time -- it is NOT free text, and it
# is never used for matching (rule 2). It exists so a reporter can pick the
# thing that matches what they feel.
CHECKLIST: dict[Category, tuple[str, ...]] = {
    Category.RESPIRATORY: (
        "cough",
        "sore throat",
        "shortness of breath",
        "runny or blocked nose",
        "fever with cough",
    ),
    Category.GASTROINTESTINAL: (
        "nausea",
        "vomiting",
        "diarrhea",
        "stomach cramps",
    ),
    Category.GENERAL: (
        "fever",
        "fatigue",
        "body aches",
        "headache",
        "chills",
    ),
    Category.OTHER: (
        "skin rash",
```

```
        "dizziness",
        "loss of taste or smell",
    ),
}


class Onset(str, Enum):
    """Coarse onset buckets. Deliberately imprecise."""

    TODAY = "today"
    ONE_TO_TWO_DAYS = "1-2 days"
    THREE_TO_SEVEN_DAYS = "3-7 days"
    ONE_WEEK_PLUS = "1+ week"


class ScopeLevel(IntEnum):
    """Hierarchy depth. Larger value == finer scope. Used to pick the
    'finest' scope in rule 9 -- there is no hardcoded tier list, only this
    ordering over whatever scopes exist."""

    CAMPUS = 1
    BUILDING = 2
    FLOOR = 3
    SUITE = 4


@dataclass(frozen=True)
class Scope:
    """One node in the location hierarchy.

    ``population`` is DECLARED at group creation (rule 7). It is never the
    number of people who reported. Every gate in the disclosure engine
    uses this number.
    """

    id: str
    label: str
    level: ScopeLevel
    population: int
    parent_id: str | None = None

    def __post_init__(self) -> None:
        if self.population <= 0:
            raise ValueError(f"scope {self.id!r}: declared population must be positiv
e")
        if self.level == ScopeLevel.CAMPUS and self.parent_id is not None:
            raise ValueError(f"scope {self.id!r}: a CAMPUS scope has no parent")
        if self.level != ScopeLevel.CAMPUS and self.parent_id is None:
            raise ValueError(f"scope {self.id!r}: non-CAMPUS scope needs a parent_id")


@dataclass(frozen=True)
class ScopeRegistry:
    """The declared set of scopes. Built once, at group creation."""

```

```
    scopes: dict[str, Scope]

    def __post_init__(self) -> None:
        for scope_id, scope in self.scopes.items():
            if scope.id != scope_id:
                raise ValueError(f"registry key {scope_id!r} != scope id {scope.id!r}")
            if scope.parent_id is not None and scope.parent_id not in self.scopes:
                raise ValueError(f"scope {scope_id!r}: unknown parent {scope.parent_id!
r}")

    def get(self, scope_id: str) -> Scope:
        try:
            return self.scopes[scope_id]
        except KeyError:
            raise KeyError(f"unknown scope id: {scope_id!r}") from None

    def chain_to_root(self, scope_id: str) -> list[Scope]:
        """``scope_id`` itself, then each ancestor, up to the campus root."""
        out: list[Scope] = []
        current: str | None = scope_id
        seen: set[str] = set()
        while current is not None:
            if current in seen:
                raise ValueError(f"scope cycle detected at {current!r}")
            seen.add(current)
            scope = self.get(current)
            out.append(scope)
            current = scope.parent_id
        return out

    def contains(self, ancestor_id: str, scope_id: str) -> bool:
        """True if ``scope_id`` is ``ancestor_id`` or sits below it."""
        return any(s.id == ancestor_id for s in self.chain_to_root(scope_id))

    def roots(self) -> list[Scope]:
        return [s for s in self.scopes.values() if s.parent_id is None]

    def campus_population(self) -> int:
        """Total declared population: the sum of the root scopes'
        declared populations. Used as the comparison denominator by both
        detectors."""
        roots = self.roots()
        if not roots:
            raise ValueError("registry has no root (CAMPUS) scope")
        return sum(s.population for s in roots)


@dataclass(frozen=True)
class Report:
    """One anonymous submission from the fixed checklist.

    ``session_id`` is an opaque, ephemeral token used only for the
    intake-time submission caps (rule 11). The engines never read it.
    """

```

```
    category: Category
    symptom: str
    onset: Onset
    location_id: str
    submitted_at: datetime
    session_id: str | None = field(default=None)

    def __post_init__(self) -> None:
        if not isinstance(self.category, Category):
            raise ValueError(f"category must be a Category, got {self.category!r}")
        if not isinstance(self.onset, Onset):
            raise ValueError(f"onset must be an Onset, got {self.onset!r}")
        if self.symptom not in CHECKLIST[self.category]:
            raise ValueError(
                f"symptom {self.symptom!r} is not in the fixed checklist for "
                f"category {self.category.value!r}"
            )
```

simulation/__init__.py

```
"""Headless outbreak simulator.

Simulates an illness spreading suite-by-suite through a small shared
building, generates anonymous symptom reports from it, and feeds them to
``microcluster``'s existing, unmodified detection engine. No rendering, no
API, no frontend.

It only ever *reads* from ``microcluster`` (``models``, ``config``,
``engine``) and never modifies it. See ``simulation.pipeline`` for the
end-to-end wiring, and CLAUDE.md's "ground-truth firewall" for what is and
is not allowed to cross from this package into the production engines.
"""

from . import agent, config, contact, infection, pipeline, population, reporting, simula
tor
from .agent import Agent, AgentSnapshot
from .config import DEFAULT_SIMULATION_CONFIG, SimulationConfig
from .infection import INFECTIOUS_STATES, STATE_ORDER, InfectionState, order_index
from .pipeline import (
    DailyRecord,
    SimulatedReports,
    SimulationRun,
    analyze_report_stream,
    run_with_detection,
    simulate_and_report,
)
from .population import StructureSpec, build_population
from .simulator import DayState, Simulation

__all__ = [
    "agent",
    "config",
    "contact",
    "infection",
    "pipeline",
    "population",
    "reporting",
    "simulator",
    "Agent",
    "AgentSnapshot",
    "SimulationConfig",
    "DEFAULT_SIMULATION_CONFIG",
    "InfectionState",
    "STATE_ORDER",
    "INFECTIOUS_STATES",
    "order_index",
    "StructureSpec",
    "build_population",
    "Simulation",
    "DayState",
    "DailyRecord",
    "SimulationRun",
```

```
    "SimulatedReports",
    "simulate_and_report",
    "analyze_report_stream",
    "run_with_detection",
]
```

simulation/agent.py

```
"""The Agent: one simulated person."""

from __future__ import annotations

from dataclasses import dataclass

from .infection import InfectionState


@dataclass
class Agent:
    """One simulated person.

    An agent belongs to exactly one suite. Every ``Report`` it produces
    (see ``simulation.reporting``) uses ``suite_id`` as the
    ``location_id`` -- always, and only that (detection.evaluate rejects
    mixed-level input).

    ``incubation_days`` / ``symptomatic_days`` are drawn when the agent is
    infected and are ``None`` before then.
    """

    id: str
    suite_id: str
    # Personal probability of filing a report while SYMPTOMATIC. Not
    # everyone reports; assigned once at population creation.
    reporting_probability: float
    state: InfectionState = InfectionState.SUSCEPTIBLE
    # Day on which ``state`` last changed (0 = start of the run).
    state_changed_day: int = 0
    incubation_days: int | None = None
    symptomatic_days: int | None = None
    # Whether this agent has already filed its (at most one) report for
    # the current illness episode. Reports are anonymous, so this flag
    # lives only here -- it never appears on a Report.
    has_reported: bool = False

    def __post_init__(self) -> None:
        if not 0.0 <= self.reporting_probability <= 1.0:
            raise ValueError(
                f"agent {self.id}: reporting_probability "
                f"{self.reporting_probability!r} is outside [0, 1]"
            )


@dataclass(frozen=True)
class AgentSnapshot:
    """Immutable view of one agent at the end of a simulated day."""

    id: str
    suite_id: str
```

```
52 |     state: InfectionState
53 |     state_changed_day: int
```

simulation/benchmark.py

```
"""Learned detector vs the hand-authored threshold rules.

Still headless -- no plotting. Builds a labelled dataset from simulated
runs, trains a small classifier on it, and scores that classifier against
the authored rules in ``microcluster`` on HELD-OUT runs.

    python -m simulation.benchmark

---------------------------------------------------------------------
WHAT "ONE ROW PER (day, scope)" MEANS HERE
---------------------------------------------------------------------

``microcluster.detection`` operates at exactly one scope: the campus, the
level its two detectors compare against (a location versus its campus
peers; total volume versus the campus background expectation). It emits
its three plain findings once per evaluation. So the dataset is one row
per (run, day): the features are that day's three findings computed on the
reports accumulated so far, and the label is ground truth for that day.
This is the unit the authored rules also produce a decision at, which is
what makes the head-to-head comparison well posed.

Features (exactly the three findings ``microcluster.detection`` reports,
rule 6 -- no composite, nothing finer than a category):

  * ``relative_excess_ratio``   -- the hotspot location's smoothed per-capita
                                   rate over its peers'
  * ``count_within_window``     -- qualifying reports in the 72h window
  * ``count_sharing_category``  -- how many of those share the dominant category

Label (from ground truth, never shown to the engines): 1 iff on that day
the outbreak is BOTH established (cumulative ever-infected >=
``ESTABLISHED_MIN_INFECTIONS``) AND still active (at least one agent
INCUBATING or SYMPTOMATIC). This is a CHOSEN definition of "an outbreak
the system ought to be flagging right now", in the same spirit as every
threshold in ``config.py``; it is not a measured quantity.

---------------------------------------------------------------------
METHODOLOGY
---------------------------------------------------------------------

* SPLIT BY RUN, never by day. Adjacent days from one outbreak are highly
  correlated; letting them straddle the split would leak the test set into
  training and flatter the model.

* CROSS-REGIME EVALUATION. A model trained on this simulator learns this
  simulator. The benchmark therefore also trains on one transmission
  regime (the shipped ``SUITE_TRANSMISSION_PROBABILITY``) and evaluates on
  runs generated with a different one (``BENCHMARK_TEST_SUITE_TRANSMISSIONS``
  -- a slower and a faster epidemic). If the model only beats the rules on
  its training distribution, ``main`` says so in plain language. A clean
  negative result is reported, not hidden.

```

```
* THE MODEL NEVER REPLACES THE RULES. Both stay in the codebase. This
  module scores them side by side and nothing more; the disclosure path
  still runs on the authored rules alone.
"""

from __future__ import annotations

import random
import sys
from collections import defaultdict
from dataclasses import dataclass
from datetime import timedelta
from pathlib import Path
from statistics import mean
from typing import Sequence

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

import numpy as np  # noqa: E402
from sklearn.linear_model import LogisticRegression  # noqa: E402
from sklearn.tree import DecisionTreeClassifier  # noqa: E402

from microcluster.config import DEFAULT_DETECTION_CONFIG, DetectionConfig  # noqa: E402
from microcluster.detection import evaluate as detection_evaluate  # noqa: E402
from microcluster.models import Report  # noqa: E402

from simulation.config import (  # noqa: E402
    BENCHMARK_DEFAULT_SEEDS,
    BENCHMARK_SPLIT_SEED,
    BENCHMARK_TEST_RUN_FRACTION,
    BENCHMARK_TEST_SUITE_TRANSMISSIONS,
    BENCHMARK_TRAIN_SUITE_TRANSMISSION,
    BENCHMARK_TREE_MAX_DEPTH,
    DEFAULT_RUN_DAYS,
    DEFAULT_SIMULATION_CONFIG,
    ESTABLISHED_MIN_INFECTIONS,
    SimulationConfig,
)
from simulation.infection import InfectionState  # noqa: E402
from simulation.pipeline import simulate_and_report  # noqa: E402

__all__ = [
    "FEATURE_NAMES",
    "DayRow",
    "build_dataset",
    "split_by_run",
    "train_model",
    "Score",
    "score_rows",
    "score_model",
    "score_rules",
    "benchmark_in_distribution",
    "benchmark_cross_regime",
]

```

```
FEATURE_NAMES: tuple[str, ...] = (
    "relative_excess_ratio",
    "count_within_window",
    "count_sharing_category",
)


# ---------------------------------------------------------------------------
# Dataset
# ---------------------------------------------------------------------------


@dataclass(frozen=True)
class DayRow:
    """One (run, day): the authored detector's three findings, the authored
    rule's own verdict, and the ground-truth label."""

    seed: int
    day: int
    relative_excess_ratio: float
    count_within_window: int
    count_sharing_category: int
    label: int  # 1 iff established AND active on this day (ground truth)
    rule_fired: bool  # microcluster.detection raw single-instant verdict


def _cumulative_infected(agents) -> int:
    return sum(1 for a in agents if a.state is not InfectionState.SUSCEPTIBLE)


def _active_infected(agents) -> int:
    return sum(
        1
        for a in agents
        if a.state in (InfectionState.INCUBATING, InfectionState.SYMPTOMATIC)
    )


def _ground_truth_label(agents, established_min_infections: int) -> int:
    established = _cumulative_infected(agents) >= established_min_infections
    active = _active_infected(agents) > 0
    return int(established and active)


def build_dataset(
    seeds: Sequence[int],
    *,
    days: int = DEFAULT_RUN_DAYS,
    config: SimulationConfig = DEFAULT_SIMULATION_CONFIG,
    detection_config: DetectionConfig = DEFAULT_DETECTION_CONFIG,
    established_min_infections: int = ESTABLISHED_MIN_INFECTIONS,
) -> list[DayRow]:
    """Simulate each seed once and emit one :class:`DayRow` per simulated
    day (day 0 .. ``days``). Ground truth is read straight from the
    simulator's history; only ``Report`` objects and the registry are ever
```

```
    passed to ``microcluster.detection`` (the ground-truth firewall)."""
    rows: list[DayRow] = []
    for seed in seeds:
        simulated = simulate_and_report(seed=seed, days=days, config=config)
        sim = simulated.simulation
        accumulated: list[Report] = []
        for day, day_reports in enumerate(simulated.daily_reports):
            accumulated.extend(day_reports)
            now = config.simulation_start + timedelta(days=day)
            result = detection_evaluate(
                accumulated, sim.registry, now=now, config=detection_config
            )
            snapshot = sim.history[day]
            rows.append(
                DayRow(
                    seed=seed,
                    day=day,
                    relative_excess_ratio=result.findings.relative_excess_ratio,
                    count_within_window=result.findings.count_within_window,
                    count_sharing_category=result.findings.count_sharing_category,
                    label=_ground_truth_label(snapshot.agents, established_min_infectio
ns),
                    rule_fired=result.raw_fired,
                )
            )
    return rows


def split_by_run(
    rows: Sequence[DayRow],
    *,
    test_fraction: float = BENCHMARK_TEST_RUN_FRACTION,
    split_seed: int = BENCHMARK_SPLIT_SEED,
) -> tuple[list[DayRow], list[DayRow]]:
    """Partition ``rows`` into (train, test) by SEED. No seed appears in
    both halves."""
    seeds = sorted({r.seed for r in rows})
    rng = random.Random(split_seed)
    rng.shuffle(seeds)
    n_test = max(1, round(len(seeds) * test_fraction))
    test_seeds = set(seeds[:n_test])
    train = [r for r in rows if r.seed not in test_seeds]
    test = [r for r in rows if r.seed in test_seeds]
    return train, test


# ---------------------------------------------------------------------------
# Model
# ---------------------------------------------------------------------------


def _feature_matrix(rows: Sequence[DayRow]) -> np.ndarray:
    return np.array(
        [
            [
```

```
                r.relative_excess_ratio,
                r.count_within_window,
                r.count_sharing_category,
            ]
            for r in rows
        ],
        dtype=float,
    )


def _labels(rows: Sequence[DayRow]) -> np.ndarray:
    return np.array([r.label for r in rows], dtype=int)


def train_model(
    train_rows: Sequence[DayRow],
    *,
    kind: str = "logistic",
    tree_max_depth: int = BENCHMARK_TREE_MAX_DEPTH,
):
    """Fit a small classifier on the three findings. ``kind`` is
    ``"logistic"`` (default) or ``"tree"`` -- both deliberately low
    capacity, so a win is a real signal and not memorisation."""
    y = _labels(train_rows)
    if len(np.unique(y)) < 2:
        raise ValueError(
            "training set has a single class; cannot fit a classifier "
            "(need both established-and-active and not days)"
        )
    X = _feature_matrix(train_rows)
    if kind == "tree":
        model = DecisionTreeClassifier(max_depth=tree_max_depth, random_state=0)
    elif kind == "logistic":
        model = LogisticRegression(max_iter=1000, class_weight="balanced")
    else:
        raise ValueError(f"unknown model kind {kind!r}")
    model.fit(X, y)
    return model


# ---------------------------------------------------------------------------
# Scoring
# ---------------------------------------------------------------------------


@dataclass(frozen=True)
class Score:
    """Row-level precision/recall plus a per-run detection-delay summary."""

    label: str  # "model (logistic)", "authored rules", ...
    precision: float
    recall: float
    n_predicted_positive: int
    n_actual_positive: int
    n_runs_with_outbreak: int
```

```
    n_runs_detected: int
    mean_detection_delay: float | None  # days, over detected runs only
    never_detected_runs: int


def score_rows(
    rows: Sequence[DayRow], predictions: Sequence[int], *, label: str
) -> Score:
    """Score a 0/1 prediction per row. Detection delay per run: first
    positive prediction on or after the run's ground-truth onset day, minus
    that onset day; runs never predicted positive after onset count as
    'never detected'."""
    preds = list(predictions)
    tp = sum(1 for r, p in zip(rows, preds) if p and r.label)
    fp = sum(1 for r, p in zip(rows, preds) if p and not r.label)
    fn = sum(1 for r, p in zip(rows, preds) if not p and r.label)
    precision = tp / (tp + fp) if (tp + fp) else 0.0
    recall = tp / (tp + fn) if (tp + fn) else 0.0

    by_run: dict[int, list[tuple[int, int, int]]] = defaultdict(list)
    for r, p in zip(rows, preds):
        by_run[r.seed].append((r.day, r.label, p))

    delays: list[int] = []
    runs_with_outbreak = 0
    never = 0
    for items in by_run.values():
        items.sort()
        onset_days = [d for d, lbl, _ in items if lbl]
        if not onset_days:
            continue
        runs_with_outbreak += 1
        onset = onset_days[0]
        fired_after = [d for d, _, p in items if p and d >= onset]
        if fired_after:
            delays.append(fired_after[0] - onset)
        else:
            never += 1

    return Score(
        label=label,
        precision=precision,
        recall=recall,
        n_predicted_positive=tp + fp,
        n_actual_positive=tp + fn,
        n_runs_with_outbreak=runs_with_outbreak,
        n_runs_detected=runs_with_outbreak - never,
        mean_detection_delay=mean(delays) if delays else None,
        never_detected_runs=never,
    )


def score_model(model, test_rows: Sequence[DayRow], *, label: str) -> Score:
    preds = model.predict(_feature_matrix(test_rows)).astype(int).tolist()
    return score_rows(test_rows, preds, label=label)
```

```
327 |
328 |
329 | def score_rules(test_rows: Sequence[DayRow], *, label: str = "authored rules") -> Scor
e:
330 |     return score_rows(
331 |         test_rows, [int(r.rule_fired) for r in test_rows], label=label
332 |     )
333 |
334 |
335 | # ---------------------------------------------------------------------------
336 | # The two benchmarks
337 | # ---------------------------------------------------------------------------
338 |
339 |
340 | @dataclass(frozen=True)
341 | class BenchmarkResult:
342 |     heading: str
343 |     n_train_rows: int
344 |     n_test_rows: int
345 |     scores: tuple[Score, ...]  # rules first, then each model
346 |
347 |
348 | def benchmark_in_distribution(
349 |     seeds: Sequence[int],
350 |     *,
351 |     days: int = DEFAULT_RUN_DAYS,
352 |     config: SimulationConfig = DEFAULT_SIMULATION_CONFIG,
353 | ) -> BenchmarkResult:
354 |     rows = build_dataset(seeds, days=days, config=config)
355 |     train, test = split_by_run(rows)
356 |     logistic = train_model(train, kind="logistic")
357 |     tree = train_model(train, kind="tree")
358 |     return BenchmarkResult(
359 |         heading="IN DISTRIBUTION (train and test are the same transmission regime)",
360 |         n_train_rows=len(train),
361 |         n_test_rows=len(test),
362 |         scores=(
363 |             score_rules(test),
364 |             score_model(logistic, test, label="model (logistic regression)"),
365 |             score_model(tree, test, label=f"model (tree, depth {BENCHMARK_TREE_MAX_DEPT
H})"),
366 |         ),
367 |     )
368 |
369 |
370 | def benchmark_cross_regime(
371 |     train_seeds: Sequence[int],
372 |     test_seeds: Sequence[int],
373 |     *,
374 |     days: int = DEFAULT_RUN_DAYS,
375 |     train_suite_transmission: float = BENCHMARK_TRAIN_SUITE_TRANSMISSION,
376 |     test_suite_transmissions: Sequence[float] = BENCHMARK_TEST_SUITE_TRANSMISSIONS,
377 | ) -> list[BenchmarkResult]:
378 |     """Train once on ``train_suite_transmission``; evaluate on held-out
379 |     seeds simulated at each of ``test_suite_transmissions``."""
```

```
    train_config = SimulationConfig(suite_transmission_probability=train_suite_transmis
sion)
    train_rows = build_dataset(train_seeds, days=days, config=train_config)
    logistic = train_model(train_rows, kind="logistic")
    tree = train_model(train_rows, kind="tree")

    results: list[BenchmarkResult] = []
    for tp in test_suite_transmissions:
        test_config = SimulationConfig(suite_transmission_probability=tp)
        test_rows = build_dataset(test_seeds, days=days, config=test_config)
        results.append(
            BenchmarkResult(
                heading=(
                    f"CROSS REGIME (trained at suite p={train_suite_transmission:g}, "
                    f"tested at suite p={tp:g})"
                ),
                n_train_rows=len(train_rows),
                n_test_rows=len(test_rows),
                scores=(
                    score_rules(test_rows),
                    score_model(logistic, test_rows, label="model (logistic regressio
n)"),
                    score_model(
                        tree, test_rows, label=f"model (tree, depth {BENCHMARK_TREE_MAX
_DEPTH})"
                    ),
                ),
            )
        )
    return results


# ---------------------------------------------------------------------------
# Reporting
# ---------------------------------------------------------------------------


def _fmt(value: float | None, decimals: int = 2) -> str:
    return "n/a" if value is None else f"{value:.{decimals}f}"


def _print_result(result: BenchmarkResult) -> None:
    print(f"--- {result.heading} ---")
    print(f"    {result.n_train_rows} train rows, {result.n_test_rows} test rows")
    cols = [
        ("detector", 30),
        ("precision", 10),
        ("recall", 8),
        ("delay", 7),
        ("detected", 12),
    ]
    header = "  ".join(f"{name:>{w}}" if name != "detector" else f"{name:<{w}}" for nam
e, w in cols)
    print(header)
    print("-" * len(header))
```

```
431 |     for s in result.scores:
432 |         detected = f"{s.n_runs_detected}/{s.n_runs_with_outbreak}"
433 |         row = (
434 |             f"{s.label:<30}",
435 |             f"{_fmt(s.precision):>10}",
436 |             f"{_fmt(s.recall):>8}",
437 |             f"{_fmt(s.mean_detection_delay, 1):>7}",
438 |             f"{detected:>12}",
439 |         )
440 |         print("  ".join(row))
441 |     print()
442 |
443 |
444 | def _tradeoff(model: Score, rules: Score) -> str:
445 |     """One plain line comparing a model score to the rules score."""
446 |     dp = model.precision - rules.precision
447 |     dr = model.recall - rules.recall
448 |     md, rd = model.mean_detection_delay, rules.mean_detection_delay
449 |     dd = None if (md is None or rd is None) else md - rd
450 |     return (
451 |         f"precision {dp:+.2f}, recall {dr:+.2f}, "
452 |         f"delay {'n/a' if dd is None else f'{dd:+.1f}d'}"
453 |     )
454 |
455 |
456 | def _verdict(in_dist: BenchmarkResult, cross: list[BenchmarkResult]) -> None:
457 |     print("=" * 78)
458 |     print("VERDICT  (logistic model vs authored rules, deltas are model minus rules)")
459 |     print("=" * 78)
460 |     print(f"  in distribution                         : {_tradeoff(in_dist.scores[1], i
n_dist.scores[0])}")
461 |     cross_deltas = []
462 |     for res in cross:
463 |         d = res.scores[1].precision - res.scores[0].precision, res.scores[1].recall - r
es.scores[0].recall
464 |         cross_deltas.append(d)
465 |         tail = res.heading.split("tested at ")[-1].rstrip(")")
466 |         print(f"  cross regime, {tail:26} : {_tradeoff(res.scores[1], res.scores[0])}")
467 |     print()
468 |     recall_gain_everywhere = all(dr > 0.02 for _, dr in cross_deltas) and (
469 |         in_dist.scores[1].recall - in_dist.scores[0].recall > 0.02
470 |     )
471 |     precision_cost = -min(
472 |         [in_dist.scores[1].precision - in_dist.scores[0].precision]
473 |         + [dp for dp, _ in cross_deltas]
474 |     )
475 |     if recall_gain_everywhere:
476 |         print("  The model consistently catches more real outbreak-days and fires about
a")
477 |         print(f"  day sooner, IN AND OUT of its training regime, at a cost of up to")
478 |         print(f"  {precision_cost:.2f} in precision. The gain survives the transmission
-probability")
479 |         print("  shift, so it is not purely memorised -- but it is small, it is bought
with")
480 |         print("  false positives, and it is still a model of THIS simulator. The author
```

```
ed")
481 |         print("  rules remain the shipped detector; the model is kept only as this benc
hmark.")
482 |     else:
483 |         print("  The model's advantage does not hold up once the transmission probabili
ty")
484 |         print("  differs from training. Reported as a negative result: the classifier")
485 |         print("  mostly learned this simulator. The three authored thresholds are the")
486 |         print("  stronger, and the shipped, choice.")
487 |     print()
488 |
489 |
490 | def main() -> None:
491 |     n = BENCHMARK_DEFAULT_SEEDS
492 |     all_seeds = list(range(1, n + 1))
493 |     bar = "=" * 78
494 |
495 |     print(bar)
496 |     print(f"LEARNED DETECTOR vs AUTHORED RULES   ({n} simulated runs, {DEFAULT_RUN_DAY
S}-day)")
497 |     print("features: relative_excess_ratio, count_within_window, count_sharing_categor
y")
498 |     print(f"label   : established (cumulative >= {ESTABLISHED_MIN_INFECTIONS}) AND acti
ve, from ground truth")
499 |     print("split   : by RUN, never by day")
500 |     print(bar)
501 |     print()
502 |
503 |     in_dist = benchmark_in_distribution(all_seeds)
504 |     _print_result(in_dist)
505 |
506 |     # Cross-regime: reuse the same split so train/test seeds never overlap.
507 |     train_seeds, test_seeds = split_by_run(
508 |         [DayRow(s, 0, 0.0, 0, 0, 0, False) for s in all_seeds]
509 |     )
510 |     train_ids = sorted({r.seed for r in train_seeds})
511 |     test_ids = sorted({r.seed for r in test_seeds})
512 |     cross = benchmark_cross_regime(train_ids, test_ids)
513 |     for res in cross:
514 |         _print_result(res)
515 |
516 |     print("  delay = mean days from ground-truth outbreak onset to first positive cal
l")
517 |     print("          (over runs that were detected at all); detected = runs called / ru
ns")
518 |     print("          with a real outbreak.")
519 |     print()
520 |     _verdict(in_dist, cross)
521 |
522 |
523 | if __name__ == "__main__":
524 |     main()
```

simulation/config.py

```
"""Tunable constants for the outbreak simulator.

EVERY value the simulation depends on lives here as a named constant with
a comment, exactly as in ``microcluster/config.py``. Nothing elsewhere in
the ``simulation`` package hard-codes a tunable number.

None of these figures is clinical or epidemiological. They are chosen to
produce a legible demonstration outbreak on a small population, not
measured from real data. Tune them before drawing any real conclusion.

PROVENANCE OF THESE NUMBERS
---------------------------------------------------------------------------
There is no real dataset behind this simulator, so there are no citations
to give for the specific values. Every transmission probability, reporting
rate, contact rate and state duration below was PICKED FOR DEMONSTRATION:
set by hand, checked by running the model, and kept at the figure that
produced a believable outbreak on a 150-person building inside a 30-day
window (see the joint-tuning note at ``SUITE_TRANSMISSION_PROBABILITY``).

Where the SHAPE of a modelling choice follows standard practice, that is
noted, and only the shape -- not any number -- is borrowed:

* compartmental progression susceptible -> incubating -> symptomatic ->
  recovered is the classic SIR / SEIR structure (Kermack & McKendrick,
  1927; and every agent-based descendant since).
* tiered per-contact transmission by shared space (suite / floor /
  building) is the ordinary agent-based-model treatment of contact
  structure.

The specific numbers are ours and illustrative only. Anything user-facing
says so in the same words (see ``api.app.DISCLAIMER`` and the on-page
limitations banner).
"""

from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime, timezone

from microcluster.config import BACKGROUND_RATE, DETECTION_WINDOW_HOURS
from microcluster.models import Category

# ---------------------------------------------------------------------------
# Infection state machine: state durations (illustrative, NOT clinical)
# ---------------------------------------------------------------------------

# On infection an agent spends this many days INCUBATING (already
# infectious, not yet reporting) before turning SYMPTOMATIC. Drawn
# uniformly per agent from this inclusive range. Kept short, together with
# SYMPTOMATIC_DAYS below, so a seeded suite's outbreak runs its course
# (and can burn out at a PARTIAL attack rate) inside the 30-day smoke
# window, rather than smouldering indefinitely -- see
```

```
# SUITE_TRANSMISSION_PROBABILITY below for how the two are tuned jointly.
INCUBATION_DAYS_MIN: int = 1
INCUBATION_DAYS_MAX: int = 2

# Once SYMPTOMATIC an agent stays symptomatic this many days before moving
# to RECOVERED. Drawn uniformly per agent from this inclusive range.
SYMPTOMATIC_DAYS_MIN: int = 1
SYMPTOMATIC_DAYS_MAX: int = 5


# ---------------------------------------------------------------------------
# Contact model: tiered by scope, reached through the ScopeRegistry
# ---------------------------------------------------------------------------
#
# Contact is not suite-only. On each daily step an infectious agent can
# transmit to a susceptible agent that shares its SUITE, its FLOOR (a
# different suite on the same floor), or its BUILDING (a different floor
# in the same building) -- at three different, decreasing per-contact
# probabilities. Campus-level contact is not modelled: two people who only
# share a campus are assumed not to meet. Which suite/floor/building an
# agent belongs to is read from the ScopeRegistry (chain_to_root), never
# hard-coded from id strings.
#
# Each probability compounds over however many infectious contacts an
# agent has at that tier:
#   P(infected today) = 1 - (1-p_suite)**k_suite
#                         * (1-p_floor)**k_floor
#                         * (1-p_building)**k_building
#
# All three, together with the state durations above, are chosen jointly
# and EMPIRICALLY (by running the model, not solved analytically) so that
# a single seeded suite typically reaches a PARTIAL, seed-varying attack
# rate -- roughly in the 40-70% range on most seeds, occasionally higher,
# occasionally fizzling out entirely from just the one index case -- rather
# than either dying out every time or deterministically infecting everyone.
# A small, fully-mixed suite is close to an all-or-nothing branching
# process: a real outbreak sometimes just fails to take off, and that is
# reported honestly (a low attack rate), not smoothed away. Floor and
# building transmission are lower than suite transmission but not
# negligible, so the outbreak reaches a second suite, and sometimes a
# second floor, well before it reaches everyone on the first floor.

# Probability that one infectious agent infects one susceptible SUITE-mate
# on a single day.
SUITE_TRANSMISSION_PROBABILITY: float = 0.008

# Probability that one infectious agent infects one susceptible agent in a
# DIFFERENT suite on the SAME FLOOR, on a single day. Lower than the suite
# rate: floor-mates share common areas but not a room.
FLOOR_TRANSMISSION_PROBABILITY: float = 0.003

# Probability that one infectious agent infects one susceptible agent on a
# DIFFERENT floor of the SAME BUILDING, on a single day. Lower still: the
# only shared contact is building-wide common space.
BUILDING_TRANSMISSION_PROBABILITY: float = 0.001
```

```
108 |
109 |
110 | # ---------------------------------------------------------------------------
111 | # Reporting behaviour
112 | # ---------------------------------------------------------------------------
113 |
114 | # Each agent is assigned, once at population creation, a personal
115 | # probability of filing a symptom report while SYMPTOMATIC. Not everyone
116 | # reports. Drawn uniformly per agent from this inclusive range. No real
117 | # source: a spread picked for demo so the detector has a partial, noisy
118 | # signal to work with rather than a full census of the sick.
119 | REPORTING_PROBABILITY_MIN: float = 0.30
120 | REPORTING_PROBABILITY_MAX: float = 0.70
121 |
122 | # The outbreak clusters at ONE category, chosen once per simulation (rule
123 | # 2: clustering is at category level only). The specific symptom text is
124 | # still drawn from the fixed CHECKLIST for realism, but -- as everywhere
125 | # else in this project -- it is never used for matching.
126 | OUTBREAK_CATEGORY: Category = Category.RESPIRATORY
127 |
128 | # BACKGROUND NOISE. Agents who are NOT currently symptomatic from the
129 | # outbreak also file unrelated reports (a sore throat that is just a sore
130 | # throat, a stomach bug that has nothing to do with any of this), each day,
131 | # independently, at this rate, in a category OTHER than OUTBREAK_CATEGORY.
132 | #
133 | # This is what makes the detection problem real. Without background noise,
134 | # every single report in the stream is outbreak signal, the absolute
135 | # detector's BACKGROUND_RATE comparison is meaningless (there is no
136 | # background to compare against), and detection is trivial. With it, the
137 | # detector has to find a category-level cluster inside a stream that also
138 | # contains ordinary, everyday, unrelated complaints.
139 | #
140 | # Chosen to land in the neighbourhood of microcluster.config.BACKGROUND_RATE
141 | # (the illustrative fraction of a population expected to report per
142 | # DETECTION_WINDOW_HOURS window under ordinary conditions), spread evenly
143 | # across that window's days: BACKGROUND_RATE / (DETECTION_WINDOW_HOURS / 24).
144 | BACKGROUND_NOISE_DAILY_RATE: float = BACKGROUND_RATE / (DETECTION_WINDOW_HOURS / 24)
145 |
146 | # Reference wall-clock instant for simulated day 0. Every Report's
147 | # ``submitted_at`` and every call to ``microcluster.engine.analyze`` is
148 | # timestamped relative to this, so the 72-hour detection window is
149 | # evaluated against SIMULATED time, never the real clock.
150 | SIMULATION_START: datetime = datetime(2026, 1, 1, tzinfo=timezone.utc)
151 |
152 |
153 | # ---------------------------------------------------------------------------
154 | # Seeding and run length
155 | # ---------------------------------------------------------------------------
156 |
157 | # Agents infected at day 0 when a run starts.
158 | DEFAULT_SEED_INFECTIONS: int = 1
159 |
160 | # Length of the smoke-test run, in days.
161 | DEFAULT_RUN_DAYS: int = 30
162 |
```

```
163 | # Random seeds the smoke-test entry point iterates over.
164 | DEFAULT_SMOKE_SEEDS: tuple[int, ...] = (1, 2, 3, 4, 5)
165 |
166 |
167 | # ---------------------------------------------------------------------------
168 | # Experiment runner (simulation.experiments): establishment and the sweep
169 | # ---------------------------------------------------------------------------
170 | #
171 | # A single seeded index case is a stochastic branching process: a real
172 | # fraction of runs fail to take off at all, purely by chance, before the
173 | # detector or the disclosure policy ever get a say. An outbreak counts as
174 | # ESTABLISHED once its cumulative (ever-infected) case count reaches this
175 | # many -- below it, the run is a fade-out from the index case, not a real
176 | # event. This is a METHODOLOGICAL CHOICE, not a modelling fact: see
177 | # simulation/experiments.py's module docstring for why it matters and how
178 | # it is applied (report the fade-out rate on its own; condition every
179 | # other statistic on establishment; never condition the false-alarm rate
180 | # on it, since that is measured over the runs that did NOT establish).
181 | ESTABLISHED_MIN_INFECTIONS: int = 5
182 |
183 | # Default number of random seeds the experiment runner sweeps over.
184 | DEFAULT_EXPERIMENT_SEEDS: int = 500
185 |
186 | # The delay-versus-infections curve sweeps the DISCLOSURE gates, not the
187 | # detection threshold. An earlier attempt swept ABSOLUTE_EXCESS_RATIO and
188 | # found time-to-disclosure completely invariant to it: with detection
189 | # firing well before disclosure clears anyway, it is the disclosure gates
190 | # that set the timing. Detection config is held at defaults for all three
191 | # sweeps below.
192 | #
193 | # Every one of these gates ultimately compares an INTEGER qualifying-report
194 | # count (or a declared population) against a threshold, so each parameter
195 | # only changes behaviour when it crosses a discrete boundary -- the scope
196 | # populations are 25 / 75 / 150 (suite / floor / building = campus, since
197 | # the default structure has one building), and ceil(sqrt(n)) is 5 / 9 / 13.
198 | # The ranges below are deliberately WIDE enough that adjacent settings land
199 | # in different regimes; a finer range would just print duplicate rows.
200 |
201 | # microcluster.config.MIN_SCOPE_POPULATION -- privacy gate: n >= this.
202 | # Regimes, given scope populations 25 / 75 / 150:
203 | #   20  -> suite (n=25) still allowed        -> finest disclosure = SUITE
204 | #   50  -> suite out, floor (n=75) allowed   -> finest = FLOOR
205 | #   100 -> floor out, building (n=150) only  -> finest = BUILDING
206 | #   200 -> nothing passes the privacy gate   -> nothing is ever disclosed
207 | # 20 is the shipped default.
208 | MIN_SCOPE_POPULATION_SWEEP: tuple[int, ...] = (20, 50, 100, 200)
209 |
210 | # microcluster.config.STATISTICAL_GATE_FLOOR -- statistical gate:
211 | # qualifying >= max(this, ceil(sqrt(n))). Below ceil(sqrt(25))=5 it does
212 | # nothing to suite-level claims (5 is the shipped default). Each step up
213 | # from there forces disclosure coarser and later: 8 and 12 keep it at
214 | # FLOOR level but demand more evidence first; 16 and 25 push it to
215 | # BUILDING. A suite is fraction-capped at 12 reports, so any floor >= 13
216 | # makes suite-level disclosure impossible outright.
217 | STATISTICAL_GATE_FLOOR_SWEEP: tuple[int, ...] = (5, 8, 12, 16, 25)
```

```
218 |
219 | # microcluster.config.MAX_REPORT_FRACTION -- privacy gate:
220 | # qualifying / n <= this. For a suite (n=25) the allowed count is
221 | # floor(25 * fraction). Only three regimes exist in this scenario:
222 | #   0.08 -> floor(2): even a floor cannot be named -> nothing disclosed
223 | #   0.15 -> floor(3): suite forbidden (needs >= 5)  -> finest = FLOOR
224 | #   0.25 -> floor(6): suite allowed                 -> finest = SUITE
225 | # The shipped default (0.5 -> floor(12)) lands in the SAME regime as 0.25:
226 | # suite report fractions never climb anywhere near it, so tightening from
227 | # 0.5 down to ~0.2 changes nothing. It only starts to bite below that.
228 | MAX_REPORT_FRACTION_SWEEP: tuple[float, ...] = (0.08, 0.15, 0.25)
229 |
230 |
231 | # ---------------------------------------------------------------------------
232 | # Benchmark (simulation.benchmark): learned detector vs the authored rules
233 | # ---------------------------------------------------------------------------
234 | #
235 | # The benchmark builds a labelled dataset from simulated runs -- one row
236 | # per (run, day), features taken from microcluster.detection's three plain
237 | # findings, label taken from ground truth -- trains a small classifier on
238 | # it, and scores that classifier against the hand-authored threshold rules
239 | # in microcluster/config.py + detection.py on HELD-OUT runs. The model
240 | # never replaces the authored rules; it is only ever scored beside them.
241 | #
242 | # Like every other figure in this file these are CHOSEN, not measured.
243 |
244 | # Fraction of simulation RUNS (never days -- adjacent days from one
245 | # outbreak are correlated and must not straddle the split) held out for
246 | # evaluation.
247 | BENCHMARK_TEST_RUN_FRACTION: float = 0.30
248 |
249 | # Fixed seed for the train/test run split, so the benchmark is reproducible.
250 | BENCHMARK_SPLIT_SEED: int = 20260906
251 |
252 | # Max depth of the shallow decision tree (the legible alternative to
253 | # logistic regression). Kept small on purpose: a deep tree just memorises
254 | # this simulator.
255 | BENCHMARK_TREE_MAX_DEPTH: int = 3
256 |
257 | # Default number of seeds the benchmark simulates for its in-distribution
258 | # dataset.
259 | BENCHMARK_DEFAULT_SEEDS: int = 300
260 |
261 | # CROSS-REGIME TEST. A model trained on this simulator learns this
262 | # simulator. To see whether it learned anything transferable, it is also
263 | # evaluated on runs generated with a DIFFERENT suite transmission
264 | # probability than it trained on. Training uses the shipped
265 | # SUITE_TRANSMISSION_PROBABILITY; evaluation additionally uses each of
266 | # these (a slower and a faster epidemic).
267 | BENCHMARK_TRAIN_SUITE_TRANSMISSION: float = SUITE_TRANSMISSION_PROBABILITY
268 | BENCHMARK_TEST_SUITE_TRANSMISSIONS: tuple[float, ...] = (0.005, 0.012)
269 |
270 |
271 | # ---------------------------------------------------------------------------
272 | # Default structure: 1 campus > 1 building > 2 floors > 3 suites > 25 agents
```

```
# ---------------------------------------------------------------------------

# 1 * 2 * 3 * 25 = 150 agents. Declared populations then come out as
# suite n=25, floor n=75, building n=150, campus n=150.
DEFAULT_BUILDINGS: int = 1
DEFAULT_FLOORS_PER_BUILDING: int = 2
DEFAULT_SUITES_PER_FLOOR: int = 3
DEFAULT_AGENTS_PER_SUITE: int = 25

# Scope id and display label for the single campus root. Building / floor /
# suite ids are derived from this by the population builder.
CAMPUS_SCOPE_ID: str = "C"
CAMPUS_LABEL: str = "Campus"


# Which reporting process generates the reports. "clean" is the shipped
# default and the one every committed result was produced with: a fixed
# per-agent probability, independent and memoryless. "realistic" is the
# stress-test alternative in ``simulation.realism`` (heterogeneous baselines,
# correlated reporting, stigma suppression after a group is named). It exists
# to test how much the detector leans on the clean assumption; it is not a
# claim about real people either -- its parameters are chosen, not measured.
REPORTING_MODEL_CLEAN = "clean"
REPORTING_MODEL_REALISTIC = "realistic"
REPORTING_MODELS = (REPORTING_MODEL_CLEAN, REPORTING_MODEL_REALISTIC)


@dataclass(frozen=True)
class SimulationConfig:
    """Tunable inputs to one simulation run. Defaults come from the module
    constants above; override per run for experiments."""

    incubation_days_min: int = INCUBATION_DAYS_MIN
    incubation_days_max: int = INCUBATION_DAYS_MAX
    symptomatic_days_min: int = SYMPTOMATIC_DAYS_MIN
    symptomatic_days_max: int = SYMPTOMATIC_DAYS_MAX
    suite_transmission_probability: float = SUITE_TRANSMISSION_PROBABILITY
    floor_transmission_probability: float = FLOOR_TRANSMISSION_PROBABILITY
    building_transmission_probability: float = BUILDING_TRANSMISSION_PROBABILITY
    reporting_probability_min: float = REPORTING_PROBABILITY_MIN
    reporting_probability_max: float = REPORTING_PROBABILITY_MAX
    outbreak_category: Category = OUTBREAK_CATEGORY
    background_noise_daily_rate: float = BACKGROUND_NOISE_DAILY_RATE
    simulation_start: datetime = SIMULATION_START
    seed_infections: int = DEFAULT_SEED_INFECTIONS
    campus_scope_id: str = CAMPUS_SCOPE_ID
    campus_label: str = CAMPUS_LABEL
    reporting_model: str = REPORTING_MODEL_CLEAN

    def __post_init__(self) -> None:
        for lo, hi, name in (
            (self.incubation_days_min, self.incubation_days_max, "incubation_days"),
            (self.symptomatic_days_min, self.symptomatic_days_max, "symptomatic_days"),
            (
                self.reporting_probability_min,
```

```
                self.reporting_probability_max,
                "reporting_probability",
            ),
        ):
            if lo > hi:
                raise ValueError(f"{name}: min {lo!r} exceeds max {hi!r}")
        if self.incubation_days_min < 1 or self.symptomatic_days_min < 1:
            raise ValueError("state durations must be at least 1 day")
        for probability in (
            self.suite_transmission_probability,
            self.floor_transmission_probability,
            self.building_transmission_probability,
            self.reporting_probability_min,
            self.reporting_probability_max,
            self.background_noise_daily_rate,
        ):
            if not 0.0 <= probability <= 1.0:
                raise ValueError(f"probability {probability!r} must be in [0, 1]")
        if not isinstance(self.outbreak_category, Category):
            raise ValueError(f"outbreak_category must be a Category, got {self.outbreak
_category!r}")
        if self.seed_infections < 0:
            raise ValueError("seed_infections must be >= 0")
        if self.reporting_model not in REPORTING_MODELS:
            raise ValueError(
                f"reporting_model must be one of {REPORTING_MODELS}, got {self.reportin
g_model!r}"
            )


DEFAULT_SIMULATION_CONFIG = SimulationConfig()
```

simulation/contact.py

```
  1 | """Contact model: tiered transmission on a single day.
  2 |
  3 | Contact is not suite-only. An infectious agent can transmit to a
  4 | susceptible agent that shares its SUITE, its FLOOR (different suite, same
  5 | floor), or its BUILDING (different floor, same building) -- at three
  6 | decreasing per-contact probabilities. Which scope an agent's suite sits
  7 | under is read from the ``ScopeRegistry`` (see :func:`suite_ancestor_ids`),
  8 | never hard-coded from id strings.
  9 | """
 10 |
 11 | from __future__ import annotations
 12 |
 13 | import random
 14 | from collections import Counter
 15 |
 16 | from microcluster.models import ScopeLevel, ScopeRegistry
 17 |
 18 | from .agent import Agent
 19 | from .config import SimulationConfig
 20 | from .infection import INFECTIOUS_STATES, InfectionState
 21 |
 22 |
 23 | def suite_ancestor_ids(registry: ScopeRegistry, suite_id: str) -> tuple[str, str]:
 24 |     """Return ``(floor_id, building_id)`` for ``suite_id``, read from the
 25 |     registry's own hierarchy rather than assumed from id strings.
 26 |
 27 |     If the registry has no FLOOR scope above this suite, ``floor_id``
 28 |     falls back to ``suite_id`` itself (so the floor tier contributes
 29 |     nothing, rather than guessing a structure that is not there). If it
 30 |     also has no BUILDING scope, ``building_id`` falls back to whatever
 31 |     ``floor_id`` resolved to, for the same reason.
 32 |     """
 33 |     by_level = {scope.level: scope.id for scope in registry.chain_to_root(suite_id)}
 34 |     floor_id = by_level.get(ScopeLevel.FLOOR, suite_id)
 35 |     building_id = by_level.get(ScopeLevel.BUILDING, floor_id)
 36 |     return floor_id, building_id
 37 |
 38 |
 39 | def new_infections(
 40 |     agents_by_suite: dict[str, list[Agent]],
 41 |     floor_of_suite: dict[str, str],
 42 |     building_of_suite: dict[str, str],
 43 |     rng: random.Random,
 44 |     config: SimulationConfig,
 45 | ) -> tuple[list[Agent], list[Agent]]:
 46 |     """Given every agent grouped by suite and their CURRENT states, return
 47 |     the susceptible agents that become infected today, across all three
 48 |     contact tiers, as ``(within_suite, cross_suite)``.
 49 |
 50 |     For a susceptible agent in a given suite, every infectious agent is one
 51 |     of three kinds of contact: a suite-mate, a floor-mate in a different
 52 |     suite, or a building-mate on a different floor. The agent escapes
```

```
    infection only by escaping every contact at every tier, so with
    per-contact probabilities ``p_suite``, ``p_floor``, ``p_building`` and
    contact counts ``k_suite``, ``k_floor``, ``k_building``:

        P(infected today) = 1 - (1-p_suite)**k_suite
                              * (1-p_floor)**k_floor
                              * (1-p_building)**k_building

    All susceptible agents within the same suite face the same probability
    (the three counts are suite/floor/building aggregates, not
    per-individual), so it is computed once per suite.

    A newly infected agent is reported as ``cross_suite`` if their OWN
    suite had zero infectious members (``k_suite == 0``) -- the infection
    can then only have come from a floor- or building-tier contact, never
    a suite-mate. Otherwise it is reported as ``within_suite``: its suite
    did have an infectious member, even though a floor/building contact
    may also have contributed to the same compounded probability (the
    tiers are not disentangled at the level of one infection event).
    """
    infectious_by_suite: dict[str, int] = {
        suite_id: sum(1 for a in agents if a.state in INFECTIOUS_STATES)
        for suite_id, agents in agents_by_suite.items()
    }

    infectious_by_floor: Counter[str] = Counter()
    infectious_by_building: Counter[str] = Counter()
    for suite_id, count in infectious_by_suite.items():
        if count:
            infectious_by_floor[floor_of_suite[suite_id]] += count
            infectious_by_building[building_of_suite[suite_id]] += count

    p_suite = config.suite_transmission_probability
    p_floor = config.floor_transmission_probability
    p_building = config.building_transmission_probability

    within_suite: list[Agent] = []
    cross_suite: list[Agent] = []
    for suite_id, agents in agents_by_suite.items():
        floor_id = floor_of_suite[suite_id]
        building_id = building_of_suite[suite_id]

        k_suite = infectious_by_suite[suite_id]
        k_floor = infectious_by_floor[floor_id] - k_suite
        k_building = infectious_by_building[building_id] - infectious_by_floor[floor_i
d]

        if k_suite == 0 and k_floor == 0 and k_building == 0:
            continue

        escape = (
            (1.0 - p_suite) ** k_suite
            * (1.0 - p_floor) ** k_floor
            * (1.0 - p_building) ** k_building
        )
```

```
        prob_infected = 1.0 - escape
        bucket = within_suite if k_suite > 0 else cross_suite

        bucket.extend(
            agent
            for agent in agents
            if agent.state is InfectionState.SUSCEPTIBLE
            and rng.random() < prob_infected
        )
    return within_suite, cross_suite
```

simulation/detection_demo.py

```
"""End-to-end demo: simulate an outbreak, generate anonymous reports from
it, and feed them to the UNCHANGED detection engine, once per simulated
day.

    python -m simulation.detection_demo

Prints, per seed and per day: the true infection count (ground truth --
never seen by the detector), how many reports were submitted that day,
whether detection fired, which detector, and the disclosed scope if any.
Then a summary of TWO delays -- time-to-first-FIRE and
time-to-first-DISCLOSURE -- and the full list of distinct scopes disclosed
over the run (scope stability, config.SIBLING_SWITCH_MARGIN, keeps this
list short).
"""

from __future__ import annotations

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from simulation.config import DEFAULT_RUN_DAYS, DEFAULT_SMOKE_SEEDS  # noqa: E402
from simulation.pipeline import DailyRecord, run_with_detection  # noqa: E402


def _detector_label(record: DailyRecord) -> str:
    if record.relative_fired and record.absolute_fired:
        return "both"
    if record.relative_fired:
        return "relative"
    if record.absolute_fired:
        return "absolute"
    return "-"


def _print_run(seed: int, days: int) -> None:
    result = run_with_detection(seed=seed, days=days)

    header = (
        f"{'day':>3}  {'true_infected':>13}  {'reports_today':>13}  "
        f"{'fired':>5}  {'detector':>8}  disclosed_scope"
    )
    print(f"SEED {seed}")
    print(header)
    print("-" * len(header))

    outbreak_started_day: int | None = None

    for record in result.daily_records:
        if outbreak_started_day is None and record.true_infected_count > 0:
            outbreak_started_day = record.day
```

```
 53 |
 54 |         print(
 55 |             f"{record.day:>3}  {record.true_infected_count:>13}  "
 56 |             f"{record.reports_submitted_today:>13}  "
 57 |             f"{str(record.detection_fired):>5}  {_detector_label(record):>8}  "
 58 |             f"{record.disclosed_scope_id or '-'}"
 59 |         )
 60 |
 61 |     last = result.daily_records[-1]
 62 |     fired_day = last.first_fired_day
 63 |     disclosed_day = last.first_disclosed_day
 64 |     distinct_scopes = result.distinct_disclosed_scopes()
 65 |
 66 |     print()
 67 |     if outbreak_started_day is None:
 68 |         print("  outbreak never took hold: nobody was ever infected")
 69 |     else:
 70 |         print(f"  outbreak began (ground truth)   : day {outbreak_started_day}")
 71 |
 72 |     if fired_day is None:
 73 |         print("  detector never fired in the run")
 74 |     else:
 75 |         print(f"  time-to-first-FIRE             : day {fired_day}"
 76 |               + (f"  (delay {fired_day - outbreak_started_day} day(s) from outbreak sta
rt)"
 77 |                  if outbreak_started_day is not None else ""))
 78 |
 79 |     if disclosed_day is None:
 80 |         print("  detector never disclosed a scope in the run")
 81 |     else:
 82 |         print(f"  time-to-first-DISCLOSURE        : day {disclosed_day}"
 83 |               + (f"  (delay {disclosed_day - outbreak_started_day} day(s) from outbreak
start)"
 84 |                  if outbreak_started_day is not None else ""))
 85 |
 86 |     if fired_day is not None and disclosed_day is not None:
 87 |         print(f"  cost of the privacy policy      : {disclosed_day - fired_day} day(s)
"
 88 |               f"(fired but could not yet be described)")
 89 |
 90 |     print(f"  distinct scopes ever disclosed  : {len(distinct_scopes)} {distinct_scope
s}")
 91 |     print()
 92 |
 93 |
 94 | def main() -> None:
 95 |     for seed in DEFAULT_SMOKE_SEEDS:
 96 |         _print_run(seed, DEFAULT_RUN_DAYS)
 97 |
 98 |
 99 | if __name__ == "__main__":
100 |     main()
```

simulation/experiments.py

```
  1 | """Experiment runner: statistics over many simulated outbreaks.
  2 |
  3 | Still headless -- no plotting, no API, no frontend. Runs the pipeline
  4 | across many random seeds, sweeping the DISCLOSURE gates one at a time,
  5 | and prints the delay-versus-infections relationship as a plain data
  6 | table.
  7 |
  8 |     python -m simulation.experiments
  9 |
 10 | ---------------------------------------------------------------------
 11 | WHY THE DISCLOSURE GATES, NOT THE DETECTION THRESHOLD
 12 | ---------------------------------------------------------------------
 13 |
 14 | An earlier version of this runner swept ``ABSOLUTE_EXCESS_RATIO`` (the
 15 | detection threshold) and found time-to-disclosure completely invariant to
 16 | it -- 14.1 days across every setting. The detector fires well before the
 17 | disclosure engine will name anything anyway, so it is the DISCLOSURE
 18 | gates, not detection, that set the timing of the first public statement.
 19 | So this runner sweeps those, holding detection config at its defaults.
 20 |
 21 | Each disclosure gate ultimately compares an INTEGER qualifying-report
 22 | count (or a declared population) against a threshold, so a parameter only
 23 | changes behaviour when it crosses a discrete boundary. In the default
 24 | structure the scope populations are 25 / 75 / 150 (suite / floor /
 25 | building = campus, one building) and ``ceil(sqrt(n))`` is 5 / 9 / 13. The
 26 | swept ranges (``config.*_SWEEP``) are deliberately wide enough that
 27 | adjacent settings land in different regimes; ``adjacent_duplicate_rows``
 28 | flags any that still collapse to an identical row.
 29 |
 30 | ---------------------------------------------------------------------
 31 | METHODOLOGICAL CHOICE: establishment conditioning
 32 | ---------------------------------------------------------------------
 33 |
 34 | A single seeded index case is a stochastic branching process (see
 35 | ``simulation/config.py`` next to ``SUITE_TRANSMISSION_PROBABILITY``): a
 36 | real fraction of runs fail to take off at all, purely by chance, before
 37 | the detector or the disclosure policy ever get a say. An outbreak counts
 38 | as ESTABLISHED once its cumulative (ever-infected) case count reaches
 39 | ``config.ESTABLISHED_MIN_INFECTIONS``.
 40 |
 41 | Mixing fizzled runs in with established ones would swamp every other
 42 | statistic with take-off variance. Standard practice in stochastic
 43 | epidemic simulation: report the fade-out (fizzle) rate on its own, then
 44 | condition every other statistic on establishment.
 45 |
 46 | The one number NOT conditioned on establishment is the FALSE-ALARM RATE:
 47 | the fraction of NON-established runs where the detector fired anyway. It
 48 | is the cost side of a loose detector and belongs in the headline results,
 49 | not a footnote. Detection is fixed here, so it is a single number.
 50 |
 51 | ---------------------------------------------------------------------
 52 | READING ``gap`` (policy cost)
```

```
 53 | ----------------------------------------------------------------------
 54 |
 55 | ``gap = disclosure_delay - fire_delay``. On its own it is ambiguous: it
 56 | shrinks when disclosure gets FASTER (good) or when detection gets SLOWER
 57 | (bad). Both delays are printed side by side, over the same subset of runs,
 58 | so the two cases cannot be confused -- and because detection config is
 59 | held fixed across every sweep, ``fire_delay`` is the control: any movement
 60 | in ``gap`` between rows is disclosure timing, not detection.
 61 | """
 62 |
 63 | from __future__ import annotations
 64 |
 65 | import math
 66 | import sys
 67 | from dataclasses import dataclass
 68 | from pathlib import Path
 69 | from statistics import mean
 70 | from typing import Callable, Iterable, Sequence
 71 |
 72 | sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
 73 |
 74 | from microcluster.config import (  # noqa: E402
 75 |     DEFAULT_DETECTION_CONFIG,
 76 |     DEFAULT_DISCLOSURE_CONFIG,
 77 |     DisclosureConfig,
 78 | )
 79 | from microcluster.disclosure import statistical_threshold  # noqa: E402
 80 | from microcluster.models import ScopeLevel  # noqa: E402
 81 |
 82 | from simulation.config import (  # noqa: E402
 83 |     DEFAULT_EXPERIMENT_SEEDS,
 84 |     DEFAULT_RUN_DAYS,
 85 |     ESTABLISHED_MIN_INFECTIONS,
 86 |     MAX_REPORT_FRACTION_SWEEP,
 87 |     MIN_SCOPE_POPULATION_SWEEP,
 88 |     STATISTICAL_GATE_FLOOR_SWEEP,
 89 | )
 90 | from simulation.infection import InfectionState  # noqa: E402
 91 | from simulation.pipeline import (  # noqa: E402
 92 |     DailyRecord,
 93 |     SimulatedReports,
 94 |     analyze_report_stream,
 95 |     run_with_detection,
 96 |     simulate_and_report,
 97 | )
 98 | from simulation.simulator import Simulation  # noqa: E402
 99 |
100 | __all__ = [
101 |     "RunOutcome",
102 |     "SettingSummary",
103 |     "DisclosureSweep",
104 |     "SweepResults",
105 |     "run_once",
106 |     "reduce_run",
107 |     "summarize_setting",
```

```
    "fizzle_rate",
    "false_alarm_rate_at_defaults",
    "overall_mean_fire_delay",
    "adjacent_duplicate_rows",
    "degenerate_reason_for",
    "run_disclosure_sweeps",
    "DISCLOSURE_SWEEPS",
]


def degenerate_reason_for(
    config: DisclosureConfig, scope_populations: Iterable[int]
) -> str | None:
    """If NO scope in ``scope_populations`` can pass both gates at ANY
    achievable qualifying count under ``config``, return a one-line reason
    the configuration is structurally impossible; otherwise ``None``.

    This is not a data point -- it is a setting under which the disclosure
    engine can never say anything regardless of the outbreak. A scope of
    declared population ``n`` is disclosable for some count iff::

        n >= min_scope_population
        and  max(statistical_gate_floor, ceil(sqrt(n)))  <=  floor(n * max_report_fract
ion)

    (the statistical gate's requirement must fit under the privacy gate's
    fraction cap). MIN_SCOPE_POPULATION greater than the whole campus is
    the obvious case; a very low MAX_REPORT_FRACTION is another.
    """
    pops = sorted(set(scope_populations))
    for n in pops:
        if n < config.min_scope_population:
            continue
        need = statistical_threshold(n, config)
        allowed = math.floor(n * config.max_report_fraction)
        if need <= allowed:
            return None
    biggest = pops[-1] if pops else 0
    if all(n < config.min_scope_population for n in pops):
        return (
            f"min_scope_population={config.min_scope_population} exceeds every "
            f"declared scope population (largest is {biggest}); the privacy "
            f"gate can never pass"
        )
    return (
        f"no scope population in {pops} admits a qualifying count that clears "
        f"the statistical gate while staying under max_report_fraction="
        f"{config.max_report_fraction:g}"
    )


# ---------------------------------------------------------------------------
# Per-run outcome
# ---------------------------------------------------------------------------

```

```
162 |
163 | @dataclass(frozen=True)
164 | class RunOutcome:
165 |     """Ground truth and detection/disclosure outcome for one simulated seed."""
166 |
167 |     seed: int
168 |     established: bool
169 |     total_infections: int  # cumulative ever-infected at the end of the run (day 30)
170 |     peak_infected: int  # largest simultaneous (INCUBATING + SYMPTOMATIC) count
171 |     first_fired_day: int | None
172 |     first_disclosed_day: int | None
173 |     infections_at_first_disclosure: int | None  # cumulative ever-infected on that day
174 |     finest_disclosed_level: ScopeLevel | None  # deepest level named anywhere in the ru
n
175 |
176 |     @property
177 |     def policy_cost(self) -> int | None:
178 |         if self.first_fired_day is None or self.first_disclosed_day is None:
179 |             return None
180 |         return self.first_disclosed_day - self.first_fired_day
181 |
182 |
183 | def _cumulative_infected(simulation: Simulation, day_index: int) -> int:
184 |     snapshot = simulation.history[day_index]
185 |     return sum(1 for a in snapshot.agents if a.state is not InfectionState.SUSCEPTIBLE)
186 |
187 |
188 | def reduce_run(
189 |     seed: int,
190 |     simulation: Simulation,
191 |     daily_records: list[DailyRecord],
192 |     *,
193 |     established_min_infections: int = ESTABLISHED_MIN_INFECTIONS,
194 | ) -> RunOutcome:
195 |     """Collapse one simulated + analysed run into a :class:`RunOutcome`."""
196 |     total_infections = _cumulative_infected(simulation, -1)
197 |     peak_infected = max(
198 |         s.count(InfectionState.INCUBATING) + s.count(InfectionState.SYMPTOMATIC)
199 |         for s in simulation.history
200 |     )
201 |
202 |     last = daily_records[-1]
203 |     fire_day = last.first_fired_day
204 |     disclose_day = last.first_disclosed_day
205 |
206 |     infections_at_disclosure = (
207 |         _cumulative_infected(simulation, disclose_day) if disclose_day is not None else
None
208 |     )
209 |
210 |     finest_level: ScopeLevel | None = None
211 |     for record in daily_records:
212 |         if record.disclosed_scope_id is None:
213 |             continue
214 |         level = simulation.registry.get(record.disclosed_scope_id).level
```

```
        finest_level = level if finest_level is None else max(finest_level, level)

    return RunOutcome(
        seed=seed,
        established=total_infections >= established_min_infections,
        total_infections=total_infections,
        peak_infected=peak_infected,
        first_fired_day=fire_day,
        first_disclosed_day=disclose_day,
        infections_at_first_disclosure=infections_at_disclosure,
        finest_disclosed_level=finest_level,
    )


def run_once(
    seed: int,
    *,
    days: int = DEFAULT_RUN_DAYS,
    detection_config=DEFAULT_DETECTION_CONFIG,
    disclosure_config: DisclosureConfig = DEFAULT_DISCLOSURE_CONFIG,
    established_min_infections: int = ESTABLISHED_MIN_INFECTIONS,
) -> RunOutcome:
    """Run one seed end to end (convenience wrapper over the pipeline)."""
    run = run_with_detection(
        seed=seed,
        days=days,
        detection_config=detection_config,
        disclosure_config=disclosure_config,
    )
    return reduce_run(
        seed,
        run.simulation,
        run.daily_records,
        established_min_infections=established_min_infections,
    )


# ---------------------------------------------------------------------------
# Aggregation for one sweep setting
# ---------------------------------------------------------------------------


def _mean(values: list[float]) -> float | None:
    return mean(values) if values else None


@dataclass(frozen=True)
class SettingSummary:
    """Aggregate statistics for one value of one swept disclosure parameter.

    ``mean_*`` fields are over ESTABLISHED runs only (the disclosure-timing
    ones additionally only over established runs that actually disclosed).
    ``false_alarm_rate`` is over NON-established runs only.
    """

```

```
    param_name: str
    value: float

    n_runs: int
    n_established: int
    n_non_established: int
    n_fired: int  # established runs that ever fired  (detection-only; constant per swe
ep)
    n_disclosed: int  # established runs that ever disclosed  (varies with the paramete
r)

    mean_total_infections: float | None  # day-30 cumulative; detection/disclosure-inde
pendent

    # All four over the SAME subset: established runs that disclosed.
    mean_fire_delay: float | None
    mean_disclosure_delay: float | None
    mean_gap: float | None  # disclosure_delay - fire_delay  == mean policy cost
    mean_infections_at_disclosure: float | None  # CONDITIONAL on disclosure (survivors
hip)

    # Over ALL established runs, disclosed or not: infections that were
    # never disclosed before the run ended. A run that disclosed
    # contributes its infection count at first disclosure; a run that never
    # disclosed contributes its full day-30 total (none of its infections
    # were ever disclosed). This is the unbiased companion to
    # ``mean_infections_at_disclosure``.
    mean_infections_before_disclosure_unbiased: float | None
    disclosed_fraction: float | None  # n_disclosed / n_established -- the selection ma
de visible

    finest_disclosed_level: ScopeLevel | None

    n_false_alarms: int
    false_alarm_rate: float | None

    # A structurally impossible configuration (see degenerate_reason_for):
    # no scope can pass both gates at any count. Not a data point.
    degenerate: bool = False
    degenerate_reason: str = ""

    def regime_key(self) -> tuple:
        """The disclosure-dependent outcome, for spotting duplicate regimes
        (adjacent settings the integer arithmetic did not separate)."""
        return (
            self.n_disclosed,
            self.mean_disclosure_delay,
            self.mean_gap,
            self.mean_infections_at_disclosure,
            self.finest_disclosed_level,
        )


def summarize_setting(
    outcomes: list[RunOutcome],
```

```
320 |     *,
321 |     param_name: str,
322 |     value: float,
323 |     degenerate: bool = False,
324 |     degenerate_reason: str = "",
325 | ) -> SettingSummary:
326 |     established = [o for o in outcomes if o.established]
327 |     non_established = [o for o in outcomes if not o.established]
328 |
329 |     fired = [o for o in established if o.first_fired_day is not None]
330 |     disclosed = [o for o in established if o.first_disclosed_day is not None]
331 |     false_alarms = [o for o in non_established if o.first_fired_day is not None]
332 |
333 |     # Unbiased: every established run contributes, a non-disclosing one at
334 |     # its full day-30 total (none of its infections were ever disclosed).
335 |     infections_before_disclosure = [
336 |         o.infections_at_first_disclosure
337 |         if o.first_disclosed_day is not None
338 |         else o.total_infections
339 |         for o in established
340 |     ]
341 |
342 |     finest: ScopeLevel | None = None
343 |     for o in disclosed:
344 |         if o.finest_disclosed_level is not None:
345 |             finest = (
346 |                 o.finest_disclosed_level
347 |                 if finest is None
348 |                 else max(finest, o.finest_disclosed_level)
349 |             )
350 |
351 |     return SettingSummary(
352 |         param_name=param_name,
353 |         value=value,
354 |         n_runs=len(outcomes),
355 |         n_established=len(established),
356 |         n_non_established=len(non_established),
357 |         n_fired=len(fired),
358 |         n_disclosed=len(disclosed),
359 |         mean_total_infections=_mean([o.total_infections for o in established]),
360 |         mean_fire_delay=_mean([o.first_fired_day for o in disclosed]),
361 |         mean_disclosure_delay=_mean([o.first_disclosed_day for o in disclosed]),
362 |         mean_gap=_mean([o.policy_cost for o in disclosed]),
363 |         mean_infections_at_disclosure=_mean(
364 |             [o.infections_at_first_disclosure for o in disclosed]
365 |         ),
366 |         mean_infections_before_disclosure_unbiased=_mean(infections_before_disclosure),
367 |         disclosed_fraction=(len(disclosed) / len(established)) if established else Non
e,
368 |         finest_disclosed_level=finest,
369 |         n_false_alarms=len(false_alarms),
370 |         false_alarm_rate=(len(false_alarms) / len(non_established)) if non_established
else None,
371 |         degenerate=degenerate,
372 |         degenerate_reason=degenerate_reason,
```

```
373 |     )
374 |
375 |
376 | def fizzle_rate(outcomes: list[RunOutcome]) -> float:
377 |     """Fraction of runs that never established. Independent of both engine
378 |     configs -- reported once, not per row."""
379 |     if not outcomes:
380 |         return 0.0
381 |     return 1 - sum(o.established for o in outcomes) / len(outcomes)
382 |
383 |
384 | def false_alarm_rate_at_defaults(outcomes: list[RunOutcome]) -> tuple[int, int, float |
None]:
385 |     """(n_false_alarms, n_non_established, rate) over a baseline (default
386 |     config) outcome set. Detection is fixed for every sweep, so this
387 |     single number is the false-alarm rate everywhere below."""
388 |     non_established = [o for o in outcomes if not o.established]
389 |     alarms = [o for o in non_established if o.first_fired_day is not None]
390 |     rate = (len(alarms) / len(non_established)) if non_established else None
391 |     return len(alarms), len(non_established), rate
392 |
393 |
394 | def overall_mean_fire_delay(outcomes: list[RunOutcome]) -> float | None:
395 |     """Mean fire delay over ALL established runs that fired (not just the
396 |     ones that went on to disclose). The detection-side control value."""
397 |     return _mean(
398 |         [o.first_fired_day for o in outcomes if o.established and o.first_fired_day is
not None]
399 |     )
400 |
401 |
402 | def adjacent_duplicate_rows(summaries: list[SettingSummary]) -> list[tuple[float, floa
t]]:
403 |     """Pairs of adjacent setting values whose disclosure-dependent outcome
404 |     is identical -- the range is too fine for the integer arithmetic
405 |     underneath at that point."""
406 |     dupes: list[tuple[float, float]] = []
407 |     for earlier, later in zip(summaries, summaries[1:]):
408 |         # Degenerate rows are labelled IMPOSSIBLE in the output, not
409 |         # compared as data -- two of them side by side is not a
410 |         # duplicate-regime problem.
411 |         if earlier.degenerate or later.degenerate:
412 |             continue
413 |         if earlier.regime_key() == later.regime_key():
414 |             dupes.append((earlier.value, later.value))
415 |     return dupes
416 |
417 |
418 | # ---------------------------------------------------------------------------
419 | # The three sweeps
420 | # ---------------------------------------------------------------------------
421 |
422 |
423 | @dataclass(frozen=True)
424 | class DisclosureSweep:
```

```
    param_name: str
    default_value: float  # the shipped default, for context in the output
    values: tuple
    config_for: Callable[[float], DisclosureConfig]
    note: str = ""


DISCLOSURE_SWEEPS: tuple[DisclosureSweep, ...] = (
    DisclosureSweep(
        "MIN_SCOPE_POPULATION",
        20,
        MIN_SCOPE_POPULATION_SWEEP,
        lambda v: DisclosureConfig(min_scope_population=int(v)),
        note="privacy gate n >= this; scope populations are 25 / 75 / 150.",
    ),
    DisclosureSweep(
        "STATISTICAL_GATE_FLOOR",
        5,
        STATISTICAL_GATE_FLOOR_SWEEP,
        lambda v: DisclosureConfig(statistical_gate_floor=int(v)),
        note="statistical gate qualifying >= max(this, ceil(sqrt(n))); ceil(sqrt) is 5
/ 9 / 13.",
    ),
    DisclosureSweep(
        "MAX_REPORT_FRACTION",
        0.5,
        MAX_REPORT_FRACTION_SWEEP,
        lambda v: DisclosureConfig(max_report_fraction=float(v)),
        note=(
            "privacy gate qualifying/n <= this. The shipped default (0.5) lands in "
            "the SAME regime as 0.25 and is not shown: suite report fractions never "
            "approach it, so it never binds. It only bites below ~0.2."
        ),
    ),
)


@dataclass
class SweepResults:
    fizzle_rate: float
    false_alarms: int
    non_established: int
    false_alarm_rate: float | None
    control_fire_delay: float | None  # mean fire delay, all established fired runs
    n_established: int
    n_fired: int
    mean_total_infections: float | None
    sweeps: dict[str, list[SettingSummary]]


def run_disclosure_sweeps(
    seeds: Sequence[int] | None = None,
    *,
    n_seeds: int = DEFAULT_EXPERIMENT_SEEDS,
    days: int = DEFAULT_RUN_DAYS,
```

```
    sweeps: Sequence[DisclosureSweep] = DISCLOSURE_SWEEPS,
) -> SweepResults:
    """Simulate every seed ONCE, then replay the analysis for every value
    of every disclosure sweep on that shared report stream."""
    seed_list = list(seeds) if seeds is not None else list(range(1, n_seeds + 1))
    simulated: dict[int, SimulatedReports] = {
        seed: simulate_and_report(seed=seed, days=days) for seed in seed_list
    }

    sample_registry = next(iter(simulated.values())).simulation.registry
    scope_populations = tuple(
        sorted({s.population for s in sample_registry.scopes.values()})
    )

    def outcomes_for(disclosure_config: DisclosureConfig) -> list[RunOutcome]:
        return [
            reduce_run(
                seed,
                sr.simulation,
                analyze_report_stream(sr, disclosure_config=disclosure_config),
            )
            for seed, sr in simulated.items()
        ]

    baseline = outcomes_for(DEFAULT_DISCLOSURE_CONFIG)
    alarms, non_est, alarm_rate = false_alarm_rate_at_defaults(baseline)

    sweep_summaries: dict[str, list[SettingSummary]] = {}
    for sweep in sweeps:
        rows: list[SettingSummary] = []
        for value in sweep.values:
            disclosure_config = sweep.config_for(value)
            reason = degenerate_reason_for(disclosure_config, scope_populations)
            rows.append(
                summarize_setting(
                    outcomes_for(disclosure_config),
                    param_name=sweep.param_name,
                    value=value,
                    degenerate=reason is not None,
                    degenerate_reason=reason or "",
                )
            )
        sweep_summaries[sweep.param_name] = rows

    return SweepResults(
        fizzle_rate=fizzle_rate(baseline),
        false_alarms=alarms,
        non_established=non_est,
        false_alarm_rate=alarm_rate,
        control_fire_delay=overall_mean_fire_delay(baseline),
        n_established=sum(o.established for o in baseline),
        n_fired=sum(
            1 for o in baseline if o.established and o.first_fired_day is not None
        ),
        mean_total_infections=_mean(
```

```
            [o.total_infections for o in baseline if o.established]
        ),
        sweeps=sweep_summaries,
    )


# ---------------------------------------------------------------------------
# Reporting
# ---------------------------------------------------------------------------


def _fmt(value: float | None, decimals: int = 1) -> str:
    return "n/a" if value is None else f"{value:.{decimals}f}"


def _level_name(level: ScopeLevel | None) -> str:
    return "none" if level is None else level.name


def _print_sweep(sweep: DisclosureSweep, summaries: list[SettingSummary]) -> None:
    print(f"--- Sweep: {sweep.param_name} (default {sweep.default_value:g}; detection f
ixed) ---")
    if sweep.note:
        print(f"    {sweep.note}")

    columns = [
        ("value", 7),
        ("disc_frac", 9),
        ("inf<disc", 9),
        ("inf@disc", 9),
        ("disc_delay", 10),
        ("fire_delay", 10),
        ("gap", 6),
        ("finest_lvl", 10),
    ]
    header = "  ".join(f"{name:>{width}}" for name, width in columns)
    print(header)
    print("-" * len(header))
    for s in summaries:
        if s.degenerate:
            print(f"{s.value:>7g}  IMPOSSIBLE -- {s.degenerate_reason}")
            continue
        frac = (
            "n/a"
            if s.disclosed_fraction is None
            else f"{s.n_disclosed}/{s.n_established}"
        )
        row = [
            f"{s.value:g}",
            frac,
            _fmt(s.mean_infections_before_disclosure_unbiased),
            _fmt(s.mean_infections_at_disclosure),
            _fmt(s.mean_disclosure_delay),
            _fmt(s.mean_fire_delay),
            _fmt(s.mean_gap),
```

```
            _level_name(s.finest_disclosed_level),
        ]
        print("  ".join(f"{value:>{width}}" for value, (_, width) in zip(row, column
s)))

    dupes = adjacent_duplicate_rows(summaries)
    if dupes:
        pairs = ", ".join(f"{a:g}<->{b:g}" for a, b in dupes)
        print(
            f"    NOTE: adjacent settings produced identical rows ({pairs}). "
            f"{sweep.param_name} acts on integer counts; these values did not "
            f"cross a discrete boundary between them."
        )
    print()


def main() -> None:
    results = run_disclosure_sweeps()
    n_seeds = next(iter(results.sweeps.values()))[0].n_runs
    bar = "=" * 78

    print(bar)
    print(f"{n_seeds} seeds, {DEFAULT_RUN_DAYS}-day runs, default structure, DETECTION
FIXED AT DEFAULTS")
    print(f"established = cumulative infections >= {ESTABLISHED_MIN_INFECTIONS} (ESTABL
ISHED_MIN_INFECTIONS)")
    print(bar)
    print()
    print(f"  FIZZLE RATE (outbreak never established) :  {results.fizzle_rate * 100:.1
f}%   "
          f"({results.non_established}/{n_seeds} runs)")
    print()
    fa = "n/a" if results.false_alarm_rate is None else f"{results.false_alarm_rate * 1
00:.1f}%"
    print(f"  >>> FALSE-ALARM RATE at shipped defaults :  {fa}   "
          f"({results.false_alarms}/{results.non_established} non-established runs fire
d anyway) <<<")
    print(f"      This is a real weakness of the shipped detector, not a footnote.")
    print(f"      With the default ABSOLUTE_EXCESS_RATIO, three in five outbreaks tha
t")
    print(f"      never established still trip the detector. Sweeping the DISCLOSURE")
    print(f"      gates below cannot touch it -- it is a detection-side cost, and it i
s")
    print(f"      the same number under every disclosure setting.")
    print()
    print(f"  established runs                  : {results.n_established} of {n_seed
s}")
    print(f"  ...of which the detector fired    : {results.n_fired}")
    print(f"  mean day-30 cumulative infections : {_fmt(results.mean_total_infections)}
"
          f"(disclosure-independent by construction -- this is why day-30")
    print(f"                                        totals cannot be the outcome axis)")
    print(f"  CONTROL mean fire delay          :  {_fmt(results.control_fire_delay)} da
ys  "
          f"(all established runs that fired)")
```

```
632 |     print()
633 |     print("  Column key for the sweep tables:")
634 |     print("    disc_frac  established runs that ever disclosed / all established runs")
635 |     print("                (the selection: shrinks as the gate tightens)")
636 |     print("    inf<disc   UNBIASED. Mean, over ALL established runs, of infections neve
r")
637 |     print("                disclosed before the run ended: a run that disclosed contribu
tes")
638 |     print("                its count at first disclosure, a run that never disclosed")
639 |     print("                contributes its full day-30 total.")
640 |     print("    inf@disc   CONDITIONAL on disclosure. Mean infections at first disclosur
e")
641 |     print("                over only the runs that disclosed -- under a tight gate those
are")
642 |     print("                systematically the largest outbreaks, so this understates the
cost.")
643 |     print("    disc_delay mean days to first disclosure   } over the same subset:")
644 |     print("    fire_delay mean days to first fire          } established runs that disc
losed")
645 |     print("    gap        disc_delay - fire_delay, the privacy-policy window (== mean p
olicy cost)")
646 |     print("    finest_lvl finest scope level ever disclosed at this setting")
647 |     print("    a row marked IMPOSSIBLE is a structurally degenerate configuration, not
data.")
648 |     print()
649 |     print("  gap alone is ambiguous (it shrinks if disclosure speeds up OR detection sl
ows)")
650 |     print("  -- but detection is fixed here, so fire_delay is flat and every move in ga
p is")
651 |     print("  disclosure timing. Read disc_delay and fire_delay together, never gap alon
e.")
652 |     print()
653 |
654 |     sweeps_by_name = {s.param_name: s for s in DISCLOSURE_SWEEPS}
655 |     for param_name, summaries in results.sweeps.items():
656 |         _print_sweep(sweeps_by_name[param_name], summaries)
657 |
658 |
659 | if __name__ == "__main__":
660 |     main()
```

simulation/infection.py

```
"""Infection state machine.

    SUSCEPTIBLE -> INCUBATING -> SYMPTOMATIC -> RECOVERED

States progress in that order, one step at a time, and never move
backwards. :func:`set_state` is the single choke point that enforces it.
"""

from __future__ import annotations

import random
from enum import Enum
from typing import TYPE_CHECKING

from .config import SimulationConfig

if TYPE_CHECKING:  # avoid an import cycle; only needed for type checkers
    from .agent import Agent


class InfectionState(Enum):
    SUSCEPTIBLE = "susceptible"
    INCUBATING = "incubating"
    SYMPTOMATIC = "symptomatic"
    RECOVERED = "recovered"


# The only permitted progression. Position in this tuple defines "forward".
STATE_ORDER: tuple[InfectionState, ...] = (
    InfectionState.SUSCEPTIBLE,
    InfectionState.INCUBATING,
    InfectionState.SYMPTOMATIC,
    InfectionState.RECOVERED,
)

# States in which an agent can transmit to a susceptible suite-mate.
INFECTIOUS_STATES: frozenset[InfectionState] = frozenset(
    {InfectionState.INCUBATING, InfectionState.SYMPTOMATIC}
)


def order_index(state: InfectionState) -> int:
    """Position of ``state`` in :data:`STATE_ORDER`."""
    return STATE_ORDER.index(state)


def set_state(agent: "Agent", new_state: InfectionState, day: int) -> None:
    """Advance ``agent`` to ``new_state`` and stamp the day of the change.

    Raises ``ValueError`` unless ``new_state`` is the state immediately
    after the agent's current one in :data:`STATE_ORDER`: the machine only
    ever steps forward, and only by one.
```

```
    """
    current = order_index(agent.state)
    target = order_index(new_state)
    if target != current + 1:
        raise ValueError(
            f"agent {agent.id}: illegal transition "
            f"{agent.state.value} -> {new_state.value} "
            f"(states move one step forward only, never back or skipping)"
        )
    agent.state = new_state
    agent.state_changed_day = day


def infect(
    agent: "Agent", day: int, rng: random.Random, config: SimulationConfig
) -> None:
    """SUSCEPTIBLE -> INCUBATING. Draws this agent's incubation and
    symptomatic durations up front, from the configured ranges."""
    set_state(agent, InfectionState.INCUBATING, day)
    agent.incubation_days = rng.randint(
        config.incubation_days_min, config.incubation_days_max
    )
    agent.symptomatic_days = rng.randint(
        config.symptomatic_days_min, config.symptomatic_days_max
    )


def advance_time_based(agent: "Agent", day: int) -> bool:
    """Apply the time-driven transitions:

    * INCUBATING -> SYMPTOMATIC once ``incubation_days`` have elapsed,
    * SYMPTOMATIC -> RECOVERED once ``symptomatic_days`` have elapsed.

    At most one transition per call. Returns True if the agent changed
    state. SUSCEPTIBLE and RECOVERED agents are untouched.
    """
    if agent.state is InfectionState.INCUBATING:
        if day - agent.state_changed_day >= agent.incubation_days:
            set_state(agent, InfectionState.SYMPTOMATIC, day)
            return True
    elif agent.state is InfectionState.SYMPTOMATIC:
        if day - agent.state_changed_day >= agent.symptomatic_days:
            set_state(agent, InfectionState.RECOVERED, day)
            return True
    return False
```

simulation/pipeline.py

```
  1 | """End-to-end wiring: simulated agents -> anonymous reports -> the
  2 | EXISTING, UNMODIFIED detection engine -> disclosure.
  3 |
  4 | This module is the one place ground truth (``Simulation`` / ``Agent``
  5 | state) and the production engines (``microcluster.engine`` /
  6 | ``detection`` / ``disclosure``) meet. Only ``Report`` objects and the
  7 | ``ScopeRegistry`` cross that line -- see CLAUDE.md, "The ground-truth
  8 | firewall". ``DailyRecord.true_infected_count`` is the one deliberate
  9 | exception: it is ground truth kept for MEASURING the detector after the
 10 | fact, and is never part of what gets fed to it.
 11 | """
 12 |
 13 | from __future__ import annotations
 14 |
 15 | import random
 16 | from dataclasses import dataclass
 17 | from datetime import timedelta
 18 |
 19 | from microcluster.config import DEFAULT_DETECTION_CONFIG, DEFAULT_DISCLOSURE_CONFIG
 20 | from microcluster.config import DetectionConfig, DisclosureConfig
 21 | from microcluster.detection import HysteresisState
 22 | from microcluster.disclosure import ScopeEvaluation
 23 | from microcluster.engine import analyze
 24 | from microcluster.models import Report
 25 |
 26 | from . import reporting
 27 | from .config import DEFAULT_SIMULATION_CONFIG, REPORTING_MODEL_CLEAN, SimulationConfig
 28 | from .infection import InfectionState
 29 | from .population import StructureSpec
 30 | from .simulator import Simulation
 31 |
 32 | __all__ = [
 33 |     "DailyRecord",
 34 |     "SimulationRun",
 35 |     "SimulatedReports",
 36 |     "simulate_and_report",
 37 |     "StreamAnalyzer",
 38 |     "analyze_report_stream",
 39 |     "run_with_detection",
 40 | ]
 41 |
 42 |
 43 | @dataclass(frozen=True)
 44 | class DailyRecord:
 45 |     """What happened on one simulated day, from both sides of the
 46 |     ground-truth firewall.
 47 |
 48 |     ``first_fired_day`` / ``first_disclosed_day`` are running totals as of
 49 |     THIS day (``None`` until the first time each has happened), recorded
 50 |     here rather than left for a caller to re-derive later by scanning the
 51 |     record list. The gap between them -- time-to-first-DISCLOSURE minus
 52 |     time-to-first-FIRE -- is the measurable cost of the privacy policy:
```

```
    how long the system knew something before it was permitted to say
    where.
    """

    day: int
    true_infected_count: int  # ground truth, kept for measurement only
    reports_submitted_today: int
    reports_accumulated: int
    detection_fired: bool
    relative_fired: bool
    absolute_fired: bool
    disclosed_scope_id: str | None
    first_fired_day: int | None
    first_disclosed_day: int | None

    # The complete disclosure gate table for this day (rule 12): a
    # ScopeEvaluation for every candidate scope, winner and rejected
    # alike, exactly as disclosure.evaluate returned it. Empty on days the
    # detector did not fire (disclosure does not run). The presentation
    # layer renders these and never recomputes a gate.
    disclosure_evaluations: tuple[ScopeEvaluation, ...] = ()


@dataclass
class SimulationRun:
    simulation: Simulation
    reports: list[Report]
    daily_records: list[DailyRecord]

    def distinct_disclosed_scopes(self) -> list[str]:
        """Every distinct scope id ever disclosed over the run, in the
        order each was first named. Small and stable is the point of
        scope-stability disclosure (config.SIBLING_SWITCH_MARGIN); a long
        list here means disclosure wandered."""
        seen: list[str] = []
        for record in self.daily_records:
            scope_id = record.disclosed_scope_id
            if scope_id is not None and scope_id not in seen:
                seen.append(scope_id)
        return seen


@dataclass
class SimulatedReports:
    """One simulated run's epidemic plus the anonymous reports it produced,
    day by day -- everything EXCEPT the detection/disclosure analysis.

    Split out from :func:`run_with_detection` so an experiment can simulate
    a seed once and then replay the analysis under many different engine
    configs cheaply (see :func:`analyze_report_stream` and
    ``simulation.experiments``). The simulated epidemic and the reports
    depend on nothing in ``microcluster`` -- neither the detection nor the
    disclosure config -- so it is always safe to reuse this across configs.
    """

```

```
    simulation: Simulation
    config: SimulationConfig
    daily_reports: list[list[Report]]  # index d -> the reports generated on day d


def simulate_and_report(
    *,
    seed: int,
    days: int,
    spec: StructureSpec | None = None,
    config: SimulationConfig = DEFAULT_SIMULATION_CONFIG,
    seed_infections: int | None = None,
) -> SimulatedReports:
    """Run the simulator for ``days`` days and generate each day's
    anonymous reports, WITHOUT analysing them.

    Only the CLEAN reporting model can be simulated ahead of analysis: the
    realistic model's stigma suppression reacts to yesterday's disclosure,
    so it needs the interleaved loop in ``simulation.realism`` (reached
    through :func:`run_with_detection`).

    Report generation uses its own RNG stream (``f"reporting-{seed}"``),
    independent of the simulation's own, and is interleaved with the daily
    steps exactly as :func:`run_with_detection` does it -- so the two
    produce byte-identical report streams for the same seed.
    """
    if config.reporting_model != REPORTING_MODEL_CLEAN:
        raise ValueError(
            "simulate_and_report only supports the clean reporting model; "
            "use run_with_detection for the realistic model (it needs the "
            "day-by-day disclosure feedback)"
        )
    sim = Simulation(seed=seed, spec=spec, config=config, seed_infections=seed_infectio
ns)
    report_rng = random.Random(f"reporting-{seed}")

    daily_reports: list[list[Report]] = [
        reporting.generate_daily_reports(sim.agents, 0, report_rng, config)
    ]
    for _ in range(days):
        day_state = sim.step()
        daily_reports.append(
            reporting.generate_daily_reports(sim.agents, day_state.day, report_rng, con
fig)
        )
    return SimulatedReports(simulation=sim, config=config, daily_reports=daily_reports)


class StreamAnalyzer:
    """One day at a time version of :func:`analyze_report_stream`.

    Holds the state the daily replay threads (accumulated reports,
    hysteresis, disclosure scope-stability anchor, first-fired /
    first-disclosed days) so a caller that must react to yesterday's
    disclosure while generating today's reports (the stigma feedback in
```

```
    ``simulation.realism``) can drive the very same analysis step. The
    batch function below is now a loop over this class, so both paths
    are byte-identical for the same report stream.
    """

    def __init__(
        self,
        simulation: Simulation,
        config: SimulationConfig,
        *,
        detection_config: DetectionConfig = DEFAULT_DETECTION_CONFIG,
        disclosure_config: DisclosureConfig = DEFAULT_DISCLOSURE_CONFIG,
        thread_scope_stability: bool = True,
    ) -> None:
        self.sim = simulation
        self.config = config
        self.detection_config = detection_config
        self.disclosure_config = disclosure_config
        self.thread_scope_stability = thread_scope_stability
        self.all_reports: list[Report] = []
        self.hysteresis_state: HysteresisState | None = None
        self.prior_disclosed_scope_id: str | None = None
        self.first_fired_day: int | None = None
        self.first_disclosed_day: int | None = None

    def step(self, day: int, new_reports: list[Report]) -> DailyRecord:
        self.all_reports.extend(new_reports)
        now = self.config.simulation_start + timedelta(days=day)
        analysis = analyze(
            self.all_reports,
            self.sim.registry,
            now=now,
            detection_config=self.detection_config,
            disclosure_config=self.disclosure_config,
            hysteresis_state=self.hysteresis_state,
            prior_disclosed_scope_id=(
                self.prior_disclosed_scope_id if self.thread_scope_stability else None
            ),
        )
        self.hysteresis_state = analysis.detection.hysteresis_state
        # The stability "anchor" only moves when a scope is actually named
        # today; a quiet day (nothing disclosed) does not reset it, so the
        # NEXT disclosure still prefers the last-named scope's lineage.
        if analysis.disclosed_scope_id is not None:
            self.prior_disclosed_scope_id = analysis.disclosed_scope_id

        if self.first_fired_day is None and analysis.detection.fired:
            self.first_fired_day = day
        if self.first_disclosed_day is None and analysis.disclosed_scope_id is not Non
e:
            self.first_disclosed_day = day

        day_state = self.sim.history[day]
        true_infected = day_state.count(InfectionState.INCUBATING) + day_state.count(
            InfectionState.SYMPTOMATIC
```

```
        )
        return DailyRecord(
            day=day,
            true_infected_count=true_infected,
            reports_submitted_today=len(new_reports),
            reports_accumulated=len(self.all_reports),
            detection_fired=analysis.detection.fired,
            relative_fired=analysis.detection.relative_detector_fired,
            absolute_fired=analysis.detection.absolute_detector_fired,
            disclosed_scope_id=analysis.disclosed_scope_id,
            first_fired_day=self.first_fired_day,
            first_disclosed_day=self.first_disclosed_day,
            disclosure_evaluations=analysis.disclosure.evaluations,
        )


def analyze_report_stream(
    simulated: SimulatedReports,
    *,
    detection_config: DetectionConfig = DEFAULT_DETECTION_CONFIG,
    disclosure_config: DisclosureConfig = DEFAULT_DISCLOSURE_CONFIG,
    thread_scope_stability: bool = True,
) -> list[DailyRecord]:
    """Replay ``microcluster.engine.analyze`` day by day over an
    already-simulated report stream, feeding the ACCUMULATED reports so
    far with ``now`` set to the simulated timestamp (never the wall
    clock). Detection hysteresis and disclosure scope stability are
    threaded day to day.

    Pure with respect to ``simulated`` -- safe to call repeatedly with
    different ``detection_config`` / ``disclosure_config`` on the same
    ``SimulatedReports``.

    ``thread_scope_stability=False`` stops threading
    ``prior_disclosed_scope_id``, so each day's disclosure is the plain
    finest-eligible pick with no continuity preference.
    """
    analyzer = StreamAnalyzer(
        simulated.simulation,
        simulated.config,
        detection_config=detection_config,
        disclosure_config=disclosure_config,
        thread_scope_stability=thread_scope_stability,
    )
    return [
        analyzer.step(day, new_reports)
        for day, new_reports in enumerate(simulated.daily_reports)
    ]


def run_with_detection(
    *,
    seed: int,
    days: int,
    spec: StructureSpec | None = None,
```

```
    config: SimulationConfig = DEFAULT_SIMULATION_CONFIG,
    seed_infections: int | None = None,
    detection_config: DetectionConfig = DEFAULT_DETECTION_CONFIG,
    disclosure_config: DisclosureConfig = DEFAULT_DISCLOSURE_CONFIG,
) -> SimulationRun:
    """Simulate ``days`` days for ``seed``, generate the anonymous report
    stream, and analyse it day by day -- the full end-to-end pipeline.

    Equivalent to ``simulate_and_report`` followed by
    ``analyze_report_stream``; kept as one call for the common case where
    a caller wants a single run under a single set of configs.
    """
    if config.reporting_model != REPORTING_MODEL_CLEAN:
        from .realism import run_realistic  # lazy: realism imports this module

        return run_realistic(
            seed=seed,
            days=days,
            spec=spec,
            config=config,
            seed_infections=seed_infections,
            detection_config=detection_config,
            disclosure_config=disclosure_config,
        )
    simulated = simulate_and_report(
        seed=seed, days=days, spec=spec, config=config, seed_infections=seed_infections
    )
    daily_records = analyze_report_stream(
        simulated,
        detection_config=detection_config,
        disclosure_config=disclosure_config,
    )
    all_reports = [report for day_reports in simulated.daily_reports for report in day_
reports]
    return SimulationRun(
        simulation=simulated.simulation,
        reports=all_reports,
        daily_records=daily_records,
    )
```

simulation/population.py

```
"""Population and scope-registry builder.

Given a structure spec, produce the agents together with a matching
``microcluster.models.ScopeRegistry`` whose every declared population is
the *actual* number of agents beneath that scope (rule 7 in CLAUDE.md:
declared populations are set at group creation, not counted from reports --
here "group creation" is this function, and it counts agents, not
reports).

Binding constraint (CLAUDE.md SIMULATION section): agents live in suites.
Any Report generated later uses the agent's SUITE id as ``location_id``.
The coarser scopes (FLOOR, BUILDING, CAMPUS) exist here only so that
``registry.chain_to_root`` can reach them.
"""

from __future__ import annotations

import random
from collections import Counter
from dataclasses import dataclass

from microcluster.config import MIN_SCOPE_POPULATION
from microcluster.models import Scope, ScopeLevel, ScopeRegistry

from . import config as simconfig
from .agent import Agent
from .config import DEFAULT_SIMULATION_CONFIG, SimulationConfig


@dataclass(frozen=True)
class StructureSpec:
    """How many of each scope, and how many agents per suite. The defaults
    give the 150-agent smoke-test structure."""

    buildings: int = simconfig.DEFAULT_BUILDINGS
    floors_per_building: int = simconfig.DEFAULT_FLOORS_PER_BUILDING
    suites_per_floor: int = simconfig.DEFAULT_SUITES_PER_FLOOR
    agents_per_suite: int = simconfig.DEFAULT_AGENTS_PER_SUITE


def _validate_spec(spec: StructureSpec) -> None:
    for name, value in vars(spec).items():
        if value <= 0:
            raise ValueError(f"StructureSpec.{name} must be positive, got {value!r}")
    if spec.agents_per_suite <= MIN_SCOPE_POPULATION:
        raise ValueError(
            f"agents_per_suite ({spec.agents_per_suite}) must exceed "
            f"MIN_SCOPE_POPULATION ({MIN_SCOPE_POPULATION}); at or below it the "
            f"privacy gate can never pass at suite level, so a suite-level "
            f"disclosure would be structurally impossible."
        )

```

```
 53 |
 54 | def build_population(
 55 |     spec: StructureSpec | None = None,
 56 |     rng: random.Random | None = None,
 57 |     config: SimulationConfig = DEFAULT_SIMULATION_CONFIG,
 58 | ) -> tuple[list[Agent], ScopeRegistry]:
 59 |     """Build the agent list and the matching ``ScopeRegistry``.
 60 |
 61 |     ``rng`` seeds the per-agent reporting-probability draws; pass an
 62 |     explicit ``random.Random`` for reproducibility.
 63 |     """
 64 |     spec = spec or StructureSpec()
 65 |     _validate_spec(spec)
 66 |     rng = rng or random.Random()
 67 |
 68 |     campus_id = config.campus_scope_id
 69 |     parent_of: dict[str, str] = {}
 70 |     building_ids: list[str] = []
 71 |     floor_ids: list[str] = []
 72 |     suite_ids: list[str] = []
 73 |     agents: list[Agent] = []
 74 |
 75 |     for b in range(1, spec.buildings + 1):
 76 |         building_id = f"{campus_id}-B{b}"
 77 |         building_ids.append(building_id)
 78 |         parent_of[building_id] = campus_id
 79 |         for f in range(1, spec.floors_per_building + 1):
 80 |             floor_id = f"{building_id}-F{f}"
 81 |             floor_ids.append(floor_id)
 82 |             parent_of[floor_id] = building_id
 83 |             for s in range(1, spec.suites_per_floor + 1):
 84 |                 suite_id = f"{floor_id}-S{s}"
 85 |                 suite_ids.append(suite_id)
 86 |                 parent_of[suite_id] = floor_id
 87 |                 for a in range(1, spec.agents_per_suite + 1):
 88 |                     agents.append(
 89 |                         Agent(
 90 |                             id=f"{suite_id}-A{a}",
 91 |                             suite_id=suite_id,
 92 |                             reporting_probability=rng.uniform(
 93 |                                 config.reporting_probability_min,
 94 |                                 config.reporting_probability_max,
 95 |                             ),
 96 |                         )
 97 |                     )
 98 |
 99 |     # Declared population of a scope == number of agents beneath it. Walk
100 |     # each agent's suite up to the campus root, counting as we go.
101 |     population: Counter[str] = Counter()
102 |     for agent in agents:
103 |         node = agent.suite_id
104 |         while True:
105 |             population[node] += 1
106 |             if node == campus_id:
107 |                 break
```

```
            node = parent_of[node]

    scopes: dict[str, Scope] = {
        campus_id: Scope(
            id=campus_id,
            label=config.campus_label,
            level=ScopeLevel.CAMPUS,
            population=population[campus_id],
        )
    }
    for building_id in building_ids:
        scopes[building_id] = Scope(
            id=building_id,
            label=building_id,
            level=ScopeLevel.BUILDING,
            population=population[building_id],
            parent_id=parent_of[building_id],
        )
    for floor_id in floor_ids:
        scopes[floor_id] = Scope(
            id=floor_id,
            label=floor_id,
            level=ScopeLevel.FLOOR,
            population=population[floor_id],
            parent_id=parent_of[floor_id],
        )
    for suite_id in suite_ids:
        scopes[suite_id] = Scope(
            id=suite_id,
            label=suite_id,
            level=ScopeLevel.SUITE,
            population=population[suite_id],
            parent_id=parent_of[suite_id],
        )

    registry = ScopeRegistry(scopes)  # runs ScopeRegistry's own validation
    return agents, registry
```

simulation/realism.py

```
"""Reporting-realism stress test.

The shipped reporting model is deliberately clean: each agent gets one fixed
probability (uniform 0.3 to 0.7), applied independently each symptomatic
day, with no memory and no feedback. Every committed result depends on it.
This module is an ALTERNATIVE, selected with
``SimulationConfig(reporting_model="realistic")``, that breaks those three
assumptions on purpose:

  * HETEROGENEITY. Baselines come from Beta(0.5, 0.5) instead of
    uniform(0.3, 0.7). Same mean (0.5), very different spread: about a third
    of agents almost never report and about a third almost always do.
  * CORRELATION. A person's probability rises after suite-mates report (the
    "everyone else is filing, so will I" effect, and worried-well filers).
    p' = 1 - (1 - p) ** (1 + gain * r), r = reports from the agent's own
    suite in the previous few days. It applies to background reports too.
  * STIGMA. Once a scope has been NAMED in a disclosure, its members report
    less (p' = p * stigma_factor) for as long as it stays named plus a
    tail. This needs a feedback loop the clean model does not have: today's
    reports depend on yesterday's disclosure, so this runner interleaves
    generation with the same daily analysis step the batch pipeline uses
    (``pipeline.StreamAnalyzer``).

The epidemic itself is untouched and, for a given seed, IDENTICAL under both
models (the simulation RNG stream never sees reporting), so every
comparison below is paired: same outbreak, different reporting behaviour.

Every constant is CHOSEN, not measured -- like the clean model it stands in
for. This tests how much the detector depends on the clean assumption; it
does not claim to describe how real residents behave.

    python -m simulation.realism
"""

from __future__ import annotations

import random
from dataclasses import dataclass, replace
from datetime import timedelta
from statistics import mean
from typing import Sequence

from microcluster.config import (
    DEFAULT_DETECTION_CONFIG,
    DEFAULT_DISCLOSURE_CONFIG,
    DetectionConfig,
    DisclosureConfig,
)
from microcluster.models import CHECKLIST, Onset, Report, ScopeRegistry

from .agent import Agent
from .config import (
```

```
 53 |     DEFAULT_RUN_DAYS,
 54 |     DEFAULT_SIMULATION_CONFIG,
 55 |     REPORTING_MODEL_REALISTIC,
 56 |     SimulationConfig,
 57 | )
 58 | from .experiments import RunOutcome, reduce_run, summarize_setting
 59 | from .infection import InfectionState
 60 | from .pipeline import SimulationRun, StreamAnalyzer, run_with_detection
 61 | from .population import StructureSpec
 62 | from .reporting import _non_outbreak_categories, _onset_for
 63 | from .simulator import Simulation
 64 |
 65 | __all__ = [
 66 |     "RealismConfig",
 67 |     "VARIANTS",
 68 |     "run_realistic",
 69 |     "run_realism_sweep",
 70 | ]
 71 |
 72 |
 73 | @dataclass(frozen=True)
 74 | class RealismConfig:
 75 |     """Which realism effects are on, and how strong. All CHOSEN figures."""
 76 |
 77 |     heterogeneity: bool = True
 78 |     correlation: bool = True
 79 |     stigma: bool = True
 80 |
 81 |     # Beta(a, b) per-agent baseline. a = b = 0.5 keeps the clean model's
 82 |     # mean of 0.5 but makes it bimodal (U-shaped).
 83 |     baseline_beta_a: float = 0.5
 84 |     baseline_beta_b: float = 0.5
 85 |
 86 |     # p' = 1 - (1 - p) ** (1 + gain * r); r counts reports from the agent's
 87 |     # own suite in the previous ``correlation_window_days`` days, capped.
 88 |     correlation_gain: float = 0.5
 89 |     correlation_window_days: int = 3
 90 |     correlation_max_reports: int = 4
 91 |
 92 |     # Inside a scope that has been named: p' = p * stigma_factor, for as
 93 |     # long as it stays named and ``stigma_days`` after it was last named.
 94 |     stigma_factor: float = 0.5
 95 |     stigma_days: int = 7
 96 |
 97 |
 98 | DEFAULT_REALISM_CONFIG = RealismConfig()
 99 |
100 | # The full model and each effect alone, for attribution.
101 | VARIANTS: dict[str, RealismConfig | None] = {
102 |     "clean (shipped)": None,
103 |     "realistic (all three)": DEFAULT_REALISM_CONFIG,
104 |     "heterogeneity only": RealismConfig(heterogeneity=True, correlation=False, stigma=F
alse),
105 |     "correlation only": RealismConfig(heterogeneity=False, correlation=True, stigma=Fal
se),
```

```
    "stigma only": RealismConfig(heterogeneity=False, correlation=False, stigma=True),
}


class _State:
    """What the generator remembers between days."""

    def __init__(self, registry: ScopeRegistry, realism: RealismConfig) -> None:
        self.realism = realism
        self._lineage: dict[str, tuple[str, ...]] = {}
        self._registry = registry
        self.suite_reports_by_day: dict[str, dict[int, int]] = {}
        self.named_until: dict[str, int] = {}

    def lineage(self, suite_id: str) -> tuple[str, ...]:
        got = self._lineage.get(suite_id)
        if got is None:
            got = tuple(s.id for s in self._registry.chain_to_root(suite_id))
            self._lineage[suite_id] = got
        return got

    def recent_suite_reports(self, suite_id: str, day: int) -> int:
        by_day = self.suite_reports_by_day.get(suite_id)
        if not by_day:
            return 0
        window = self.realism.correlation_window_days
        total = sum(by_day.get(d, 0) for d in range(day - window, day))
        return min(total, self.realism.correlation_max_reports)

    def note_reports(self, day: int, reports: Sequence[Report]) -> None:
        for r in reports:
            d = self.suite_reports_by_day.setdefault(r.location_id, {})
            d[day] = d.get(day, 0) + 1

    def note_disclosed(self, day: int, scope_id: str | None) -> None:
        if scope_id is not None:
            self.named_until[scope_id] = day + self.realism.stigma_days

    def suppressed(self, suite_id: str, day: int) -> bool:
        return any(self.named_until.get(s, -1) >= day for s in self.lineage(suite_id))


def _effective(p: float, suite_id: str, day: int, state: _State) -> float:
    rc = state.realism
    if rc.correlation:
        r = state.recent_suite_reports(suite_id, day)
        if r:
            p = 1.0 - (1.0 - p) ** (1.0 + rc.correlation_gain * r)
    if rc.stigma and state.suppressed(suite_id, day):
        p *= rc.stigma_factor
    return p


def _generate_daily_reports(
    agents: list[Agent],
```

```
    day: int,
    rng: random.Random,
    config: SimulationConfig,
    state: _State,
) -> list[Report]:
    """Same structure as ``reporting.generate_daily_reports`` (same
    categories, same one-outbreak-report-per-episode rule, same suite-only
    location); only the probabilities differ."""
    submitted_at = config.simulation_start + timedelta(days=day)
    noise_categories = _non_outbreak_categories(config.outbreak_category)
    reports: list[Report] = []
    for agent in agents:
        if agent.state is InfectionState.SYMPTOMATIC:
            if agent.has_reported:
                continue
            p = _effective(agent.reporting_probability, agent.suite_id, day, state)
            if rng.random() < p:
                symptom = rng.choice(CHECKLIST[config.outbreak_category])
                reports.append(
                    Report(
                        category=config.outbreak_category,
                        symptom=symptom,
                        onset=_onset_for(day - agent.state_changed_day),
                        location_id=agent.suite_id,
                        submitted_at=submitted_at,
                    )
                )
                agent.has_reported = True
            continue
        noise = _effective(config.background_noise_daily_rate, agent.suite_id, day, sta
te)
        if rng.random() < noise:
            category = rng.choice(noise_categories)
            reports.append(
                Report(
                    category=category,
                    symptom=rng.choice(CHECKLIST[category]),
                    onset=rng.choice(list(Onset)),
                    location_id=agent.suite_id,
                    submitted_at=submitted_at,
                )
            )
    return reports


def run_realistic(
    *,
    seed: int,
    days: int,
    spec: StructureSpec | None = None,
    config: SimulationConfig = DEFAULT_SIMULATION_CONFIG,
    seed_infections: int | None = None,
    detection_config: DetectionConfig = DEFAULT_DETECTION_CONFIG,
    disclosure_config: DisclosureConfig = DEFAULT_DISCLOSURE_CONFIG,
    realism: RealismConfig = DEFAULT_REALISM_CONFIG,
```

```
) -> SimulationRun:
    """Simulate + report + analyse one run under the realistic model."""
    sim = Simulation(seed=seed, spec=spec, config=config, seed_infections=seed_infectio
ns)
    if realism.heterogeneity:
        # own stream: never perturbs the epidemic's RNG
        baseline_rng = random.Random(f"realism-baseline-{seed}")
        for agent in sim.agents:
            agent.reporting_probability = baseline_rng.betavariate(
                realism.baseline_beta_a, realism.baseline_beta_b
            )
    rng = random.Random(f"reporting-{seed}")
    state = _State(sim.registry, realism)
    analyzer = StreamAnalyzer(
        sim, config, detection_config=detection_config, disclosure_config=disclosure_co
nfig
    )

    all_reports: list[Report] = []
    records = []
    for day in range(days + 1):
        if day > 0:
            sim.step()
        todays = _generate_daily_reports(sim.agents, day, rng, config, state)
        state.note_reports(day, todays)
        all_reports.extend(todays)
        record = analyzer.step(day, todays)
        state.note_disclosed(day, record.disclosed_scope_id)
        records.append(record)
    return SimulationRun(simulation=sim, reports=all_reports, daily_records=records)


# ---------------------------------------------------------------------------
# The paired sweep
# ---------------------------------------------------------------------------


def _continuity(records) -> float | None:
    """Of the days from first disclosure to the end of the run, the share on
    which something was still being disclosed."""
    first = records[-1].first_disclosed_day
    if first is None:
        return None
    tail = records[first:]
    return sum(1 for r in tail if r.disclosed_scope_id is not None) / len(tail)


def run_realism_sweep(
    seeds: Sequence[int] | None = None,
    *,
    days: int = DEFAULT_RUN_DAYS,
    spec: StructureSpec | None = None,
) -> dict[str, dict]:
    seed_list = list(seeds) if seeds is not None else list(range(1, 501))
    out: dict[str, dict] = {}
```

```
    for name, realism in VARIANTS.items():
        outcomes: list[RunOutcome] = []
        continuity: list[float] = []
        reports_total: list[int] = []
        for seed in seed_list:
            if realism is None:
                run = run_with_detection(seed=seed, days=days, spec=spec)
            else:
                run = run_realistic(
                    seed=seed,
                    days=days,
                    spec=spec,
                    config=replace(DEFAULT_SIMULATION_CONFIG, reporting_model=REPORTING
_MODEL_REALISTIC),
                    realism=realism,
                )
            outcome = reduce_run(seed, run.simulation, run.daily_records)
            outcomes.append(outcome)
            reports_total.append(len(run.reports))
            if outcome.established:
                c = _continuity(run.daily_records)
                if c is not None:
                    continuity.append(c)
        s = summarize_setting(outcomes, param_name="reporting_model", value=0)
        non_est = [o for o in outcomes if not o.established]
        false_disclosures = [o for o in non_est if o.first_disclosed_day is not None]
        out[name] = {
            "n_runs": s.n_runs,
            "n_established": s.n_established,
            "n_disclosed": s.n_disclosed,
            "disclosed_fraction": s.disclosed_fraction,
            "fire_delay": s.mean_fire_delay,
            "disclosure_delay": s.mean_disclosure_delay,
            "gap": s.mean_gap,
            "infections_before_disclosure": s.mean_infections_before_disclosure_unbiase
d,
            "false_alarm_rate": s.false_alarm_rate,
            "false_disclosure_rate": (len(false_disclosures) / len(non_est)) if non_est
else None,
            "disclosure_continuity": mean(continuity) if continuity else None,
            "mean_reports_per_run": mean(reports_total),
            "finest_level": s.finest_disclosed_level.name if s.finest_disclosed_level e
lse None,
        }
    return out


GRID_BETA = (None, 2.0, 0.5, 0.25)        # None = the clean uniform(0.3, 0.7) baseline
GRID_GAIN = (0.0, 0.25, 0.5, 1.0)         # correlation strength; 0 = none


def run_sensitivity_grid(
    seeds: Sequence[int] | None = None, *, days: int = DEFAULT_RUN_DAYS
) -> list[dict]:
    """Heterogeneity x correlation grid (stigma on throughout). The full
```

```
    model above is one cell. Its point: the timing metrics can look robust
    only because opposing effects cancel, so show how far they move when the
    strengths are changed."""
    seed_list = list(seeds) if seeds is not None else list(range(1, 501))
    cfg = replace(DEFAULT_SIMULATION_CONFIG, reporting_model=REPORTING_MODEL_REALISTIC)
    rows: list[dict] = []
    for beta in GRID_BETA:
        for gain in GRID_GAIN:
            realism = RealismConfig(
                heterogeneity=beta is not None,
                correlation=gain > 0,
                stigma=True,
                baseline_beta_a=beta or 0.5,
                baseline_beta_b=beta or 0.5,
                correlation_gain=gain,
            )
            outcomes = []
            for seed in seed_list:
                run = run_realistic(seed=seed, days=days, config=cfg, realism=realism)
                outcomes.append(reduce_run(seed, run.simulation, run.daily_records))
            s = summarize_setting(outcomes, param_name="grid", value=0)
            non_est = [o for o in outcomes if not o.established]
            rows.append(
                {
                    "beta": beta,
                    "correlation_gain": gain,
                    "disclosure_delay": s.mean_disclosure_delay,
                    "infections_before_disclosure": s.mean_infections_before_disclosure
_unbiased,
                    "false_alarm_rate": s.false_alarm_rate,
                    "false_disclosure_rate": (
                        sum(1 for o in non_est if o.first_disclosed_day is not None) /
len(non_est)
                        if non_est else None
                    ),
                }
            )
    return rows


def _f(v, d=2, pct=False):
    if v is None:
        return "n/a"
    return f"{v * 100:.0f}%" if pct else f"{v:.{d}f}"


def format_sweep(results: dict[str, dict]) -> str:
    head = (
        f"{'model':<24}{'fire d':>8}{'disc d':>8}{'gap':>7}{'inf<disc':>10}"
        f"{'discl%':>8}{'false-alarm':>13}{'false-disc':>12}{'continuity':>12}{'report
s':>9}"
    )
    lines = ["REPORTING-REALISM STRESS TEST (paired: same outbreaks, different reportin
g)", "", head]
    for name, r in results.items():
```

```
370 |         lines.append(
371 |             f"{name:<24}{_f(r['fire_delay'],1):>8}{_f(r['disclosure_delay'],1):>8}{_f(r
['gap'],1):>7}"
372 |             f"{_f(r['infections_before_disclosure'],1):>10}{_f(r['disclosed_fraction'],
pct=True):>8}"
373 |             f"{_f(r['false_alarm_rate'],pct=True):>13}{_f(r['false_disclosure_rate'],pc
t=True):>12}"
374 |             f"{_f(r['disclosure_continuity'],pct=True):>12}{_f(r['mean_reports_per_ru
n'],0):>9}"
375 |         )
376 |     return "\n".join(lines)
377 |
378 |
379 | def main() -> None:
380 |     print(format_sweep(run_realism_sweep()))
381 |
382 |
383 | if __name__ == "__main__":
384 |     main()
```

simulation/reporting.py

```
"""Turns simulated infection state into anonymous Reports.

This is the ONLY bridge between the simulator's ground truth and the
production ``microcluster`` engines. Only two facts survive the crossing:
which SUITE an agent is in, and whether they feel like filing a report
today. Everything else the simulator knows -- who is actually infected,
since when, who infected whom -- stays on this side of the line (see
CLAUDE.md, "The ground-truth firewall").

Two kinds of reports are generated each day:

  * OUTBREAK reports -- from agents SYMPTOMATIC with the real illness, at
    most once per episode, in the single configured outbreak category.
  * BACKGROUND NOISE -- from agents who are NOT currently symptomatic,
    independently each day, in a category other than the outbreak's. This
    is what makes the detection problem real: without it, every report in
    the stream would be outbreak signal.
"""

from __future__ import annotations

import random
from datetime import timedelta

from microcluster.models import CHECKLIST, Category, Onset, Report

from .agent import Agent
from .config import SimulationConfig
from .infection import InfectionState

# Every fixed-checklist category except whichever one the outbreak is
# using, for drawing a background-noise report's category.
_ALL_CATEGORIES: tuple[Category, ...] = tuple(Category)


def _non_outbreak_categories(outbreak_category: Category) -> tuple[Category, ...]:
    return tuple(c for c in _ALL_CATEGORIES if c != outbreak_category)


def _onset_for(days_symptomatic: int) -> Onset:
    """Coarse onset bucket (rule 1), computed from how many days an agent
    has been symptomatic -- what a real reporter would say if asked."""
    if days_symptomatic <= 0:
        return Onset.TODAY
    if days_symptomatic <= 2:
        return Onset.ONE_TO_TWO_DAYS
    if days_symptomatic <= 7:
        return Onset.THREE_TO_SEVEN_DAYS
    return Onset.ONE_WEEK_PLUS


def generate_daily_reports(
```

```
    agents: list[Agent],
    day: int,
    rng: random.Random,
    config: SimulationConfig,
) -> list[Report]:
    """Every report filed on simulated ``day``.

    Mutates ``agent.has_reported`` in place when an outbreak report is
    filed, so the same episode is never reported twice. ``rng`` should be
    a stream dedicated to reporting, kept separate from the simulation's
    own transmission/progression stream so that adding report generation
    never perturbs the epidemic trajectory itself.
    """
    submitted_at = config.simulation_start + timedelta(days=day)
    noise_categories = _non_outbreak_categories(config.outbreak_category)
    reports: list[Report] = []

    for agent in agents:
        if agent.state is InfectionState.SYMPTOMATIC:
            if agent.has_reported:
                continue
            if rng.random() < agent.reporting_probability:
                days_symptomatic = day - agent.state_changed_day
                symptom = rng.choice(CHECKLIST[config.outbreak_category])
                reports.append(
                    Report(
                        category=config.outbreak_category,
                        symptom=symptom,
                        onset=_onset_for(days_symptomatic),
                        location_id=agent.suite_id,
                        submitted_at=submitted_at,
                    )
                )
                agent.has_reported = True
            continue

        # Not currently symptomatic with the real illness: a chance of an
        # ordinary, unrelated report, independent of any episode.
        if rng.random() < config.background_noise_daily_rate:
            category = rng.choice(noise_categories)
            reports.append(
                Report(
                    category=category,
                    symptom=rng.choice(CHECKLIST[category]),
                    onset=rng.choice(list(Onset)),
                    location_id=agent.suite_id,
                    submitted_at=submitted_at,
                )
            )

    return reports
```

simulation/scale.py

```
"""Scale validation: does the pipeline still behave at 500 / 1,000 / 2,000
people with four scope levels (campus > building > floor > suite)?

The population builder was already parametric; what was never exercised was
a structure with more than one building, so the BUILDING scope always
equalled the CAMPUS scope and the hierarchy had three DISTINCT levels. The
specs below give four distinct levels at every size. No engine, gate or
simulator logic is touched -- this module only builds bigger structures,
runs the existing pipeline over a seeded sweep, and CHECKS the results
against the gate formulas independently of the engine that produced them.

    python -m simulation.scale            # prints the table
    python -m scripts.stress_tests        # writes results/scale.json too

Every number here is a property of this simulator with CHOSEN parameters
(see config.py); it says whether the machinery scales, not what a real
building would do.
"""

from __future__ import annotations

import math
import time
from dataclasses import dataclass, field
from statistics import mean, median

from microcluster.config import DEFAULT_DISCLOSURE_CONFIG
from microcluster.models import ScopeLevel

from .config import DEFAULT_RUN_DAYS
from .experiments import reduce_run, summarize_setting
from .infection import InfectionState
from .pipeline import run_with_detection
from .population import StructureSpec

__all__ = ["SCALE_SPECS", "SizeReport", "measure_size", "run_scale_validation"]

# (buildings, floors per building, suites per floor, agents per suite).
# Suite size stays 25 so the suite-level gates are identical at every size
# and any change is attributable to the HIERARCHY growing, not the suite.
SCALE_SPECS: dict[int, StructureSpec] = {
    150: StructureSpec(1, 2, 3, 25),    # the shipped baseline (3 distinct levels)
    500: StructureSpec(2, 2, 5, 25),
    1000: StructureSpec(2, 4, 5, 25),
    2000: StructureSpec(4, 4, 5, 25),
}


@dataclass
class SizeReport:
    n_agents: int
    n_suites: int
```

```
 53 |     n_distinct_populations: int      # distinct declared populations across the hierarc
hy
 54 |     n_runs: int
 55 |     seconds_per_run: float
 56 |
 57 |     # plausibility of the simulated epidemic
 58 |     established_fraction: float
 59 |     median_attack_rate: float | None          # of ALL agents, established runs
 60 |     median_attack_rate_in_touched: float | None  # of the buildings that got any case
 61 |     median_buildings_touched: float | None     # buildings with >= 1 ever-infected
 62 |     peaks_and_declines_fraction: float | None  # established runs whose curve turns dow
n
 63 |
 64 |     # evidence gate vs its formula
 65 |     scope_evaluations_checked: int
 66 |     threshold_formula_violations: int          # engine threshold != max(5, ceil(sqrt
n))
 67 |     pass_flag_violations: int                  # stat_pass != (qualifying >= threshold)
 68 |     selected_without_both_gates: int
 69 |     thresholds_by_level: dict[str, int] = field(default_factory=dict)
 70 |
 71 |     # privacy gate
 72 |     roster_refusals: int = 0                   # stat PASS but privacy FAIL (scope-day
s)
 73 |     roster_refusal_levels: dict[str, int] = field(default_factory=dict)
 74 |     runs_with_a_roster_refusal: int = 0
 75 |
 76 |     # timing (established runs that disclosed)
 77 |     n_established: int = 0
 78 |     n_disclosed: int = 0
 79 |     mean_fire_delay: float | None = None
 80 |     mean_disclosure_delay: float | None = None
 81 |     mean_gap: float | None = None
 82 |     infections_before_disclosure: float | None = None       # unbiased
 83 |     infections_before_disclosure_pct: float | None = None   # of all agents
 84 |     finest_level_histogram: dict[str, int] = field(default_factory=dict)
 85 |     false_alarm_rate: float | None = None
 86 |     fizzle_rate: float = 0.0
 87 |     relative_fired_days: int = 0
 88 |     absolute_fired_days: int = 0
 89 |     errors: int = 0
 90 |
 91 |
 92 | def _ceil_sqrt(n: int) -> int:
 93 |     root = math.isqrt(n)
 94 |     return root if root * root == n else root + 1
 95 |
 96 |
 97 | def measure_size(
 98 |     n_agents: int, seeds: list[int], *, days: int = DEFAULT_RUN_DAYS
 99 | ) -> SizeReport:
100 |     spec = SCALE_SPECS[n_agents]
101 |     floor = DEFAULT_DISCLOSURE_CONFIG.statistical_gate_floor
102 |     outcomes = []
103 |     attack_rates: list[float] = []
```

```
    attack_touched: list[float] = []
    buildings_touched: list[int] = []
    peaked = established = 0
    checked = thr_bad = flag_bad = sel_bad = 0
    thresholds: dict[str, int] = {}
    roster = roster_runs = 0
    roster_levels: dict[str, int] = {}
    rel_days = abs_days = 0
    errors = 0
    n_suites = 0
    populations_seen: set[int] = set()
    t0 = time.perf_counter()

    for seed in seeds:
        try:
            run = run_with_detection(seed=seed, days=days, spec=spec)
        except Exception:  # a component failing outright is a RESULT, not a crash
            errors += 1
            continue
        sim, records = run.simulation, run.daily_records
        n_suites = sum(1 for s in sim.registry.scopes.values() if s.level is ScopeLeve
l.SUITE)
        populations_seen = {s.population for s in sim.registry.scopes.values()}
        outcome = reduce_run(seed, sim, records)
        outcomes.append(outcome)

        if outcome.established:
            established += 1
            attack_rates.append(outcome.total_infections / len(sim.agents))
            final = sim.history[-1]
            touched = {
                a.suite_id.rsplit("-F", 1)[0]
                for a in final.agents
                if a.state is not InfectionState.SUSCEPTIBLE
            }
            buildings_touched.append(len(touched))
            bpop = sim.registry.get(sorted(touched)[0]).population
            attack_touched.append(outcome.total_infections / (len(touched) * bpop))
            active = [
                s.count(InfectionState.INCUBATING) + s.count(InfectionState.SYMPTOMATI
C)
                for s in sim.history
            ]
            if max(active) > 0 and active.index(max(active)) < len(active) - 1 and acti
ve[-1] < max(active):
                peaked += 1

        run_refused = False
        for rec in records:
            rel_days += rec.relative_fired
            abs_days += rec.absolute_fired
            for ev in rec.disclosure_evaluations:
                checked += 1
                expected = max(floor, _ceil_sqrt(ev.population))
                thresholds[ev.scope_level.name] = expected
```

```
                if ev.statistical_threshold != expected:
                    thr_bad += 1
                if ev.statistical_pass != (ev.qualifying_reports >= ev.statistical_thre
shold):
                    flag_bad += 1
                if ev.selected and not (ev.statistical_pass and ev.privacy_pass):
                    sel_bad += 1
                if ev.statistical_pass and not ev.privacy_pass:
                    roster += 1
                    run_refused = True
                    roster_levels[ev.scope_level.name] = roster_levels.get(ev.scope_lev
el.name, 0) + 1
        roster_runs += run_refused

    elapsed = time.perf_counter() - t0
    summary = summarize_setting(outcomes, param_name="size", value=n_agents)
    finest: dict[str, int] = {}
    for o in outcomes:
        if o.established and o.finest_disclosed_level is not None:
            name = o.finest_disclosed_level.name
            finest[name] = finest.get(name, 0) + 1

    def _pct(x: float | None) -> float | None:
        return None if x is None else x / n_agents * 100

    return SizeReport(
        n_agents=n_agents,
        n_suites=n_suites,
        n_distinct_populations=len(populations_seen),
        n_runs=len(outcomes),
        seconds_per_run=elapsed / max(1, len(seeds)),
        established_fraction=(established / len(outcomes)) if outcomes else 0.0,
        median_attack_rate=median(attack_rates) if attack_rates else None,
        median_attack_rate_in_touched=median(attack_touched) if attack_touched else Non
e,
        median_buildings_touched=median(buildings_touched) if buildings_touched else No
ne,
        peaks_and_declines_fraction=(peaked / established) if established else None,
        scope_evaluations_checked=checked,
        threshold_formula_violations=thr_bad,
        pass_flag_violations=flag_bad,
        selected_without_both_gates=sel_bad,
        thresholds_by_level=thresholds,
        roster_refusals=roster,
        roster_refusal_levels=roster_levels,
        runs_with_a_roster_refusal=roster_runs,
        n_established=summary.n_established,
        n_disclosed=summary.n_disclosed,
        mean_fire_delay=summary.mean_fire_delay,
        mean_disclosure_delay=summary.mean_disclosure_delay,
        mean_gap=summary.mean_gap,
        infections_before_disclosure=summary.mean_infections_before_disclosure_unbiase
d,
        infections_before_disclosure_pct=_pct(summary.mean_infections_before_disclosure
_unbiased),
```

```
        finest_level_histogram=finest,
        false_alarm_rate=summary.false_alarm_rate,
        fizzle_rate=1 - (established / len(outcomes)) if outcomes else 0.0,
        relative_fired_days=rel_days,
        absolute_fired_days=abs_days,
        errors=errors,
    )


def run_scale_validation(
    seeds: list[int] | None = None, sizes: tuple[int, ...] = (150, 500, 1000, 2000)
) -> list[SizeReport]:
    seed_list = seeds if seeds is not None else list(range(1, 201))
    return [measure_size(n, seed_list) for n in sizes]


def _f(v: float | None, d: int = 1) -> str:
    return "n/a" if v is None else f"{v:.{d}f}"


def format_reports(reports: list[SizeReport]) -> str:
    lines = ["SCALE VALIDATION (same pipeline, same gates, bigger structures)", ""]
    for r in reports:
        lines += [
            f"--- {r.n_agents} agents, {r.n_suites} suites, {r.n_distinct_populations} distinct scope populations "
            f"({r.n_runs} runs, {r.seconds_per_run:.2f}s/run, {r.errors} errors)",
            f"  epidemic : established {r.established_fraction:.0%}, median attack rate "
            f"{_f(None if r.median_attack_rate is None else r.median_attack_rate * 100)}% of all agents, "
            f"{_f(None if r.median_attack_rate_in_touched is None else r.median_attack_rate_in_touched * 100)}% of the building(s) touched, "
            f"median buildings touched {_f(r.median_buildings_touched)}, "
            f"peak-then-decline {_f(None if r.peaks_and_declines_fraction is None else r.peaks_and_declines_fraction * 100, 0)}%",
            f"  evidence : {r.scope_evaluations_checked} scope evaluations checked; formula violations "
            f"{r.threshold_formula_violations}, pass-flag violations {r.pass_flag_violations}, "
            f"selected-without-both-gates {r.selected_without_both_gates}; thresholds {r.thresholds_by_level}",
            f"  privacy  : roster refusals {r.roster_refusals} in {r.runs_with_a_roster_refusal} runs, by level {r.roster_refusal_levels}",
            f"  timing   : fire day {_f(r.mean_fire_delay)}, disclosure day {_f(r.mean_disclosure_delay)}, gap {_f(r.mean_gap)}, "
            f"infected before disclosure {_f(r.infections_before_disclosure)} "
            f"({_f(r.infections_before_disclosure_pct)}% of agents); disclosed {r.n_disclosed}/{r.n_established}; "
            f"false-alarm rate {_f(None if r.false_alarm_rate is None else r.false_alarm_rate * 100, 0)}%",
            f"  finest level named (established runs): {r.finest_level_histogram}",
            "",
        ]
    return "\n".join(lines)
```

```
248 |
249 |
250 | def main() -> None:
251 |     print(format_reports(run_scale_validation()))
252 |
253 |
254 | if __name__ == "__main__":
255 |     main()
```

simulation/simulator.py

```
"""The run loop.

Step one simulated day at a time. After every step the full world state
is captured as a :class:`DayState` (per-state counts plus an immutable
snapshot of every agent) and appended to ``history`` so a run can be
recorded, replayed, or rendered later.

Contact is tiered (suite / floor / building), not suite-only -- see
``simulation.contact``. Each suite's floor and building ids are resolved
once, through the ``ScopeRegistry``, and reused every step.
"""

from __future__ import annotations

import random
from dataclasses import dataclass

from microcluster.models import ScopeRegistry

from . import contact, infection
from .contact import suite_ancestor_ids
from .agent import Agent, AgentSnapshot
from .config import DEFAULT_SIMULATION_CONFIG, SimulationConfig
from .infection import STATE_ORDER, InfectionState
from .population import StructureSpec, build_population


@dataclass(frozen=True)
class DayState:
    """The whole simulated world at the end of one day."""

    day: int
    counts: dict[InfectionState, int]
    agents: tuple[AgentSnapshot, ...]

    @property
    def total(self) -> int:
        return sum(self.counts.values())

    def count(self, state: InfectionState) -> int:
        return self.counts[state]


class Simulation:
    """One outbreak run. Fully determined by ``seed`` (given the same
    ``spec`` and ``config``)."""

    def __init__(
        self,
        *,
        seed: int,
        spec: StructureSpec | None = None,
```

```
        config: SimulationConfig = DEFAULT_SIMULATION_CONFIG,
        seed_infections: int | None = None,
    ) -> None:
        self.config = config
        self.spec = spec or StructureSpec()
        self.rng = random.Random(seed)
        self.agents: list[Agent]
        self.registry: ScopeRegistry
        self.agents, self.registry = build_population(self.spec, self.rng, config)

        self._suites: dict[str, list[Agent]] = {}
        for agent in self.agents:
            self._suites.setdefault(agent.suite_id, []).append(agent)

        # Where each suite sits in the hierarchy, read once from the
        # registry (never hard-coded from id strings) and reused every step.
        self._floor_of_suite: dict[str, str] = {}
        self._building_of_suite: dict[str, str] = {}
        for suite_id in self._suites:
            floor_id, building_id = suite_ancestor_ids(self.registry, suite_id)
            self._floor_of_suite[suite_id] = floor_id
            self._building_of_suite[suite_id] = building_id

        self.day = 0
        requested = (
            config.seed_infections if seed_infections is None else seed_infections
        )
        self._seed_infections(requested)

        # Running totals, split by contact tier that caused the infection
        # (see contact.new_infections). Useful for measuring how much of an
        # outbreak stayed within its starting suite versus spread further.
        self.total_within_suite_infections = 0
        self.total_cross_suite_infections = 0

        self.history: list[DayState] = [self._snapshot()]

    # -- setup ---------------------------------------------------------

    def _seed_infections(self, n: int) -> None:
        if n < 0:
            raise ValueError("seed_infections must be >= 0")
        if n > len(self.agents):
            raise ValueError(
                f"cannot seed {n} infections in a population of {len(self.agents)}"
            )
        for agent in self.rng.sample(self.agents, n):
            infection.infect(agent, self.day, self.rng, self.config)

    # -- stepping ------------------------------------------------------

    def step(self) -> DayState:
        """Advance one day.

        Order within the step:
```

```
        1. Decide new infections from the START-OF-DAY state, so an agent
           infected today cannot also transmit or progress today.
        2. Apply time-based progression (INCUBATING -> SYMPTOMATIC ->
           RECOVERED) to everyone.
        3. Apply the new infections (SUSCEPTIBLE -> INCUBATING).

        Returns the resulting :class:`DayState`, also appended to
        ``history``.
        """
        self.day += 1
        day = self.day

        within_suite, cross_suite = contact.new_infections(
            self._suites, self._floor_of_suite, self._building_of_suite,
            self.rng, self.config,
        )
        self.total_within_suite_infections += len(within_suite)
        self.total_cross_suite_infections += len(cross_suite)
        newly_infected = within_suite + cross_suite

        for agent in self.agents:
            infection.advance_time_based(agent, day)

        for agent in newly_infected:
            infection.infect(agent, day, self.rng, self.config)

        snapshot = self._snapshot()
        self.history.append(snapshot)
        return snapshot

    def run(self, days: int) -> list[DayState]:
        """Step ``days`` times; return the full history (day 0 .. day
        ``days``)."""
        if days < 0:
            raise ValueError("days must be >= 0")
        for _ in range(days):
            self.step()
        return self.history

    # -- snapshot ---------------------------------------------------

    def _snapshot(self) -> DayState:
        counts: dict[InfectionState, int] = {state: 0 for state in STATE_ORDER}
        agent_snapshots: list[AgentSnapshot] = []
        for agent in self.agents:
            counts[agent.state] += 1
            agent_snapshots.append(
                AgentSnapshot(
                    id=agent.id,
                    suite_id=agent.suite_id,
                    state=agent.state,
                    state_changed_day=agent.state_changed_day,
                )
```

```
            )
        return DayState(day=self.day, counts=counts, agents=tuple(agent_snapshots))
```

simulation/smoke.py

```
  1 | """Smoke test entry point.
  2 |
  3 | Run the default 150-agent structure, seeded with one infection, for
  4 | DEFAULT_RUN_DAYS days, across DEFAULT_SMOKE_SEEDS random seeds. For each
  5 | seed, print the daily per-state counts, then a one-line summary: peak
  6 | infected and the day it occurred, the seeded suite's final attack rate,
  7 | and how many distinct suites and floors were ever touched.
  8 |
  9 |     python -m simulation.smoke
 10 | """
 11 |
 12 | from __future__ import annotations
 13 |
 14 | import sys
 15 | from pathlib import Path
 16 |
 17 | sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
 18 |
 19 | from simulation.config import (  # noqa: E402
 20 |     DEFAULT_RUN_DAYS,
 21 |     DEFAULT_SEED_INFECTIONS,
 22 |     DEFAULT_SMOKE_SEEDS,
 23 | )
 24 | from simulation.contact import suite_ancestor_ids  # noqa: E402
 25 | from simulation.infection import STATE_ORDER, InfectionState  # noqa: E402
 26 | from simulation.simulator import DayState, Simulation  # noqa: E402
 27 |
 28 |
 29 | def _peak(history: list[DayState]) -> tuple[int, int]:
 30 |     """(peak simultaneously-infectious count, day it was first reached)."""
 31 |     peak_count = -1
 32 |     peak_day = 0
 33 |     for snapshot in history:
 34 |         infectious_now = snapshot.count(InfectionState.INCUBATING) + snapshot.count(
 35 |             InfectionState.SYMPTOMATIC
 36 |         )
 37 |         if infectious_now > peak_count:
 38 |             peak_count = infectious_now
 39 |             peak_day = snapshot.day
 40 |     return peak_count, peak_day
 41 |
 42 |
 43 | def _summary(sim: Simulation, history: list[DayState]) -> str:
 44 |     # Identify the seeded suite from day 0, BEFORE any spread -- by the
 45 |     # end of the run other suites may also have non-susceptible agents.
 46 |     seeded_suite = next(
 47 |         a.suite_id for a in history[0].agents if a.state is not InfectionState.SUSCEPTI
BLE
 48 |     )
 49 |     peak_count, peak_day = _peak(history)
 50 |
 51 |     final = history[-1]
```

```
    suite_population = sum(1 for s in final.agents if s.suite_id == seeded_suite)
    suite_ever_infected = sum(
        1
        for s in final.agents
        if s.suite_id == seeded_suite and s.state is not InfectionState.SUSCEPTIBLE
    )
    attack_rate = suite_ever_infected / suite_population * 100

    touched_suites: set[str] = set()
    touched_floors: set[str] = set()
    for snapshot in history:
        for agent_snapshot in snapshot.agents:
            if agent_snapshot.state is not InfectionState.SUSCEPTIBLE:
                touched_suites.add(agent_snapshot.suite_id)
    for suite_id in touched_suites:
        floor_id, _building_id = suite_ancestor_ids(sim.registry, suite_id)
        touched_floors.add(floor_id)

    return (
        f"  peak infected      : {peak_count} on day {peak_day}\n"
        f"  attack rate (seeded suite, n={suite_population}): {attack_rate:.1f}%\n"
        f"  suites ever touched: {len(touched_suites)}\n"
        f"  floors ever touched: {len(touched_floors)}"
    )


def _print_run(seed: int, days: int) -> None:
    sim = Simulation(seed=seed, seed_infections=DEFAULT_SEED_INFECTIONS)
    history = sim.run(days)

    columns = [state.name for state in STATE_ORDER]
    header = "day  " + "  ".join(f"{name:>11}" for name in columns)
    print(
        f"SEED {seed}   {len(sim.agents)} agents, "
        f"{DEFAULT_SEED_INFECTIONS} seeded, {days} days"
    )
    print(header)
    print("-" * len(header))
    for snapshot in history:
        cells = "  ".join(f"{snapshot.count(state):>11}" for state in STATE_ORDER)
        print(f"{snapshot.day:>3}  {cells}")
    print()
    print(_summary(sim, history))
    print()


def main() -> None:
    for seed in DEFAULT_SMOKE_SEEDS:
        _print_run(seed, DEFAULT_RUN_DAYS)


if __name__ == "__main__":
    main()
```

api/__init__.py

```
"""FastAPI layer for the MicroCluster outbreak simulator.

Detection and disclosure run SERVER SIDE only (CLAUDE.md, SECURITY
POSTURE). The API returns exactly what the floor-plan animation renders --
per-agent suite, position and infection state, plus that day's
detection/disclosure verdict -- and never a raw ``Report`` or any ground
truth beyond what is drawn on screen.

The ASGI app is ``api.app:app``; import it from there. ``api.bands`` is
kept dependency-light (no FastAPI import) so other packages can reuse the
count-banding rules.
"""
```

api/app.py

```
"""HTTP API: start a run, then read one day at a time.

Endpoints
---------
POST /api/run
    Body (all fields optional except nothing -- every field has a default):
        seed, days, seed_infections,
        suite_transmission_probability, floor_transmission_probability,
        building_transmission_probability,
        reporting_probability_min, reporting_probability_max,
        background_noise_daily_rate
    Returns: { run_id, seed, days, disclaimer, params, layout }

GET /api/run/{run_id}/day/{n}
    Returns the world at day n: every agent's suite, index within that
    suite, and infection state; the day's counts; and the
    detection/disclosure verdict. Nothing else.

The simulation's ground-truth firewall still holds: detection and
disclosure see only anonymous Reports and the ScopeRegistry (built inside
``simulation.pipeline``), never agent state. This module then exposes the
agent states for *rendering* -- the same states the animation paints --
and the engine verdicts, and stops there.
"""

from __future__ import annotations

import base64
import json
from dataclasses import dataclass
from pathlib import Path

from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field

from simulation.config import (
    DEFAULT_RUN_DAYS,
    DEFAULT_SIMULATION_CONFIG,
    SimulationConfig,
)

from microcluster.config import DEFAULT_DETECTION_CONFIG, DetectionConfig

from .bands import band, sanitize_reason
from simulation.contact import suite_ancestor_ids
from simulation.pipeline import (
    DailyRecord,
    SimulatedReports,
    analyze_report_stream,
    simulate_and_report,
```

```
 53 | )
 54 | from microcluster.config import DEFAULT_DISCLOSURE_CONFIG
 55 | from microcluster.disclosure import statistical_threshold
 56 | from simulation.population import StructureSpec
 57 |
 58 | WEB_DIR = Path(__file__).resolve().parent.parent / "web"
 59 | RESULTS_DIR = Path(__file__).resolve().parent.parent / "results"
 60 | RESULT_FILES = ("disclosure_sweep", "benchmark", "adversarial")
 61 |
 62 | MAX_DAYS = 120
 63 | MAX_RUNS_KEPT = 64  # in-memory only; oldest evicted past this
 64 |
 65 | MIN_SUITE_SIZE = 21   # StructureSpec rejects agents_per_suite <= MIN_SCOPE_POPULATION
(20)
 66 | MAX_SUITE_SIZE = 40
 67 | MIN_WINDOW_HOURS = 24
 68 | MAX_WINDOW_HOURS = 168
 69 |
 70 | DISCLAIMER = (
 71 |     "Illustrative simulation. Transmission, reporting, contact and incubation values "
 72 |     "are chosen for demonstration, not measured from any real population. Tested at 150
to "
 73 |     "2,000 simulated people and against a harsher simulated reporting model, never agai
nst "
 74 |     "real reports. This predicts nothing about any real building, and no one is diagnos
ed "
 75 |     "or treated by this tool."
 76 | )
 77 |
 78 |
 79 | # ---------------------------------------------------------------------------
 80 | # In-memory run store
 81 | # ---------------------------------------------------------------------------
 82 |
 83 |
 84 | @dataclass
 85 | class StoredRun:
 86 |     seed: int
 87 |     days: int
 88 |     params: dict
 89 |     simulated: SimulatedReports
 90 |     daily_records: list[DailyRecord]
 91 |     # agent id -> (suite_id, floor_id, index within suite), computed once.
 92 |     agent_slot: dict[str, tuple[str, str, int]]
 93 |     layout: dict
 94 |
 95 |
 96 | RUNS: "dict[str, StoredRun]" = {}
 97 | _RUN_ORDER: list[str] = []
 98 |
 99 |
100 | def _remember(run_id: str, run: StoredRun) -> None:
101 |     if run_id in RUNS:
102 |         RUNS[run_id] = run
103 |         return
```

```
    RUNS[run_id] = run
    _RUN_ORDER.append(run_id)
    while len(_RUN_ORDER) > MAX_RUNS_KEPT:
        RUNS.pop(_RUN_ORDER.pop(0), None)


# run id carries its own params so any process can rebuild the run
# deterministically after a restart or free-tier spin-down. RUNS stays a
# pure cache
def _encode_run_id(payload: dict) -> str:
    raw = json.dumps(payload, separators=(",", ":"), sort_keys=True).encode()
    return base64.urlsafe_b64encode(raw).decode().rstrip("=")


def _decode_run_id(run_id: str) -> "dict | None":
    try:
        pad = "=" * (-len(run_id) % 4)
        data = json.loads(base64.urlsafe_b64decode(run_id + pad))
    except Exception:
        return None
    return data if isinstance(data, dict) else None


# results json read once at import, not per request
_RESULTS_CACHE: "dict[str, dict]" = {}
for _name in RESULT_FILES:
    _p = RESULTS_DIR / f"{_name}.json"
    if _p.is_file():
        try:
            _RESULTS_CACHE[_name] = json.loads(_p.read_text())
        except Exception:
            pass


# ---------------------------------------------------------------------------
# Request / response models
# ---------------------------------------------------------------------------


_OVERRIDABLE = (
    "suite_transmission_probability",
    "floor_transmission_probability",
    "building_transmission_probability",
    "reporting_probability_min",
    "reporting_probability_max",
    "background_noise_daily_rate",
)


class RunRequest(BaseModel):
    seed: int = 1
    days: int = Field(default=DEFAULT_RUN_DAYS, ge=1, le=MAX_DAYS)
    seed_infections: int = Field(default=1, ge=0)

    population: int | None = Field(default=None, ge=MIN_SUITE_SIZE, le=MAX_SUITE_SIZE)
```

```
159 |     detection_window_hours: int | None = Field(
160 |         default=None, ge=MIN_WINDOW_HOURS, le=MAX_WINDOW_HOURS
161 |     )
162 |
163 |     suite_transmission_probability: float | None = Field(default=None, ge=0.0, le=1.0)
164 |     floor_transmission_probability: float | None = Field(default=None, ge=0.0, le=1.0)
165 |     building_transmission_probability: float | None = Field(default=None, ge=0.0, le=1.
0)
166 |     reporting_probability_min: float | None = Field(default=None, ge=0.0, le=1.0)
167 |     reporting_probability_max: float | None = Field(default=None, ge=0.0, le=1.0)
168 |     background_noise_daily_rate: float | None = Field(default=None, ge=0.0, le=1.0)
169 |
170 |     def to_config(self) -> SimulationConfig:
171 |         base = DEFAULT_SIMULATION_CONFIG
172 |         values = {name: getattr(base, name) for name in _OVERRIDABLE}
173 |         for name in _OVERRIDABLE:
174 |             supplied = getattr(self, name)
175 |             if supplied is not None:
176 |                 values[name] = supplied
177 |         values["seed_infections"] = self.seed_infections
178 |         try:
179 |             return SimulationConfig(**values)
180 |         except ValueError as exc:
181 |             raise HTTPException(status_code=422, detail=str(exc)) from exc
182 |
183 |     def to_spec(self) -> StructureSpec | None:
184 |         if self.population is None:
185 |             return None
186 |         return StructureSpec(agents_per_suite=self.population)
187 |
188 |     def to_detection_config(self) -> DetectionConfig:
189 |         if self.detection_window_hours is None:
190 |             return DEFAULT_DETECTION_CONFIG
191 |         return DetectionConfig(detection_window_hours=self.detection_window_hours)
192 |
193 |
194 | # ---------------------------------------------------------------------------
195 | # App
196 | # ---------------------------------------------------------------------------
197 |
198 | app = FastAPI(title="MicroCluster simulator", version="0.1")
199 | app.add_middleware(
200 |     CORSMiddleware,
201 |     allow_origins=["*"],
202 |     allow_methods=["*"],
203 |     allow_headers=["*"],
204 | )
205 |
206 |
207 | @app.middleware("http")
208 | async def _no_stale_assets(request, call_next):
209 |     # single-process demo, no CDN: make the browser revalidate every asset
210 |     # so an edited web/ file is never served from a stale cache
211 |     response = await call_next(request)
212 |     if not request.url.path.startswith("/api/"):
```

```
        response.headers["Cache-Control"] = "no-cache, must-revalidate"
    return response


def _build_layout_and_slots(
    simulated: SimulatedReports,
) -> tuple[dict, dict[str, tuple[str, str, int]]]:
    registry = simulated.simulation.registry

    # Group agents by suite in their stable creation order.
    suite_members: dict[str, list[str]] = {}
    for agent in simulated.simulation.agents:
        suite_members.setdefault(agent.suite_id, []).append(agent.id)

    agent_slot: dict[str, tuple[str, str, int]] = {}
    suite_floor: dict[str, str] = {}
    for suite_id, members in suite_members.items():
        floor_id, _building_id = suite_ancestor_ids(registry, suite_id)
        suite_floor[suite_id] = floor_id
        for index, agent_id in enumerate(members):
            agent_slot[agent_id] = (suite_id, floor_id, index)

    floors: dict[str, list[str]] = {}
    for suite_id, floor_id in suite_floor.items():
        floors.setdefault(floor_id, []).append(suite_id)

    layout = {
        "floors": [
            {
                "id": floor_id,
                "label": registry.get(floor_id).label if floor_id in registry.scopes el
se floor_id,
                "suites": [
                    {
                        "id": suite_id,
                        "label": registry.get(suite_id).label,
                        "size": len(suite_members[suite_id]),
                    }
                    for suite_id in sorted(suite_ids)
                ],
            }
            for floor_id, suite_ids in sorted(floors.items())
        ]
    }
    return layout, agent_slot


def _materialize(req: RunRequest) -> StoredRun:
    config = req.to_config()
    detection_config = req.to_detection_config()
    try:
        simulated = simulate_and_report(
            seed=req.seed, days=req.days, spec=req.to_spec(), config=config
        )
    except ValueError as exc:
```

```
267 |         raise HTTPException(status_code=422, detail=str(exc)) from exc
268 |     daily_records = analyze_report_stream(simulated, detection_config=detection_config)
269 |
270 |     layout, agent_slot = _build_layout_and_slots(simulated)
271 |     params = {
272 |         "population_per_suite": simulated.simulation.spec.agents_per_suite,
273 |         "total_population": len(simulated.simulation.agents),
274 |         "detection_window_hours": detection_config.detection_window_hours,
275 |         "suite_transmission_probability": config.suite_transmission_probability,
276 |         "floor_transmission_probability": config.floor_transmission_probability,
277 |         "building_transmission_probability": config.building_transmission_probability,
278 |         "reporting_probability_min": config.reporting_probability_min,
279 |         "reporting_probability_max": config.reporting_probability_max,
280 |         "background_noise_daily_rate": config.background_noise_daily_rate,
281 |         "seed_infections": config.seed_infections,
282 |     }
283 |     return StoredRun(
284 |         seed=req.seed,
285 |         days=req.days,
286 |         params=params,
287 |         simulated=simulated,
288 |         daily_records=daily_records,
289 |         agent_slot=agent_slot,
290 |         layout=layout,
291 |     )
292 |
293 |
294 | @app.post("/api/run")
295 | def start_run(req: RunRequest) -> dict:
296 |     run_id = _encode_run_id(req.model_dump(exclude_none=True))
297 |     run = RUNS.get(run_id) or _materialize(req)
298 |     _remember(run_id, run)
299 |     return {
300 |         "run_id": run_id,
301 |         "seed": req.seed,
302 |         "days": req.days,
303 |         "disclaimer": DISCLAIMER,
304 |         "params": run.params,
305 |         "layout": run.layout,
306 |     }
307 |
308 |
309 | @app.get("/api/layout")
310 | def get_layout(population: int | None = None) -> dict:
311 |     # pre-run empty state: the building structure before anything happens.
312 |     # 1-day sim just to build the population n registry, no run stored
313 |     fields: dict = {"days": 1}
314 |     if population is not None:
315 |         fields["population"] = population
316 |     try:
317 |         req = RunRequest(**fields)
318 |     except Exception as exc:
319 |         raise HTTPException(status_code=422, detail=str(exc)) from exc
320 |     run = _materialize(req)
321 |     return {"layout": run.layout, "params": run.params, "disclaimer": DISCLAIMER}
```

```
322 |
323 |
324 | @app.get("/api/run/{run_id}/day/{n}")
325 | def get_day(run_id: str, n: int) -> dict:
326 |     run = RUNS.get(run_id)
327 |     if run is None:
328 |         # cache miss: rebuild from the params baked into the id
329 |         payload = _decode_run_id(run_id)
330 |         if payload is None:
331 |             raise HTTPException(status_code=404, detail="unknown run id (start a new ru
n)")
332 |         try:
333 |             run = _materialize(RunRequest(**payload))
334 |         except HTTPException:
335 |             raise
336 |         except Exception:
337 |             raise HTTPException(status_code=404, detail="unknown run id (start a new ru
n)")
338 |         _remember(run_id, run)
339 |     if n < 0 or n > run.days:
340 |         raise HTTPException(status_code=404, detail=f"day {n} out of range 0..{run.day
s}")
341 |
342 |     snapshot = run.simulated.simulation.history[n]
343 |     record = run.daily_records[n]
344 |     registry = run.simulated.simulation.registry
345 |
346 |     agents = []
347 |     for agent_snapshot in snapshot.agents:
348 |         suite_id, floor_id, index = run.agent_slot[agent_snapshot.id]
349 |         agents.append(
350 |             {
351 |                 "suite": suite_id,
352 |                 "floor": floor_id,
353 |                 "i": index,
354 |                 "state": agent_snapshot.state.value,
355 |             }
356 |         )
357 |
358 |     disclosure = None
359 |     if record.disclosed_scope_id is not None:
360 |         scope = registry.get(record.disclosed_scope_id)
361 |         disclosure = {
362 |             "scope_id": scope.id,
363 |             "label": scope.label,
364 |             "level": scope.level.name,
365 |         }
366 |
367 |     # The gate table (rule 12), ordered coarsest -> finest so the
368 |     # resolution slider can walk it directly. Exact qualifying counts and
369 |     # the exact report fraction are dropped here and never sent; only the
370 |     # band leaves the server (see api/bands.py).
371 |     evaluations = [
372 |         {
373 |             "scope_id": ev.scope_id,
```

```
            "label": ev.scope_label,
            "level": ev.scope_level.name,
            "population": ev.population,
            "qualifying_band": band(ev.qualifying_reports),
            "statistical_threshold": ev.statistical_threshold,
            "statistical_pass": ev.statistical_pass,
            "min_population_pass": ev.min_population_pass,
            "report_fraction_pass": ev.report_fraction_pass,
            "privacy_pass": ev.privacy_pass,
            "eligible": ev.disclosure_eligible,
            "selected": ev.selected,
            "reason": sanitize_reason(ev.disclosure_reason),
        }
        for ev in sorted(
            record.disclosure_evaluations,
            key=lambda e: (int(e.scope_level), -e.population, e.scope_id),
        )
    ]

    return {
        "day": n,
        "counts": {state.value: snapshot.counts[state] for state in snapshot.counts},
        "agents": agents,
        "detection": {
            "fired": record.detection_fired,
            "relative": record.relative_fired,
            "absolute": record.absolute_fired,
        },
        "disclosure": disclosure,
        "evaluations": evaluations,
        "first_fired_day": record.first_fired_day,
        "first_disclosed_day": record.first_disclosed_day,
        "disclaimer": DISCLAIMER,
    }


@app.get("/api/results/{name}")
def get_results(name: str) -> dict:
    if name not in RESULT_FILES:
        raise HTTPException(status_code=404, detail=f"unknown results file: {name!r}")
    if name in _RESULTS_CACHE:
        return _RESULTS_CACHE[name]
    path = RESULTS_DIR / f"{name}.json"
    if not path.is_file():
        raise HTTPException(
            status_code=503,
            detail="results not precomputed; run python -m scripts.precompute_results",
        )
    _RESULTS_CACHE[name] = json.loads(path.read_text())
    return _RESULTS_CACHE[name]


@app.get("/api/gates")
def get_gates() -> dict:
    """Worked examples for the gate explanations, computed by the engine's
```

```
    own threshold function and the configured privacy constants -- never
    typed into the page. Populations are the default structure's scope
    sizes (suite / floor / building)."""
    spec = StructureSpec()
    suite = spec.agents_per_suite
    floor = suite * spec.suites_per_floor
    building = floor * spec.floors_per_building
    cfg = DEFAULT_DISCLOSURE_CONFIG
    scopes = (("Suite", suite), ("Floor", floor), ("Building", building))

    def first_refused(n: int) -> int:
        # smallest report count the privacy gate refuses: the same
        # comparison disclosure.py makes, qualifying / n <= max fraction
        q = 0
        while q / n <= cfg.max_report_fraction:
            q += 1
        return q

    return {
        "statistical": {
            "floor": cfg.statistical_gate_floor,
            "rows": [
                {"scope": name, "population": n, "required": statistical_threshold(n, c
fg)}
                for name, n in scopes
            ],
        },
        "privacy": {
            "min_population": cfg.min_scope_population,
            "max_fraction": cfg.max_report_fraction,
            "rows": [
                {
                    "scope": name,
                    "population": n,
                    "meets_minimum": n >= cfg.min_scope_population,
                    "refused_from": first_refused(n),
                }
                for name, n in scopes
            ],
        },
    }


@app.get("/api/health")
def health() -> dict:
    return {"ok": True, "runs_in_memory": len(RUNS)}


# The plain web UI. Mounted last so /api/* wins.
if WEB_DIR.is_dir():
    app.mount("/", StaticFiles(directory=str(WEB_DIR), html=True), name="web")
```

api/bands.py

```
"""Report-count banding for the API's presentation boundary.

PRIVACY REQUIREMENT, not a styling choice. A ``ScopeEvaluation`` carries
the exact qualifying-report count for a scope. If that exact integer is
visible in the UI on two consecutive days that both name the same scope,
an observer subtracts the two and learns that exactly one identifiable
person became ill (or recovered) overnight -- a differencing attack that
would be live in the shipped product.

So the exact count never leaves the server. Every count the API emits is
first mapped to a BAND. The band edges below are a privacy policy, named
here the same way every threshold in ``microcluster/config.py`` is:
wider bands are safer and coarser; these are a chosen starting point, not
a proven-safe value.

The same rule reaches the human-readable ``disclosure_reason`` strings,
which sometimes embed a raw count ("7 qualifying reports < required 45"):
:func:`sanitize_reason` rewrites those to the band before the string is
sent.
"""

from __future__ import annotations

import re

# Lower bound of each band. A count of N lands in the band [lo, next_lo - 1]
# for the largest lo <= N; at or above the last edge it is "<edge>+".
# Chosen so the smallest bands ("1-4", "5-9", "10-19") are narrow enough
# to stay informative and wide enough that a one-person night-to-night
# change cannot be read off two consecutive disclosures.
BAND_LOWER_BOUNDS: tuple[int, ...] = (1, 5, 10, 20, 50, 100)


def band(count: int) -> str:
    """Map an exact count to its band label ("0", "1-4", ..., "100+")."""
    if count <= 0:
        return "0"
    edges = BAND_LOWER_BOUNDS
    if count >= edges[-1]:
        return f"{edges[-1]}+"
    for lo, hi in zip(edges, edges[1:]):
        if lo <= count < hi:
            return f"{lo}-{hi - 1}" if hi - 1 > lo else f"{lo}"
    # count in [1, edges[0]) is impossible while edges[0] == 1, but stay safe.
    return f"1-{edges[0] - 1}"


def band_interval(label: str) -> tuple[int, int | None]:
    """Inverse of :func:`band`: the inclusive ``[lo, hi]`` a label stands
    for. ``hi`` is ``None`` for the open-topped ``"<n>+"`` band."""
    if label == "0":
        return (0, 0)
```

```
    if label.endswith("+"):
        return (int(label[:-1]), None)
    lo, hi = label.split("-")
    return (int(lo), int(hi))


_QUALIFYING_PAREN = re.compile(r"\((\d+) qualifying reports")
_QUALIFYING_COUNT = re.compile(r"qualifying count \((\d+)\)")
_SCOPES_COUNT = re.compile(r"scope's \((\d+)\)")
_FRACTION = re.compile(r"report fraction \d+(?:\.\d+)? exceeds maximum (\d+(?:\.\d+)?)")


def sanitize_reason(reason: str) -> str:
    """Rewrite every raw report count inside a ``disclosure_reason`` to its
    band, and blur the exact report fraction. Population, statistical
    thresholds and the switch margin are public policy numbers and are
    left untouched.

    Targets the exact phrasings ``microcluster.disclosure`` produces; a
    reason with no embedded count passes through unchanged.
    """
    out = _QUALIFYING_PAREN.sub(
        lambda m: f"({band(int(m.group(1)))} qualifying reports", reason
    )
    out = _QUALIFYING_COUNT.sub(
        lambda m: f"qualifying count ({band(int(m.group(1)))})", out
    )
    out = _SCOPES_COUNT.sub(lambda m: f"scope's ({band(int(m.group(1)))})", out)
    out = _FRACTION.sub(
        r"report fraction over the limit of \1", out
    )
    return out
```

adversarial/__init__.py

```
"""Adversarial self-test for the privacy claim.

Models an observer who sees only what the shipped UI shows -- day by day,
the disclosed scope and its reported report-count band -- and runs two
attacks against that stream:

  * the DIFFERENCING attack (adversarial.differencing): infer the
    overnight change in a named group from consecutive disclosures. Run
    against exact counts (the pre-band behaviour) and against bands; the
    contrast is the result.
  * SCOPE WANDERING (adversarial.wandering): how many distinct groups get
    named across a run, scope stability on versus off.
  * REPORT INJECTION (adversarial.injection): how many fabricated reports,
    and how many sessions, it takes to manufacture a false disclosure --
    with and without the intake submission cap.

The point is to make the privacy claim demonstrable. Where a defence only
reduces leakage rather than removing it, that is reported as such.
"""

from .differencing import DifferencingOutcome, attack
from .injection import InjectionResult, measure_injection
from .observer import DayView, observe
from .wandering import WanderingOutcome, measure_wandering

__all__ = [
    "DayView",
    "observe",
    "DifferencingOutcome",
    "attack",
    "WanderingOutcome",
    "measure_wandering",
    "InjectionResult",
    "measure_injection",
]
```

adversarial/__main__.py

```
from .selftest import main

if __name__ == "__main__":
    main()
```

adversarial/differencing.py

```
"""The differencing attack.

The observer takes two consecutive days that disclose the SAME scope and
asks how many individuals' reports changed overnight. The disclosed
per-scope qualifying count is a running total over the 72h detection
window, so the overnight change is::

    delta = count(day d+1) - count(day d)

With EXACT counts the observer learns ``delta`` as a single integer; when
``abs(delta) == 1`` that is one identifiable person's report appearing or
ageing out of the window overnight -- a pinned one-person change.

With BANDS the observer knows only ``[lo, hi]`` for each day, so ``delta``
is an interval ``[lo(d+1) - hi(d), hi(d+1) - lo(d)]`` -- normally many
integers wide, and never a pinned single person.
"""

from __future__ import annotations

from dataclasses import dataclass
from statistics import median

from api.bands import band_interval

from .observer import DayView

_NEG_INF = float("-inf")
_POS_INF = float("inf")


@dataclass(frozen=True)
class _Pair:
    scope_label: str
    day_from: int
    delta_low: float
    delta_high: float

    @property
    def known_exactly(self) -> bool:
        return self.delta_low == self.delta_high

    @property
    def unbounded(self) -> bool:
        return self.delta_low == _NEG_INF or self.delta_high == _POS_INF

    @property
    def possible_values(self) -> float:
        return _POS_INF if self.unbounded else self.delta_high - self.delta_low + 1

    @property
    def one_person_pinned(self) -> bool:
```

```
        return self.known_exactly and abs(self.delta_low) == 1

    @property
    def direction_known(self) -> bool:
        return not self.unbounded and not (self.delta_low <= 0 <= self.delta_high)


@dataclass(frozen=True)
class DifferencingOutcome:
    mode: str
    same_scope_pairs: int
    pinned_exactly: int
    one_person_pinned: int
    direction_known: int
    unbounded_pairs: int
    median_possible_values: float
    residual: str


def _delta_bounds(a: DayView, b: DayView) -> tuple[float, float]:
    if a.exact_count is not None and b.exact_count is not None:
        d = float(b.exact_count - a.exact_count)
        return d, d
    lo_a, hi_a = band_interval(a.band)
    lo_b, hi_b = band_interval(b.band)
    low = _NEG_INF if hi_a is None else float(lo_b - hi_a)
    high = _POS_INF if hi_b is None else float(hi_b - lo_a)
    return low, high


def _pairs(views: list[DayView]) -> list[_Pair]:
    out: list[_Pair] = []
    for a, b in zip(views, views[1:]):
        if a.scope_id is None or a.scope_id != b.scope_id:
            continue
        low, high = _delta_bounds(a, b)
        out.append(_Pair(a.scope_label, a.day, low, high))
    return out


def _residual(mode: str, pairs: list[_Pair]) -> str:
    if not pairs:
        return "no scope was disclosed on two consecutive days"
    if mode == "exact":
        return "overnight change in the named group is known to the exact person"
    return (
        "the named group is still named; its count stays bracketed to a band; "
        "the overnight change stays a wide interval"
    )


def attack(views: list[DayView], *, mode: str) -> DifferencingOutcome:
    """Run the differencing attack over one observed run.

    ``mode`` is ``"exact"`` or ``"band"`` and only labels the result --
```

```
    which one actually applies is decided by whichever field
    :func:`adversarial.observer.observe` populated.
    """
    pairs = _pairs(views)
    finite = [p.possible_values for p in pairs if not p.unbounded]
    return DifferencingOutcome(
        mode=mode,
        same_scope_pairs=len(pairs),
        pinned_exactly=sum(1 for p in pairs if p.known_exactly),
        one_person_pinned=sum(1 for p in pairs if p.one_person_pinned),
        direction_known=sum(1 for p in pairs if p.direction_known),
        unbounded_pairs=sum(1 for p in pairs if p.unbounded),
        median_possible_values=median(finite) if finite else _POS_INF,
        residual=_residual(mode, pairs),
    )
```

adversarial/injection.py

```
"""What the intake submission cap actually buys against report injection.

An attacker wants the system to disclose a cluster that is not real, at a
scope of their choosing. They fabricate reports: a fixed category, tagged
to suites under the target scope, all inside one detection window, each
carrying a ``session_id`` the attacker picks freely.

This module measures the cost of that attack with and without
``microcluster.intake``'s per-session cap in force. The result is
deliberately unflattering: the cap does not reduce the number of
fabricated reports required, it only forces the attacker to spread them
across ``ceil(reports / session_cap)`` sessions instead of one. A
``session_id`` is an unauthenticated, client-chosen string, so that is a
near-zero cost for a determined attacker. The cap stops a naive
single-session flood and nothing more.
"""

from __future__ import annotations

import math
import random
from dataclasses import dataclass
from datetime import datetime, timezone

from microcluster.config import DEFAULT_INTAKE_CONFIG, IntakeConfig
from microcluster.engine import analyze
from microcluster.models import CHECKLIST, Category, Onset, Report, ScopeLevel, ScopeRe
gistry
from simulation.population import StructureSpec, build_population

_WINDOW_INSTANT = datetime(2026, 1, 10, 12, 0, tzinfo=timezone.utc)
_FAKE_CATEGORY = Category.GASTROINTESTINAL
_FAKE_SYMPTOM = CHECKLIST[_FAKE_CATEGORY][0]

# Cap per suite when the target is a FLOOR, so no single suite clears its
# own (finer) statistical gate and the disclosure that appears is the
# floor -- not a suite the attacker did not aim at. ceil(sqrt(25)) = 5, so
# 4 per suite is the most that keeps every suite gate failing.
_PER_SUITE_MAX_FOR_FLOOR = 4


@dataclass(frozen=True)
class InjectionResult:
    target_scope_id: str
    target_level: str
    target_population: int

    reports_to_disclose: int          # fabricated reports needed, cap or no cap
    sessions_without_cap: int         # = 1
    sessions_with_cap: int            # = ceil(reports / session_submission_cap)
    session_cap: int

```

```
    single_session_attack_succeeds: bool   # attacker uses ONE session, cap on
    multi_session_attack_succeeds: bool     # attacker uses ceil(k/cap) sessions, cap o
n


def _suites_under(registry: ScopeRegistry, scope_id: str) -> list[str]:
    target = registry.get(scope_id)
    if target.level is ScopeLevel.SUITE:
        return [scope_id]
    return sorted(
        s.id for s in registry.scopes.values()
        if s.level is ScopeLevel.SUITE and registry.contains(scope_id, s.id)
    )


def _fabricate(
    suites: list[str], n_reports: int, per_suite_max: int, session_ids: list[str | Non
e]
) -> list[Report]:
    reports: list[Report] = []
    per_suite = {s: 0 for s in suites}
    si = 0
    while len(reports) < n_reports:
        suite = suites[si % len(suites)]
        si += 1
        if per_suite[suite] >= per_suite_max:
            if all(per_suite[s] >= per_suite_max for s in suites):
                break
            continue
        per_suite[suite] += 1
        reports.append(
            Report(
                category=_FAKE_CATEGORY,
                symptom=_FAKE_SYMPTOM,
                onset=Onset.TODAY,
                location_id=suite,
                submitted_at=_WINDOW_INSTANT,
                session_id=session_ids[len(reports) % len(session_ids)],
            )
        )
    return reports


def _discloses_target(
    reports: list[Report], registry: ScopeRegistry, target_scope_id: str,
    intake_config: IntakeConfig | None,
) -> bool:
    result = analyze(reports, registry, now=_WINDOW_INSTANT, intake_config=intake_confi
g)
    return result.disclosure.disclosed_scope_id == target_scope_id


def measure_injection(
    target_scope_id: str = "C-B1-F2",
    *,
```

```
    intake_config: IntakeConfig = DEFAULT_INTAKE_CONFIG,
    max_reports: int = 60,
) -> InjectionResult:
    """Measure the injection attack against ``target_scope_id`` (a FLOOR by
    default -- the representative mid scope, declared population 75)."""
    _, registry = build_population(spec=StructureSpec(), rng=random.Random(0))
    target = registry.get(target_scope_id)
    suites = _suites_under(registry, target_scope_id)
    per_suite_max = (
        max_reports if target.level is ScopeLevel.SUITE else _PER_SUITE_MAX_FOR_FLOOR
    )
    cap = intake_config.session_submission_cap

    # 1. reports needed when the attacker is NOT rate-limited (one session,
    #    or none): smallest k whose fabricated reports name the target.
    reports_needed = 0
    for k in range(1, max_reports + 1):
        fab = _fabricate(suites, k, per_suite_max, session_ids=[None])
        if len(fab) < k:
            break  # ran out of room under per_suite_max
        if _discloses_target(fab, registry, target_scope_id, intake_config=None):
            reports_needed = k
            break
    if reports_needed == 0:
        raise RuntimeError(f"could not manufacture a disclosure at {target_scope_id}")

    sessions_with_cap = math.ceil(reports_needed / cap)

    # 2. cap ON, attacker uses ONE session -> only `cap` reports survive.
    one_session = _fabricate(
        suites, reports_needed, per_suite_max, session_ids=["attacker-1"]
    )
    single_ok = _discloses_target(
        one_session, registry, target_scope_id, intake_config=intake_config
    )

    # 3. cap ON, attacker uses ceil(k/cap) sessions -> all reports survive.
    sids: list[str | None] = [
        f"attacker-{i // cap}" for i in range(reports_needed)
    ]
    many_sessions = _fabricate(suites, reports_needed, per_suite_max, session_ids=sids)
    multi_ok = _discloses_target(
        many_sessions, registry, target_scope_id, intake_config=intake_config
    )

    return InjectionResult(
        target_scope_id=target_scope_id,
        target_level=target.level.name,
        target_population=target.population,
        reports_to_disclose=reports_needed,
        sessions_without_cap=1,
        sessions_with_cap=sessions_with_cap,
        session_cap=cap,
        single_session_attack_succeeds=single_ok,
        multi_session_attack_succeeds=multi_ok,
```

```
    )


def format_result(r: InjectionResult) -> str:
    return "\n".join([
        "=" * 78,
        f"REPORT-INJECTION ATTACK   target {r.target_scope_id} "
        f"({r.target_level}, declared population {r.target_population})",
        "=" * 78,
        "",
        f"  fabricated reports to manufacture the disclosure : {r.reports_to_disclos
e}",
        f"    (the statistical gate for this scope; the cap does not change it)",
        "",
        f"  sources needed, NO per-session cap               : "
        f"{r.sessions_without_cap}",
        f"  sources needed, per-session cap = {r.session_cap}              : "
        f"{r.sessions_with_cap}   (= ceil({r.reports_to_disclose} / {r.session_cap}))",
        "",
        f"  cap on, attacker uses 1 session  -> disclosure?  : "
        f"{'YES' if r.single_session_attack_succeeds else 'NO (attack blocked)'}",
        f"  cap on, attacker uses {r.sessions_with_cap} sessions -> disclosure?  : "
        f"{'YES (attack succeeds)' if r.multi_session_attack_succeeds else 'NO'}",
        "",
        "HONEST READ",
        "  The cap stops a single-session flood. It does not stop the attack: a",
        f"  session_id is a free, unauthenticated string, and ceil(k / {r.session_cap})
of them",
        "  is a near-zero cost. Injection is possible for any attacker willing to",
        "  rotate sessions. Closing this needs identity or session binding, which the",
        "  system deliberately does not have.",
    ])


def main() -> None:
    print(format_result(measure_injection()))


if __name__ == "__main__":
    main()
```

adversarial/observer.py

```
"""What a day-by-day observer of the shipped UI can see, and nothing else."""

from __future__ import annotations

from dataclasses import dataclass

from api.bands import band
from simulation.pipeline import DailyRecord


@dataclass(frozen=True)
class DayView:
    """One day as the observer sees it.

    Exactly one of ``exact_count`` / ``band`` is populated: ``exact_count``
    for the pre-band scenario, ``band`` for the shipped one. A day with
    nothing disclosed leaves every field but ``day`` as ``None``.
    """

    day: int
    scope_id: str | None
    scope_label: str | None
    scope_level: str | None
    exact_count: int | None
    band: str | None


def _selected_evaluation(record: DailyRecord):
    for ev in record.disclosure_evaluations:
        if ev.scope_id == record.disclosed_scope_id:
            return ev
    return None


def observe(records: list[DailyRecord], *, leak_exact_counts: bool) -> list[DayView]:
    """Reduce a run's daily records to the observer's view.

    With ``leak_exact_counts=True`` the observer is handed the exact
    qualifying-report count of the disclosed scope (what the UI showed
    before banding). With ``False`` they get only the band.
    """
    views: list[DayView] = []
    for record in records:
        ev = _selected_evaluation(record)
        if ev is None:
            views.append(DayView(record.day, None, None, None, None, None))
            continue
        views.append(
            DayView(
                day=record.day,
                scope_id=ev.scope_id,
                scope_label=ev.scope_label,
```

```
                scope_level=ev.scope_level.name,
                exact_count=ev.qualifying_reports if leak_exact_counts else None,
                band=None if leak_exact_counts else band(ev.qualifying_reports),
            )
        )
    return views
```

adversarial/selftest.py

```
"""Run the adversarial self-test across many seeds and summarise."""

from __future__ import annotations

from dataclasses import dataclass
from statistics import mean, median
from typing import Sequence

from simulation.config import DEFAULT_RUN_DAYS
from simulation.pipeline import analyze_report_stream, simulate_and_report

from .differencing import attack
from .injection import format_result, measure_injection
from .observer import observe
from .wandering import measure_wandering

DEFAULT_SEEDS = 200


@dataclass(frozen=True)
class SelfTestSummary:
    n_runs: int
    days: int

    same_scope_pairs: int
    exact_one_person_pinned: int
    band_one_person_pinned: int
    exact_direction_known: int
    band_direction_known: int
    band_median_possible_values: float
    band_residual: str

    wandering_on_mean: float
    wandering_off_mean: float
    wandering_on_max: int
    wandering_off_max: int


def run_self_test(
    seeds: Sequence[int] | None = None,
    *,
    n_seeds: int = DEFAULT_SEEDS,
    days: int = DEFAULT_RUN_DAYS,
) -> SelfTestSummary:
    seed_list = list(seeds) if seeds is not None else list(range(1, n_seeds + 1))

    pairs = 0
    exact_pins = band_pins = 0
    exact_dir = band_dir = 0
    band_spans: list[float] = []
    band_residual = "no consecutive same-scope disclosures in any run"
    wander_on: list[int] = []
```

```
    wander_off: list[int] = []

    for seed in seed_list:
        simulated = simulate_and_report(seed=seed, days=days)
        records = analyze_report_stream(simulated)

        exact = attack(observe(records, leak_exact_counts=True), mode="exact")
        band = attack(observe(records, leak_exact_counts=False), mode="band")

        pairs += exact.same_scope_pairs
        exact_pins += exact.one_person_pinned
        band_pins += band.one_person_pinned
        exact_dir += exact.direction_known
        band_dir += band.direction_known
        if band.same_scope_pairs and band.median_possible_values != float("inf"):
            band_spans.append(band.median_possible_values)
            band_residual = band.residual

        w = measure_wandering(simulated)
        wander_on.append(w.stability_on)
        wander_off.append(w.stability_off)

    return SelfTestSummary(
        n_runs=len(seed_list),
        days=days,
        same_scope_pairs=pairs,
        exact_one_person_pinned=exact_pins,
        band_one_person_pinned=band_pins,
        exact_direction_known=exact_dir,
        band_direction_known=band_dir,
        band_median_possible_values=median(band_spans) if band_spans else float("inf"),
        band_residual=band_residual,
        wandering_on_mean=mean(wander_on) if wander_on else 0.0,
        wandering_off_mean=mean(wander_off) if wander_off else 0.0,
        wandering_on_max=max(wander_on) if wander_on else 0,
        wandering_off_max=max(wander_off) if wander_off else 0,
    )


def format_summary(s: SelfTestSummary) -> str:
    lines = [
        "=" * 78,
        f"ADVERSARIAL SELF-TEST   ({s.n_runs} runs, {s.days}-day)",
        "observer sees only: the disclosed scope, and its report count as the UI shows
it",
        "=" * 78,
        "",
        "DIFFERENCING ATTACK",
        f"  consecutive same-scope disclosures           : {s.same_scope_pairs} day-pa
irs",
        "",
        f"  >>> pre-band (EXACT counts) one-person pins   : {s.exact_one_person_pinne
d}",
        f"  >>> shipped  (BANDS)        one-person pins   : {s.band_one_person_pinne
d}",
```

```
104 |         "",
105 |         f"  exact  : overnight change forced to a known sign on {s.exact_direction_know
n}/{s.same_scope_pairs} pairs",
106 |         f"  bands  : overnight change forced to a known sign on {s.band_direction_know
n}/{s.same_scope_pairs} pairs",
107 |         f"  bands  : residual per pair -- {s.band_residual}",
108 |         f"           median count of still-possible overnight deltas : "
109 |         f"{_fmt(s.band_median_possible_values)}",
110 |         "",
111 |         "SCOPE WANDERING   (distinct groups named across a run)",
112 |         f"  scope stability ON   : mean {s.wandering_on_mean:.2f}   max {s.wandering_on
_max}",
113 |         f"  scope stability OFF  : mean {s.wandering_off_mean:.2f}   max {s.wandering_o
ff_max}",
114 |         "",
115 |         "HONEST READ",
116 |         "  Bands remove the one-person differencing pin entirely. They do NOT make th
e",
117 |         "  observer learn nothing: the named group is still named, its activity is stil
l",
118 |         "  bracketed to a band, and on many pairs the direction of change is still forc
ed.",
119 |         "  Scope stability shrinks the set of groups ever named but does not hide the",
120 |         "  lineage the disclosure walks. This is leakage reduced, not leakage remove
d.",
121 |     ]
122 |     return "\n".join(lines)
123 |
124 |
125 | def _fmt(value: float) -> str:
126 |     return "unbounded" if value == float("inf") else f"{value:.0f}"
127 |
128 |
129 | def main() -> None:
130 |     print(format_summary(run_self_test()))
131 |     print()
132 |     print(format_result(measure_injection()))
133 |
134 |
135 | if __name__ == "__main__":
136 |     main()
```

adversarial/wandering.py

```
"""What scope wandering leaks: distinct groups named across a run.

Every distinct scope the system ever discloses is a group the observer
learns about. Scope stability (config.SIBLING_SWITCH_MARGIN) exists to
keep that set small. This measures it against the same run analysed with
stability threading turned off.
"""

from __future__ import annotations

from dataclasses import dataclass

from simulation.pipeline import DailyRecord, SimulatedReports, analyze_report_stream


@dataclass(frozen=True)
class WanderingOutcome:
    stability_on: int
    stability_off: int
    names_on: tuple[str, ...]
    names_off: tuple[str, ...]


def _distinct_named(records: list[DailyRecord]) -> list[str]:
    seen: list[str] = []
    for record in records:
        sid = record.disclosed_scope_id
        if sid is not None and sid not in seen:
            seen.append(sid)
    return seen


def measure_wandering(simulated: SimulatedReports) -> WanderingOutcome:
    on = _distinct_named(analyze_report_stream(simulated))
    off = _distinct_named(
        analyze_report_stream(simulated, thread_scope_stability=False)
    )
    return WanderingOutcome(len(on), len(off), tuple(on), tuple(off))
```

web/app.js

```
   1 | "use strict";
   2 |
   3 | // MicroCluster outbreak animation. Plain DOM, no framework, no build step.
   4 | // Detection and disclosure happen server-side; this file only draws what
   5 | // /api/run and /api/run/{id}/day/{n} return.
   6 |
   7 | // render free tier edge returns intermittent 404/502 (x-render-routing:
   8 | // no-server) before the request reaches the app. retry those a few times
   9 | async function apiFetch(url, opts, tries = 4) {
  10 |   let lastErr;
  11 |   for (let attempt = 0; attempt < tries; attempt++) {
  12 |     if (attempt > 0) await new Promise((r) => setTimeout(r, 300 * 2 ** (attempt -
1)));
  13 |     try {
  14 |       // a stalled connection should surface as a plain "timed out" error,
  15 |       // not hang the UI (and the "Running..." label) forever
  16 |       const res = await fetch(url, { ...opts, signal: AbortSignal.timeout(20000) });
  17 |       if (res.ok) return res;
  18 |       if (![404, 502, 503, 504].includes(res.status) || attempt === tries - 1) return
res;
  19 |       lastErr = new Error(res.status + " " + (await res.clone().text()).slice(0, 12
0));
  20 |     } catch (err) {
  21 |       lastErr = err;
  22 |     }
  23 |   }
  24 |   throw lastErr;
  25 | }
  26 |
  27 | const els = {
  28 |   seed: document.getElementById("seed"),
  29 |   days: document.getElementById("days"),
  30 |   run: document.getElementById("run"),
  31 |   runGuided: document.getElementById("run-guided"),
  32 |   refusal: document.getElementById("refusal"),
  33 |   planPop: document.getElementById("plan-pop"),
  34 |   summaryList: document.getElementById("summary-list"),
  35 |   play: document.getElementById("play"),
  36 |   pause: document.getElementById("pause"),
  37 |   step: document.getElementById("step"),
  38 |   reset: document.getElementById("reset"),
  39 |   speed: document.getElementById("speed"),
  40 |   scrub: document.getElementById("scrub"),
  41 |   dayLabel: document.getElementById("day-label"),
  42 |   dayMax: document.getElementById("day-max"),
  43 |   status: document.getElementById("status"),
  44 |   counts: document.getElementById("counts"),
  45 |   plan: document.getElementById("plan"),
  46 |   emptyNote: document.getElementById("empty-note"),
  47 |   disclaimer: document.getElementById("disclaimer"),
  48 |   resSlider: document.getElementById("res-slider"),
  49 |   resReadout: document.getElementById("res-readout"),
```

```
  50 |   resEndCoarse: document.getElementById("res-end-coarse"),
  51 |   resEndFine: document.getElementById("res-end-fine"),
  52 |   runSummary: document.getElementById("run-summary"),
  53 |   sumDetected: document.getElementById("sum-detected"),
  54 |   sumDetectedDay: document.getElementById("sum-detected-day"),
  55 |   sumDisclosure: document.getElementById("sum-disclosure"),
  56 |   sumRefused: document.getElementById("sum-refused"),
  57 |   sumRefusedDay: document.getElementById("sum-refused-day"),
  58 |   gateDay: document.getElementById("gate-day"),
  59 |   gateBody: document.getElementById("gate-body"),
  60 |   pPopulation: document.getElementById("p-population"),
  61 |   pTransmission: document.getElementById("p-transmission"),
  62 |   pReporting: document.getElementById("p-reporting"),
  63 |   pWindow: document.getElementById("p-window"),
  64 |   pPopulationHint: document.getElementById("p-population-hint"),
  65 |   pTransmissionHint: document.getElementById("p-transmission-hint"),
  66 |   pReportingHint: document.getElementById("p-reporting-hint"),
  67 |   paramsDirty: document.getElementById("params-dirty"),
  68 | };
  69 |
  70 | // captured once at load, before either button's label can be swapped to
  71 | // "Running..." while a fetch is in flight
  72 | const RUN_LABEL = els.run.textContent;
  73 | const GUIDED_LABEL = els.runGuided.textContent;
  74 |
  75 | const state = {
  76 |   runId: null,
  77 |   totalDays: 0,
  78 |   frames: [],          // frames[n] = day payload
  79 |   currentDay: 0,
  80 |   timer: null,
  81 |   agentCells: new Map(), // key `${suite}#${i}` -> <div class="agent">
  82 |   suiteEls: new Map(),   // suite id -> <div class="suite">
  83 |   evals: [],           // current day's candidate scopes, coarsest -> finest
  84 |   wallIdx: -1,         // first index that FAILED the privacy gate, or -1
  85 |   maxResIdx: 0,        // furthest the slider may travel (the wall, inclusive)
  86 |   resIdx: 0,           // slider position
  87 |   lastDisclosedScopeId: undefined, // undefined = no paint yet; null = disclosed nothi
ng
  88 |   summary: null,        // running end-of-run summary, built while frames load
  89 | };
  90 |
  91 | // SUITE is the finest scope level; used to compare disclosures across a run
  92 | // to find the most specific one ever made (never hardcoded against a tier list)
  93 | const LEVEL_RANK = { CAMPUS: 1, BUILDING: 2, FLOOR: 3, SUITE: 4 };
  94 |
  95 | // scope ids nest by "-" prefix: "C-B1-F2" covers "C-B1-F2-S1".
  96 | function scopeCoversSuite(scopeId, suiteId) {
  97 |   return scopeId === suiteId || String(suiteId).startsWith(scopeId + "-");
  98 | }
  99 |
 100 | function setPlaybackEnabled(on) {
 101 |   for (const b of [els.play, els.pause, els.step, els.reset]) b.disabled = !on;
 102 |   els.scrub.disabled = !on;
 103 | }
```

```
 104 |
 105 | function stopTimer() {
 106 |   if (state.timer !== null) {
 107 |     clearInterval(state.timer);
 108 |     state.timer = null;
 109 |   }
 110 | }
 111 |
 112 | // ---- rendering -----------------------------------------------------------
 113 |
 114 | function renderSkeleton() {
 115 |   els.plan.classList.add("loading");
 116 |   els.plan.innerHTML = "";
 117 |   els.plan.setAttribute("aria-label", "Loading the floor plan.");
 118 |   for (let f = 0; f < 2; f++) {
 119 |     const floor = document.createElement("div");
 120 |     floor.className = "floor";
 121 |     const label = document.createElement("div");
 122 |     label.className = "floor-label";
 123 |     label.textContent = "loading";
 124 |     const suites = document.createElement("div");
 125 |     suites.className = "suites";
 126 |     for (let s = 0; s < 3; s++) {
 127 |       const sk = document.createElement("div");
 128 |       sk.className = "skeleton-suite";
 129 |       suites.appendChild(sk);
 130 |     }
 131 |     floor.append(label, suites);
 132 |     els.plan.appendChild(floor);
 133 |   }
 134 | }
 135 |
 136 | function buildPlan(layout) {
 137 |   els.plan.classList.remove("loading");
 138 |   els.plan.innerHTML = "";
 139 |   els.plan.setAttribute("aria-label", "Floor plan. Everyone susceptible; the simulatio
n has not run yet.");
 140 |   state.agentCells.clear();
 141 |   state.suiteEls.clear();
 142 |
 143 |   for (const floor of layout.floors) {
 144 |     const floorEl = document.createElement("div");
 145 |     floorEl.className = "floor";
 146 |
 147 |     const label = document.createElement("div");
 148 |     label.className = "floor-label";
 149 |     label.textContent = "Floor " + floor.label;
 150 |
 151 |     const suitesEl = document.createElement("div");
 152 |     suitesEl.className = "suites";
 153 |
 154 |     for (const suite of floor.suites) {
 155 |       const suiteEl = document.createElement("div");
 156 |       suiteEl.className = "suite";
 157 |       suiteEl.dataset.suite = suite.id;
```

```
 158 |
 159 |       const sLabel = document.createElement("div");
 160 |       sLabel.className = "suite-label";
 161 |       sLabel.textContent = suite.label + "  n=" + suite.size;
 162 |
 163 |       const agentsEl = document.createElement("div");
 164 |       agentsEl.className = "agents";
 165 |       for (let i = 0; i < suite.size; i++) {
 166 |         const cell = document.createElement("div");
 167 |         cell.className = "agent s-susceptible";
 168 |         agentsEl.appendChild(cell);
 169 |         state.agentCells.set(suite.id + "#" + i, cell);
 170 |       }
 171 |
 172 |       suiteEl.append(sLabel, agentsEl);
 173 |       suitesEl.appendChild(suiteEl);
 174 |       state.suiteEls.set(suite.id, suiteEl);
 175 |     }
 176 |
 177 |     floorEl.append(label, suitesEl);
 178 |     els.plan.appendChild(floorEl);
 179 |   }
 180 | }
 181 |
 182 | // a one-shot ring, nothing else, on the instant a circle turns symptomatic.
 183 | // restart-safe (remove, force reflow, re-add) so a rapid re-trigger still plays.
 184 | function triggerPulse(cell) {
 185 |   cell.classList.remove("pulse");
 186 |   void cell.offsetWidth;
 187 |   cell.classList.add("pulse");
 188 |   setTimeout(() => cell.classList.remove("pulse"), 400);
 189 | }
 190 |
 191 | function triggerStatusChange() {
 192 |   els.status.classList.remove("change-in");
 193 |   void els.status.offsetWidth;
 194 |   els.status.classList.add("change-in");
 195 |   setTimeout(() => els.status.classList.remove("change-in"), 250);
 196 | }
 197 |
 198 | // `stepped` is true only for a genuine single forward step during real
 199 | // playback (advance(), driven by Step or the timer) -- never on initial
 200 | // load, a scrub jump, or ?auto=dayN. That is what keeps every screenshot
 201 | // (which always jumps straight to a day) free of the pulse / row-flash
 202 | // motion below: there is no previous rendered day to diff against.
 203 | function paintDay(n, opts = {}) {
 204 |   const frame = state.frames[n];
 205 |   if (!frame) return;
 206 |   const stepped = Boolean(opts.stepped) && n === state.currentDay + 1 && Boolean(stat
e.frames[n - 1]);
 207 |   const prevFrame = stepped ? state.frames[n - 1] : null;
 208 |   state.currentDay = n;
 209 |
 210 |   const prevAgentState = new Map();
 211 |   if (prevFrame) {
```

```
 212 |     for (const a of prevFrame.agents) prevAgentState.set(a.suite + "#" + a.i, a.stat
e);
 213 |   }
 214 |   // diffed, not reset-then-set: every agent is present in every frame, so
 215 |   // only cells whose state actually changed get a new class -- a colour
 216 |   // transition on every cell every day would read as the grid flashing
 217 |   for (const a of frame.agents) {
 218 |     const key = a.suite + "#" + a.i;
 219 |     const cell = state.agentCells.get(key);
 220 |     if (!cell) continue;
 221 |     const cls = "agent s-" + a.state;
 222 |     if (cell.className !== cls) cell.className = cls;
 223 |     if (prevFrame && a.state === "symptomatic" && prevAgentState.get(key) !== "symptom
atic") {
 224 |       triggerPulse(cell);
 225 |     }
 226 |   }
 227 |
 228 |   renderResolution(frame);
 229 |   renderGateTable(frame, prevFrame);
 230 |
 231 |   const d = frame.disclosure;
 232 |   const resScope = state.evals[state.resIdx];
 233 |   for (const [suiteId, suiteEl] of state.suiteEls) {
 234 |     suiteEl.classList.toggle(
 235 |       "disclosed", Boolean(d && scopeCoversSuite(d.scope_id, suiteId))
 236 |     );
 237 |     suiteEl.classList.toggle(
 238 |       "resolution", Boolean(resScope && scopeCoversSuite(resScope.scope_id, suiteId))
 239 |     );
 240 |   }
 241 |
 242 |   const c = frame.counts;
 243 |   els.counts.textContent =
 244 |     `susceptible ${c.susceptible} · incubating ${c.incubating} · ` +
 245 |     `symptomatic ${c.symptomatic} · recovered ${c.recovered}`;
 246 |   // text alternative for the role=img plan -- the numbers, not the pixels
 247 |   els.plan.setAttribute(
 248 |     "aria-label",
 249 |     `Floor plan, day ${n}. ${c.susceptible} susceptible, ${c.incubating} incubating, `
+
 250 |     `${c.symptomatic} symptomatic, ${c.recovered} recovered` +
 251 |     (d ? `. Disclosed scope: ${d.level} ${d.label}.` : ".")
 252 |   );
 253 |
 254 |   els.dayLabel.textContent = String(n);
 255 |   els.scrub.value = String(n);
 256 |
 257 |   // undefined only before the very first paint -- a fresh load (including
 258 |   // every ?auto=dayN screenshot) never sees this as a "change"
 259 |   const newScopeId = d ? d.scope_id : null;
 260 |   const scopeChanged =
 261 |     state.lastDisclosedScopeId !== undefined && state.lastDisclosedScopeId !== newScop
eId;
 262 |   state.lastDisclosedScopeId = newScopeId;
```

```
 263 |
 264 |   if (d) {
 265 |     els.status.textContent =
 266 |       `System says: ${d.level} ${d.label} disclosed` +
 267 |       (frame.first_disclosed_day !== null ? ` (first on day ${frame.first_disclosed_da
y})` : "");
 268 |     els.status.classList.add("flagged");
 269 |   } else if (frame.detection.fired) {
 270 |     els.status.textContent =
 271 |       "System says: activity detected, nothing disclosed (no scope passes both gate
s)";
 272 |     els.status.classList.remove("flagged");
 273 |   } else {
 274 |     els.status.textContent = "System says: nothing";
 275 |     els.status.classList.remove("flagged");
 276 |   }
 277 |   if (scopeChanged) triggerStatusChange();
 278 |
 279 |   renderRefusal(frame);
 280 | }
 281 |
 282 | // the money moment: a scope with enough evidence that the privacy gate
 283 | // refuses. wording comes from the engine's own reason text, not invented here
 284 | function renderRefusal(frame) {
 285 |   const evals = frame.evaluations || [];
 286 |   const refused = evals.find((e) => e.statistical_pass && !e.privacy_pass);
 287 |   const d = frame.disclosure;
 288 |
 289 |   // :empty hides the box; writing into it is the change role=status announces
 290 |   function show(title, text) {
 291 |     const head = document.createElement("strong");
 292 |     head.textContent = title;
 293 |     const body = document.createElement("span");
 294 |     body.textContent = text;
 295 |     els.refusal.replaceChildren(head, body);
 296 |   }
 297 |
 298 |   if (refused) {
 299 |     const named = d ? `${d.level} ${d.label}` : "no scope at all";
 300 |     // strip the engine's "Rejected: " prefix n trailing punctuation, frame it here
 301 |     const reason = refused.reason.replace(/^Rejected:\s*/, "").replace(/[.\s]+$/, "");
 302 |     show(
 303 |       "Privacy refusal",
 304 |       `The system has enough evidence to name ${refused.level} ${refused.label}, ` +
 305 |       `but will not: ${reason}. It disclosed ${named} instead.`
 306 |     );
 307 |     return;
 308 |   }
 309 |
 310 |   if (frame.detection.fired && !d) {
 311 |     show(
 312 |       "Nothing disclosed",
 313 |       "Activity was detected, but no scope has both enough evidence and a safe " +
 314 |       "population to name, so the system says nothing."
 315 |     );
```

```
    return;
  }

  els.refusal.replaceChildren();
}

// ---- resolution slider ------------------------------------------------

// The slider walks the candidate scopes from coarsest to finest. It may
// travel up to and including the first scope that FAILED the privacy gate
// (state.wallIdx); dragging further snaps back to it. The engine already
// decided pass/fail -- this only reads frame.evaluations.
function renderResolution(frame) {
  const evals = frame.evaluations || [];
  state.evals = evals;

  if (evals.length === 0) {
    state.wallIdx = -1;
    state.maxResIdx = 0;
    state.resIdx = 0;
    els.resSlider.disabled = true;
    els.resSlider.min = "0";
    els.resSlider.max = "0";
    els.resSlider.value = "0";
    els.resReadout.className = "res-readout";
    els.resReadout.textContent = frame.detection.fired
      ? "Detection fired but no scope had a qualifying report this day."
      : "No candidate scopes. Detection has not fired on this day.";
    return;
  }

  // label the slider ends with the actual scope levels this day offers,
  // coarsest -> finest (the server already orders evaluations that way),
  // instead of the generic "coarsest"/"finest" placeholder shown pre-run
  els.resEndCoarse.textContent = evals[0].level.toLowerCase();
  els.resEndFine.textContent = evals[evals.length - 1].level.toLowerCase();

  state.wallIdx = evals.findIndex((e) => !e.privacy_pass);
  state.maxResIdx = state.wallIdx === -1 ? evals.length - 1 : state.wallIdx;
  const selIdx = evals.findIndex((e) => e.selected);
  state.resIdx = selIdx !== -1 ? selIdx : state.maxResIdx;

  els.resSlider.disabled = false;
  els.resSlider.min = "0";
  els.resSlider.max = String(evals.length - 1);
  els.resSlider.value = String(state.resIdx);
  renderResReadout();
}

function renderResReadout() {
  const e = state.evals[state.resIdx];
  if (!e) return;
  const atWall = state.wallIdx !== -1 && state.resIdx === state.wallIdx;
  els.resReadout.classList.toggle("wall", atWall);

```

```
  if (atWall) {
    els.resReadout.textContent =
      `Resolution stops at ${e.level} ${e.label}. ${e.reason}`;
  } else if (e.selected) {
    els.resReadout.textContent =
      `Resolution: ${e.level} ${e.label}. The scope the system disclosed.`;
  } else if (e.eligible) {
    els.resReadout.textContent =
      `Resolution: ${e.level} ${e.label}. Passes both gates; a finer scope was disclos
ed.`;
  } else {
    els.resReadout.textContent =
      `Resolution: ${e.level} ${e.label}. Passes the privacy gate. ${e.reason}`;
  }
}

function applyResolutionHighlight() {
  const resScope = state.evals[state.resIdx];
  for (const [suiteId, suiteEl] of state.suiteEls) {
    suiteEl.classList.toggle(
      "resolution", Boolean(resScope && scopeCoversSuite(resScope.scope_id, suiteId))
    );
  }
}

els.resSlider.addEventListener("input", () => {
  let v = Number(els.resSlider.value);
  if (v > state.maxResIdx) {
    v = state.maxResIdx;                // hard stop at the privacy wall
    els.resSlider.value = String(v);
  }
  state.resIdx = v;
  renderResReadout();
  applyResolutionHighlight();
});

// ---- gate table ------------------------------------------------------

function td(text) {
  const el = document.createElement("td");
  el.textContent = text;
  return el;
}

function gateCell(pass, sub) {
  const el = document.createElement("td");
  el.className = "gate-cell";
  const verdict = document.createElement("span");
  verdict.className = pass ? "pass" : "fail";
  verdict.textContent = pass ? "PASS" : "FAIL";
  el.appendChild(verdict);
  if (sub) {
    const note = document.createElement("small");
    note.textContent = sub;
    el.appendChild(note);
```

```
  425 |   }
  426 |   return el;
  427 | }
  428 |
  429 | function _verdictSignature(e) {
  430 |   return `${e.statistical_pass}|${e.privacy_pass}|${e.selected}`;
  431 | }
  432 |
  433 | // prevFrame is only passed for a genuine single forward step (see paintDay);
  434 | // rows are always rebuilt fresh either way, so a jump/scrub/screenshot load
  435 | // never sees the flash (there is nothing to diff against, prevFrame is null)
  436 | function renderGateTable(frame, prevFrame) {
  437 |   els.gateDay.textContent = String(frame.day);
  438 |   const body = els.gateBody;
  439 |   body.innerHTML = "";
  440 |   const evals = frame.evaluations || [];
  441 |
  442 |   if (evals.length === 0) {
  443 |     const tr = document.createElement("tr");
  444 |     const cell = document.createElement("td");
  445 |     cell.colSpan = 6;
  446 |     cell.className = "muted";
  447 |     cell.textContent =
  448 |       "No candidate scopes evaluated" +
  449 |       (frame.detection.fired ? "." : " (detection did not fire).");
  450 |     tr.appendChild(cell);
  451 |     body.appendChild(tr);
  452 |     return;
  453 |   }
  454 |
  455 |   const prevVerdict = new Map();
  456 |   if (prevFrame) {
  457 |     for (const e of prevFrame.evaluations || []) prevVerdict.set(e.scope_id, _verdictS
ignature(e));
  458 |   }
  459 |
  460 |   const flashRows = [];
  461 |   for (const e of evals) {
  462 |     const tr = document.createElement("tr");
  463 |     if (e.selected) tr.className = "selected";
  464 |     tr.appendChild(td(`${e.level} ${e.label}`));
  465 |     tr.appendChild(td(String(e.population)));
  466 |     tr.appendChild(td(e.qualifying_band));
  467 |     tr.appendChild(gateCell(e.statistical_pass, `needs at least ${e.statistical_thresh
old}`));
  468 |     tr.appendChild(gateCell(e.privacy_pass, ""));
  469 |     tr.appendChild(td(e.reason));
  470 |     body.appendChild(tr);
  471 |
  472 |     if (prevFrame) {
  473 |       const before = prevVerdict.get(e.scope_id);
  474 |       if (before !== undefined && before !== _verdictSignature(e)) {
  475 |         tr.classList.add("verdict-changed");
  476 |         flashRows.push(tr);
  477 |       }
```

```
    }
  }
  // double rAF: let the browser paint the flash once, THEN remove the class,
  // so the removal (not the arrival) is what transitions -- a fade out, not a snap
  if (flashRows.length) {
    requestAnimationFrame(() => {
      requestAnimationFrame(() => {
        for (const tr of flashRows) tr.classList.remove("verdict-changed");
      });
    });
  }
}

// ---- end-of-run summary ------------------------------------------------

// Built while every day's frame is fetched in startRun's prefetch loop, not
// recomputed from a single day's frame -- "most specific disclosure" and
// "was a finer scope ever refused" are properties of the whole run.
function noteSummaryFrame(summary, n, frame) {
  if (frame.detection.fired && summary.firedDay === null) {
    summary.fired = true;
    summary.firedDay = n;
  }
  const d = frame.disclosure;
  if (d) {
    const rank = LEVEL_RANK[d.level] || 0;
    if (!summary.finest || rank > LEVEL_RANK[summary.finest.level]) {
      summary.finest = d;
      summary.finestDay = n;
    }
  }
  if (!summary.refused) {
    const r = (frame.evaluations || []).find((e) => e.statistical_pass && !e.privacy_p
ass);
    if (r) {
      summary.refused = true;
      summary.refusedDay = n;
    }
  }
}

function renderRunSummary() {
  const s = state.summary;
  if (!s) return;
  els.sumDetected.textContent = s.fired ? "Yes" : "No";
  els.sumDetectedDay.textContent = s.fired ? String(s.firedDay) : "n/a";
  els.sumDisclosure.textContent = s.finest
    ? `${s.finest.level} ${s.finest.label} (day ${s.finestDay})`
    : "Nothing disclosed";
  els.sumRefused.textContent = s.refused ? "Yes" : "No";
  els.sumRefusedDay.textContent = s.refused ? String(s.refusedDay) : "n/a";
  els.runSummary.hidden = false;
}

// ---- run + playback ----------------------------------------------------
```

```
 532 |
 533 | // Both run buttons are disabled and the clicked one relabelled for the
 534 | // couple of seconds the fetch takes, so the page never just sits there
 535 | // looking inert. Always restored together, win or lose.
 536 | function setRunningFeedback(triggerButton) {
 537 |   els.run.disabled = true;
 538 |   els.runGuided.disabled = true;
 539 |   if (triggerButton) triggerButton.textContent = "Running…";
 540 | }
 541 | function clearRunningFeedback() {
 542 |   els.run.disabled = false;
 543 |   els.runGuided.disabled = false;
 544 |   els.run.textContent = RUN_LABEL;
 545 |   els.runGuided.textContent = GUIDED_LABEL;
 546 | }
 547 |
 548 | // opts.triggerButton: the button clicked, so its label can say "Running...".
 549 | // opts.autoplay (default true): start playback once frames land, same as
 550 | // the page-load autoplay -- the ?auto= smoke-test path below passes false
 551 | // because it drives its own jump-to-day / play().
 552 | async function startRun(opts = {}) {
 553 |   const autoplay = opts.autoplay !== false;
 554 |   stopTimer();
 555 |   setPlaybackEnabled(false);
 556 |   setRunningFeedback(opts.triggerButton);
 557 |   els.emptyNote && (els.emptyNote.style.display = "none");
 558 |   renderSkeleton();
 559 |   els.status.textContent = "System says: starting the simulation…";
 560 |   els.status.classList.remove("flagged");
 561 |   els.refusal.replaceChildren();
 562 |   els.resSlider.disabled = true;
 563 |   els.resReadout.className = "res-readout";
 564 |   els.resReadout.textContent = "Loading";
 565 |   els.gateBody.innerHTML = "";
 566 |   els.gateDay.textContent = "0";
 567 |   els.runSummary.hidden = true;
 568 |   state.summary = { fired: false, firedDay: null, finest: null, finestDay: null, refus
ed: false, refusedDay: null };
 569 |   // reset the scrubber immediately, not just once frames arrive -- otherwise
 570 |   // a disabled slider sits at the previous run's end position while this
 571 |   // fetch is still in flight
 572 |   els.scrub.value = "0";
 573 |   els.dayLabel.textContent = "0";
 574 |   els.dayMax.textContent = "0";
 575 |
 576 |   const reporting = Number(els.pReporting.value);
 577 |   const body = {
 578 |     seed: Number(els.seed.value) || 0,
 579 |     days: Math.max(1, Math.min(120, Number(els.days.value) || 30)),
 580 |     population: Number(els.pPopulation.value) || 25,
 581 |     suite_transmission_probability: Number(els.pTransmission.value),
 582 |     reporting_probability_min: reporting,
 583 |     reporting_probability_max: reporting,
 584 |     detection_window_hours: Number(els.pWindow.value) || 72,
 585 |   };
```

```
  const qs = new URLSearchParams(location.search);
  for (const key of [
    "suite_transmission_probability", "floor_transmission_probability",
    "building_transmission_probability", "reporting_probability_min",
    "reporting_probability_max", "background_noise_daily_rate", "seed_infections",
    "population", "detection_window_hours",
  ]) {
    if (qs.has(key)) body[key] = Number(qs.get(key));
  }
  if (qs.has("seed")) { body.seed = Number(qs.get("seed")); els.seed.value = body.see
d; }
  if (qs.has("days")) { body.days = Number(qs.get("days")); els.days.value = body.day
s; }
  if (qs.has("population")) els.pPopulation.value = body.population;
  if (qs.has("suite_transmission_probability")) els.pTransmission.value = body.suite_t
ransmission_probability;
  if (qs.has("reporting_probability_min")) els.pReporting.value = body.reporting_proba
bility_min;
  if (qs.has("detection_window_hours")) els.pWindow.value = body.detection_window_hour
s;
  refreshParamHints();
  els.paramsDirty.hidden = true;
  els.status.textContent = `System says: simulating ${body.days} days…`;

  let meta;
  try {
    const res = await apiFetch("/api/run", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(body),
    });
    if (!res.ok) throw new Error("run failed: " + res.status + " " + (await res.text
()));
    meta = await res.json();
  } catch (err) {
    els.plan.classList.remove("loading");
    els.plan.innerHTML = "";
    els.status.textContent = "System says: the simulation could not be started (" + er
r.message + "). Try again.";
    clearRunningFeedback();
    return;
  }

  state.runId = meta.run_id;
  state.totalDays = meta.days;
  state.frames = new Array(meta.days + 1).fill(null);
  if (meta.disclaimer) els.disclaimer.textContent = meta.disclaimer;
  els.dayMax.textContent = String(meta.days);
  els.scrub.max = String(meta.days);

  buildPlan(meta.layout);

  // Prefetch every day up front so playback is smooth.
  for (let n = 0; n <= meta.days; n++) {
    let res;
```

```
 634 |     try {
 635 |       res = await apiFetch(`/api/run/${state.runId}/day/${n}`);
 636 |     } catch (err) {
 637 |       els.status.textContent = "System says: the simulation stopped partway through (d
ay " + n + ": " + err.message + "). Try again.";
 638 |       clearRunningFeedback();
 639 |       return;
 640 |     }
 641 |     if (!res.ok) {
 642 |       els.status.textContent = "System says: the simulation stopped partway through (d
ay " + n + ": HTTP " + res.status + "). Try again.";
 643 |       clearRunningFeedback();
 644 |       return;
 645 |     }
 646 |     state.frames[n] = await res.json();
 647 |     noteSummaryFrame(state.summary, n, state.frames[n]);
 648 |     if (n === 0) paintDay(0);
 649 |   }
 650 |
 651 |   setPlaybackEnabled(true);
 652 |   clearRunningFeedback();
 653 |   paintDay(0);
 654 |   renderRunSummary();
 655 |   if (autoplay) play();
 656 | }
 657 |
 658 | function advance() {
 659 |   if (state.currentDay >= state.totalDays) {
 660 |     stopTimer();
 661 |     return;
 662 |   }
 663 |   paintDay(state.currentDay + 1, { stepped: true });
 664 | }
 665 |
 666 | function play() {
 667 |   if (state.timer !== null) return;
 668 |   if (state.currentDay >= state.totalDays) paintDay(0);
 669 |   const interval = Number(els.speed.value) || 500;
 670 |   state.timer = setInterval(advance, interval);
 671 | }
 672 |
 673 | // guided example: the exact scenario from docs/demo_video_script.md.
 674 | // Shared by the button and the autoplay-on-load below.
 675 | function runGuidedScenario() {
 676 |   els.seed.value = "4";
 677 |   els.days.value = "30";
 678 |   els.pPopulation.value = "25";
 679 |   els.pTransmission.value = "0.025";
 680 |   els.pReporting.value = "0.8";
 681 |   els.pWindow.value = "72";
 682 |   refreshParamHints();
 683 |   return startRun({ triggerButton: els.runGuided });
 684 | }
 685 |
 686 | // once the user has touched Run, the guided button, or any Playback
```

```
 687 | // control, the scheduled autoplay (below) must not start a run on top of
 688 | // whatever they are doing
 689 | let userActed = false;
 690 | function noteUserAction() { userActed = true; }
 691 | for (const el of [els.run, els.runGuided, els.play, els.pause, els.step, els.reset]) {
 692 |   el.addEventListener("click", noteUserAction);
 693 | }
 694 | els.scrub.addEventListener("input", noteUserAction);
 695 |
 696 | els.run.addEventListener("click", () => startRun({ triggerButton: els.run }));
 697 | els.runGuided.addEventListener("click", () => runGuidedScenario());
 698 | els.play.addEventListener("click", play);
 699 | els.pause.addEventListener("click", stopTimer);
 700 | els.step.addEventListener("click", () => { stopTimer(); advance(); });
 701 | els.reset.addEventListener("click", () => { stopTimer(); paintDay(0); });
 702 | els.scrub.addEventListener("input", () => { stopTimer(); paintDay(Number(els.scrub.val
ue)); });
 703 | els.speed.addEventListener("change", () => {
 704 |   if (state.timer !== null) { stopTimer(); play(); }
 705 | });
 706 |
 707 | // ---- model parameter controls --------------------------------------
 708 |
 709 | function refreshParamHints() {
 710 |   const perSuite = Number(els.pPopulation.value) || 25;
 711 |   els.pPopulationHint.textContent = `6 suites, ${perSuite * 6} people`;
 712 |   els.pTransmissionHint.textContent = Number(els.pTransmission.value).toFixed(3);
 713 |   els.pReportingHint.textContent = Number(els.pReporting.value).toFixed(2);
 714 | }
 715 |
 716 | for (const el of [
 717 |   els.pPopulation, els.pTransmission, els.pReporting, els.pWindow, els.seed, els.days,
 718 | ]) {
 719 |   el.addEventListener("input", () => {
 720 |     refreshParamHints();
 721 |     if (state.runId) els.paramsDirty.hidden = false;
 722 |   });
 723 | }
 724 | refreshParamHints();
 725 |
 726 | // ts keeps the pre-run building drawing in sync w suite size
 727 | let preloadTimer = null;
 728 | els.pPopulation.addEventListener("input", () => {
 729 |   if (state.runId) return;
 730 |   clearTimeout(preloadTimer);
 731 |   preloadTimer = setTimeout(preloadLayout, 250);
 732 | });
 733 |
 734 | // ---- pre-run empty state: draw the building before anything happens ----
 735 |
 736 | function layoutQuery() {
 737 |   const qs = new URLSearchParams(location.search);
 738 |   const pop = qs.get("population") || els.pPopulation.value;
 739 |   return pop ? `?population=${encodeURIComponent(pop)}` : "";
 740 | }
```

```
 741 |
 742 | async function preloadLayout() {
 743 |   if (state.runId) return false;
 744 |   try {
 745 |     const res = await apiFetch("/api/layout" + layoutQuery());
 746 |     if (!res.ok) return false;
 747 |     const data = await res.json();
 748 |     if (state.runId) return false;            // a real run started meanwhile
 749 |     buildPlan(data.layout);
 750 |     if (data.params && data.params.total_population) {
 751 |       els.planPop.textContent = String(data.params.total_population);
 752 |     }
 753 |     if (data.disclaimer) els.disclaimer.textContent = data.disclaimer;
 754 |     return true;
 755 |   } catch (_) {
 756 |     const note = document.getElementById("empty-note");
 757 |     if (note) note.textContent = "Press Run to draw the building and start an outbrea
k.";
 758 |     return false;
 759 |   }
 760 | }
 761 |
 762 | // ---- precomputed results panel ---------------------------------------
 763 |
 764 | const SVG_NS = "http://www.w3.org/2000/svg";
 765 |
 766 | function svg(name, attrs, text) {
 767 |   const el = document.createElementNS(SVG_NS, name);
 768 |   for (const [k, v] of Object.entries(attrs || {})) el.setAttribute(k, String(v));
 769 |   if (text != null) el.textContent = text;
 770 |   return el;
 771 | }
 772 |
 773 | function renderCostChart(data) {
 774 |   const host = document.getElementById("cost-chart");
 775 |   host.innerHTML = "";
 776 |   const rows = (data.sweeps[data.primary_sweep].rows || []).filter(
 777 |     (r) => !r.degenerate && r.disc_delay != null && r.inf_before_disclosure_unbiased !
= null
 778 |   );
 779 |   if (rows.length < 2) {
 780 |     host.textContent = "not enough sweep points to plot.";
 781 |     return;
 782 |   }
 783 |   rows.sort((a, b) => a.disc_delay - b.disc_delay);
 784 |
 785 |   const W = 760, H = 430, L = 66, Rm = 46, T = 22, Bm = 52;
 786 |   const xs = rows.map((r) => r.disc_delay);
 787 |   const ys = rows.map((r) => r.inf_before_disclosure_unbiased);
 788 |   const xMin = Math.floor(Math.min(...xs)) - 0.5;
 789 |   const xMax = Math.ceil(Math.max(...xs)) + 0.5;
 790 |   const yMax = Math.max(15, Math.ceil(Math.max(...ys) / 15) * 15);
 791 |   const px = (x) => L + ((x - xMin) / (xMax - xMin)) * (W - L - Rm);
 792 |   const py = (y) => H - Bm - (y / yMax) * (H - T - Bm);
 793 |
```

```
  const root = svg("svg", { viewBox: `0 0 ${W} ${H}`, preserveAspectRatio: "xMidYMid m
eet",
    role: "img", "aria-label": "cost of the privacy policy" });

  root.appendChild(svg("line", { class: "axis", x1: L, y1: H - Bm, x2: W - Rm, y2: H -
Bm }));
  root.appendChild(svg("line", { class: "axis", x1: L, y1: T, x2: L, y2: H - Bm }));

  for (let x = Math.ceil(xMin); x <= Math.floor(xMax); x += 2) {
    root.appendChild(svg("line", { class: "axis", x1: px(x), y1: H - Bm, x2: px(x), y
2: H - Bm + 5 }));
    root.appendChild(svg("text", { class: "tick", x: px(x), y: H - Bm + 18, "text-anch
or": "middle" }, String(x)));
  }
  for (let i = 0; i <= 3; i++) {
    const y = (yMax / 3) * i;
    root.appendChild(svg("line", { class: "axis", x1: L - 5, y1: py(y), x2: L, y2: py
(y) }));
    root.appendChild(svg("text", { class: "tick", x: L - 9, y: py(y) + 4, "text-ancho
r": "end" }, String(Math.round(y))));
  }

  const d = rows.map((r, i) => `${i ? "L" : "M"}${px(r.disc_delay).toFixed(1)},${py(r.
inf_before_disclosure_unbiased).toFixed(1)}`).join(" ");
  root.appendChild(svg("path", { class: "series", d }));

  rows.forEach((r, i) => {
    const cx = px(r.disc_delay), cy = py(r.inf_before_disclosure_unbiased);
    root.appendChild(svg("circle", { class: "point", cx, cy, r: 4 }));
    const first = i === 0;
    root.appendChild(svg("text", {
      class: "point-label",
      x: cx + (first ? 8 : -8),
      y: cy - 8,
      "text-anchor": first ? "start" : "end",
    }, String(r.value)));
  });

  root.appendChild(svg("text", { x: (L + W - Rm) / 2, y: H - 10, "text-anchor": "middl
e" },
    "mean days to first disclosure"));
  root.appendChild(svg("text", {
    x: 16, y: (T + H - Bm) / 2, "text-anchor": "middle",
    transform: `rotate(-90 16 ${(T + H - Bm) / 2})`,
  }, "mean infections before disclosure"));

  host.appendChild(root);
  const caption = document.createElement("p");
  caption.className = "results-note";
  caption.textContent =
    `point labels are the ${data.primary_sweep} value; the shipped default is ${rows
[0].value}. Independent reporting.`;
  host.appendChild(caption);
}

```

```
function _cell(text, cls) {
  const td = document.createElement("td");
  td.textContent = text;
  if (cls) td.className = cls;
  return td;
}

function _fmtScore(v) {
  return v == null ? "n/a" : v.toFixed(2);
}

// precision/recall are fractions of 1 -- also show the percentage so the
// number reads without translating it in your head
function _fmtFraction(v) {
  return v == null ? "n/a" : `${v.toFixed(2)} (${Math.round(v * 100)}%)`;
}

function _detectorRows(scores, cols) {
  // scores: {rules, model}; cols: extra leading cell text per row or null
  const rules = document.createElement("tr");
  const model = document.createElement("tr");
  model.className = "model";
  if (cols) {
    const lead = _cell(cols);
    lead.rowSpan = 2;
    rules.appendChild(lead);
  }
  rules.append(
    _cell("authored rules"),
    _cell(_fmtFraction(scores.rules.precision)),
    _cell(_fmtFraction(scores.rules.recall)),
    _cell(_fmtScore(scores.rules.delay))
  );
  model.append(
    _cell("learned model"),
    _cell(_fmtFraction(scores.model.precision)),
    _cell(_fmtFraction(scores.model.recall)),
    _cell(_fmtScore(scores.model.delay))
  );
  return [rules, model];
}

function renderBenchmark(data) {
  const body = document.getElementById("benchmark-body");
  body.innerHTML = "";
  body.append(..._detectorRows(data.in_distribution, null));
  document.getElementById("benchmark-conclusion").textContent = data.conclusion;
}

function renderRobustness(data) {
  const body = document.getElementById("robustness-body");
  body.innerHTML = "";
  for (const regime of data.cross_regime) {
    body.append(..._detectorRows(regime, regime.label));
  }
```

```
  document.getElementById("robustness-conclusion").textContent = data.cross_regime_conclusion;
}

function renderAdversarial(data) {
  document.getElementById("adv-exact").textContent = String(data.exact_one_person_pins);
  document.getElementById("adv-band").textContent = String(data.band_one_person_pins);
  document.getElementById("adv-residual").textContent =
    `Bands remove the one-person pin, not every inference. What the observer still gets: ` +
    `${data.residual}. The direction of the overnight change is still forced on ` +
    `${data.band_direction_known} of ${data.same_scope_pairs.toLocaleString()} day-pairs, ` +
    `and with scope stability off a run names about ${data.wandering_off_mean} distinct ` +
    `groups instead of ${data.wandering_on_mean}.`;
}

// headline numbers, straight from results/*.json, nothing hardcoded
function renderSummary(sweep, bench, adv) {
  const rows = (sweep.sweeps[sweep.primary_sweep].rows || []).filter(
    (r) => !r.degenerate && r.disc_delay != null && r.fire_delay != null
  );
  const shipped = rows[0];
  const items = [];

  if (shipped) {
    const gap = shipped.disc_delay - shipped.fire_delay;
    items.push(
      `Averaged over ${sweep.seeds.toLocaleString()} simulated outbreaks at the default ` +
      `transmission rate, with independent reporting, the detector fires around day ` +
      `${shipped.fire_delay.toFixed(0)} ` +
      `but the privacy rules hold disclosure until about day ${shipped.disc_delay.toFixed(0)}. ` +
      `That gap of roughly ${gap.toFixed(0)} days, during which about ` +
      `${Math.round(shipped.inf_before_disclosure_unbiased)} people are infected, ` +
      `is the measurable cost of the policy.`
    );
  }

  const r = bench.in_distribution.rules, m = bench.in_distribution.model;
  items.push(
    `On held-out runs in the training regime, with independent reporting, the authored ` +
    `rules score precision ` +
    `${r.precision.toFixed(2)}, recall ${r.recall.toFixed(2)}. A learned model is about ` +
    `${(r.delay - m.delay).toFixed(1)} day faster but less precise (${m.precision.toFixed(2)}), ` +
    `so the rules ship and the model stays a benchmark.`
  );

```

```
   items.push(
     `In simulated runs with independent reporting, banding the report counts stops an
` +
     `observer pinning an exact one-person overnight ` +
     `change: ${adv.exact_one_person_pins} days against exact counts, ` +
     `${adv.band_one_person_pins} against the bands. On about ` +
     `${adv.band_direction_known} of ${adv.same_scope_pairs.toLocaleString()} same-scop
e ` +
     `day-pairs the bands still reveal which way the count moved, though not by how muc
h.`
   );

   els.summaryList.innerHTML = "";
   for (const text of items) {
     const li = document.createElement("li");
     li.textContent = text;
     els.summaryList.appendChild(li);
   }
 }

 async function loadResults() {
   try {
     const [sweep, bench, adv] = await Promise.all([
       apiFetch("/api/results/disclosure_sweep").then((r) => r.json()),
       apiFetch("/api/results/benchmark").then((r) => r.json()),
       apiFetch("/api/results/adversarial").then((r) => r.json()),
     ]);
     renderSummary(sweep, bench, adv);
     renderCostChart(sweep);
     renderBenchmark(bench);
     renderRobustness(bench);
     renderAdversarial(adv);
     return true;
   } catch (err) {
     document.getElementById("cost-chart").textContent =
       "precomputed results unavailable: run python -m scripts.precompute_results";
     els.summaryList.innerHTML = "<li class=\"muted\">Findings unavailable.</li>";
     return false;
   }
 }

 // Autoplay the guided example once the layout and the precomputed results
 // have both loaded, so a visitor sees motion with no click. Does nothing if:
 //   - ?auto=... is present (that query param already drives its own
 //     playback below -- used for every gallery/verification screenshot),
 //   - ?autoplay=0 is present (escape hatch to capture the plain pre-run
 //     state deliberately),
 //   - loading is slow (over AUTOPLAY_TIMEOUT_MS) or either fetch failed --
 //     falls back to the static pre-run state rather than a broken run,
 //   - the visitor has already clicked Run, the guided button, or anything
 //     in the Playback panel first.
 const AUTOPLAY_TIMEOUT_MS = 5000;
 async function autoplayOnLoad(layoutReady, resultsReady) {
   const qs = new URLSearchParams(location.search);
   if (qs.get("auto") !== null || qs.get("autoplay") === "0") return;
```

```
 991 |   let ok = false;
 992 |   try {
 993 |     ok = await Promise.race([
 994 |       Promise.all([layoutReady, resultsReady]).then((results) => results.every(Boolea
n)),
 995 |       new Promise((resolve) => setTimeout(() => resolve(false), AUTOPLAY_TIMEOUT_MS)),
 996 |     ]);
 997 |   } catch (_) {
 998 |     ok = false;
 999 |   }
1000 |   if (!ok || userActed || state.runId) return;
1001 |   runGuidedScenario();
1002 | }
1003 |
1004 | const layoutReady = preloadLayout();
1005 | const resultsReady = loadResults();
1006 | autoplayOnLoad(layoutReady, resultsReady);
1007 |
1008 | // Optional: ?auto runs a seed on load (used for smoke screenshots).
1009 | // ?auto=day30 also jumps to the final day. Harmless otherwise.
1010 | const auto = new URLSearchParams(location.search).get("auto");
1011 | if (auto !== null) {
1012 |   startRun({ autoplay: false }).then(() => {
1013 |     const m = /^day(\d+)$/.exec(auto);
1014 |     if (auto === "end") paintDay(state.totalDays);
1015 |     else if (m) paintDay(Math.min(state.totalDays, Number(m[1])));
1016 |     else play();
1017 |     // ?res=wall drives the slider to its hard stop (for screenshots).
1018 |     if (new URLSearchParams(location.search).get("res") === "wall") {
1019 |       els.resSlider.value = String(state.maxResIdx);
1020 |       els.resSlider.dispatchEvent(new Event("input"));
1021 |     }
1022 |   });
1023 | }
```

web/gates.js

```
"use strict";

// Fills the worked-example tables under the gate explanations. The numbers
// come from GET /api/gates, which calls the engine's own threshold function,
// so nothing here (or in the HTML) is a typed-in copy of a threshold.
(function () {
  const hosts = document.querySelectorAll("[data-gate-examples]");
  if (!hosts.length) return;

  function cell(tag, text) {
    const el = document.createElement(tag);
    el.textContent = text;
    return el;
  }

  function table(caption, head, rows) {
    const t = document.createElement("table");
    t.className = "gate-example";
    const cap = document.createElement("caption");
    cap.textContent = caption;
    t.appendChild(cap);
    const thead = document.createElement("thead");
    const hr = document.createElement("tr");
    for (const h of head) hr.appendChild(cell("th", h));
    thead.appendChild(hr);
    t.appendChild(thead);
    const tbody = document.createElement("tbody");
    for (const r of rows) {
      const tr = document.createElement("tr");
      for (const c of r) tr.appendChild(cell("td", String(c)));
      tbody.appendChild(tr);
    }
    t.appendChild(tbody);
    return t;
  }

  // constants typed in the notation come from the same response, so the
  // page cannot drift from the configured thresholds
  function fillConstants(data) {
    const p = data.privacy;
    const values = {
      "statistical.floor": data.statistical.floor,
      "privacy.min_population": p.min_population,
      "privacy.max_fraction": p.max_fraction,
      "privacy.max_fraction_words":
        p.max_fraction === 0.5 ? "half" : Math.round(p.max_fraction * 100) + "%",
    };
    for (const el of document.querySelectorAll("[data-gate-const]")) {
      const v = values[el.dataset.gateConst];
      if (v !== undefined) el.textContent = String(v);
    }
  }
```

```
53 |
54 |   function render(data) {
55 |     fillConstants(data);
56 |     for (const host of hosts) {
57 |       host.textContent = "";
58 |       const scroll = document.createElement("div");
59 |       scroll.className = "table-scroll";
60 |       if (host.dataset.gateExamples === "statistical") {
61 |         scroll.appendChild(table(
62 |           "Reports needed before a group can be named",
63 |           ["Group", "People", "Reports needed"],
64 |           data.statistical.rows.map((r) => [r.scope, r.population, r.required])
65 |         ));
66 |       } else {
67 |         const p = data.privacy;
68 |         scroll.appendChild(table(
69 |           `Minimum ${p.min_population} people; refused once more than ` +
70 |           `${Math.round(p.max_fraction * 100)}% of the group has reported`,
71 |           ["Group", "People", "Big enough to name?", "Refused once reports reach"],
72 |           p.rows.map((r) => [
73 |             r.scope, r.population, r.meets_minimum ? "yes" : "no", r.refused_from,
74 |           ])
75 |         ));
76 |       }
77 |       host.appendChild(scroll);
78 |     }
79 |   }
80 |
81 |   async function load(tries) {
82 |     for (let i = 0; i < tries; i++) {
83 |       try {
84 |         const res = await fetch("/api/gates");
85 |         if (res.ok) return render(await res.json());
86 |       } catch (_) { /* retry */ }
87 |       await new Promise((r) => setTimeout(r, 400 * (i + 1)));
88 |     }
89 |     for (const host of hosts) {
90 |       host.textContent = "Worked numbers load from the running server and are unavailabl
e right now.";
91 |     }
92 |   }
93 |
94 |   load(4);
95 | })();
```

web/index.html

```
  1 | <!doctype html>
  2 | <html lang="en">
  3 | <head>
  4 | <meta charset="utf-8">
  5 | <meta name="viewport" content="width=device-width, initial-scale=1">
  6 | <title>MicroCluster simulator</title>
  7 | <link rel="stylesheet" href="style.css?v=16">
  8 | </head>
  9 | <body>
 10 | <a class="skip-link" href="#main">Skip to main content</a>
 11 | <p id="disclaimer" class="limitations" role="note">
 12 |   Illustrative simulation. Transmission, reporting, contact and incubation values are
 13 |   chosen for demonstration, not measured from any real population. Tested at 150 to 2,0
00
 14 |   simulated people and against a harsher simulated reporting model, never against real
 15 |   reports. This predicts nothing about any real building, and no one is diagnosed or
 16 |   treated by this tool.
 17 | </p>
 18 |
 19 | <header class="intro">
 20 |   <h1>MicroCluster</h1>
 21 |   <p class="tagline">It detects the signal without identifying the reporter, and won't
name a group too small or too covered to name safely.</p>
 22 |   <p class="explainer">
 23 |     An illness spreads through a simulated dorm floor. A detector that sees only
 24 |     anonymous symptom reports, never a name or a room, tries to find the outbreak it
 25 |     was never told about. When it finds one, it names the finest scope that is still
 26 |     safe to name, never one small enough to identify a reporter. Everything on this
 27 |     page is simulated.
 28 |   </p>
 29 |   <details class="how">
 30 |     <summary>How it works</summary>
 31 |     <ol>
 32 |       <li>Anonymous reports flow in: one symptom category from a fixed checklist, a
 33 |         rough onset, and which group the reporter is in. No name, no free text, no
 34 |         room number.</li>
 35 |       <li>The detector flags a statistical excess of reports inside a rolling time
 36 |         window, either against a location's neighbours or against a background rate.</l
i>
 37 |       <li>Two independent gates then decide whether anything may be said. Both must pas
s.
 38 |         <div class="gate-rule">
 39 |           <p><strong>Evidence gate.</strong> The bigger the group you want to name, the
more
 40 |           reports you need before the system will name it, and never fewer than
 41 |           <span data-gate-const="statistical.floor">5</span>.</p>
 42 |           <p class="gate-formula">In symbols:
 43 |           <math aria-label="at least five, or the square root of the population rounded
up, whichever is larger"><mrow><mi>needed</mi><mo>=</mo><mi>max</mi><mo>(</mo><mn data-gate-c
onst="statistical.floor">5</mn><mo>,</mo><mo>&lceil;</mo><msqrt><mi>n</mi></msqrt><mo>&rceil;
</mo><mo>)</mo></mrow></math>,
 44 |           where <math aria-label="n"><mi>n</mi></math> is how many people are in the gr
```

```
oup.</p>
 45 |           <div class="gate-examples" data-gate-examples="statistical">Worked numbers lo
ad from the running server.</div>
 46 |         </div>
 47 |         <div class="gate-rule">
 48 |           <p><strong>Privacy gate.</strong> The system never names a group with fewer t
han
 49 |           <span data-gate-const="privacy.min_population">20</span> people, and never na
mes a group where
 50 |           more than <span data-gate-const="privacy.max_fraction_words">half</span> of t
he members have
 51 |           reported, because that announcement would be close to a list of who is ill.</
p>
 52 |           <p class="gate-formula">In symbols, with <math aria-label="q"><mi>q</mi></mat
h> reports from a group of
 53 |           <math aria-label="n"><mi>n</mi></math> people:
 54 |           <math aria-label="the group has at least twenty people"><mrow><mi>n</mi><mo>&
ge;</mo><mn data-gate-const="privacy.min_population">20</mn></mrow></math>
 55 |           and
 56 |           <math aria-label="the reports divided by the group size is at most one half">
<mrow><mfrac><mi>q</mi><mi>n</mi></mfrac><mo>&le;</mo><mn data-gate-const="privacy.max_fracti
on">0.5</mn></mrow></math>.</p>
 57 |           <div class="gate-examples" data-gate-examples="privacy">Worked numbers load f
rom the running server.</div>
 58 |         </div>
 59 |       </li>
 60 |       <li>The system names the finest scope that clears both gates. If no scope
 61 |         clears them, it discloses nothing.</li>
 62 |     </ol>
 63 |   </details>
 64 | </header>
 65 |
 66 | <main id="main">
 67 | <div class="workbench">
 68 |   <div class="col-controls">
 69 |
 70 |     <fieldset class="panel">
 71 |       <legend>Run</legend>
 72 |       <div class="btn-row">
 73 |         <button type="button" id="run-guided" class="primary">Run example settings</but
ton>
 74 |         <button type="button" id="run">Run custom simulation</button>
 75 |       </div>
 76 |       <p class="run-hint" id="guided-hint">
 77 |         The guided example is the scenario to watch. In this single run, at these exact
 78 |         settings (seed 4, transmission 0.025, reporting 0.80), the detector fires on
 79 |         day 3, one suite is named on day 4, then by day 8 the privacy gate forces the
 80 |         disclosure back up to the whole floor. The headline numbers below are averages
 81 |         over many runs and will not match this one.
 82 |       </p>
 83 |       <div class="field-row">
 84 |         <label for="seed">Seed
 85 |           <input type="number" id="seed" value="4" min="0" step="1" aria-describedby="s
eed-hint"></label>
 86 |         <label for="days">Simulation length (days)
```

```
 87 |           <input type="number" id="days" value="30" min="1" max="120" step="1" aria-des
cribedby="days-hint"></label>
 88 |       </div>
 89 |       <div class="field-row">
 90 |         <span class="hint" id="seed-hint">fixes the random numbers, so the same seed al
ways reproduces the same outbreak</span>
 91 |         <span class="hint" id="days-hint">how many days the simulation runs before stop
ping</span>
 92 |       </div>
 93 |       <p id="params-dirty" class="dirty" hidden>Parameters changed. Press &ldquo;Run cu
stom simulation&rdquo; to apply.</p>
 94 |     </fieldset>
 95 |
 96 |     <fieldset class="panel">
 97 |       <legend>Playback</legend>
 98 |       <div class="btn-row">
 99 |         <button type="button" id="play" disabled>Play</button>
100 |         <button type="button" id="pause" disabled>Pause</button>
101 |         <button type="button" id="step" disabled>Step</button>
102 |         <button type="button" id="reset" disabled>Reset</button>
103 |       </div>
104 |       <div class="field-row">
105 |         <label for="speed">Speed
106 |           <select id="speed">
107 |             <option value="1000">0.5x</option>
108 |             <option value="500" selected>1x</option>
109 |             <option value="250">2x</option>
110 |             <option value="120">4x</option>
111 |           </select>
112 |         </label>
113 |       </div>
114 |       <div class="daycount">
115 |         <span>Day <output id="day-label">0</output> / <output id="day-max">0</output></
span>
116 |         <input type="range" id="scrub" min="0" max="0" value="0" disabled
117 |           aria-label="Playback day">
118 |       </div>
119 |     </fieldset>
120 |
121 |     <fieldset class="panel params">
122 |       <legend>Model parameters (applied on Run)</legend>
123 |       <label class="param">Suite size
124 |         <input type="number" id="p-population" value="25" min="21" max="40" step="1"
125 |           aria-describedby="p-population-hint p-population-help">
126 |         <span class="hint" id="p-population-hint">6 suites, 150 people</span>
127 |         <span class="hint" id="p-population-help">people sharing one suite &ndash; also
the detector's smallest group</span>
128 |       </label>
129 |       <label class="param">Transmission (per contact / day)
130 |         <input type="range" id="p-transmission" min="0.001" max="0.04" step="0.001" val
ue="0.008"
131 |           aria-describedby="p-transmission-hint p-transmission-help">
132 |         <span class="hint" id="p-transmission-hint">0.008</span>
133 |         <span class="hint" id="p-transmission-help">chance an infected person passes it
to each person they're near, per day</span>
```

```
      </label>
      <label class="param">Reporting rate (while symptomatic)
        <input type="range" id="p-reporting" min="0.1" max="1" step="0.05" value="0.5"
          aria-describedby="p-reporting-hint p-reporting-help">
        <span class="hint" id="p-reporting-hint">0.50</span>
        <span class="hint" id="p-reporting-help">chance a symptomatic person files a re
port at all</span>
      </label>
      <label class="param">Detection window (hours)
        <input type="number" id="p-window" value="72" min="24" max="168" step="12"
          aria-describedby="p-window-hint p-window-help">
        <span class="hint" id="p-window-hint">rolling window for &ldquo;current&rdquo;
reports</span>
        <span class="hint" id="p-window-help">hours of recent reports the detector trea
ts as happening right now</span>
      </label>
    </fieldset>

    <section class="panel resolution" aria-label="privacy / resolution control">
      <h2>Location detail shown</h2>
      <p class="panel-caption">Drag toward finer detail. It stops at the point where na
ming the group would expose its members.</p>
      <div class="res-row">
        <span class="res-end" id="res-end-coarse" aria-hidden="true">coarsest</span>
        <input type="range" id="res-slider" min="0" max="0" value="0" step="1" disabled
          aria-label="Location detail shown, coarsest to finest">
        <span class="res-end" id="res-end-fine" aria-hidden="true">finest</span>
      </div>
      <p id="res-readout" class="res-readout" role="status">Run a simulation to explore
the candidate scopes.</p>
    </section>

  </div>

  <div class="col-stage">
    <h2 class="visually-hidden">Floor plan and detector output</h2>

    <section class="legend" aria-label="colour key">
      <span class="key"><span class="dot s-susceptible"></span>susceptible</span>
      <span class="key"><span class="dot s-incubating"></span>incubating</span>
      <span class="key"><span class="dot s-symptomatic"></span>symptomatic</span>
      <span class="key"><span class="dot s-recovered"></span>recovered</span>
      <span class="key counts" id="counts"></span>
    </section>

    <p id="status" class="status" role="status">System says: no run yet</p>

    <div id="refusal" class="refusal" role="status" aria-live="polite"></div>

    <p class="plan-caption" id="plan-caption">
      Each circle is one person; colour is that person's state on the shown day.
      Symptomatic circles carry a heavy ring and recovered circles a centre dot, so
      the four states read without colour. Circles are grouped into suites, suites into
      floors. The building holds <output id="plan-pop">150</output> people. The detecto
r
```

```
183 |       never sees this picture, only the anonymous reports it produces.
184 |     </p>
185 |
186 |     <div id="plan" class="plan" role="img"
187 |       aria-label="Floor plan. Run a simulation to populate it." aria-describedby="plan-
caption">
188 |       <p class="empty-note" id="empty-note">Loading the building layout.</p>
189 |     </div>
190 |
191 |     <section id="run-summary" class="panel run-summary" aria-label="end of run summary"
hidden>
192 |       <h2>This run, summarised</h2>
193 |       <dl class="summary-grid">
194 |         <div class="summary-row"><dt>Outbreak detected</dt><dd id="sum-detected">n/a</d
d></div>
195 |         <div class="summary-row"><dt>Detected on day</dt><dd id="sum-detected-day">n/a
</dd></div>
196 |         <div class="summary-row"><dt>Most specific safe disclosure</dt><dd id="sum-disc
losure">n/a</dd></div>
197 |         <div class="summary-row"><dt>Finer scope refused</dt><dd id="sum-refused">n/a</
dd></div>
198 |         <div class="summary-row"><dt>Refused on day</dt><dd id="sum-refused-day">n/a</d
d></div>
199 |       </dl>
200 |     </section>
201 |
202 |     <section class="gates" aria-label="gate table for the current day">
203 |       <h2>Gate table, day <output id="gate-day">0</output></h2>
204 |       <p class="panel-caption">
205 |         One row per group the engine considered, refused ones included. The first gate
206 |         passes when the group has enough reports for its size; the second passes when
207 |         the group is big enough to name and not mostly reporters. A group is named only
208 |         when both read PASS. Report counts show as bands, never exact numbers.
209 |       </p>
210 |       <div class="table-scroll">
211 |       <table id="gate-table">
212 |         <thead>
213 |           <tr>
214 |             <th>Scope</th>
215 |             <th>Population</th>
216 |             <th>Qualifying</th>
217 |             <th>Statistical gate</th>
218 |             <th>Privacy gate</th>
219 |             <th>Reason</th>
220 |           </tr>
221 |         </thead>
222 |         <tbody id="gate-body">
223 |           <tr><td colspan="6" class="muted">Run a simulation. Each row will be one grou
p the engine checked.</td></tr>
224 |         </tbody>
225 |       </table>
226 |       </div>
227 |     </section>
228 |
229 |   </div>
```

```
</div>

<div class="precomputed-divider" role="note">
  <p>Below: precomputed results from many independent simulated runs &ndash; not the run you just watched above.</p>
</div>

<section class="results-summary" aria-label="headline findings">
  <h2>What the experiments found</h2>
  <p class="panel-caption">Three findings from the precomputed runs, all with reports generated independently of one another.</p>
  <ul id="summary-list">
    <li class="muted">Loading findings.</li>
  </ul>
  <p class="results-conclusion" id="reporting-scope">These figures assume independent, memoryless reporting. Under correlated reporting the measured effect is a large rise in false alarms, from 60% to 78 to 100% of outbreaks that never take off, with about 3% of those then naming a group falsely.</p>
</section>

<section class="results" id="result-cost" aria-label="the cost of the privacy policy">
  <h2>The cost of the privacy policy</h2>
  <p class="panel-caption">
    How many more people are infected for each extra day the privacy rules delay
    disclosure. A flatter line means restraint is cheap; a steep one means it is
    expensive.
  </p>
  <p class="results-note">
    Each point is one setting of the statistical gate, over 500 simulated outbreaks
    with detection held fixed and independent reporting. Non-disclosing outbreaks count their full day-30 total.
  </p>
  <div id="cost-chart" class="chart"></div>
</section>

<section class="results" id="result-benchmark" aria-label="rules versus learned model">
  <h2>Rules vs learned model</h2>
  <p class="panel-caption">
    Whether a model fitted to the data beats the hand-written rules. A good result for
    the rules is comparable precision and recall without the model's extra false
    alarms.
  </p>
  <p class="results-note">
    A logistic-regression detector trained on the three detection findings, scored
    against the authored threshold rules on held-out simulation runs with independent
    reporting. Both stay in the product; the model never replaces the rules.
  </p>
  <p class="results-note">
    Precision is how often an alert is correct. Recall is how many of the real outbreaks
    get caught. Mean detection delay is measured between the outbreak actually taking
    hold in the simulation and the detector firing.
  </p>
  <div class="table-scroll">
  <table id="benchmark-table">
```

```
278 |     <thead>
279 |       <tr>
280 |         <th>Detector</th><th>Precision</th><th>Recall</th><th>Mean detection delay (day
s)</th>
281 |       </tr>
282 |     </thead>
283 |     <tbody id="benchmark-body">
284 |       <tr><td colspan="4" class="muted">Loading</td></tr>
285 |     </tbody>
286 |   </table>
287 |   </div>
288 |   <p id="benchmark-conclusion" class="results-conclusion"></p>
289 | </section>
290 |
291 | <section class="results" id="result-robustness" aria-label="cross-parameter robustnes
s">
292 |   <h2>Does the result hold up on data it wasn't trained on?<span class="h2-sub">Cross-p
arameter robustness check</span></h2>
293 |   <p class="panel-caption">
294 |     Whether the model's small edge is real or just memorised: it is trained at one
295 |     transmission probability and tested at another. If the edge disappears here, it was
296 |     overfitting.
297 |   </p>
298 |   <p class="results-note">
299 |     The circularity risk stated plainly: a classifier trained on this simulator learns
300 |     this simulator. This panel reports what happens when the transmission regime shift
s;
301 |     reporting stays independent in every regime.
302 |   </p>
303 |   <p class="results-note">
304 |     Precision is how often an alert is correct. Recall is how many of the real outbreak
s
305 |     get caught. Mean delay is measured between the outbreak actually taking hold and th
e
306 |     detector firing.
307 |   </p>
308 |   <div class="table-scroll">
309 |   <table id="robustness-table">
310 |     <thead>
311 |       <tr>
312 |         <th>Test regime</th><th>Detector</th><th>Precision</th><th>Recall</th><th>Mean
delay (days)</th>
313 |       </tr>
314 |     </thead>
315 |     <tbody id="robustness-body">
316 |       <tr><td colspan="5" class="muted">Loading</td></tr>
317 |     </tbody>
318 |   </table>
319 |   </div>
320 |   <p id="robustness-conclusion" class="results-conclusion"></p>
321 | </section>
322 |
323 | <section class="results" id="result-adversarial" aria-label="attacking our own system">
324 |   <h2>What can a bystander still figure out about one person?<span class="h2-sub">Attac
king our own system</span></h2>
```

```
  <p class="panel-caption">
    What someone watching only this page can still work out about individuals. A good
    result is that the banded output leaves them almost nothing.
  </p>
  <p class="results-note">
    An observer who sees only the disclosed scope and its report-count band, day after
    day, in simulated runs with independent reporting, runs the differencing attack: su
btract consecutive disclosures of the same
    scope to infer how many people changed state overnight.
  </p>
  <div class="bignums">
    <div class="bignum"><span class="bignum-value" id="adv-exact">0</span>
      <span class="bignum-label">days an outsider could pin the exact overnight change,
if exact report counts were shown</span></div>
    <div class="bignum"><span class="bignum-value" id="adv-band">0</span>
      <span class="bignum-label">the same measure, against the rounded bands this page
actually ships</span></div>
  </div>
  <p id="adv-residual" class="results-conclusion"></p>
</section>


</main>

<footer class="site-footer">
  <a href="terms.html">Terms of Service</a>
  <span aria-hidden="true">&middot;</span>
  <a href="privacy.html">Privacy policy</a>
</footer>

<script src="app.js?v=16"></script>
<script src="gates.js?v=16"></script>
</body>
</html>
```

web/privacy.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>MicroCluster Privacy policy</title>
<link rel="stylesheet" href="style.css?v=16">
</head>
<body>
<p class="limitations" role="note">
  Illustrative simulation. Transmission, reporting, contact and incubation values are
  chosen for demonstration, not measured from any real population. Tested at 150 to 2,0
00
  simulated people and against a harsher simulated reporting model, never against real
  reports. This predicts nothing about any real building, and no one is diagnosed or
  treated by this tool.
</p>
<article class="static-page">
  <p class="back"><a href="index.html">Back to the simulator</a></p>
  <h1>Privacy policy</h1>
  <p class="updated">This describes the mechanism as built. It is not legal boilerplat
e.</p>

  <p>
    MicroCluster is a privacy-preserving health-signal system for small shared
    communities. It looks for statistically unusual symptom activity and discloses
    it only at a scope that is both statistically supported and privacy-safe. The
    running page on this site is an illustrative simulation of that system: the
    people, the outbreak, and every parameter are synthetic.
  </p>

  <h2>What a real deployment collects</h2>
  <p>Each report is a single submission from a fixed checklist:</p>
  <ul>
    <li>one of four <strong>symptom categories</strong> (respiratory, gastrointestinal,
general, other);</li>
    <li>a specific symptom from that category, recorded for the reporter's own clarity
and <strong>never used for matching</strong>;</li>
    <li>a <strong>coarse onset bucket</strong>: today, 1&ndash;2 days, 3&ndash;7 days,
1+ week;</li>
    <li>one <strong>location group</strong> from the community's scope hierarchy, at th
e finest level the reporter belongs to.</li>
  </ul>

  <h2>What it never collects</h2>
  <ul>
    <li>No name, account, login, device identifier, or contact information.</li>
    <li>No free text. Every field is a choice from a closed vocabulary, so there is not
hing to interpret and nothing to leak in prose.</li>
    <li>No precise timestamp. Onset is a bucket; submission time is only used to decide
whether a report falls inside the current detection window.</li>
    <li>No precise location. The finest location is a declared group, not a coordinate
```

```
or a room.</li>
    <li>No medical concept finer than a category. The system has no code path that reas
ons about a specific diagnosis.</li>
  </ul>
  <p>
    The privacy strategy is <strong>data minimisation, not encryption</strong>. The sys
tem
    never holds identity, free text, precise time, or precise location, so there is ver
y
    little to encrypt. You cannot leak what you did not collect.
  </p>

  <h2>How the gates work</h2>
  <p>
    Detection and disclosure are separate. Detection decides whether something unusual
is
    happening. Disclosure decides what, if anything, may be said, and at what scope. A
    scope is disclosed only if it passes <strong>both</strong> of two gates, which are
    evaluated independently and never combined into one number:
  </p>
  <div class="gate-rule">
    <p><strong>Evidence gate.</strong> The bigger the group you want to name, the more
    reports you need before the system will name it, and never fewer than
    <span data-gate-const="statistical.floor">5</span>.</p>
    <p class="gate-formula">In symbols:
    <math aria-label="at least five, or the square root of the population rounded up, w
hichever is larger"><mrow><mi>needed</mi><mo>=</mo><mi>max</mi><mo>(</mo><mn data-gate-const
="statistical.floor">5</mn><mo>,</mo><mo>&lceil;</mo><msqrt><mi>n</mi></msqrt><mo>&rceil;</mo
><mo>)</mo></mrow></math>,
    where <math aria-label="n"><mi>n</mi></math> is how many people are in the group.</
p>
    <div class="gate-examples" data-gate-examples="statistical">Worked numbers load fro
m the running server.</div>
  </div>
  <div class="gate-rule">
    <p><strong>Privacy gate.</strong> The system never names a group with fewer than
    <span data-gate-const="privacy.min_population">20</span> people, and never names a
group where
    more than <span data-gate-const="privacy.max_fraction_words">half</span> of the mem
bers have
    reported, because that announcement would be close to a list of who is ill.</p>
    <p class="gate-formula">In symbols, with <math aria-label="q"><mi>q</mi></math> rep
orts from a group of
    <math aria-label="n"><mi>n</mi></math> people:
    <math aria-label="the group has at least twenty people"><mrow><mi>n</mi><mo>&ge;</m
o><mn data-gate-const="privacy.min_population">20</mn></mrow></math>
    and
    <math aria-label="the reports divided by the group size is at most one half"><mrow>
<mfrac><mi>q</mi><mi>n</mi></mfrac><mo>&le;</mo><mn data-gate-const="privacy.max_fraction">0.
5</mn></mrow></math>.</p>
    <div class="gate-examples" data-gate-examples="privacy">Worked numbers load from th
e running server.</div>
  </div>
  <p>
    The two gates pull in opposite directions by design. The system discloses the fines
```

```
t
 83 |     scope that clears both; if none does, it discloses nothing. Every candidate scope,
 84 |     including the rejected ones, gets a full evaluation record with the reason for its
 85 |     outcome. The disclosure engine is only ever handed aggregate per-scope counts and
 86 |     declared populations, never individual reports.
 87 |   </p>
 88 |
 89 |   <h2>What the gates guarantee, and what they do not</h2>
 90 |   <p>
 91 |     In established terms, the two gates together enforce three things about every group
the
 92 |     system names, every time, by construction:
 93 |   </p>
 94 |   <ul>
 95 |     <li><strong>A group-size floor.</strong> The named group has at least 20 people. Th
is is a
 96 |       k-anonymity-style property (k of at least 20): the statement is about a group, an
d
 97 |       any member is indistinguishable from the others in it, because reports carry noth
ing
 98 |       finer than a group.</li>
 99 |     <li><strong>Minimum support.</strong> The statement rests on at least 5 reports, mo
re for
100 |       bigger groups. This is a standard small-count threshold rule.</li>
101 |     <li><strong>A prevalence cap.</strong> At most half of the group can be among the r
eporters,
102 |       so knowing someone is in the named group never makes them more than 50% likely to
be a
103 |       reporter. This limits the homogeneity weakness of k-anonymity. It is a simple ca
p, not
104 |       a full l-diversity or t-closeness guarantee.</li>
105 |   </ul>
106 |   <p>
107 |     That is all the gates promise. They do <strong>not</strong> provide differential pr
ivacy. Differential
108 |     privacy adds calibrated random noise so that any one person's data changes what is
109 |     released by at most a bounded amount. These gates add no noise and use hard thresho
lds on
110 |     exact counts: one extra report can be the difference between naming a group and say
ing
111 |     nothing, so the change in output from one person is not bounded. They also keep no
112 |     privacy budget, so many daily releases over overlapping data are not accounted for,
and
113 |     they give no protection against someone who already knows almost all of the report
s.
114 |   </p>
115 |   <p>
116 |     They also do not guarantee protection from the operator who holds the reports, do n
ot
117 |     stop an attacker who injects reports, do not verify that a group's declared size is
118 |     true, and do not make being named as a group with unusual illness harmless to its
119 |     members. The full write-up is in docs/privacy_guarantee.md in the repository.
120 |   </p>
121 |
122 |   <h2>What the bands are for</h2>
```

```
123 |   <p>
124 |     The interface shows qualifying report counts as ranges (&amp;ldquo;5&amp;ndash;9&amp;rdquo;,
125 |     &amp;ldquo;10&amp;ndash;19&amp;rdquo;), never exact integers. This is a defence against a
126 |     differencing attack: if the exact count for a named group were visible on two
127 |     consecutive days, an observer could subtract them and learn that exactly one
128 |     identifiable person changed state overnight. The exact count stays on the server;
129 |     only the band is sent to the page. This reduces that leak; it does not eliminate
130 |     every inference. A named group with sustained activity in a known band still tells
an
131 |     observer something, and the system's own adversarial self-test reports exactly wha
t.
132 |   </p>
133 |
134 |   <h2>Abuse</h2>
135 |   <p>
136 |     Because reports are anonymous, someone can submit reports to manufacture a signal.
The
137 |     system accepts this as the cost of anonymity. It rate-limits submissions per sessio
n
138 |     (a per-session and a per-session-per-day cap, enforced server side), but it does no
t
139 |     authenticate reporters, so a determined attacker using multiple sessions can still
140 |     inject reports. No identity-based control is used, because identity is the thing th
e
141 |     system exists to protect.
142 |   </p>
143 |
144 |   <p class="back"><a href="index.html">Back to the simulator</a></p>
145 | </article>
146 | <script src="gates.js?v=16"></script>
147 | </body>
148 | </html>
```

web/style.css

```
  1 | /* Flat, high-contrast, documentary/clinical. Off-white ground, no
  2 |    gradients, no shadows. Square panels; the only circles are agent /
  3 |    legend data marks. Motion is permitted only for state-change and
  4 |    interaction feedback, capped at 400ms, never looping -- see the MOTION
  5 |    section near the end of this file and CLAUDE.md UI CONSTRAINTS.
  6 |    System UI / IBM Plex stack. */
  7 |
  8 | :root {
  9 |   --ground: #f4f3ef;
 10 |   --field: #faf9f6;            /* off-white ground for form controls, never pure white
*/
 11 |   --panel-head: #e7e4db;
 12 |   --ink: #1a1a1a;
 13 |   --line: #1a1a1a;
 14 |   --line-soft: #847d72;        /* borders / dividers -- >=3:1 on the ground */
 15 |   --muted: #68635a;            /* secondary text -- >=4.5:1 on ground n field */
 16 |   --susceptible: #d9d6cc;
 17 |   --incubating: #c77b00;
 18 |   --symptomatic: #b3121b;
 19 |   --recovered: #6a6f77;
 20 |   --flag: #b3121b;
 21 |
 22 |   /* one spacing scale, used everywhere */
 23 |   --sp-1: 8px;
 24 |   --sp-2: 16px;
 25 |   --sp-3: 24px;
 26 |   --sp-4: 40px;
 27 |   --edge: 24px;               /* page gutter */
 28 | }
 29 |
 30 | * { box-sizing: border-box; }
 31 |
 32 | /* visible, high-contrast focus ring for keyboard users (>=3:1 vs both grounds) */
 33 | a:focus-visible, button:focus-visible, input:focus-visible,
 34 | select:focus-visible, summary:focus-visible {
 35 |   outline: 2px solid var(--ink);
 36 |   outline-offset: 2px;
 37 | }
 38 |
 39 | .visually-hidden {
 40 |   position: absolute;
 41 |   width: 1px; height: 1px;
 42 |   margin: -1px; padding: 0; border: 0;
 43 |   clip: rect(0 0 0 0);
 44 |   clip-path: inset(50%);
 45 |   overflow: hidden;
 46 |   white-space: nowrap;
 47 | }
 48 |
 49 | .skip-link {
 50 |   position: absolute;
 51 |   left: -9999px;
```

```
  top: 0;
  z-index: 20;
  background: var(--ground);
  color: var(--ink);
  border: 2px solid var(--ink);
  padding: var(--sp-1) var(--sp-2);
  font-size: 13px;
}
.skip-link:focus {
  left: var(--edge);
  top: var(--sp-1);
}

html, body {
  margin: 0;
  background: var(--ground);
  color: var(--ink);
  font-family: "IBM Plex Sans", ui-sans-serif, system-ui, -apple-system,
    "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
  font-size: 15px;
  line-height: 1.45;
}

.limitations {
  position: sticky;
  top: 0;
  z-index: 10;
  margin: 0;
  padding: var(--sp-1) var(--edge);
  background: var(--panel-head);
  border-bottom: 1px solid var(--line);
  font-size: 13px;
}

/* ---- intro ------------------------------------------------------------ */

.intro {
  padding: var(--sp-3) var(--edge) var(--sp-3);
  border-bottom: 1px solid var(--line-soft);
  max-width: 900px;
}
h1 {
  font-size: 20px;
  font-weight: 600;
  letter-spacing: 0.01em;
  margin: 0 0 var(--sp-1);
}
.tagline {
  margin: 0 0 var(--sp-2);
  font-size: 15px;
  font-weight: 600;
}
.explainer {
  margin: 0 0 var(--sp-2);
  font-size: 14px;
```

```
  max-width: 68ch;
}
.how {
  border: 1px solid var(--line-soft);
  padding: 0 var(--sp-2);
}
.how summary {
  cursor: pointer;
  padding: var(--sp-1) 0;
  font-size: 13px;
  font-weight: 600;
  letter-spacing: 0.04em;
  text-transform: uppercase;
}
.how[open] summary { border-bottom: 1px solid var(--line-soft); }
.how ol {
  margin: var(--sp-2) 0;
  padding-left: 20px;
  font-size: 14px;
  max-width: 66ch;
}
.how li { margin-bottom: var(--sp-1); }

/* ---- workbench: two columns above 1000px ---------------------------- */

.workbench {
  padding: var(--sp-3) var(--edge);
  display: flex;
  flex-direction: column;
  gap: var(--sp-3);
}
@media (min-width: 1000px) {
  .workbench {
    display: grid;
    grid-template-columns: minmax(300px, 340px) minmax(0, 1fr);
    gap: var(--sp-4);
    align-items: start;
  }
}

.col-controls, .col-stage {
  display: flex;
  flex-direction: column;
  gap: var(--sp-3);
  min-width: 0;
}

.panel {
  border: 1px solid var(--line-soft);
  margin: 0;
  padding: var(--sp-2);
}
.panel legend, .panel > h2 {
  font-size: 12px;
  font-weight: 600;
```

```
  letter-spacing: 0.05em;
  text-transform: uppercase;
  padding: 0;
  margin: 0 0 var(--sp-2);
}
.panel > h2 { margin: 0 0 var(--sp-1); }

.panel-caption {
  margin: 0 0 var(--sp-2);
  font-size: 12px;
  font-style: italic;
  color: var(--muted);
  max-width: 72ch;
}

/* ---- form controls ------------------------------------------------ */

label { font-size: 13px; }

input[type="number"], select {
  width: auto;
  font: inherit;
  padding: 4px 6px;
  border: 1px solid var(--line);
  background: var(--field);
  border-radius: 0;
}
input[type="number"] { width: 66px; }

button {
  font: inherit;
  padding: 6px 14px;
  border: 1px solid var(--line);
  background: var(--field);
  color: var(--ink);
  border-radius: 0;
  cursor: pointer;
}
button:hover:not(:disabled) { background: var(--ink); color: var(--ground); }
button:disabled { color: var(--muted); border-color: var(--muted); cursor: default; }

/* guided example: filled by default, inverts on hover. same palette, no new colour */
button.primary { background: var(--ink); color: var(--ground); border-color: var(--in
k); }
button.primary:hover:not(:disabled) { background: var(--field); color: var(--ink); }

input[type="range"] { accent-color: var(--ink); }

.btn-row {
  display: flex;
  flex-wrap: wrap;
  gap: var(--sp-1);
  margin-bottom: var(--sp-2);
}
.field-row {
```

```
  display: flex;
  flex-wrap: wrap;
  gap: var(--sp-1) var(--sp-3);
  align-items: center;
}
.field-row label { display: inline-flex; align-items: center; gap: 6px; }

.run-hint {
  margin: 0 0 var(--sp-2);
  font-size: 12px;
  color: var(--muted);
  max-width: 60ch;
}
.dirty {
  margin: var(--sp-2) 0 0;
  padding: var(--sp-1) 10px;
  font-size: 12px;
  color: var(--flag);
  font-weight: 600;
  border: 2px solid var(--flag);
}

.daycount {
  margin-top: var(--sp-2);
  display: flex;
  flex-direction: column;
  gap: 6px;
  font-variant-numeric: tabular-nums;
  font-size: 13px;
}
.daycount input[type="range"] { width: 100%; }

.params {
  display: flex;
  flex-wrap: wrap;
  gap: var(--sp-2) var(--sp-3);
  align-items: end;
}
.param {
  display: flex;
  flex-direction: column;
  gap: 3px;
  font-size: 13px;
}
.param .hint { font-size: 11px; color: var(--muted); font-variant-numeric: tabular-num
s; }
.param input[type="range"] { width: 150px; }

/* ---- resolution ------------------------------------------------- */

.resolution .res-row {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: var(--sp-1);
```

```
}
.res-end { font-size: 12px; color: var(--muted); }
#res-slider { flex: 1; min-width: 120px; }
.res-readout {
  margin: var(--sp-2) 0 0;
  font-size: 13px;
  border: 1px solid var(--line);
  padding: var(--sp-1) 10px;
  min-height: 1.45em;
}
.res-readout.wall { border-color: var(--flag); border-width: 2px; }

/* ---- stage: legend, status, plan ---------------------------------- */

.legend {
  display: flex;
  flex-wrap: wrap;
  gap: 6px var(--sp-3);
  font-size: 13px;
}
.key { display: flex; align-items: center; gap: 6px; }
.key.counts { color: var(--muted); font-variant-numeric: tabular-nums; }
.dot {
  width: 12px; height: 12px;
  border: 1px solid var(--line);
  border-radius: 50%;            /* a data mark, not a rounded panel */
  display: inline-block;
}
/* state also carries a non-colour channel so it survives colour blindness
   n greyscale: symptomatic = heavy rim, recovered = centre hole. see
   matching .agent rules below */
.dot { position: relative; }
.dot.s-susceptible { background: var(--susceptible); }
.dot.s-incubating { background: var(--incubating); }
.dot.s-symptomatic { background: var(--symptomatic); border-width: 2px; }
.dot.s-recovered { background: var(--recovered); }
.dot.s-recovered::after {
  content: "";
  position: absolute;
  top: 50%; left: 50%;
  width: 3px; height: 3px;
  margin: -1.5px 0 0 -1.5px;
  background: var(--ground);
  border-radius: 50%;
}

.status {
  margin: 0;
  font-size: 15px;
  font-weight: 600;
  border: 1px solid var(--line);
  padding: var(--sp-1) 10px;
}
.status.flagged { border-color: var(--flag); border-width: 2px; }

```

```
/* privacy refusal: the one moment that must not read as a table state.
   :empty hides it (so role=status announces the fill as a change) */
.refusal {
  margin: 0;
  border: 2px solid var(--flag);
  padding: var(--sp-2);
  font-size: 14px;
  line-height: 1.5;
}
.refusal:empty { display: none; }
.refusal strong { display: block; margin-bottom: 4px; text-transform: uppercase; letter-spacing: 0.05em; font-size: 12px; }

.plan-caption {
  margin: 0;
  font-size: 12px;
  color: var(--muted);
  max-width: 74ch;
}

.plan {
  margin: 0;
  padding: 0;
  display: flex;
  flex-direction: column;
  gap: var(--sp-2);
}
.empty-note { color: var(--muted); font-size: 13px; margin: 0; }

.floor { border: 1px solid var(--line); }
.floor-label {
  font-size: 12px;
  font-weight: 600;
  letter-spacing: 0.06em;
  text-transform: uppercase;
  padding: 5px 8px;
  border-bottom: 1px solid var(--line);
  background: var(--panel-head);
}
.suites {
  display: flex;
  flex-wrap: wrap;
  gap: var(--sp-2);
  padding: var(--sp-2);
}
.suite {
  border: 1px solid var(--line-soft);
  padding: var(--sp-1);
  min-width: 110px;
}
.suite.disclosed { border-color: var(--flag); border-width: 2px; padding: 7px; }
.suite.resolution { outline: 2px solid var(--ink); outline-offset: 1px; }
.suite-label {
  font-size: 11px;
  color: var(--muted);
```

```
  margin-bottom: 6px;
  font-variant-numeric: tabular-nums;
}
.agents {
  display: grid;
  grid-template-columns: repeat(5, 14px);
  gap: 4px;
}
.agent {
  position: relative;
  width: 14px; height: 14px;
  border: 1px solid var(--line);
  border-radius: 50%;            /* agents are circles; a data mark, not a panel */
  background: var(--susceptible);
}
.agent.s-incubating { background: var(--incubating); }
/* redundant non-colour channel -- heavy rim vs centre hole -- so the four
   states stay distinct in greyscale n under deuter/protan/tritanopia */
.agent.s-symptomatic { background: var(--symptomatic); border-width: 2px; }
.agent.s-recovered { background: var(--recovered); }
.agent.s-recovered::after {
  content: "";
  position: absolute;
  top: 50%; left: 50%;
  width: 4px; height: 4px;
  margin: -2px 0 0 -2px;
  background: var(--ground);
  border-radius: 50%;
}

/* ---- end-of-run summary card ---------------------------------------- */

.run-summary dl, .run-summary dt, .run-summary dd { margin: 0; }
.summary-row {
  display: flex;
  justify-content: space-between;
  gap: var(--sp-2);
  padding: 4px 0;
  border-bottom: 1px solid var(--line-soft);
  font-size: 13px;
}
.summary-row:last-child { border-bottom: none; }
.summary-row dt { color: var(--muted); font-weight: 600; }
.summary-row dd { font-variant-numeric: tabular-nums; text-align: right; }

/* ---- gate table ----------------------------------------------------- */

.gates h2 {
  font-size: 13px;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.06em;
  margin: 0 0 var(--sp-1);
}
#gate-table {
```

```
  border-collapse: collapse;
  width: 100%;
  font-size: 13px;
}
#gate-table th, #gate-table td {
  border: 1px solid var(--line-soft);
  padding: 5px 8px;
  text-align: left;
  vertical-align: top;
}
#gate-table th {
  background: var(--panel-head);
  border-color: var(--line);
  font-weight: 600;
}
#gate-table td.muted { color: var(--muted); }
#gate-table tr.selected td { border-color: var(--flag); }
/* PASS/FAIL is spelled out; colour n weight are redundant reinforcement */
.fail { color: var(--flag); font-weight: 700; }
.pass { font-weight: 700; }
.gate-cell small { display: block; color: var(--muted); }

/* ---- results ------------------------------------------------- */

/* unmistakable break between the run just watched and the precomputed
   validation panels below it -- same bg token as .limitations, so no new
   colour, just a second use of the existing "this is a system note" bar */
.precomputed-divider {
  margin: 0;
  padding: var(--sp-2) var(--edge);
  background: var(--panel-head);
  border-top: 2px solid var(--line);
  border-bottom: 1px solid var(--line-soft);
}
.precomputed-divider p {
  margin: 0;
  font-size: 13px;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.04em;
}

/* plain-language title first, technical term demoted to a second line */
.results h2 .h2-sub {
  display: block;
  margin-top: 2px;
  font-size: 11px;
  font-weight: 600;
  letter-spacing: 0.04em;
  text-transform: none;
  color: var(--muted);
}

.results-summary {
  padding: var(--sp-3) var(--edge);
```

```
489 |   border-top: 1px solid var(--line-soft);
490 |   max-width: 1100px;
491 | }
492 | .results-summary h2,
493 | .results h2 {
494 |   font-size: 14px;
495 |   font-weight: 600;
496 |   text-transform: uppercase;
497 |   letter-spacing: 0.06em;
498 |   margin: 0 0 var(--sp-1);
499 | }
500 | #summary-list {
501 |   margin: 0;
502 |   padding-left: 20px;
503 |   max-width: 82ch;
504 | }
505 | #summary-list li { margin-bottom: var(--sp-1); font-size: 14px; }
506 | #summary-list li.muted { color: var(--muted); }
507 |
508 | .results {
509 |   padding: var(--sp-3) var(--edge);
510 |   border-top: 1px solid var(--line-soft);
511 |   max-width: 1100px;
512 | }
513 | .results-note {
514 |   margin: 0 0 var(--sp-3);
515 |   font-size: 13px;
516 |   color: var(--muted);
517 |   max-width: 82ch;
518 | }
519 | .results-conclusion {
520 |   margin: var(--sp-3) 0 0;
521 |   font-size: 14px;
522 |   max-width: 82ch;
523 |   border: 1px solid var(--line);
524 |   padding: 10px 12px;
525 | }
526 |
527 | .chart { overflow-x: auto; }
528 | .chart svg { display: block; width: 100%; min-width: 460px; max-width: 760px; height: a
uto; }
529 | .chart .axis { stroke: var(--ink); stroke-width: 1; }
530 | .chart .series { fill: none; stroke: var(--ink); stroke-width: 1.5; }
531 | .chart .point { fill: var(--ink); }
532 | .chart text { fill: var(--ink); font-size: 12px; }
533 | .chart .tick { fill: var(--muted); font-size: 11px; }
534 | .chart .point-label { font-size: 11px; font-weight: 600; }
535 |
536 | .table-scroll { overflow-x: auto; }
537 |
538 | #benchmark-table, #robustness-table {
539 |   border-collapse: collapse;
540 |   width: 100%;
541 |   max-width: 760px;
542 |   font-size: 13px;
```

```
543 |   font-variant-numeric: tabular-nums;
544 | }
545 | #benchmark-table th, #benchmark-table td,
546 | #robustness-table th, #robustness-table td {
547 |   border: 1px solid var(--line-soft);
548 |   padding: 5px 10px;
549 |   text-align: left;
550 | }
551 | #benchmark-table th, #robustness-table th {
552 |   background: var(--panel-head);
553 |   border-color: var(--line);
554 |   font-weight: 600;
555 | }
556 | #benchmark-table td.muted { color: var(--muted); }
557 | #benchmark-table tr.model td,
558 | #robustness-table tr.model td { color: var(--muted); }
559 |
560 | .bignums {
561 |   display: flex;
562 |   flex-wrap: wrap;
563 |   gap: var(--sp-2);
564 | }
565 | .bignum {
566 |   border: 1px solid var(--line);
567 |   padding: var(--sp-2) var(--sp-3);
568 |   min-width: 220px;
569 |   flex: 1 1 220px;
570 | }
571 | .bignum-value {
572 |   display: block;
573 |   font-size: 48px;
574 |   font-weight: 700;
575 |   line-height: 1;
576 |   font-variant-numeric: tabular-nums;
577 | }
578 | .bignum-label {
579 |   display: block;
580 |   margin-top: var(--sp-1);
581 |   font-size: 13px;
582 |   color: var(--muted);
583 | }
584 |
585 | /* ---- footer + static pages ---------------------------------- */
586 |
587 | .site-footer {
588 |   padding: var(--sp-2) var(--edge) var(--sp-4);
589 |   border-top: 1px solid var(--line-soft);
590 |   font-size: 13px;
591 |   display: flex;
592 |   gap: 10px;
593 | }
594 | .site-footer a { color: var(--ink); }
595 |
596 | .static-page {
597 |   max-width: 74ch;
```

```
598 |   margin: 0 auto;
599 |   padding: 28px 20px 60px;
600 | }
601 | .static-page h1 { font-size: 20px; margin: 0 0 4px; }
602 | .static-page h2 { font-size: 15px; margin: 26px 0 6px; text-transform: none; letter-spa
cing: 0; }
603 | .static-page p, .static-page li { font-size: 14px; }
604 | .static-page a { color: var(--ink); }
605 | .static-page .back { font-size: 13px; }
606 | .static-page .updated { color: var(--muted); font-size: 12px; margin-top: 0; }
607 |
608 | /* ---- narrow screens ---------------------------------------- */
609 |
610 | @media (max-width: 560px) {
611 |   :root { --edge: 14px; }
612 |   .limitations { font-size: 12px; }
613 |   .param input[type="range"] { width: 130px; }
614 |   .bignum-value { font-size: 38px; }
615 | }
616 |
617 | /* skeleton loader for the async run -- flat gray placeholder, no motion */
618 | .plan.loading .floor { border-color: var(--muted); }
619 | .skeleton-suite {
620 |   min-width: 110px;
621 |   height: 96px;
622 |   border: 1px solid var(--line-soft);
623 |   background: var(--panel-head);
624 | }
625 |
626 |
627 | /* ---- gate explanations: plain rule first, typeset formula second ---- */
628 |
629 | math {
630 |   font-family: "STIX Two Math", "Cambria Math", "Latin Modern Math", math, serif;
631 |   font-size: 1.15em;
632 |   math-style: compact;   /* keep an inline fraction from ballooning the line */
633 | }
634 | .gate-rule {
635 |   border: 1px solid var(--line-soft);
636 |   padding: var(--sp-1) var(--sp-2);
637 |   margin: var(--sp-2) 0;
638 | }
639 | .gate-rule p { margin: 0 0 var(--sp-1); }
640 | .gate-formula { overflow-x: auto; max-width: 100%; line-height: 1.9; }
641 | .gate-examples { margin: var(--sp-1) 0; font-size: 13px; color: var(--muted); }
642 | .gate-example {
643 |   border-collapse: collapse;
644 |   font-size: 13px;
645 |   font-variant-numeric: tabular-nums;
646 |   color: var(--ink);
647 | }
648 | .gate-example caption {
649 |   text-align: left;
650 |   font-size: 12px;
651 |   font-style: italic;
```

```
652 |   color: var(--muted);
653 |   padding-bottom: 4px;
654 | }
655 | .gate-example th, .gate-example td {
656 |   border: 1px solid var(--line-soft);
657 |   padding: 4px 10px;
658 |   text-align: left;
659 | }
660 | .gate-example th { background: var(--panel-head); border-color: var(--line); font-weigh
t: 600; }
661 |
662 |
663 | /* ---- motion: state-change and interaction feedback only ---------------
664 |    CLAUDE.md UI CONSTRAINTS, "Motion" -- added because the page was fully
665 |    static and nothing moved until a judge clicked a button they might not
666 |    click. Every rule below either (a) carries a state change already
667 |    happening in the simulation, or (b) is ordinary interaction feedback.
668 |    Nothing loops, nothing exceeds 400ms, nothing here is purely decorative
669 |    except the single symptomatic-onset ring. Every rule is neutralised
670 |    under prefers-reduced-motion: reduce, at the bottom of this section.
671 |    This section is purely additive -- it sets no property any other rule
672 |    in this file also sets, so no existing rest-state appearance changes. */
673 |
674 | .agent { transition: background-color 180ms ease-out, border-color 180ms ease-out; }
675 |
676 | /* one-shot ring, monochrome, no repeat -- app.js adds .pulse for exactly
677 |    one frame on the instant a circle turns symptomatic during a real
678 |    forward playback step, and only then (never on load, scrub, or
679 |    ?auto=dayN), so no screenshot ever shows it mid-flight */
680 | .agent.pulse::before {
681 |   content: "";
682 |   position: absolute;
683 |   top: 50%; left: 50%;
684 |   width: 16px; height: 16px;
685 |   margin: -8px 0 0 -8px;
686 |   border: 1px solid var(--ink);
687 |   border-radius: 50%;
688 |   opacity: 0.5;
689 |   pointer-events: none;
690 |   animation: agent-ring 350ms ease-out;
691 | }
692 | @keyframes agent-ring {
693 |   from { transform: scale(0.9); opacity: 0.5; }
694 |   to   { transform: scale(1.9); opacity: 0; }
695 | }
696 |
697 | @keyframes fade-rise-in {
698 |   from { opacity: 0; transform: translateY(4px); }
699 |   to   { opacity: 1; transform: translateY(0); }
700 | }
701 | /* :not(:empty) starting to match IS the refusal box appearing; animation-name
702 |    goes none -> fade-rise-in right then, so this needs no JS trigger */
703 | .refusal:not(:empty) { animation: fade-rise-in 200ms ease-out; }
704 | /* app.js toggles .change-in for one tick when the disclosed scope changes */
705 | .status.change-in { animation: fade-rise-in 200ms ease-out; }
```

```
706 |
707 | /* app.js toggles .verdict-changed on a row for one tick, only on a real
708 |    forward step, only when that scope's PASS/FAIL actually changed; the
709 |    removal (not the arrival) is what transitions, so it reads as a fade,
710 |    and it always settles back to no background -- nothing persists */
711 | #gate-table tr { transition: background-color 150ms ease-out; }
712 | #gate-table tr.verdict-changed { background-color: var(--panel-head); }
713 |
714 | button, input[type="number"], input[type="range"], select {
715 |   transition: background-color 120ms ease-out, border-color 120ms ease-out;
716 | }
717 |
718 | @media (prefers-reduced-motion: reduce) {
719 |   .agent, #gate-table tr, button, input[type="number"], input[type="range"], select {
720 |     transition: none !important;
721 |   }
722 |   .agent.pulse::before { content: none; }
723 |   #gate-table tr.verdict-changed { background-color: transparent; }
724 |   .refusal:not(:empty), .status.change-in { animation: none !important; }
725 | }
```

web/terms.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>MicroCluster Terms of Service</title>
<link rel="stylesheet" href="style.css?v=16">
</head>
<body>
<p class="limitations" role="note">
  Illustrative simulation. Transmission, reporting, contact and incubation values are
  chosen for demonstration, not measured from any real population. Tested at 150 to 2,00
0
  simulated people and against a harsher simulated reporting model, never against real
  reports. This predicts nothing about any real building, and no one is diagnosed or
  treated by this tool.
</p>
<article class="static-page">
  <p class="back"><a href="index.html">Back to the simulator</a></p>
  <h1>Terms of Service</h1>
  <p class="updated">Plain terms for the demonstration site.</p>

  <h2>What this is</h2>
  <p>
    This site runs an illustrative simulation of the MicroCluster health-signal system.
    The population, the outbreak, the reports, and every parameter are synthetic and are
    chosen to produce a legible demonstration. The simulation does not model, predict, o
r
    describe real disease transmission in any real community.
  </p>

  <h2>Not medical advice</h2>
  <p>
    Nothing on this site is medical, epidemiological, or public-health advice. The syste
m
    does not diagnose anyone and is not designed to. Do not use any output here to make
a
    decision about your health or anyone else's.
  </p>

  <h2>Acceptable use</h2>
  <p>
    You may run simulations, read the precomputed results, and inspect the code. Do not
    attempt to overload the service, and do not present its output as describing a real
    outbreak or a real group of people.
  </p>

  <h2>Data you submit</h2>
  <p>
    Starting a simulation sends a seed and a set of numeric parameters to the server. No
    personal information is requested or required. What the system does and does not
    collect in a real deployment is described in the
```

```
    <a href="privacy.html">privacy policy</a>.
  </p>

  <h2>Availability and changes</h2>
  <p>
    The service is provided as is, with no warranty and no guarantee of availability.
    Runs are held in memory and are discarded over time. These terms and the underlying
    system may change without notice.
  </p>

  <p class="back"><a href="index.html">Back to the simulator</a></p>
</article>
</body>
</html>
```

tests/__init__.py

(empty file)

tests/fixtures.py

```
"""Shared test fixtures: a reference scope registry and report builders."""

from __future__ import annotations

from datetime import datetime, timedelta, timezone

from microcluster.models import (
    CHECKLIST,
    Category,
    Onset,
    Report,
    Scope,
    ScopeLevel,
    ScopeRegistry,
)

# A fixed evaluation instant so every test is deterministic.
NOW = datetime(2026, 9, 1, 12, 0, 0, tzinfo=timezone.utc)


def standard_registry() -> ScopeRegistry:
    """Campus n=2000: five buildings of 400; building B1 has three floors
    (~130 each); floor F1 has five suites (~26 each)."""
    scopes: dict[str, Scope] = {}

    def add(scope: Scope) -> None:
        scopes[scope.id] = scope

    add(Scope("C", "Campus", ScopeLevel.CAMPUS, population=2000))
    for b in range(1, 6):
        add(Scope(f"B{b}", f"Building {b}", ScopeLevel.BUILDING, 400, parent_id="C"))

    add(Scope("F1", "Bldg 1 / Floor 1", ScopeLevel.FLOOR, 130, parent_id="B1"))
    add(Scope("F2", "Bldg 1 / Floor 2", ScopeLevel.FLOOR, 130, parent_id="B1"))
    add(Scope("F3", "Bldg 1 / Floor 3", ScopeLevel.FLOOR, 140, parent_id="B1"))

    for s in range(1, 6):
        add(Scope(f"S{s}", f"Bldg 1 / Fl 1 / Suite {s}", ScopeLevel.SUITE, 26, parent_id
="F1"))

    return ScopeRegistry(scopes)


def small_registry_floor18() -> ScopeRegistry:
    """A campus with a floor of declared population 18 whose parent
    building is declared 25. Used for the privacy-refusal scenario."""
    return ScopeRegistry(
        {
            "C": Scope("C", "Campus", ScopeLevel.CAMPUS, population=300),
            "B": Scope("B", "Annex building", ScopeLevel.BUILDING, 25, parent_id="C"),
            "F": Scope("F", "Annex / top floor", ScopeLevel.FLOOR, 18, parent_id="B"),
        }
```

```
    )


def roster_registry() -> ScopeRegistry:
    """A large campus (n=2000) with one small declared group (n=25).
    Used for the "disclosure would be a roster" scenario: enough reports
    to clear the statistical gate and the population minimum, but so many
    relative to n=25 that naming the group names its members."""
    return ScopeRegistry(
        {
            "C": Scope("C", "Campus", ScopeLevel.CAMPUS, population=2000),
            "G": Scope("G", "Studio group", ScopeLevel.SUITE, 25, parent_id="C"),
        }
    )


def single_scope_registry(n: int) -> ScopeRegistry:
    """A campus-only registry of declared population ``n``. Used for the
    across-populations gate table."""
    return ScopeRegistry({"C": Scope("C", f"Campus n={n}", ScopeLevel.CAMPUS, population
=n)})


def report(
    location_id: str,
    *,
    category: Category = Category.RESPIRATORY,
    hours_ago: float = 1.0,
    onset: Onset = Onset.TODAY,
    now: datetime = NOW,
) -> Report:
    """One report at ``location_id``, submitted ``hours_ago`` before
    ``now``. Symptom is the first in the category's checklist -- it is
    never used for matching, so its value does not matter to the engines."""
    return Report(
        category=category,
        symptom=CHECKLIST[category][0],
        onset=onset,
        location_id=location_id,
        submitted_at=now - timedelta(hours=hours_ago),
    )


def reports(location_id: str, count: int, **kwargs) -> list[Report]:
    return [report(location_id, **kwargs) for _ in range(count)]
```

tests/test_adversarial.py

```
"""Adversarial self-test tests.

The important ones are on the ATTACK: the differencing attack must
provably pin a one-person overnight change when it is handed exact counts,
otherwise the banded-vs-exact contrast demonstrates nothing.
"""

from __future__ import annotations

import unittest

from api.bands import band
from adversarial.differencing import attack
from adversarial.observer import DayView, observe
from adversarial.selftest import run_self_test
from adversarial.wandering import measure_wandering
from simulation.pipeline import run_with_detection, simulate_and_report


def _views(scope, counts, *, exact):
    out = []
    for i, c in enumerate(counts):
        out.append(
            DayView(
                day=i,
                scope_id=scope,
                scope_label=scope,
                scope_level="SUITE",
                exact_count=c if exact else None,
                band=None if exact else band(c),
            )
        )
    return out


class DifferencingAgainstExactCountsTest(unittest.TestCase):
    def test_pins_every_one_person_overnight_change(self) -> None:
        out = attack(_views("S", [7, 8, 9, 8, 8], exact=True), mode="exact")
        self.assertEqual(out.same_scope_pairs, 4)
        self.assertEqual(out.pinned_exactly, 4)
        self.assertEqual(out.one_person_pinned, 3)
        self.assertEqual(out.direction_known, 3)
        self.assertEqual(out.unbounded_pairs, 0)

    def test_two_person_change_is_pinned_but_not_one_person(self) -> None:
        out = attack(_views("S", [10, 12], exact=True), mode="exact")
        self.assertEqual(out.pinned_exactly, 1)
        self.assertEqual(out.one_person_pinned, 0)

    def test_a_scope_change_between_days_yields_no_pair(self) -> None:
        views = [
            DayView(0, "A", "A", "SUITE", 7, None),
```

```
            DayView(1, "B", "B", "SUITE", 8, None),
        ]
        self.assertEqual(attack(views, mode="exact").same_scope_pairs, 0)


class DifferencingAgainstBandsTest(unittest.TestCase):
    def test_one_person_pin_is_impossible(self) -> None:
        out = attack(_views("S", [7, 8, 9, 8, 8], exact=False), mode="band")
        self.assertEqual(out.same_scope_pairs, 4)
        self.assertEqual(out.one_person_pinned, 0)
        self.assertEqual(out.pinned_exactly, 0)

    def test_direction_can_still_leak_across_a_band_edge(self) -> None:
        out = attack(_views("S", [9, 10], exact=False), mode="band")
        self.assertEqual(out.one_person_pinned, 0)
        self.assertEqual(out.direction_known, 1)

    def test_within_a_band_nothing_leaks(self) -> None:
        out = attack(_views("S", [5, 9], exact=False), mode="band")
        self.assertEqual(out.direction_known, 0)
        self.assertGreater(out.median_possible_values, 1)


class ObserverTest(unittest.TestCase):
    def test_exact_and_band_views_populate_exactly_one_count_field(self) -> None:
        run = run_with_detection(seed=4, days=30)
        exact = observe(run.daily_records, leak_exact_counts=True)
        banded = observe(run.daily_records, leak_exact_counts=False)
        self.assertEqual(len(exact), len(run.daily_records))
        for ev, bv, rec in zip(exact, banded, run.daily_records):
            if rec.disclosed_scope_id is None:
                self.assertIsNone(ev.scope_id)
                continue
            self.assertIsNotNone(ev.exact_count)
            self.assertIsNone(ev.band)
            self.assertIsNone(bv.exact_count)
            self.assertEqual(bv.band, band(ev.exact_count))

    def test_exact_count_matches_the_selected_scope_evaluation(self) -> None:
        run = run_with_detection(seed=4, days=30)
        for rec, view in zip(run.daily_records, observe(run.daily_records, leak_exact_c
ounts=True)):
            if rec.disclosed_scope_id is None:
                continue
            selected = next(
                e for e in rec.disclosure_evaluations if e.scope_id == rec.disclosed_sc
ope_id
            )
            self.assertEqual(view.exact_count, selected.qualifying_reports)


class WanderingTest(unittest.TestCase):
    def test_stability_off_names_more_groups_in_aggregate(self) -> None:
        total_on = total_off = 0
        for seed in range(1, 16):
```

```
            w = measure_wandering(simulate_and_report(seed=seed, days=30))
            total_on += w.stability_on
            total_off += w.stability_off
        self.assertLess(total_on, total_off)


class SelfTestIntegrationTest(unittest.TestCase):
    def test_attack_succeeds_on_exact_and_fails_on_bands(self) -> None:
        s = run_self_test(seeds=list(range(1, 25)))
        self.assertGreater(s.same_scope_pairs, 0)
        self.assertGreater(
            s.exact_one_person_pinned, 0, "differencing attack proved nothing against e
xact counts"
        )
        self.assertEqual(s.band_one_person_pinned, 0)
        self.assertGreaterEqual(s.wandering_off_mean, s.wandering_on_mean)


class InjectionAttackTest(unittest.TestCase):
    """The intake cap must raise the attacker's session count without
    claiming to close the hole -- exactly what the docs say it does."""

    def test_cap_blocks_one_session_but_not_a_session_rotating_attacker(self) -> None:
        from adversarial.injection import measure_injection

        r = measure_injection()  # target FLOOR C-B1-F2, declared population 75
        self.assertEqual(r.target_level, "FLOOR")

        # the report count is set by the statistical gate, not the cap
        self.assertEqual(r.reports_to_disclose, 9)  # ceil(sqrt(75)) = 9
        self.assertEqual(r.sessions_without_cap, 1)
        self.assertEqual(
            r.sessions_with_cap,
            -(-r.reports_to_disclose // r.session_cap),  # ceil(9 / 3) = 3
        )

        # the cap stops a single-session flood ...
        self.assertFalse(r.single_session_attack_succeeds)
        # ... and does nothing against an attacker who rotates sessions
        self.assertTrue(r.multi_session_attack_succeeds)


if __name__ == "__main__":
    unittest.main()
```

tests/test_api.py

```
  1 | """API-layer tests.
  2 |
  3 | The engines and the simulator have their own suites; these only check the
  4 | HTTP surface: shapes, the day range, the parameter passthrough, and -- the
  5 | security-relevant one -- that a day payload carries no raw reports or
  6 | ground truth beyond the agent states the animation renders.
  7 | """
  8 |
  9 | from __future__ import annotations
 10 |
 11 | import unittest
 12 |
 13 | import re
 14 |
 15 | from fastapi.testclient import TestClient
 16 |
 17 | from api.app import DISCLAIMER, app
 18 |
 19 | # High transmission + near-certain reporting: a suite reliably blows past
 20 | # the report-fraction cap and fails the privacy gate (the "roster" refusal).
 21 | _ROSTER_PARAMS = {
 22 |     "suite_transmission_probability": 0.03,
 23 |     "floor_transmission_probability": 0.002,
 24 |     "building_transmission_probability": 0.0005,
 25 |     "reporting_probability_min": 0.85,
 26 |     "reporting_probability_max": 1.0,
 27 |     "background_noise_daily_rate": 0.003,
 28 | }
 29 | _BAND_RE = re.compile(r"^(0|\d+-\d+|\d+\+)$")
 30 |
 31 |
 32 | class ApiTest(unittest.TestCase):
 33 |     def setUp(self) -> None:
 34 |         self.client = TestClient(app)
 35 |
 36 |     def _run(self, **body) -> dict:
 37 |         body.setdefault("seed", 4)
 38 |         body.setdefault("days", 12)
 39 |         res = self.client.post("/api/run", json=body)
 40 |         self.assertEqual(res.status_code, 200, res.text)
 41 |         return res.json()
 42 |
 43 |     def test_run_returns_id_layout_and_always_visible_disclaimer(self) -> None:
 44 |         data = self._run()
 45 |         self.assertIn("run_id", data)
 46 |         self.assertEqual(data["disclaimer"], DISCLAIMER)
 47 |         self.assertIn("predicts nothing about any real building", data["disclaimer"])
 48 |         self.assertIn("no one is diagnosed or treated", data["disclaimer"])
 49 |         floors = data["layout"]["floors"]
 50 |         self.assertEqual(len(floors), 2)
 51 |         self.assertEqual(sum(len(f["suites"]) for f in floors), 6)
 52 |         self.assertTrue(all(s["size"] == 25 for f in floors for s in f["suites"]))
```

```
 53 |
 54 |     def test_population_and_detection_window_controls_take_effect(self) -> None:
 55 |         data = self._run(population=30, detection_window_hours=48, days=10)
 56 |         self.assertEqual(data["params"]["population_per_suite"], 30)
 57 |         self.assertEqual(data["params"]["total_population"], 180)
 58 |         self.assertEqual(data["params"]["detection_window_hours"], 48)
 59 |         day = self.client.get(f"/api/run/{data['run_id']}/day/5").json()
 60 |         self.assertEqual(len(day["agents"]), 180)
 61 |
 62 |     def test_out_of_range_controls_are_422_not_500(self) -> None:
 63 |         self.assertEqual(self.client.post("/api/run", json={"seed": 1, "population": 1
5}).status_code, 422)
 64 |         self.assertEqual(
 65 |             self.client.post("/api/run", json={"seed": 1, "detection_window_hours":
6}).status_code,
 66 |             422,
 67 |         )
 68 |
 69 |     def test_assets_are_served_no_cache(self) -> None:
 70 |         res = self.client.get("/style.css")
 71 |         self.assertEqual(res.status_code, 200)
 72 |         self.assertIn("no-cache", res.headers.get("cache-control", ""))
 73 |         api_res = self.client.get("/api/health")
 74 |         self.assertNotIn("no-cache", api_res.headers.get("cache-control", ""))
 75 |
 76 |     def test_day_zero_and_last_day_are_in_range_and_beyond_is_404(self) -> None:
 77 |         data = self._run(days=10)
 78 |         rid = data["run_id"]
 79 |         self.assertEqual(self.client.get(f"/api/run/{rid}/day/0").status_code, 200)
 80 |         self.assertEqual(self.client.get(f"/api/run/{rid}/day/10").status_code, 200)
 81 |         self.assertEqual(self.client.get(f"/api/run/{rid}/day/11").status_code, 404)
 82 |         self.assertEqual(self.client.get(f"/api/run/{rid}/day/-1").status_code, 404)
 83 |
 84 |     def test_unknown_run_is_404(self) -> None:
 85 |         self.assertEqual(self.client.get("/api/run/deadbeef/day/0").status_code, 404)
 86 |
 87 |     def test_day_payload_shape(self) -> None:
 88 |         data = self._run(days=8)
 89 |         day = self.client.get(f"/api/run/{data['run_id']}/day/5").json()
 90 |         self.assertEqual(day["day"], 5)
 91 |         self.assertEqual(len(day["agents"]), 150)
 92 |         for agent in day["agents"]:
 93 |             self.assertEqual(set(agent), {"suite", "floor", "i", "state"})
 94 |             self.assertIn(
 95 |                 agent["state"],
 96 |                 {"susceptible", "incubating", "symptomatic", "recovered"},
 97 |             )
 98 |         self.assertEqual(set(day["detection"]), {"fired", "relative", "absolute"})
 99 |         self.assertEqual(
100 |             sum(day["counts"].values()), 150, "counts must cover every agent once"
101 |         )
102 |
103 |     def test_day_payload_leaks_no_reports_or_extra_ground_truth(self) -> None:
104 |         # Whitelist exactly the keys the animation consumes. Anything else
105 |         # -- a report list, per-agent infection timing, who-infected-whom --
```

```
        # would be a firewall break.
        data = self._run(days=6)
        day = self.client.get(f"/api/run/{data['run_id']}/day/3").json()
        self.assertEqual(
            set(day),
            {
                "day",
                "counts",
                "agents",
                "detection",
                "disclosure",
                "evaluations",
                "first_fired_day",
                "first_disclosed_day",
                "disclaimer",
            },
        )
        for agent in day["agents"]:
            self.assertNotIn("id", agent)
            self.assertNotIn("state_changed_day", agent)
            self.assertNotIn("incubation_days", agent)

    def test_seed_4_establishes_and_eventually_discloses_a_scope(self) -> None:
        data = self._run(seed=4, days=30)
        rid = data["run_id"]
        last = self.client.get(f"/api/run/{rid}/day/30").json()
        self.assertGreater(last["counts"]["recovered"], 50)  # a real outbreak ran
        self.assertIsNotNone(last["first_fired_day"])
        self.assertIsNotNone(last["first_disclosed_day"])
        self.assertLessEqual(last["first_fired_day"], last["first_disclosed_day"])
        self.assertIsNotNone(last["disclosure"])
        self.assertIn(last["disclosure"]["level"], {"SUITE", "FLOOR", "BUILDING", "CAMP
US"})

    def test_parameter_passthrough_and_validation(self) -> None:
        echoed = self._run(background_noise_daily_rate=0.02)["params"]
        self.assertAlmostEqual(echoed["background_noise_daily_rate"], 0.02)
        # min > max is rejected by SimulationConfig -> 422, not a 500.
        bad = self.client.post(
            "/api/run",
            json={"seed": 1, "reporting_probability_min": 0.9, "reporting_probability_m
ax": 0.1},
        )
        self.assertEqual(bad.status_code, 422)

    def test_evaluations_shape_and_ordering(self) -> None:
        data = self._run(seed=4, days=30)
        rid = data["run_id"]
        levels_rank = {"CAMPUS": 1, "BUILDING": 2, "FLOOR": 3, "SUITE": 4}
        saw_a_firing_day = False
        for n in range(31):
            day = self.client.get(f"/api/run/{rid}/day/{n}").json()
            evs = day["evaluations"]
            if day["detection"]["fired"] and day["disclosure"] is not None:
                saw_a_firing_day = saw_a_firing_day or len(evs) > 0
```

```
            order_key = [(levels_rank[e["level"]], -e["population"]) for e in evs]
            self.assertEqual(
                order_key, sorted(order_key),
                "evaluations must be ordered coarsest -> finest",
            )
            for e in evs:
                self.assertEqual(
                    set(e),
                    {
                        "scope_id", "label", "level", "population", "qualifying_band",
                        "statistical_threshold", "statistical_pass", "min_population_pa
ss",
                        "report_fraction_pass", "privacy_pass", "eligible", "selected",
                        "reason",
                    },
                )
                self.assertNotIn("qualifying_reports", e)
                self.assertNotIn("report_fraction", e)
                self.assertRegex(e["qualifying_band"], _BAND_RE)
        self.assertTrue(saw_a_firing_day)

    def test_exact_qualifying_count_never_appears_in_a_reason(self) -> None:
        # Cross-check against the engine's own numbers, computed here.
        from simulation.config import SimulationConfig
        from simulation.pipeline import run_with_detection

        run = run_with_detection(seed=4, days=30, config=SimulationConfig())
        data = self._run(seed=4, days=30)
        rid = data["run_id"]
        for record in run.daily_records:
            if not record.disclosure_evaluations:
                continue
            day = self.client.get(f"/api/run/{rid}/day/{record.day}").json()
            api_reasons = " || ".join(e["reason"] for e in day["evaluations"])
            for ev in record.disclosure_evaluations:
                exact = ev.qualifying_reports
                self.assertNotIn(f"({exact} qualifying reports", api_reasons)
                self.assertNotIn(f"qualifying count ({exact})", api_reasons)

    def test_slider_wall_data_exists_when_a_suite_fails_privacy(self) -> None:
        data = self._run(seed=4, days=30, **_ROSTER_PARAMS)
        rid = data["run_id"]
        found_wall = False
        for n in range(31):
            evs = self.client.get(f"/api/run/{rid}/day/{n}").json()["evaluations"]
            passed_then_failed = any(
                evs[i]["privacy_pass"] and not evs[j]["privacy_pass"]
                for i in range(len(evs))
                for j in range(i + 1, len(evs))
            )
            suite_roster_fail = [
                e for e in evs
                if e["level"] == "SUITE"
                and not e["privacy_pass"]
                and e["min_population_pass"]
```

```
                and not e["report_fraction_pass"]
            ]
            if passed_then_failed and suite_roster_fail:
                found_wall = True
                self.assertIn("over the limit", suite_roster_fail[0]["reason"])
                break
        self.assertTrue(found_wall, "expected a day with a coarser pass then a suite pr
ivacy fail")

    def test_layout_endpoint_serves_prerun_structure(self) -> None:
        res = self.client.get("/api/layout")
        self.assertEqual(res.status_code, 200)
        data = res.json()
        floors = data["layout"]["floors"]
        self.assertEqual(len(floors), 2)
        self.assertEqual(sum(len(f["suites"]) for f in floors), 6)
        self.assertEqual(data["params"]["total_population"], 150)
        self.assertIn("predicts nothing about any real building", data["disclaimer"])

    def test_layout_endpoint_honours_population_and_rejects_bad_values(self) -> None:
        data = self.client.get("/api/layout?population=30").json()
        self.assertEqual(data["params"]["total_population"], 180)
        self.assertEqual(self.client.get("/api/layout?population=5").status_code, 422)

    def test_gates_endpoint_uses_the_real_threshold_function(self) -> None:
        from microcluster.disclosure import statistical_threshold

        data = self.client.get("/api/gates").json()
        rows = data["statistical"]["rows"]
        self.assertEqual([r["population"] for r in rows], [25, 75, 150])
        for r in rows:
            self.assertEqual(r["required"], statistical_threshold(r["population"]))
        self.assertEqual([r["required"] for r in rows], [5, 9, 13])
        privacy = data["privacy"]
        self.assertEqual(privacy["min_population"], 20)
        self.assertEqual(privacy["max_fraction"], 0.5)
        # 12 of 25 is exactly half (allowed); the 13th report is refused
        self.assertEqual([r["refused_from"] for r in privacy["rows"]], [13, 38, 76])
        self.assertTrue(all(r["meets_minimum"] for r in privacy["rows"]))

    def test_gate_constants_typed_in_the_html_match_the_config(self) -> None:
        # the pages ship fallback text for the thresholds; gates.js overwrites
        # it from /api/gates, but the fallback must not be able to drift
        import re
        from pathlib import Path

        from microcluster.config import DEFAULT_DISCLOSURE_CONFIG as cfg

        expected = {
            "statistical.floor": str(cfg.statistical_gate_floor),
            "privacy.min_population": str(cfg.min_scope_population),
            "privacy.max_fraction": str(cfg.max_report_fraction),
            "privacy.max_fraction_words": "half",
        }
        web = Path(__file__).resolve().parent.parent / "web"
```

```
        for page in ("index.html", "privacy.html"):
            html = (web / page).read_text(encoding="utf-8")
            found = re.findall(r'data-gate-const="([^"]+)">([^<]*)<', html)
            self.assertTrue(found, page)
            for key, text in found:
                self.assertEqual(text, expected[key], f"{page}: {key}")

    def test_web_index_is_served(self) -> None:
        res = self.client.get("/")
        self.assertEqual(res.status_code, 200)
        self.assertIn("outbreak", res.text.lower())

    def test_static_pages_are_served(self) -> None:
        for page, needle in (("terms.html", "terms of service"), ("privacy.html", "priv
acy")):
            res = self.client.get(f"/{page}")
            self.assertEqual(res.status_code, 200, page)
            self.assertIn(needle, res.text.lower())

    def test_precomputed_results_are_served_and_shaped(self) -> None:
        sweep = self.client.get("/api/results/disclosure_sweep")
        self.assertEqual(sweep.status_code, 200)
        sweep = sweep.json()
        self.assertIn(sweep["primary_sweep"], sweep["sweeps"])
        self.assertTrue(sweep["sweeps"][sweep["primary_sweep"]]["rows"])

        bench = self.client.get("/api/results/benchmark").json()
        self.assertEqual(set(bench["in_distribution"]), {"rules", "model"})
        self.assertEqual(len(bench["cross_regime"]), 2)
        self.assertIn("cross_regime_conclusion", bench)
        blocks = [bench["in_distribution"]] + bench["cross_regime"]
        for block in blocks:
            for who in ("rules", "model"):
                self.assertLessEqual(block[who]["precision"], 1.0)
                self.assertLessEqual(block[who]["recall"], 1.0)

        adv = self.client.get("/api/results/adversarial").json()
        self.assertGreater(adv["exact_one_person_pins"], adv["band_one_person_pins"])
        self.assertEqual(adv["band_one_person_pins"], 0)

    def test_unknown_results_file_is_404(self) -> None:
        self.assertEqual(self.client.get("/api/results/secrets").status_code, 404)


if __name__ == "__main__":
    unittest.main()
```

tests/test_bands.py

```
"""api.bands: count banding and reason sanitisation.

The differencing-attack defence (CLAUDE.md rule 3 for the UI): an exact
qualifying-report count must never leave the API, in a field or embedded
in a reason string.
"""

from __future__ import annotations

import unittest

from api.bands import BAND_LOWER_BOUNDS, band, band_interval, sanitize_reason


class BandTest(unittest.TestCase):
    def test_zero_and_small_bands(self) -> None:
        self.assertEqual(band(0), "0")
        self.assertEqual(band(1), "1-4")
        self.assertEqual(band(4), "1-4")
        self.assertEqual(band(5), "5-9")
        self.assertEqual(band(9), "5-9")
        self.assertEqual(band(10), "10-19")
        self.assertEqual(band(19), "10-19")

    def test_open_top_band(self) -> None:
        top = BAND_LOWER_BOUNDS[-1]
        self.assertEqual(band(top), f"{top}+")
        self.assertEqual(band(top + 1000), f"{top}+")

    def test_every_count_maps_somewhere_and_hides_plus_or_minus_one(self) -> None:
        # No two counts one apart are ever distinguishable unless they
        # straddle a band edge -- and even then only "which side", never
        # the value. Concretely: many consecutive counts share a label.
        for lo in BAND_LOWER_BOUNDS[:-1]:
            self.assertEqual(band(lo), band(lo + 1))

    def test_negative_is_zero(self) -> None:
        self.assertEqual(band(-3), "0")

    def test_band_interval_inverts_band(self) -> None:
        self.assertEqual(band_interval("0"), (0, 0))
        self.assertEqual(band_interval("1-4"), (1, 4))
        self.assertEqual(band_interval("10-19"), (10, 19))
        top = BAND_LOWER_BOUNDS[-1]
        self.assertEqual(band_interval(f"{top}+"), (top, None))
        for c in (1, 4, 5, 12, 40, 99):
            lo, hi = band_interval(band(c))
            self.assertLessEqual(lo, c)
            self.assertGreaterEqual(hi, c)


class SanitizeReasonTest(unittest.TestCase):
```

```
    def test_statistical_failure_count_becomes_a_band(self) -> None:
        out = sanitize_reason(
            "Rejected: statistical gate failed (7 qualifying reports < required 45 for n
=2000)"
        )
        self.assertNotIn("(7 qualifying", out)
        self.assertIn("(5-9 qualifying reports", out)
        self.assertIn("required 45", out)  # threshold untouched
        self.assertIn("n=2000", out)       # declared population untouched

    def test_report_fraction_is_blurred(self) -> None:
        out = sanitize_reason(
            "Rejected: report fraction 0.60 exceeds maximum 0.50 (roster)"
        )
        self.assertNotIn("0.60", out)          # the exact fraction is gone
        self.assertIn("over the limit of 0.50", out)  # the policy max stays
        self.assertNotIn("exceeds maximum", out)      # no redundant phrasing

    def test_stability_note_counts_become_bands(self) -> None:
        out = sanitize_reason(
            "Disclosed: qualifying count (14) exceeded the previously disclosed "
            "scope's (8) by more than the switch margin (5); disclosure moved here."
        )
        self.assertIn("qualifying count (10-19)", out)
        self.assertIn("scope's (5-9)", out)
        self.assertIn("switch margin (5)", out)  # public policy number, kept

    def test_reason_without_a_count_is_unchanged(self) -> None:
        original = "Rejected: population n=18 below minimum privacy scope (20)"
        self.assertEqual(sanitize_reason(original), original)


if __name__ == "__main__":
    unittest.main()
```

tests/test_benchmark.py

```
"""Benchmark tests.

The dataset builder and the scoring logic are exercised on small, short
simulated runs -- enough seeds for both label classes to appear, few
enough to stay fast. The cross-regime harness is run once at a tiny size
to prove it wires together.
"""

from __future__ import annotations

import unittest

from simulation.benchmark import (
    FEATURE_NAMES,
    DayRow,
    benchmark_cross_regime,
    build_dataset,
    score_model,
    score_rows,
    score_rules,
    split_by_run,
    train_model,
)


class DatasetShapeTest(unittest.TestCase):
    def test_one_row_per_run_day_including_day_zero(self) -> None:
        rows = build_dataset([1, 2, 3], days=10)
        self.assertEqual(len(rows), 3 * 11)
        for seed in (1, 2, 3):
            days = sorted(r.day for r in rows if r.seed == seed)
            self.assertEqual(days, list(range(11)))

    def test_features_and_label_are_well_formed(self) -> None:
        for r in build_dataset([1, 2], days=10):
            self.assertIn(r.label, (0, 1))
            self.assertIsInstance(r.rule_fired, bool)
            self.assertGreaterEqual(r.relative_excess_ratio, 0.0)
            self.assertGreaterEqual(r.count_within_window, 0)
            self.assertGreaterEqual(r.count_within_window, r.count_sharing_category)

    def test_three_named_features(self) -> None:
        self.assertEqual(
            FEATURE_NAMES,
            ("relative_excess_ratio", "count_within_window", "count_sharing_category"),
        )

    def test_deterministic(self) -> None:
        self.assertEqual(build_dataset([1, 2], days=8), build_dataset([1, 2], days=8))

    def test_day_zero_is_never_an_established_outbreak(self) -> None:
        for r in build_dataset([1, 2, 3, 4, 5], days=6):
```

```
            if r.day == 0:
                self.assertEqual(r.label, 0)


class SplitByRunTest(unittest.TestCase):
    def test_no_seed_in_both_halves(self) -> None:
        rows = build_dataset(list(range(1, 11)), days=6)
        train, test = split_by_run(rows)
        self.assertTrue(train and test)
        self.assertEqual(
            set(r.seed for r in train) & set(r.seed for r in test), set()
        )

    def test_stable_given_the_split_seed(self) -> None:
        rows = build_dataset(list(range(1, 11)), days=6)
        a = split_by_run(rows, split_seed=99)
        b = split_by_run(rows, split_seed=99)
        self.assertEqual([r.seed for r in a[1]], [r.seed for r in b[1]])


class TrainingTest(unittest.TestCase):
    def test_single_class_training_set_is_rejected(self) -> None:
        rows = [DayRow(1, d, 0.0, 0, 0, 0, False) for d in range(5)]
        with self.assertRaises(ValueError):
            train_model(rows, kind="logistic")

    def test_unknown_kind_is_rejected(self) -> None:
        rows = build_dataset([1, 2, 3, 4], days=12)
        with self.assertRaises(ValueError):
            train_model(rows, kind="forest")

    def test_both_model_kinds_fit_and_score(self) -> None:
        rows = build_dataset(list(range(1, 13)), days=15)
        train, test = split_by_run(rows)
        for kind in ("logistic", "tree"):
            model = train_model(train, kind=kind)
            score = score_model(model, test, label=kind)
            self.assertGreaterEqual(score.precision, 0.0)
            self.assertLessEqual(score.precision, 1.0)
            self.assertGreaterEqual(score.recall, 0.0)
            self.assertLessEqual(score.recall, 1.0)


class ScoreRowsTest(unittest.TestCase):
    def _rows(self) -> list[DayRow]:
        # Seed 1: outbreak days 2..4. Seed 2: no outbreak.
        return [
            DayRow(1, 0, 0.0, 0, 0, 0, False),
            DayRow(1, 1, 0.0, 0, 0, 0, False),
            DayRow(1, 2, 0.0, 0, 0, 1, False),
            DayRow(1, 3, 0.0, 0, 0, 1, False),
            DayRow(1, 4, 0.0, 0, 0, 1, False),
            DayRow(2, 0, 0.0, 0, 0, 0, False),
            DayRow(2, 1, 0.0, 0, 0, 0, False),
        ]
```

```
    def test_perfect_prediction(self) -> None:
        rows = self._rows()
        preds = [r.label for r in rows]
        s = score_rows(rows, preds, label="perfect")
        self.assertEqual((s.precision, s.recall), (1.0, 1.0))
        self.assertEqual(s.mean_detection_delay, 0.0)
        self.assertEqual((s.n_runs_with_outbreak, s.n_runs_detected), (1, 1))

    def test_late_and_missed_detection(self) -> None:
        rows = self._rows()
        # fire only on day 4 for seed 1 -> delay 2; never for seed 2 (fine).
        preds = [1 if (r.seed == 1 and r.day == 4) else 0 for r in rows]
        s = score_rows(rows, preds, label="late")
        self.assertEqual(s.mean_detection_delay, 2.0)
        self.assertEqual(s.never_detected_runs, 0)

    def test_never_detected_run_is_counted(self) -> None:
        rows = self._rows()
        s = score_rows(rows, [0] * len(rows), label="silent")
        self.assertEqual(s.never_detected_runs, 1)
        self.assertIsNone(s.mean_detection_delay)
        self.assertEqual(s.recall, 0.0)

    def test_score_rules_uses_the_rule_fired_flag(self) -> None:
        rows = [
            DayRow(1, 0, 0.0, 0, 0, 0, False),
            DayRow(1, 1, 0.0, 0, 0, 1, True),
        ]
        s = score_rules(rows)
        self.assertEqual(s.n_predicted_positive, 1)
        self.assertEqual(s.recall, 1.0)


class CrossRegimeHarnessTest(unittest.TestCase):
    def test_runs_end_to_end_at_a_tiny_size(self) -> None:
        results = benchmark_cross_regime(
            train_seeds=list(range(1, 11)),
            test_seeds=list(range(11, 17)),
            days=15,
            test_suite_transmissions=(0.012,),
        )
        self.assertEqual(len(results), 1)
        res = results[0]
        self.assertEqual(len(res.scores), 3)
        self.assertEqual(res.scores[0].label, "authored rules")
        self.assertGreater(res.n_test_rows, 0)


if __name__ == "__main__":
    unittest.main()
```

tests/test_detection.py

```
  1 | """Detection engine tests -- independent of the disclosure engine."""
  2 |
  3 | from __future__ import annotations
  4 |
  5 | import math
  6 | import unittest
  7 |
  8 | from microcluster import detection
  9 | from microcluster.config import DetectionConfig
 10 | from microcluster.detection import HysteresisState
 11 | from microcluster.models import Category
 12 |
 13 | from tests.fixtures import NOW, report, reports, single_scope_registry, standard_regist
ry
 14 |
 15 |
 16 | class LocalizedClusterTest(unittest.TestCase):
 17 |     """A burst in one suite: the relative detector fires, the absolute
 18 |     detector does not."""
 19 |
 20 |     def setUp(self) -> None:
 21 |         self.reg = standard_registry()
 22 |         # All reports at suite level (see the mixed-level assumption in
 23 |         # detection.evaluate): the cluster in S1, scattered peers in the
 24 |         # sibling suites.
 25 |         self.reports = (
 26 |             reports("S1", 9, category=Category.RESPIRATORY)
 27 |             + reports("S2", 2, category=Category.GENERAL)
 28 |             + reports("S3", 1, category=Category.GASTROINTESTINAL)
 29 |             + reports("S4", 1, category=Category.OTHER)
 30 |             + reports("S5", 1, category=Category.GENERAL)
 31 |             # Old reports in the same suite: outside the 72h window, ignored.
 32 |             + reports("S1", 6, category=Category.RESPIRATORY, hours_ago=500)
 33 |         )
 34 |         self.result = detection.evaluate(self.reports, self.reg, now=NOW)
 35 |
 36 |     def test_relative_fires_absolute_does_not(self) -> None:
 37 |         self.assertTrue(self.result.relative_detector_fired)
 38 |         self.assertFalse(self.result.absolute_detector_fired)
 39 |         self.assertTrue(self.result.fired)
 40 |
 41 |     def test_three_findings_reported_plainly(self) -> None:
 42 |         f = self.result.findings
 43 |         self.assertEqual(f.count_within_window, 14)
 44 |         self.assertEqual(f.count_sharing_category, 9)
 45 |         self.assertGreater(f.relative_excess_ratio, 2.0)
 46 |
 47 |     def test_hotspot_and_dominant_category(self) -> None:
 48 |         self.assertEqual(self.result.hotspot_location_id, "S1")
 49 |         self.assertEqual(self.result.dominant_category, Category.RESPIRATORY)
 50 |
 51 |     def test_absolute_expectation_is_from_background_rate(self) -> None:
```

```
        # 0.02 * 2000 declared campus population.
        self.assertAlmostEqual(self.result.absolute_expected, 40.0)

    def test_aggregate_for_disclosure_is_per_scope_counts_only(self) -> None:
        self.assertEqual(
            self.result.qualifying_scope_counts,
            {"S1": 9, "F1": 9, "B1": 9, "C": 9},
        )


class CampusWideRiseTest(unittest.TestCase):
    """An even rise across every building: the absolute detector fires,
    the relative detector does not (no location stands out)."""

    def setUp(self) -> None:
        self.reg = standard_registry()
        self.reports = []
        for b in ("B1", "B2", "B3", "B4", "B5"):
            self.reports += reports(b, 14, category=Category.RESPIRATORY)
        self.result = detection.evaluate(self.reports, self.reg, now=NOW)

    def test_absolute_fires_relative_does_not(self) -> None:
        self.assertTrue(self.result.absolute_detector_fired)
        self.assertFalse(self.result.relative_detector_fired)
        self.assertTrue(self.result.fired)

    def test_relative_excess_ratio_is_about_one(self) -> None:
        # Every building reports at the same per-capita rate, so no
        # location stands out: the smoothed ratio sits close to 1.
        ratio = self.result.findings.relative_excess_ratio
        self.assertGreater(ratio, 0.9)
        self.assertLess(ratio, 1.15)

    def test_counts(self) -> None:
        self.assertEqual(self.result.findings.count_within_window, 70)
        self.assertEqual(self.result.findings.count_sharing_category, 70)
        self.assertEqual(self.result.qualifying_scope_counts["C"], 70)
        for b in ("B1", "B2", "B3", "B4", "B5"):
            self.assertEqual(self.result.qualifying_scope_counts[b], 14)


class ScatteredNoiseTest(unittest.TestCase):
    """Low volume, spread across locations and categories: neither
    detector fires."""

    def setUp(self) -> None:
        self.reg = standard_registry()
        # Uniform building level; three reports per building, no category
        # concentrated anywhere.
        self.reports = (
            reports("B1", 3, category=Category.GENERAL)
            + reports("B2", 3, category=Category.RESPIRATORY)
            + reports("B3", 3, category=Category.GASTROINTESTINAL)
            + reports("B4", 3, category=Category.OTHER)
            + reports("B5", 3, category=Category.GENERAL)
```

```
        )
        self.result = detection.evaluate(self.reports, self.reg, now=NOW)

    def test_neither_detector_fires(self) -> None:
        self.assertFalse(self.result.relative_detector_fired)
        self.assertFalse(self.result.absolute_detector_fired)
        self.assertFalse(self.result.fired)


class DetectorEdgeCaseTest(unittest.TestCase):
    def test_zero_reports(self) -> None:
        reg = standard_registry()
        result = detection.evaluate([], reg, now=NOW)
        self.assertFalse(result.fired)
        self.assertFalse(result.relative_detector_fired)
        self.assertFalse(result.absolute_detector_fired)
        self.assertIsNone(result.dominant_category)
        self.assertIsNone(result.hotspot_location_id)
        self.assertEqual(result.findings.count_within_window, 0)
        self.assertEqual(result.findings.count_sharing_category, 0)
        self.assertEqual(result.findings.relative_excess_ratio, 0.0)
        self.assertEqual(result.qualifying_scope_counts, {})

    def test_reports_outside_window_are_excluded(self) -> None:
        reg = standard_registry()
        old = reports("S1", 9, category=Category.RESPIRATORY, hours_ago=200)
        result = detection.evaluate(old, reg, now=NOW)
        self.assertEqual(result.findings.count_within_window, 0)
        self.assertIsNone(result.dominant_category)
        self.assertFalse(result.fired)
        self.assertEqual(result.qualifying_scope_counts, {})

    def test_relative_detector_has_a_minimum_cluster_size(self) -> None:
        # Two reports in a tiny suite produce a huge ratio but must not
        # fire -- MIN_RELATIVE_CLUSTER_REPORTS is 3.
        reg = standard_registry()
        result = detection.evaluate(
            reports("S1", 2, category=Category.RESPIRATORY), reg, now=NOW
        )
        self.assertGreater(result.findings.relative_excess_ratio, 2.0)
        self.assertFalse(result.relative_detector_fired)

    def test_location_that_is_the_whole_campus_cannot_fire_relative(self) -> None:
        reg = standard_registry()
        result = detection.evaluate(
            reports("C", 50, category=Category.RESPIRATORY), reg, now=NOW
        )
        # No peers to compare against -> relative ratio stays 0, not inf.
        self.assertEqual(result.findings.relative_excess_ratio, 0.0)
        self.assertFalse(result.relative_detector_fired)
        # Absolute still works: 50 >= 1.5 * (0.02 * 2000) = 60 is False here.
        self.assertFalse(result.absolute_detector_fired)

    def test_unknown_location_is_rejected(self) -> None:
        reg = standard_registry()
```

```
        bad = [report("does-not-exist")]
        with self.assertRaises(KeyError):
            detection.evaluate(bad, reg, now=NOW)

    def test_relative_ratio_stays_finite_when_peers_are_silent(self) -> None:
        # Smoothing keeps the ratio finite even when the control rate
        # would otherwise be exactly zero.
        reg = standard_registry()
        result = detection.evaluate(
            reports("S1", 5, category=Category.RESPIRATORY), reg, now=NOW
        )
        ratio = result.findings.relative_excess_ratio
        self.assertTrue(math.isfinite(ratio))
        self.assertGreater(ratio, 2.0)
        self.assertTrue(result.relative_detector_fired)

    def test_smoothing_constant_damps_the_ratio(self) -> None:
        # Same data, larger smoothing constant -> ratio pulled toward 1.
        reg = standard_registry()
        data = reports("S1", 9, category=Category.RESPIRATORY)
        light = detection.evaluate(
            data, reg, now=NOW, config=DetectionConfig(relative_smoothing=1.0)
        ).findings.relative_excess_ratio
        heavy = detection.evaluate(
            data, reg, now=NOW, config=DetectionConfig(relative_smoothing=25.0)
        ).findings.relative_excess_ratio
        self.assertLess(heavy, light)


class AbsoluteDetectorCategoryScopingTest(unittest.TestCase):
    """The absolute detector counts only dominant-category reports, so a
    spike made of several unrelated illnesses does not fire it."""

    def test_mixed_category_volume_spike_does_not_fire_absolute(self) -> None:
        reg = standard_registry()
        # 70 within-window reports campus-wide -- above the raw-volume
        # threshold of 1.5 * (0.02 * 2000) = 60 -- but split evenly
        # between two unrelated categories, so the dominant-category
        # count is only 35.
        data = []
        for b in ("B1", "B2", "B3", "B4", "B5"):
            data += reports(b, 7, category=Category.RESPIRATORY)
            data += reports(b, 7, category=Category.GASTROINTESTINAL)
        result = detection.evaluate(data, reg, now=NOW)

        self.assertEqual(result.findings.count_within_window, 70)
        self.assertFalse(result.absolute_detector_fired)
        self.assertFalse(result.relative_detector_fired)
        self.assertFalse(result.fired)

    def test_single_category_spike_of_the_same_size_does_fire_absolute(self) -> None:
        reg = standard_registry()
        data = []
        for b in ("B1", "B2", "B3", "B4", "B5"):
            data += reports(b, 14, category=Category.RESPIRATORY)
```

```
        result = detection.evaluate(data, reg, now=NOW)

        self.assertEqual(result.findings.count_within_window, 70)
        self.assertTrue(result.absolute_detector_fired)


class RelativeDetectorClusterMinimumIsCategoryScopedTest(unittest.TestCase):
    """The minimum-cluster-size gate counts only the hotspot's
    dominant-category reports."""

    def test_location_with_three_different_categories_fails_the_minimum(self) -> None:
        reg = standard_registry()
        # One suite, three reports, three categories. Volume ratio is
        # high, but no single category reaches
        # MIN_RELATIVE_CLUSTER_REPORTS (3).
        data = (
            reports("S1", 1, category=Category.RESPIRATORY)
            + reports("S1", 1, category=Category.GASTROINTESTINAL)
            + reports("S1", 1, category=Category.GENERAL)
        )
        result = detection.evaluate(data, reg, now=NOW)

        self.assertEqual(result.findings.count_within_window, 3)
        self.assertGreater(result.findings.relative_excess_ratio, 2.0)
        self.assertFalse(result.relative_detector_fired)
        self.assertFalse(result.fired)

    def test_same_location_one_category_meets_the_minimum(self) -> None:
        reg = standard_registry()
        data = reports("S1", 3, category=Category.RESPIRATORY)
        result = detection.evaluate(data, reg, now=NOW)

        self.assertGreater(result.findings.relative_excess_ratio, 2.0)
        self.assertTrue(result.relative_detector_fired)


class MixedLevelReportsAssumptionTest(unittest.TestCase):
    """evaluate() enforces that all reports resolve to one ScopeLevel."""

    def test_mixed_level_reports_are_rejected(self) -> None:
        reg = standard_registry()
        data = (
            reports("S1", 3, category=Category.RESPIRATORY)  # SUITE
            + reports("B2", 3, category=Category.RESPIRATORY)  # BUILDING
        )
        with self.assertRaises(ValueError) as ctx:
            detection.evaluate(data, reg, now=NOW)
        self.assertIn("same scope level", str(ctx.exception))

    def test_uniform_level_reports_are_accepted(self) -> None:
        reg = standard_registry()
        data = reports("S1", 3, category=Category.RESPIRATORY) + reports(
            "S2", 3, category=Category.RESPIRATORY
        )
        detection.evaluate(data, reg, now=NOW)  # does not raise
```

```
|
|     def test_level_check_sees_reports_outside_the_window_too(self) -> None:
|         # The old report is outside the 72h window and would be ignored by
|         # both detectors, but it still counts for the level check.
|         reg = standard_registry()
|         data = reports("S1", 3, category=Category.RESPIRATORY) + reports(
|             "B2", 1, category=Category.RESPIRATORY, hours_ago=500
|         )
|         with self.assertRaises(ValueError):
|             detection.evaluate(data, reg, now=NOW)
|
|
| class HysteresisTest(unittest.TestCase):
|     """Detection hysteresis (config.HYSTERESIS_MIN_FIRED_DAYS /
|     HYSTERESIS_RELEASE_THRESHOLD), threaded via HysteresisState across a
|     sequence of evaluate() calls.
|
|     A campus-only registry isolates these to the absolute detector alone:
|     with a single scope, the relative detector has no peers and can never
|     fire (see DetectorEdgeCaseTest), so ``raw_fired`` here is exactly
|     ``absolute_detector_fired``, driven entirely by how many reports we
|     put in the window.
|     """
|
|     def setUp(self) -> None:
|         self.reg = single_scope_registry(2000)  # expected = 0.02*2000 = 40
|         self.config = DetectionConfig(
|             background_rate=0.02,
|             absolute_excess_ratio=1.5,  # fires at >= 60
|             hysteresis_min_fired_days=3,
|             hysteresis_release_threshold=3,
|         )
|
|     def _evaluate(self, count: int, state: HysteresisState | None):
|         data = reports("C", count, category=Category.RESPIRATORY, hours_ago=0, now=NOW)
|         return detection.evaluate(
|             data, self.reg, now=NOW, config=self.config, hysteresis_state=state
|         )
|
|     def test_single_call_with_no_state_is_unaffected_by_hysteresis(self) -> None:
|         # 70 >= 60 -> raw fires; no prior state -> fired == raw_fired.
|         fired_result = self._evaluate(70, None)
|         self.assertTrue(fired_result.raw_fired)
|         self.assertEqual(fired_result.fired, fired_result.raw_fired)
|
|         # 10 < 60 -> raw does not fire; still no prior state -> False.
|         quiet_result = self._evaluate(10, None)
|         self.assertFalse(quiet_result.raw_fired)
|         self.assertEqual(quiet_result.fired, quiet_result.raw_fired)
|
|     def test_quiet_stretch_with_no_prior_activation_never_fires(self) -> None:
|         state = None
|         for _ in range(4):
|             result = self._evaluate(5, state)
|             self.assertFalse(result.fired)
```

```
327 |             self.assertFalse(result.hysteresis_state.active)
328 |             state = result.hysteresis_state
329 |
330 |     def test_hold_continues_while_raw_still_fires(self) -> None:
331 |         state = None
332 |         result = self._evaluate(70, state)  # raw fires
333 |         self.assertTrue(result.fired)
334 |         self.assertEqual(result.hysteresis_state, HysteresisState(active=True, consecut
ive_days=1))
335 |
336 |         state = result.hysteresis_state
337 |         result = self._evaluate(65, state)  # still raw-fires
338 |         self.assertTrue(result.fired)
339 |         self.assertEqual(result.hysteresis_state, HysteresisState(active=True, consecut
ive_days=2))
340 |
341 |     def test_hold_survives_a_raw_dip_until_the_minimum_is_served(self) -> None:
342 |         # Day 0: raw fires (70 >= 60).
343 |         state = None
344 |         result = self._evaluate(70, state)
345 |         self.assertTrue(result.fired)
346 |         state = result.hysteresis_state
347 |
348 |         # Day 1: raw fires again (65 >= 60); streak now 2.
349 |         result = self._evaluate(65, state)
350 |         self.assertTrue(result.fired)
351 |         self.assertEqual(result.hysteresis_state.consecutive_days, 2)
352 |         state = result.hysteresis_state
353 |
354 |         # Day 2: raw does NOT fire (50 < 60), but the window is nowhere
355 |         # near the release threshold (3) and the minimum hold (3 days)
356 |         # has not yet been served (streak is only 2) -> HELD fired=True.
357 |         result = self._evaluate(50, state)
358 |         self.assertFalse(result.raw_fired)
359 |         self.assertTrue(result.fired, "hysteresis should have held fired=True")
360 |         self.assertEqual(result.hysteresis_state, HysteresisState(active=True, consecut
ive_days=3))
361 |         state = result.hysteresis_state
362 |
363 |         # Day 3: raw still does not fire (45 < 60); the minimum (3) has
364 |         # now been served -> releases on schedule.
365 |         result = self._evaluate(45, state)
366 |         self.assertFalse(result.raw_fired)
367 |         self.assertFalse(result.fired, "hysteresis should have released on schedule")
368 |         self.assertEqual(result.hysteresis_state, HysteresisState(active=False, consecu
tive_days=0))
369 |
370 |     def test_release_threshold_overrides_the_minimum_hold(self) -> None:
371 |         # Day 0: raw fires (70 >= 60); streak = 1, well under the minimum.
372 |         state = None
373 |         result = self._evaluate(70, state)
374 |         self.assertTrue(result.fired)
375 |         state = result.hysteresis_state
376 |
377 |         # Day 1: the window has nearly emptied out (2 <= release threshold
```

```
        # of 3) -- releases IMMEDIATELY, even though the minimum hold (3)
        # was never served.
        result = self._evaluate(2, state)
        self.assertFalse(result.raw_fired)
        self.assertFalse(result.fired, "a near-empty window should release early")
        self.assertEqual(result.hysteresis_state, HysteresisState(active=False, consecu
tive_days=0))

    def test_a_fresh_raw_fire_restarts_the_streak_at_one(self) -> None:
        state = HysteresisState(active=False, consecutive_days=0)
        result = self._evaluate(70, state)
        self.assertEqual(result.hysteresis_state, HysteresisState(active=True, consecut
ive_days=1))


if __name__ == "__main__":
    unittest.main()
```

tests/test_disclosure.py

```
  1 | """Disclosure engine tests -- independent of the detection engine.
  2 |
  3 | The disclosure engine is fed aggregate per-scope counts directly, exactly
  4 | as the detection engine would hand them over (rule 10).
  5 | """
  6 |
  7 | from __future__ import annotations
  8 |
  9 | import unittest
 10 |
 11 | from microcluster import disclosure
 12 | from microcluster.config import DisclosureConfig
 13 | from microcluster.models import Scope, ScopeLevel, ScopeRegistry
 14 |
 15 | from tests.fixtures import (
 16 |     roster_registry,
 17 |     single_scope_registry,
 18 |     small_registry_floor18,
 19 |     standard_registry,
 20 | )
 21 |
 22 |
 23 | class StatisticalThresholdTest(unittest.TestCase):
 24 |     def test_threshold_scales_up_with_population(self) -> None:
 25 |         expected = {12: 5, 18: 5, 40: 7, 100: 10, 500: 23, 2000: 45}
 26 |         for n, threshold in expected.items():
 27 |             self.assertEqual(disclosure.statistical_threshold(n), threshold, msg=f"n=
{n}")
 28 |
 29 |     def test_floor_applies_to_small_populations(self) -> None:
 30 |         # ceil(sqrt(4)) == 2, but the floor is 5.
 31 |         self.assertEqual(disclosure.statistical_threshold(4), 5)
 32 |
 33 |     def test_perfect_squares_are_exact(self) -> None:
 34 |         self.assertEqual(disclosure.statistical_threshold(10_000), 100)
 35 |
 36 |
 37 | class BothGatesAcrossPopulationsTest(unittest.TestCase):
 38 |     """n in {12, 18, 40, 100, 500, 2000}: the statistical threshold rises
 39 |     with n, while the privacy minimum rejects the two smallest scopes."""
 40 |
 41 |     POPULATIONS = (12, 18, 40, 100, 500, 2000)
 42 |     STAT_THRESHOLD = {12: 5, 18: 5, 40: 7, 100: 10, 500: 23, 2000: 45}
 43 |
 44 |     def test_qualifying_at_exactly_the_statistical_threshold(self) -> None:
 45 |         for n in self.POPULATIONS:
 46 |             q = self.STAT_THRESHOLD[n]
 47 |             result = disclosure.evaluate(single_scope_registry(n), {"C": q})
 48 |             ev = result.evaluations[0]
 49 |             with self.subTest(n=n):
 50 |                 self.assertEqual(ev.statistical_threshold, q)
 51 |                 self.assertTrue(ev.statistical_pass)
```

```
                # Privacy minimum (20) rejects n=12 and n=18.
                if n < 20:
                    self.assertFalse(ev.min_population_pass)
                    self.assertFalse(ev.privacy_pass)
                    self.assertFalse(ev.disclosure_eligible)
                    self.assertIsNone(result.disclosed_scope_id)
                    self.assertIn("below minimum privacy scope", ev.disclosure_reason)
                else:
                    self.assertTrue(ev.min_population_pass)
                    self.assertTrue(ev.report_fraction_pass)
                    self.assertTrue(ev.privacy_pass)
                    self.assertTrue(ev.disclosure_eligible)
                    self.assertEqual(result.disclosed_scope_id, "C")

    def test_report_fraction_gate_rejects_a_dense_small_scope(self) -> None:
        # n=40, 25 qualifying reports: statistical gate passes (25 >= 7),
        # but 25/40 = 0.625 > MAX_REPORT_FRACTION (0.5).
        result = disclosure.evaluate(single_scope_registry(40), {"C": 25})
        ev = result.evaluations[0]
        self.assertTrue(ev.statistical_pass)
        self.assertTrue(ev.min_population_pass)
        self.assertFalse(ev.report_fraction_pass)
        self.assertFalse(ev.privacy_pass)
        self.assertFalse(ev.disclosure_eligible)
        self.assertIsNone(result.disclosed_scope_id)
        self.assertIn("report fraction", ev.disclosure_reason)

    def test_gates_pull_in_opposite_directions(self) -> None:
        # Same qualifying count (10). Small scope fails statistics-free but
        # privacy; large scope fails statistics. Only the middle passes.
        small = disclosure.evaluate(single_scope_registry(15), {"C": 10}).evaluations
[0]
        mid = disclosure.evaluate(single_scope_registry(60), {"C": 10}).evaluations[0]
        large = disclosure.evaluate(single_scope_registry(400), {"C": 10}).evaluations
[0]

        self.assertFalse(small.privacy_pass)      # n=15 < 20
        self.assertTrue(small.statistical_pass)   # 10 >= 5

        self.assertTrue(mid.disclosure_eligible)  # 10 >= 8 and privacy ok

        self.assertFalse(large.statistical_pass)  # 10 < 20
        self.assertTrue(large.privacy_pass)       # n=400, fraction tiny


class OverclaimRefusalTest(unittest.TestCase):
    """7 qualifying reports, campus n=2000: the statistical gate fails
    (7 < 45) at every scope, so nothing is disclosed."""

    def test_campus_scope_fails_statistical_gate(self) -> None:
        reg = standard_registry()
        result = disclosure.evaluate(reg, {"B1": 7, "C": 7})

        campus = next(ev for ev in result.evaluations if ev.scope_id == "C")
        self.assertEqual(campus.statistical_threshold, 45)
```

```
        self.assertEqual(campus.qualifying_reports, 7)
        self.assertFalse(campus.statistical_pass)
        self.assertFalse(campus.disclosure_eligible)
        self.assertIn("statistical gate failed", campus.disclosure_reason)

        self.assertIsNone(result.disclosed_scope_id)


class PrivacyRefusalTest(unittest.TestCase):
    """5 qualifying reports, floor n=18: statistical gate passes (5 >= 5)
    but the privacy gate fails (18 < 20). A coarser scope is disclosed."""

    def setUp(self) -> None:
        self.reg = small_registry_floor18()
        self.result = disclosure.evaluate(self.reg, {"F": 5, "B": 5, "C": 5})

    def test_floor_passes_statistics_but_fails_privacy(self) -> None:
        floor = next(ev for ev in self.result.evaluations if ev.scope_id == "F")
        self.assertEqual(floor.statistical_threshold, 5)
        self.assertTrue(floor.statistical_pass)
        self.assertFalse(floor.min_population_pass)
        self.assertFalse(floor.privacy_pass)
        self.assertFalse(floor.disclosure_eligible)
        self.assertFalse(floor.selected)
        self.assertIn("below minimum privacy scope", floor.disclosure_reason)

    def test_coarser_scope_is_disclosed_instead(self) -> None:
        self.assertEqual(self.result.disclosed_scope_id, "B")
        building = self.result.disclosed()
        self.assertIsNotNone(building)
        self.assertEqual(building.scope_id, "B")
        self.assertTrue(building.selected)
        self.assertTrue(building.disclosure_eligible)

    def test_campus_scope_now_fails_statistics(self) -> None:
        campus = next(ev for ev in self.result.evaluations if ev.scope_id == "C")
        self.assertFalse(campus.statistical_pass)  # ceil(sqrt(300)) == 18 > 5


class RosterRefusalTest(unittest.TestCase):
    """~15 qualifying reports in a declared group of n=25: clears the
    statistical gate and the population minimum, but 15/25 = 0.60 exceeds
    MAX_REPORT_FRACTION -- naming the group would be naming its members."""

    def setUp(self) -> None:
        self.reg = roster_registry()
        self.result = disclosure.evaluate(self.reg, {"G": 15, "C": 15})
        self.group = next(ev for ev in self.result.evaluations if ev.scope_id == "G")

    def test_passes_statistical_gate(self) -> None:
        self.assertEqual(self.group.statistical_threshold, 5)
        self.assertTrue(self.group.statistical_pass)

    def test_passes_population_minimum(self) -> None:
        self.assertTrue(self.group.min_population_pass)  # 25 >= 20
```

```
    def test_refused_on_report_fraction(self) -> None:
        self.assertAlmostEqual(self.group.report_fraction, 0.60)
        self.assertFalse(self.group.report_fraction_pass)
        self.assertFalse(self.group.privacy_pass)
        self.assertFalse(self.group.disclosure_eligible)

    def test_disclosure_reason_names_the_fraction_ceiling_specifically(self) -> None:
        reason = self.group.disclosure_reason
        self.assertIn("report fraction 0.60 exceeds maximum 0.50", reason)
        self.assertIn("roster", reason)

    def test_nothing_is_disclosed(self) -> None:
        self.assertIsNone(self.result.disclosed_scope_id)


class EvaluationTableTest(unittest.TestCase):
    def test_every_candidate_scope_gets_a_record_including_rejected(self) -> None:
        reg = standard_registry()
        result = disclosure.evaluate(reg, {"S1": 9, "F1": 9, "B1": 9, "C": 9})
        ids = [ev.scope_id for ev in result.evaluations]
        self.assertEqual(set(ids), {"S1", "F1", "B1", "C"})

    def test_records_are_ordered_finest_scope_first(self) -> None:
        reg = standard_registry()
        result = disclosure.evaluate(reg, {"S1": 9, "F1": 9, "B1": 9, "C": 9})
        levels = [ev.scope_level for ev in result.evaluations]
        self.assertEqual(
            levels,
            [ScopeLevel.SUITE, ScopeLevel.FLOOR, ScopeLevel.BUILDING, ScopeLevel.CAMPU
S],
        )

    def test_finest_eligible_scope_wins_and_others_are_marked_not_selected(self) -> Non
e:
        reg = standard_registry()
        # Enough reports for the suite AND the floor to pass both gates.
        result = disclosure.evaluate(reg, {"S1": 12, "F1": 12, "B1": 12, "C": 12})
        selected = [ev.scope_id for ev in result.evaluations if ev.selected]
        self.assertEqual(selected, ["S1"])
        floor = next(ev for ev in result.evaluations if ev.scope_id == "F1")
        self.assertTrue(floor.disclosure_eligible)
        self.assertFalse(floor.selected)
        self.assertIn("a finer scope was disclosed", floor.disclosure_reason)

    def test_winner_is_selected_from_the_table(self) -> None:
        reg = standard_registry()
        result = disclosure.evaluate(reg, {"S1": 9, "F1": 9, "B1": 9, "C": 9})
        self.assertIs(result.disclosed(), next(e for e in result.evaluations if e.selec
ted))

    def test_empty_count_table_discloses_nothing(self) -> None:
        reg = standard_registry()
        result = disclosure.evaluate(reg, {})
        self.assertEqual(result.evaluations, ())
```

```
        self.assertIsNone(result.disclosed_scope_id)
        self.assertIsNone(result.disclosed())

    def test_nothing_disclosed_yields_no_selection_but_full_table(self) -> None:
        reg = standard_registry()
        result = disclosure.evaluate(reg, {"B1": 3, "C": 3})
        self.assertIsNone(result.disclosed_scope_id)
        self.assertIsNone(result.disclosed())
        self.assertEqual(len(result.evaluations), 2)
        self.assertTrue(all(not ev.selected for ev in result.evaluations))


class ConfigurableThresholdsTest(unittest.TestCase):
    def test_max_report_fraction_is_configurable(self) -> None:
        strict = DisclosureConfig(max_report_fraction=0.1)
        result = disclosure.evaluate(single_scope_registry(100), {"C": 15}, config=stri
ct)
        ev = result.evaluations[0]
        self.assertTrue(ev.statistical_pass)     # 15 >= 10
        self.assertFalse(ev.report_fraction_pass)  # 0.15 > 0.10
        self.assertIsNone(result.disclosed_scope_id)

    def test_min_scope_population_is_configurable(self) -> None:
        lax = DisclosureConfig(min_scope_population=10)
        result = disclosure.evaluate(single_scope_registry(15), {"C": 6}, config=lax)
        ev = result.evaluations[0]
        self.assertTrue(ev.min_population_pass)   # 15 >= 10 now
        self.assertTrue(ev.disclosure_eligible)
        self.assertEqual(result.disclosed_scope_id, "C")


class ScopeStabilityTest(unittest.TestCase):
    """config.SIBLING_SWITCH_MARGIN: once a scope has been disclosed,
    evaluate() prefers to keep disclosing it (or a coarser ancestor of it)
    across a SEQUENCE of calls, threaded via ``prior_disclosed_scope_id``.
    A single call with no prior id is unaffected (all other disclosure
    tests exercise exactly that default path).
    """

    def setUp(self) -> None:
        self.reg = standard_registry()  # S1, S2 are sibling suites under F1

    def test_stays_at_the_prior_scope_when_a_sibling_is_within_the_margin(self) -> Non
e:
        # S2's count (11) exceeds S1's (6) by only 5 -- not MORE than the
        # margin (5) -- so disclosure must stay at S1.
        counts = {"S1": 6, "S2": 11, "F1": 6, "B1": 6, "C": 6}
        result = disclosure.evaluate(self.reg, counts, prior_disclosed_scope_id="S1")

        self.assertEqual(result.disclosed_scope_id, "S1")
        s1 = next(e for e in result.evaluations if e.scope_id == "S1")
        self.assertIn("kept disclosing the same scope", s1.disclosure_reason)

    def test_switches_only_once_a_sibling_clears_the_margin(self) -> None:
        # S2's count (12) exceeds S1's (6) by 6 -- MORE than the margin
```

```
        # (5) -- so disclosure is allowed to move.
        counts = {"S1": 6, "S2": 12, "F1": 6, "B1": 6, "C": 6}
        result = disclosure.evaluate(self.reg, counts, prior_disclosed_scope_id="S1")

        self.assertEqual(result.disclosed_scope_id, "S2")
        s2 = next(e for e in result.evaluations if e.scope_id == "S2")
        self.assertIn("exceeded the previously disclosed scope", s2.disclosure_reason)
        self.assertIn("switch margin", s2.disclosure_reason)

    def test_never_silently_hops_between_siblings_across_a_sequence(self) -> None:
        # Simulate three consecutive days. Day 1 names S1. Day 2, S2 briefly
        # edges ahead but not past the margin -- must NOT switch. Day 3, S2
        # pulls decisively ahead -- switching is now allowed.
        day1 = disclosure.evaluate(
            self.reg, {"S1": 6, "F1": 6, "B1": 6, "C": 6}, prior_disclosed_scope_id=Non
e
        )
        self.assertEqual(day1.disclosed_scope_id, "S1")

        day2 = disclosure.evaluate(
            self.reg,
            {"S1": 6, "S2": 10, "F1": 6, "B1": 6, "C": 6},
            prior_disclosed_scope_id=day1.disclosed_scope_id,
        )
        self.assertEqual(day2.disclosed_scope_id, "S1")

        day3 = disclosure.evaluate(
            self.reg,
            {"S1": 6, "S2": 12, "F1": 6, "B1": 6, "C": 6},
            prior_disclosed_scope_id=day2.disclosed_scope_id,
        )
        self.assertEqual(day3.disclosed_scope_id, "S2")

    def test_falls_back_to_a_coarser_ancestor_when_the_prior_scope_drops_out(self) -> N
one:
        # S1 no longer clears its own statistical gate; its ancestor F1
        # does. A different, unrelated suite (S2) is ALSO eligible with a
        # smaller count than F1 -- disclosure must still prefer F1 (S1's
        # own ancestor) over hopping to the unrelated sibling S2.
        counts = {"S1": 3, "S2": 8, "F1": 15, "B1": 15, "C": 15}
        result = disclosure.evaluate(self.reg, counts, prior_disclosed_scope_id="S1")

        self.assertEqual(result.disclosed_scope_id, "F1")
        f1 = next(e for e in result.evaluations if e.scope_id == "F1")
        self.assertIn("no longer qualifies", f1.disclosure_reason)
        self.assertIn("coarser ancestor", f1.disclosure_reason)

    def test_no_prior_scope_ignores_stability_entirely(self) -> None:
        # Same counts as the "switches" test, but with no prior id: plain
        # finest-eligible selection (rule 9) -- S1 and S2 are both SUITE
        # level with the same declared population, so the qualifying count
        # is irrelevant to the tie-break; S1 wins on scope id alone,
        # exactly as every other disclosure test in this module exercises.
        counts = {"S1": 6, "S2": 12, "F1": 6, "B1": 6, "C": 6}
        result = disclosure.evaluate(self.reg, counts)
```

```
        self.assertEqual(result.disclosed_scope_id, "S1")


if __name__ == "__main__":
    unittest.main()
```

tests/test_engine.py

```
"""End-to-end tests: detection wired to disclosure (rule 10 ordering)."""

from __future__ import annotations

import unittest

from microcluster.engine import analyze
from microcluster.models import Category

from tests.fixtures import (
    NOW,
    reports,
    roster_registry,
    small_registry_floor18,
    standard_registry,
)


class LocalizedClusterEndToEndTest(unittest.TestCase):
    def setUp(self) -> None:
        reg = standard_registry()
        # Uniform suite level (see detection.evaluate's mixed-level rule).
        data = (
            reports("S1", 9, category=Category.RESPIRATORY)
            + reports("S2", 2, category=Category.GENERAL)
            + reports("S3", 1, category=Category.GASTROINTESTINAL)
            + reports("S4", 1, category=Category.OTHER)
            + reports("S5", 1, category=Category.GENERAL)
        )
        self.result = analyze(data, reg, now=NOW)

    def test_discloses_the_suite(self) -> None:
        self.assertEqual(self.result.disclosed_scope_id, "S1")
        disclosed = self.result.disclosure.disclosed()
        self.assertEqual(disclosed.scope_id, "S1")
        self.assertTrue(disclosed.disclosure_eligible)
        self.assertEqual(disclosed.qualifying_reports, 9)

    def test_table_includes_the_rejected_coarser_scopes(self) -> None:
        ids = {ev.scope_id for ev in self.result.disclosure.evaluations}
        self.assertEqual(ids, {"S1", "F1", "B1", "C"})
        rejected = [ev for ev in self.result.disclosure.evaluations if not ev.selected]
        self.assertTrue(all(not ev.disclosure_eligible for ev in rejected))


class CampusWideEndToEndTest(unittest.TestCase):
    def test_discloses_the_campus(self) -> None:
        reg = standard_registry()
        data = []
        for b in ("B1", "B2", "B3", "B4", "B5"):
            data += reports(b, 14, category=Category.RESPIRATORY)
        result = analyze(data, reg, now=NOW)
```

```
        self.assertTrue(result.detection.absolute_detector_fired)
        self.assertFalse(result.detection.relative_detector_fired)
        self.assertEqual(result.disclosed_scope_id, "C")


class ScatteredNoiseEndToEndTest(unittest.TestCase):
    def test_detects_nothing_and_discloses_nothing(self) -> None:
        reg = standard_registry()
        data = (
            reports("B1", 3, category=Category.GENERAL)
            + reports("B2", 3, category=Category.RESPIRATORY)
            + reports("B3", 3, category=Category.GASTROINTESTINAL)
            + reports("B4", 3, category=Category.OTHER)
            + reports("B5", 3, category=Category.GENERAL)
        )
        result = analyze(data, reg, now=NOW)

        self.assertFalse(result.detection.fired)
        # Disclosure engine does not run when nothing is detected (rule 10).
        self.assertEqual(result.disclosure.evaluations, ())
        self.assertIsNone(result.disclosed_scope_id)


class OverclaimRefusalEndToEndTest(unittest.TestCase):
    def test_seven_reports_cannot_name_a_scope_of_two_thousand(self) -> None:
        reg = standard_registry()
        data = reports("B1", 7, category=Category.RESPIRATORY)
        result = analyze(data, reg, now=NOW)

        # Something looks unusual in B1 relative to its peers...
        self.assertTrue(result.detection.fired)
        # ...but no scope has enough evidence to be named.
        self.assertIsNone(result.disclosed_scope_id)

        campus = next(ev for ev in result.disclosure.evaluations if ev.scope_id == "C")
        self.assertEqual(campus.statistical_threshold, 45)
        self.assertFalse(campus.statistical_pass)
        building = next(ev for ev in result.disclosure.evaluations if ev.scope_id == "B
1")
        self.assertFalse(building.statistical_pass)  # 7 < ceil(sqrt(400)) == 20


class RosterRefusalEndToEndTest(unittest.TestCase):
    def test_dense_small_group_is_detected_but_never_named(self) -> None:
        reg = roster_registry()
        data = reports("G", 15, category=Category.RESPIRATORY)
        result = analyze(data, reg, now=NOW)

        self.assertTrue(result.detection.fired)  # the signal is real

        group = next(ev for ev in result.disclosure.evaluations if ev.scope_id == "G")
        self.assertTrue(group.statistical_pass)
        self.assertTrue(group.min_population_pass)
        self.assertFalse(group.report_fraction_pass)
```

```
        self.assertIn("roster", group.disclosure_reason)

        self.assertIsNone(result.disclosed_scope_id)  # ...but nothing is disclosed


class ZeroReportsEndToEndTest(unittest.TestCase):
    def test_no_reports_detects_nothing_and_discloses_nothing(self) -> None:
        result = analyze([], standard_registry(), now=NOW)
        self.assertFalse(result.detection.fired)
        self.assertEqual(result.disclosure.evaluations, ())
        self.assertIsNone(result.disclosed_scope_id)


class PrivacyRefusalEndToEndTest(unittest.TestCase):
    def test_five_reports_on_an_eighteen_person_floor_disclose_the_building(self) -> No
ne:
        reg = small_registry_floor18()
        data = reports("F", 5, category=Category.RESPIRATORY)
        result = analyze(data, reg, now=NOW)

        self.assertTrue(result.detection.fired)

        floor = next(ev for ev in result.disclosure.evaluations if ev.scope_id == "F")
        self.assertTrue(floor.statistical_pass)   # 5 >= 5
        self.assertFalse(floor.privacy_pass)      # 18 < 20
        self.assertFalse(floor.selected)

        self.assertEqual(result.disclosed_scope_id, "B")


if __name__ == "__main__":
    unittest.main()
```

tests/test_experiments.py

```
  1 | """Experiment-runner tests.
  2 |
  3 | Aggregation logic (summarize_setting, fizzle_rate, adjacent_duplicate_rows)
  4 | is tested against hand-built RunOutcome fixtures -- fast, exact, no
  5 | simulation needed. A couple of small, short integration runs exercise the
  6 | sweep end to end without paying for the full 500-seed default.
  7 | """
  8 |
  9 | from __future__ import annotations
 10 |
 11 | import unittest
 12 |
 13 | from microcluster.models import ScopeLevel
 14 |
 15 | from microcluster.config import DisclosureConfig
 16 |
 17 | from simulation.experiments import (
 18 |     RunOutcome,
 19 |     adjacent_duplicate_rows,
 20 |     degenerate_reason_for,
 21 |     false_alarm_rate_at_defaults,
 22 |     fizzle_rate,
 23 |     run_disclosure_sweeps,
 24 |     run_once,
 25 |     summarize_setting,
 26 | )
 27 |
 28 |
 29 | def _outcome(
 30 |     seed,
 31 |     *,
 32 |     established,
 33 |     total_infections=0,
 34 |     peak_infected=0,
 35 |     first_fired_day=None,
 36 |     first_disclosed_day=None,
 37 |     infections_at_first_disclosure=None,
 38 |     finest_disclosed_level=None,
 39 | ) -> RunOutcome:
 40 |     return RunOutcome(
 41 |         seed=seed,
 42 |         established=established,
 43 |         total_infections=total_infections,
 44 |         peak_infected=peak_infected,
 45 |         first_fired_day=first_fired_day,
 46 |         first_disclosed_day=first_disclosed_day,
 47 |         infections_at_first_disclosure=infections_at_first_disclosure,
 48 |         finest_disclosed_level=finest_disclosed_level,
 49 |     )
 50 |
 51 |
 52 | class PolicyCostPropertyTest(unittest.TestCase):
```

```
    def test_defined_only_when_both_events_happened(self) -> None:
        self.assertEqual(
            _outcome(1, established=True, first_fired_day=3, first_disclosed_day=8).pol
icy_cost,
            5,
        )
        self.assertIsNone(_outcome(2, established=True, first_fired_day=3).policy_cost)
        self.assertIsNone(_outcome(3, established=True, first_disclosed_day=8).policy_c
ost)
        self.assertIsNone(_outcome(4, established=False).policy_cost)


class FizzleRateTest(unittest.TestCase):
    def test_fraction_never_established(self) -> None:
        outcomes = [
            _outcome(1, established=False),
            _outcome(2, established=True, total_infections=10),
            _outcome(3, established=False),
            _outcome(4, established=True, total_infections=20),
        ]
        self.assertAlmostEqual(fizzle_rate(outcomes), 0.5)

    def test_empty_is_zero(self) -> None:
        self.assertEqual(fizzle_rate([]), 0.0)


class SummarizeSettingConditioningTest(unittest.TestCase):
    def setUp(self) -> None:
        self.outcomes = [
            # established, fired + disclosed
            _outcome(
                1, established=True, total_infections=40,
                first_fired_day=3, first_disclosed_day=7,
                infections_at_first_disclosure=20, finest_disclosed_level=ScopeLevel.SU
ITE,
            ),
            _outcome(
                2, established=True, total_infections=60,
                first_fired_day=5, first_disclosed_day=9,
                infections_at_first_disclosure=30, finest_disclosed_level=ScopeLevel.FL
OOR,
            ),
            # established, detector never fired (a miss)
            _outcome(3, established=True, total_infections=15),
            # not established, one fired (false alarm), one silent
            _outcome(4, established=False, first_fired_day=2),
            _outcome(5, established=False),
        ]
        self.s = summarize_setting(self.outcomes, param_name="MIN_SCOPE_POPULATION", va
lue=20)

    def test_counts(self) -> None:
        self.assertEqual(self.s.n_established, 3)
        self.assertEqual(self.s.n_non_established, 2)
        self.assertEqual(self.s.n_fired, 2)
```

```
        self.assertEqual(self.s.n_disclosed, 2)

    def test_total_infections_is_over_all_established(self) -> None:
        self.assertAlmostEqual(self.s.mean_total_infections, (40 + 60 + 15) / 3)

    def test_delays_are_over_the_disclosed_subset_only(self) -> None:
        self.assertAlmostEqual(self.s.mean_fire_delay, (3 + 5) / 2)
        self.assertAlmostEqual(self.s.mean_disclosure_delay, (7 + 9) / 2)
        self.assertAlmostEqual(self.s.mean_gap, ((7 - 3) + (9 - 5)) / 2)
        self.assertAlmostEqual(self.s.mean_infections_at_disclosure, (20 + 30) / 2)

    def test_finest_level_is_the_deepest_ever_disclosed(self) -> None:
        self.assertEqual(self.s.finest_disclosed_level, ScopeLevel.SUITE)

    def test_disclosed_fraction_is_over_established(self) -> None:
        self.assertAlmostEqual(self.s.disclosed_fraction, 2 / 3)

    def test_unbiased_infections_include_the_non_disclosing_established_run(self) -> No
ne:
        # runs 1 and 2 disclosed (20, 30); run 3 established but never
        # disclosed -> contributes its full total (15).
        self.assertAlmostEqual(
            self.s.mean_infections_before_disclosure_unbiased, (20 + 30 + 15) / 3
        )

    def test_not_degenerate_by_default(self) -> None:
        self.assertFalse(self.s.degenerate)

    def test_false_alarm_rate_is_over_non_established_only(self) -> None:
        self.assertEqual(self.s.n_false_alarms, 1)
        self.assertAlmostEqual(self.s.false_alarm_rate, 1 / 2)


class SummarizeSettingEmptyBucketsTest(unittest.TestCase):
    def test_no_disclosures_leaves_disclosure_stats_none(self) -> None:
        s = summarize_setting(
            [_outcome(1, established=True, total_infections=10)],
            param_name="x", value=1,
        )
        self.assertEqual(s.n_disclosed, 0)
        self.assertIsNone(s.mean_disclosure_delay)
        self.assertIsNone(s.mean_gap)
        self.assertIsNone(s.mean_infections_at_disclosure)
        self.assertIsNone(s.finest_disclosed_level)
        self.assertIsNotNone(s.mean_total_infections)  # still established

    def test_all_established_yields_no_false_alarm_rate(self) -> None:
        s = summarize_setting(
            [_outcome(1, established=True, total_infections=10)], param_name="x", value
=1
        )
        self.assertIsNone(s.false_alarm_rate)


class FalseAlarmAtDefaultsTest(unittest.TestCase):
```

```
    def test_over_non_established_runs(self) -> None:
        outcomes = [
            _outcome(1, established=True, first_fired_day=1),
            _outcome(2, established=False, first_fired_day=2),  # false alarm
            _outcome(3, established=False),  # silent
        ]
        alarms, non_est, rate = false_alarm_rate_at_defaults(outcomes)
        self.assertEqual((alarms, non_est), (1, 2))
        self.assertAlmostEqual(rate, 0.5)


class AdjacentDuplicateRowsTest(unittest.TestCase):
    def test_identical_regime_rows_are_flagged(self) -> None:
        same = [
            _outcome(
                1, established=True, total_infections=40,
                first_fired_day=3, first_disclosed_day=7,
                infections_at_first_disclosure=20, finest_disclosed_level=ScopeLevel.SU
ITE,
            )
        ]
        different = [
            _outcome(
                1, established=True, total_infections=40,
                first_fired_day=3, first_disclosed_day=12,
                infections_at_first_disclosure=55, finest_disclosed_level=ScopeLevel.FL
OOR,
            )
        ]
        a = summarize_setting(same, param_name="p", value=0.25)
        b = summarize_setting(same, param_name="p", value=0.50)
        c = summarize_setting(different, param_name="p", value=0.75)

        self.assertEqual(adjacent_duplicate_rows([a, b, c]), [(0.25, 0.50)])
        self.assertEqual(adjacent_duplicate_rows([a, c]), [])


class DegenerateReasonTest(unittest.TestCase):
    POPS = (25, 75, 150)  # suite / floor / building=campus in the default structure

    def test_normal_config_is_not_degenerate(self) -> None:
        self.assertIsNone(degenerate_reason_for(DisclosureConfig(), self.POPS))

    def test_min_population_above_whole_campus_is_degenerate(self) -> None:
        reason = degenerate_reason_for(
            DisclosureConfig(min_scope_population=200), self.POPS
        )
        self.assertIsNotNone(reason)
        self.assertIn("200", reason)

    def test_tiny_report_fraction_is_degenerate(self) -> None:
        # floor(25*0.08)=2, floor(75*0.08)=6 < ceil(sqrt(75))=9,
        # floor(150*0.08)=12 < 13 -- no scope can clear both gates.
        self.assertIsNotNone(
            degenerate_reason_for(DisclosureConfig(max_report_fraction=0.08), self.POP
```

```
S)
209 |         )
210 |
211 |     def test_high_statistical_floor_still_leaves_floor_disclosable(self) -> None:
212 |         # floor n=75 admits up to floor(75*0.5)=37 reports and needs 25 -> OK.
213 |         self.assertIsNone(
214 |             degenerate_reason_for(DisclosureConfig(statistical_gate_floor=25), self.POP
S)
215 |         )
216 |
217 |     def test_adjacent_duplicate_rows_skips_degenerate_rows(self) -> None:
218 |         a = summarize_setting([], param_name="p", value=1, degenerate=True, degenerate_
reason="x")
219 |         b = summarize_setting([], param_name="p", value=2, degenerate=True, degenerate_
reason="x")
220 |         self.assertEqual(adjacent_duplicate_rows([a, b]), [])
221 |
222 |
223 | class RunOnceIntegrationTest(unittest.TestCase):
224 |     def test_disclosure_fields_are_consistent(self) -> None:
225 |         outcome = run_once(2, days=20)
226 |         if outcome.first_disclosed_day is None:
227 |             self.assertIsNone(outcome.infections_at_first_disclosure)
228 |             self.assertIsNone(outcome.finest_disclosed_level)
229 |         else:
230 |             self.assertIsNotNone(outcome.infections_at_first_disclosure)
231 |             self.assertIsInstance(outcome.finest_disclosed_level, ScopeLevel)
232 |             # disclosure cannot precede firing
233 |             self.assertLessEqual(outcome.first_fired_day, outcome.first_disclosed_day)
234 |
235 |
236 | class SweepIntegrationTest(unittest.TestCase):
237 |     def setUp(self) -> None:
238 |         self.results = run_disclosure_sweeps(seeds=list(range(1, 13)), days=18)
239 |
240 |     def test_three_sweeps_each_with_the_right_row_count(self) -> None:
241 |         self.assertEqual(
242 |             set(self.results.sweeps),
243 |             {"MIN_SCOPE_POPULATION", "STATISTICAL_GATE_FLOOR", "MAX_REPORT_FRACTION"},
244 |         )
245 |         self.assertEqual(len(self.results.sweeps["MIN_SCOPE_POPULATION"]), 4)
246 |         self.assertEqual(len(self.results.sweeps["STATISTICAL_GATE_FLOOR"]), 5)
247 |         self.assertEqual(len(self.results.sweeps["MAX_REPORT_FRACTION"]), 3)
248 |
249 |     def test_establishment_is_disclosure_independent(self) -> None:
250 |         # Every row of every sweep must reflect the same established count
251 |         # (establishment depends only on the simulated epidemic).
252 |         n_established = {
253 |             s.n_established
254 |             for summaries in self.results.sweeps.values()
255 |             for s in summaries
256 |         }
257 |         self.assertEqual(len(n_established), 1)
258 |         only = n_established.pop()
259 |         self.assertAlmostEqual(
```

```
            1 - only / 12, self.results.fizzle_rate
        )

    def test_false_alarm_and_fire_delay_are_detection_side_constants(self) -> None:
        self.assertEqual(self.results.n_established + self.results.non_established, 12)
        # control fire delay is defined whenever at least one established
        # run fired
        if self.results.n_fired:
            self.assertIsNotNone(self.results.control_fire_delay)

    def test_impossible_min_population_row_is_marked_degenerate(self) -> None:
        rows = self.results.sweeps["MIN_SCOPE_POPULATION"]
        self.assertTrue(rows[-1].degenerate)  # value 200, campus is 150
        self.assertFalse(rows[0].degenerate)  # value 20

    def test_tighter_gate_never_discloses_more_than_a_looser_one(self) -> None:
        # Within the MIN_SCOPE_POPULATION sweep (monotone tightening),
        # n_disclosed should be non-increasing.
        counts = [s.n_disclosed for s in self.results.sweeps["MIN_SCOPE_POPULATION"]]
        self.assertEqual(counts, sorted(counts, reverse=True))


if __name__ == "__main__":
    unittest.main()
```

tests/test_intake.py

```
  1 | """Intake submission-cap tests (rule 11).
  2 |
  3 | Boundary coverage: exactly at the cap, one over, the reset conditions
  4 | (a fresh session_id, and the per-day counter rolling at midnight), the
  5 | session_id=None exemption, and that a rejection is surfaced (report kept
  6 | plus a reason) rather than silently dropped.
  7 | """
  8 |
  9 | from __future__ import annotations
 10 |
 11 | import unittest
 12 | from datetime import datetime, timedelta, timezone
 13 |
 14 | from microcluster.config import IntakeConfig
 15 | from microcluster.engine import analyze
 16 | from microcluster.intake import enforce_submission_caps
 17 | from microcluster.models import (
 18 |     Category,
 19 |     Onset,
 20 |     Report,
 21 |     Scope,
 22 |     ScopeLevel,
 23 |     ScopeRegistry,
 24 | )
 25 |
 26 | _DAY0 = datetime(2026, 1, 1, 9, 0, tzinfo=timezone.utc)
 27 | _CFG = IntakeConfig(session_submission_cap=3, daily_submission_cap=10)
 28 |
 29 |
 30 | def _report(session_id: str | None, *, when: datetime = _DAY0, loc: str = "S1") -> Repo
rt:
 31 |     return Report(
 32 |         category=Category.GENERAL,
 33 |         symptom="fever",
 34 |         onset=Onset.TODAY,
 35 |         location_id=loc,
 36 |         submitted_at=when,
 37 |         session_id=session_id,
 38 |     )
 39 |
 40 |
 41 | class SessionCapTest(unittest.TestCase):
 42 |     def test_reports_up_to_the_cap_are_all_accepted(self) -> None:
 43 |         reports = [_report("s") for _ in range(_CFG.session_submission_cap)]
 44 |         result = enforce_submission_caps(reports, config=_CFG)
 45 |         self.assertEqual(len(result.accepted), 3)
 46 |         self.assertEqual(result.rejected, ())
 47 |
 48 |     def test_one_over_the_cap_is_rejected_not_dropped(self) -> None:
 49 |         reports = [_report("s") for _ in range(_CFG.session_submission_cap + 1)]
 50 |         result = enforce_submission_caps(reports, config=_CFG)
 51 |         self.assertEqual(len(result.accepted), 3)
```

```
        self.assertEqual(result.n_rejected, 1)
        rej = result.rejected[0]
        self.assertIs(rej.report, reports[-1])          # the report is kept
        self.assertIn("session submission cap", rej.reason)  # with a reason

    def test_the_earliest_reports_win_when_a_session_is_over_cap(self) -> None:
        early = [_report("s", when=_DAY0 + timedelta(minutes=i)) for i in range(3)]
        late = _report("s", when=_DAY0 + timedelta(hours=5))
        result = enforce_submission_caps([*early, late], config=_CFG)
        self.assertEqual(list(result.accepted), early)
        self.assertEqual(result.rejected[0].report, late)

    def test_a_fresh_session_id_resets_the_allowance(self) -> None:
        reports = [_report("s1") for _ in range(4)] + [_report("s2") for _ in range(3)]
        result = enforce_submission_caps(reports, config=_CFG)
        # s1: 3 in, 1 out.  s2: a different key, all 3 in.
        self.assertEqual(len(result.accepted), 6)
        self.assertEqual(result.n_rejected, 1)

    def test_none_session_id_is_never_rate_limited(self) -> None:
        reports = [_report(None) for _ in range(50)]
        result = enforce_submission_caps(reports, config=_CFG)
        self.assertEqual(len(result.accepted), 50)
        self.assertEqual(result.rejected, ())

    def test_accepted_order_is_preserved(self) -> None:
        reports = [
            _report(None, loc="A"),
            _report("s", loc="B"),
            _report(None, loc="C"),
            _report("s", loc="D"),
        ]
        accepted = enforce_submission_caps(reports, config=_CFG).accepted
        self.assertEqual([r.location_id for r in accepted], ["A", "B", "C", "D"])


class DailyCapResetTest(unittest.TestCase):
    def test_daily_cap_bites_within_one_day(self) -> None:
        cfg = IntakeConfig(session_submission_cap=100, daily_submission_cap=4)
        reports = [_report("s", when=_DAY0 + timedelta(minutes=i)) for i in range(6)]
        result = enforce_submission_caps(reports, config=cfg)
        self.assertEqual(len(result.accepted), 4)
        self.assertEqual(result.n_rejected, 2)
        self.assertIn("daily submission cap", result.rejected[0].reason)

    def test_daily_counter_resets_on_the_next_calendar_day(self) -> None:
        cfg = IntakeConfig(session_submission_cap=100, daily_submission_cap=4)
        day1 = [_report("s", when=_DAY0 + timedelta(minutes=i)) for i in range(6)]
        day2 = [_report("s", when=_DAY0 + timedelta(days=1, minutes=i)) for i in range
(3)]
        result = enforce_submission_caps([*day1, *day2], config=cfg)
        # day 1: 4 of 6.  day 2: fresh allowance, all 3.
        self.assertEqual(len(result.accepted), 7)
        self.assertEqual(result.n_rejected, 2)

```

```
|
| class EngineWiringTest(unittest.TestCase):
|     """The cap is a normal engine outcome, not a frontend concern."""
|
|     def _registry(self) -> ScopeRegistry:
|         return ScopeRegistry({
|             "C": Scope("C", "Campus", ScopeLevel.CAMPUS, 40),
|             "C-S": Scope("C-S", "Suite", ScopeLevel.SUITE, 40, parent_id="C"),
|         })
|
|     def test_analyze_reports_intake_rejections(self) -> None:
|         reg = self._registry()
|         reports = [_report("flood", loc="C-S") for _ in range(9)]
|         result = analyze(reports, reg, now=_DAY0, intake_config=_CFG)
|         self.assertIsNotNone(result.intake)
|         self.assertEqual(len(result.intake.accepted), 3)
|         self.assertEqual(len(result.intake_rejected), 6)
|
|     def test_intake_config_none_skips_the_caps(self) -> None:
|         reg = self._registry()
|         reports = [_report("flood", loc="C-S") for _ in range(9)]
|         result = analyze(reports, reg, now=_DAY0, intake_config=None)
|         self.assertIsNone(result.intake)
|
|     def test_anonymous_simulator_style_stream_is_untouched(self) -> None:
|         # every simulator Report has session_id=None -> nothing is capped
|         reg = self._registry()
|         reports = [_report(None, loc="C-S") for _ in range(30)]
|         with_caps = analyze(reports, reg, now=_DAY0)
|         without = analyze(reports, reg, now=_DAY0, intake_config=None)
|         self.assertEqual(with_caps.intake_rejected, ())
|         self.assertEqual(
|             with_caps.detection.qualifying_scope_counts,
|             without.detection.qualifying_scope_counts,
|         )
|
|
| if __name__ == "__main__":
|     unittest.main()
```

tests/test_models.py

```
"""Model-level validation: the fixed checklist is the only accepted input."""

from __future__ import annotations

import unittest
from datetime import timedelta

from microcluster.models import (
    CHECKLIST,
    Category,
    Onset,
    Report,
    Scope,
    ScopeLevel,
    ScopeRegistry,
)

from tests.fixtures import NOW


def _report(**overrides) -> Report:
    kwargs = dict(
        category=Category.RESPIRATORY,
        symptom="cough",
        onset=Onset.TODAY,
        location_id="X",
        submitted_at=NOW - timedelta(hours=1),
    )
    kwargs.update(overrides)
    return Report(**kwargs)


class ReportValidationTest(unittest.TestCase):
    def test_symptom_not_in_the_fixed_checklist_is_rejected(self) -> None:
        with self.assertRaises(ValueError) as ctx:
            _report(category=Category.RESPIRATORY, symptom="chest x-ray abnormality")
        self.assertIn("not in the fixed checklist", str(ctx.exception))

    def test_symptom_from_a_different_category_is_rejected(self) -> None:
        # "diarrhea" is a real checklist symptom, but not under RESPIRATORY.
        self.assertIn("diarrhea", CHECKLIST[Category.GASTROINTESTINAL])
        with self.assertRaises(ValueError):
            _report(category=Category.RESPIRATORY, symptom="diarrhea")

    def test_free_text_category_is_rejected(self) -> None:
        with self.assertRaises(ValueError):
            _report(category="respiratory-ish")  # type: ignore[arg-type]

    def test_free_text_onset_is_rejected(self) -> None:
        with self.assertRaises(ValueError):
            _report(onset="a while ago")  # type: ignore[arg-type]

```

```
    def test_every_valid_checklist_symptom_is_accepted(self) -> None:
        for category, symptoms in CHECKLIST.items():
            for symptom in symptoms:
                _report(category=category, symptom=symptom)  # no raise


class ScopeValidationTest(unittest.TestCase):
    def test_declared_population_must_be_positive(self) -> None:
        with self.assertRaises(ValueError):
            Scope("z", "Zero", ScopeLevel.CAMPUS, population=0)

    def test_non_campus_scope_needs_a_parent(self) -> None:
        with self.assertRaises(ValueError):
            Scope("b", "Orphan building", ScopeLevel.BUILDING, 100)

    def test_registry_rejects_unknown_parent(self) -> None:
        with self.assertRaises(ValueError):
            ScopeRegistry(
                {"b": Scope("b", "B", ScopeLevel.BUILDING, 100, parent_id="missing")}
            )


if __name__ == "__main__":
    unittest.main()
```

tests/test_motion_constraints.py

```
 1 | """CLAUDE.md UI CONSTRAINTS, "Motion" -- the durable version of the ad hoc
 2 | grep that used to just ban transition/animation outright.
 3 |
 4 | Motion is now permitted narrowly (state-change and interaction feedback,
 5 | capped at 400ms, never looping; one decorative exception). What must still
 6 | be impossible -- gradients, shadows, glassmorphism/blur, and anything that
 7 | loops -- is checked here instead of by a blanket "no transition" grep, so
 8 | this suite has to be updated if the rule changes, not silently bypassed.
 9 | """
10 |
11 | from __future__ import annotations
12 |
13 | import re
14 | import unittest
15 | from pathlib import Path
16 |
17 | WEB = Path(__file__).resolve().parent.parent / "web"
18 |
19 | # still an absolute ban, with or without motion.
20 | _BANNED_RAW = (
21 |     r"linear-gradient",
22 |     r"radial-gradient",
23 |     r"conic-gradient",
24 |     r"box-shadow",
25 |     r"text-shadow",
26 |     r"backdrop-filter",
27 |     r"filter:\s*blur",
28 |     r"\binfinite\b",          # looping animation/animation-iteration-count
29 | )
30 | _BANNED_FONT = r"font-family:[^;]*(Inter|Geist|Space Grotesk)"
31 |
32 | _FOCUS_RING_RULE = (
33 |     "a:focus-visible, button:focus-visible, input:focus-visible,\n"
34 |     "select:focus-visible, summary:focus-visible {\n"
35 |     "  outline: 2px solid var(--ink);\n"
36 |     "  outline-offset: 2px;\n"
37 |     "}"
38 | )
39 |
40 |
41 | class BannedPatternsTest(unittest.TestCase):
42 |     def test_css_has_no_banned_decoration_or_looping_motion(self) -> None:
43 |         css = (WEB / "style.css").read_text(encoding="utf-8")
44 |         for pattern in _BANNED_RAW:
45 |             self.assertIsNone(re.search(pattern, css, re.I), f"found banned pattern: {pa
ttern}")
46 |         self.assertIsNone(re.search(_BANNED_FONT, css, re.I), "banned font-family")
47 |
48 |     def test_html_pages_have_no_banned_decoration(self) -> None:
49 |         for name in ("index.html", "privacy.html", "terms.html"):
50 |             html = (WEB / name).read_text(encoding="utf-8")
51 |             for pattern in _BANNED_RAW:
```

```
                self.assertIsNone(re.search(pattern, html, re.I), f"{name}: {pattern}")

    def test_no_em_dash_in_ui_copy_or_code(self) -> None:
        for path in list(WEB.glob("*.html")) + list(WEB.glob("*.css")) + list(WEB.glob("*.js")):
            self.assertNotIn("—", path.read_text(encoding="utf-8"), path.name)


class MotionCapTest(unittest.TestCase):
    def test_every_transition_and_animation_duration_is_capped_at_400ms(self) -> None:
        css = (WEB / "style.css").read_text(encoding="utf-8")
        durations = [int(m) for m in re.findall(r"(\d{2,4})ms", css)]
        # sanity check the test itself is looking at the motion section at all
        self.assertGreaterEqual(len(durations), 5, "expected the new motion rules to be present")
        for ms in durations:
            self.assertLessEqual(ms, 400, f"{ms}ms exceeds the 400ms cap")

    def test_reduced_motion_neutralises_every_new_transition_and_animation(self) -> None:
        css = (WEB / "style.css").read_text(encoding="utf-8")
        match = re.search(
            r"@media \(prefers-reduced-motion:\s*reduce\)\s*\{(.*?)\n\}", css, re.S
        )
        self.assertIsNotNone(match, "no prefers-reduced-motion: reduce block found")
        block = match.group(1)
        self.assertIn("transition: none", block)
        self.assertIn("animation: none", block)
        # the pulse ring and the flashed gate row must be fully suppressed,
        # not just left with an un-eased snap
        self.assertIn(".agent.pulse::before", block)
        self.assertIn("tr.verdict-changed", block)


class FocusRingUntouchedTest(unittest.TestCase):
    def test_focus_visible_ring_is_byte_identical(self) -> None:
        css = (WEB / "style.css").read_text(encoding="utf-8")
        self.assertIn(_FOCUS_RING_RULE, css, "the focus-visible ring rule was changed")


if __name__ == "__main__":
    unittest.main()
```

tests/test_pipeline.py

```
"""End-to-end wiring tests: simulation -> reports -> the real detection
engine, unmodified.
"""

from __future__ import annotations

import unittest

from microcluster.models import ScopeLevel

from simulation.config import DEFAULT_SMOKE_SEEDS, SimulationConfig
from simulation.infection import InfectionState
from simulation.pipeline import (
    analyze_report_stream,
    run_with_detection,
    simulate_and_report,
)
from simulation.population import StructureSpec
from simulation.simulator import Simulation


class ReportsAreSuiteLevelTest(unittest.TestCase):
    def test_every_report_resolves_to_a_suite_scope(self) -> None:
        result = run_with_detection(seed=2, days=20)
        for report in result.reports:
            scope = result.simulation.registry.get(report.location_id)
            self.assertEqual(scope.level, ScopeLevel.SUITE)


class DailyRecordShapeTest(unittest.TestCase):
    def test_one_record_per_day_including_day_zero(self) -> None:
        result = run_with_detection(seed=1, days=15)
        self.assertEqual(len(result.daily_records), 16)
        self.assertEqual([r.day for r in result.daily_records], list(range(16)))

    def test_reports_accumulated_is_the_running_total(self) -> None:
        result = run_with_detection(seed=3, days=15)
        running = 0
        for record in result.daily_records:
            running += record.reports_submitted_today
            self.assertEqual(record.reports_accumulated, running)
        self.assertEqual(running, len(result.reports))

    def test_true_infected_count_matches_ground_truth_history(self) -> None:
        result = run_with_detection(seed=4, days=15)
        for record, day_state in zip(result.daily_records, result.simulation.history):
            expected = day_state.count(InfectionState.INCUBATING) + day_state.count(
                InfectionState.SYMPTOMATIC
            )
            self.assertEqual(record.true_infected_count, expected)


```

```
class TrajectoryUnperturbedTest(unittest.TestCase):
    def test_reporting_does_not_change_the_epidemic_trajectory(self) -> None:
        # Report generation uses its own RNG stream; the same seed must
        # produce the identical infection trajectory with or without it.
        plain = Simulation(seed=6, seed_infections=1).run(20)
        piped = run_with_detection(seed=6, days=20)
        self.assertEqual(
            [s.counts for s in plain],
            [s.counts for s in piped.simulation.history],
        )


class DetectionCanFireAndDiscloseTest(unittest.TestCase):
    def test_favorable_outbreak_is_detected_and_disclosed(self) -> None:
        # Deliberately easy: certain transmission, certain reporting, no
        # background noise -- if the wiring works at all, this must fire
        # and eventually name a scope.
        config = SimulationConfig(
            suite_transmission_probability=1.0,
            reporting_probability_min=1.0,
            reporting_probability_max=1.0,
            background_noise_daily_rate=0.0,
        )
        result = run_with_detection(seed=1, days=20, config=config)

        self.assertTrue(any(r.detection_fired for r in result.daily_records))
        self.assertTrue(any(r.disclosed_scope_id for r in result.daily_records))

    def test_no_reports_at_all_never_fires(self) -> None:
        config = SimulationConfig(
            reporting_probability_min=0.0,
            reporting_probability_max=0.0,
            background_noise_daily_rate=0.0,
        )
        result = run_with_detection(seed=1, days=20, config=config)

        self.assertEqual(result.reports, [])
        self.assertTrue(all(not r.detection_fired for r in result.daily_records))
        self.assertTrue(all(r.disclosed_scope_id is None for r in result.daily_record
s))


class DelayRecordingTest(unittest.TestCase):
    def test_first_fired_and_first_disclosed_are_recorded_and_ordered(self) -> None:
        config = SimulationConfig(
            suite_transmission_probability=1.0,
            reporting_probability_min=1.0,
            reporting_probability_max=1.0,
            background_noise_daily_rate=0.0,
        )
        result = run_with_detection(seed=1, days=20, config=config)

        fired_day = next(r.day for r in result.daily_records if r.detection_fired)
        disclosed_day = next(
            r.day for r in result.daily_records if r.disclosed_scope_id is not None
```

```
        )
        last = result.daily_records[-1]

        # The running fields on the final record must match what actually
        # happened, and disclosure can never precede firing.
        self.assertEqual(last.first_fired_day, fired_day)
        self.assertEqual(last.first_disclosed_day, disclosed_day)
        self.assertLessEqual(fired_day, disclosed_day)

    def test_before_either_happens_both_fields_are_none(self) -> None:
        result = run_with_detection(seed=1, days=1)
        first = result.daily_records[0]
        if not first.detection_fired:
            self.assertIsNone(first.first_fired_day)
        if first.disclosed_scope_id is None:
            self.assertIsNone(first.first_disclosed_day)

    def test_no_detection_ever_leaves_both_fields_none(self) -> None:
        config = SimulationConfig(
            reporting_probability_min=0.0,
            reporting_probability_max=0.0,
            background_noise_daily_rate=0.0,
        )
        result = run_with_detection(seed=1, days=20, config=config)
        last = result.daily_records[-1]
        self.assertIsNone(last.first_fired_day)
        self.assertIsNone(last.first_disclosed_day)


class ScopeStabilityAcrossARunTest(unittest.TestCase):
    """config.SIBLING_SWITCH_MARGIN, exercised over a full simulated run
    rather than a single evaluate() call."""

    def test_distinct_disclosed_scopes_stay_small_across_the_default_seeds(self) -> Non
e:
        # The default structure has 6 suites, 2 floors, 1 building, 1
        # campus -- 10 possible scopes. Without stability, a wandering
        # outbreak can (and, before this change, did) name most of them
        # over a 30-day run. With it, each run should settle on very few.
        for seed in DEFAULT_SMOKE_SEEDS:
            with self.subTest(seed=seed):
                result = run_with_detection(seed=seed, days=30)
                distinct = result.distinct_disclosed_scopes()
                self.assertLessEqual(
                    len(distinct), 3, f"seed {seed} disclosed too many scopes: {distinc
t}"
                )

    def test_a_named_scope_persists_across_consecutive_disclosing_days(self) -> None:
        # Once something is disclosed, most consecutive disclosing days
        # should repeat it rather than naming something new each time.
        found_a_run_with_multiple_disclosures = False
        for seed in DEFAULT_SMOKE_SEEDS:
            result = run_with_detection(seed=seed, days=30)
            sequence = [r.disclosed_scope_id for r in result.daily_records if r.disclos
```

```
ed_scope_id]
            if len(sequence) < 2:
                continue
            found_a_run_with_multiple_disclosures = True
            changes = sum(1 for a, b in zip(sequence, sequence[1:]) if a != b)
            self.assertLess(
                changes, len(sequence), f"seed {seed}: every disclosing day changed sco
pe"
            )
        self.assertTrue(found_a_run_with_multiple_disclosures)


class TwoPhaseEquivalenceTest(unittest.TestCase):
    """run_with_detection == simulate_and_report + analyze_report_stream.
    The split exists so an experiment can simulate a seed once and replay
    the analysis under many configs; it must not change the result."""

    def test_split_matches_the_combined_call(self) -> None:
        combined = run_with_detection(seed=3, days=25)

        simulated = simulate_and_report(seed=3, days=25)
        replayed = analyze_report_stream(simulated)

        self.assertEqual(combined.daily_records, replayed)
        self.assertEqual(
            [s.counts for s in combined.simulation.history],
            [s.counts for s in simulated.simulation.history],
        )

    def test_one_simulation_replayed_under_two_configs(self) -> None:
        from microcluster.config import DisclosureConfig

        simulated = simulate_and_report(seed=4, days=25)
        loose = analyze_report_stream(
            simulated, disclosure_config=DisclosureConfig(min_scope_population=10)
        )
        strict = analyze_report_stream(
            simulated, disclosure_config=DisclosureConfig(min_scope_population=200)
        )
        # Same epidemic, different disclosure gate -> the strict one names
        # nothing, the loose one may.
        self.assertTrue(all(r.disclosed_scope_id is None for r in strict))
        self.assertEqual(len(loose), len(strict))


class SmallStructureStillPassesPrivacyGateTest(unittest.TestCase):
    def test_custom_structure_runs_end_to_end(self) -> None:
        spec = StructureSpec(
            buildings=1, floors_per_building=1, suites_per_floor=2, agents_per_suite=25
        )
        result = run_with_detection(seed=1, days=10, spec=spec)
        self.assertEqual(len(result.simulation.agents), 50)


```

```
212 | if __name__ == "__main__":
213 |     unittest.main()
```

tests/test_reporting.py

```
  1 | """Report generation: the one place simulated agents turn into anonymous
  2 | ``microcluster.models.Report`` objects."""
  3 |
  4 | from __future__ import annotations
  5 |
  6 | import random
  7 | import unittest
  8 |
  9 | from microcluster.models import CHECKLIST, Category
 10 |
 11 | from simulation.agent import Agent
 12 | from simulation.config import SimulationConfig
 13 | from simulation.infection import InfectionState
 14 | from simulation.reporting import generate_daily_reports
 15 |
 16 |
 17 | def _symptomatic_agent(**overrides) -> Agent:
 18 |     kwargs = dict(
 19 |         id="A1",
 20 |         suite_id="S1",
 21 |         reporting_probability=1.0,
 22 |         state=InfectionState.SYMPTOMATIC,
 23 |         state_changed_day=0,
 24 |         incubation_days=2,
 25 |         symptomatic_days=5,
 26 |     )
 27 |     kwargs.update(overrides)
 28 |     return Agent(**kwargs)
 29 |
 30 |
 31 | class OutbreakReportTest(unittest.TestCase):
 32 |     def test_symptomatic_agent_reports_in_the_suite_and_outbreak_category(self) -> Non
e:
 33 |         config = SimulationConfig()
 34 |         agent = _symptomatic_agent(reporting_probability=1.0)
 35 |         reports = generate_daily_reports([agent], 0, random.Random(0), config)
 36 |
 37 |         self.assertEqual(len(reports), 1)
 38 |         report = reports[0]
 39 |         self.assertEqual(report.location_id, agent.suite_id)
 40 |         self.assertEqual(report.category, config.outbreak_category)
 41 |         self.assertIn(report.symptom, CHECKLIST[config.outbreak_category])
 42 |         self.assertTrue(agent.has_reported)
 43 |
 44 |     def test_zero_reporting_probability_never_reports(self) -> None:
 45 |         config = SimulationConfig()
 46 |         agent = _symptomatic_agent(reporting_probability=0.0)
 47 |         reports = generate_daily_reports([agent], 0, random.Random(0), config)
 48 |
 49 |         self.assertEqual(reports, [])
 50 |         self.assertFalse(agent.has_reported)
 51 |
```

```
    def test_agent_reports_at_most_once_per_episode(self) -> None:
        config = SimulationConfig()
        agent = _symptomatic_agent(reporting_probability=1.0, state_changed_day=0)
        rng = random.Random(0)

        all_reports = []
        for day in range(4):  # still SYMPTOMATIC every one of these days
            all_reports += generate_daily_reports([agent], day, rng, config)

        self.assertEqual(len(all_reports), 1)

    def test_onset_bucket_reflects_days_symptomatic(self) -> None:
        config = SimulationConfig()
        agent = _symptomatic_agent(reporting_probability=1.0, state_changed_day=5)
        # Report generated on day 12 -> 7 days symptomatic -> "3-7 days".
        reports = generate_daily_reports([agent], 12, random.Random(0), config)
        self.assertEqual(reports[0].onset.value, "3-7 days")


class BackgroundNoiseTest(unittest.TestCase):
    def test_noise_report_uses_a_non_outbreak_category(self) -> None:
        config = SimulationConfig(background_noise_daily_rate=1.0)
        for state in (
            InfectionState.SUSCEPTIBLE,
            InfectionState.INCUBATING,
            InfectionState.RECOVERED,
        ):
            with self.subTest(state=state):
                agent = Agent(
                    id="A2", suite_id="S1", reporting_probability=0.5, state=state
                )
                reports = generate_daily_reports([agent], 3, random.Random(1), config)
                self.assertEqual(len(reports), 1)
                self.assertNotEqual(reports[0].category, config.outbreak_category)
                self.assertIn(reports[0].symptom, CHECKLIST[reports[0].category])
                self.assertEqual(reports[0].location_id, agent.suite_id)

    def test_zero_rate_never_generates_noise(self) -> None:
        config = SimulationConfig(background_noise_daily_rate=0.0)
        agent = Agent(id="A3", suite_id="S1", reporting_probability=0.5)
        reports = generate_daily_reports([agent], 0, random.Random(2), config)
        self.assertEqual(reports, [])

    def test_symptomatic_agent_does_not_also_generate_noise(self) -> None:
        # A symptomatic agent's chance is entirely the outbreak-report
        # roll; it must not separately roll for background noise too.
        config = SimulationConfig(background_noise_daily_rate=1.0)
        agent = _symptomatic_agent(reporting_probability=0.0)  # never files
        reports = generate_daily_reports([agent], 0, random.Random(3), config)
        self.assertEqual(reports, [])


if __name__ == "__main__":
    unittest.main()
```

tests/test_scale_and_realism.py

```
  1 | """Scale validation and reporting-realism tests.
  2 |
  3 | Scale: the largest structure that works (2,000 agents, four distinct scope
  4 | populations) must run end to end, keep the evidence gate equal to its
  5 | formula on every evaluation, and still trigger the privacy gate.
  6 |
  7 | Realism: the alternative reporting model must be strictly opt-in (the
  8 | default config is unchanged and every committed result stays valid), must
  9 | reduce to the clean model when all three effects are off, and each effect
 10 | must push the reporting process in the direction it claims to.
 11 | """
 12 |
 13 | from __future__ import annotations
 14 |
 15 | import math
 16 | import unittest
 17 | from dataclasses import replace
 18 |
 19 | from microcluster.models import ScopeLevel
 20 | from simulation.config import (
 21 |     DEFAULT_SIMULATION_CONFIG,
 22 |     REPORTING_MODEL_CLEAN,
 23 |     REPORTING_MODEL_REALISTIC,
 24 |     SimulationConfig,
 25 | )
 26 | from simulation.pipeline import run_with_detection, simulate_and_report
 27 | from simulation.realism import RealismConfig, run_realistic
 28 | from simulation.scale import SCALE_SPECS, measure_size
 29 |
 30 |
 31 | def _sig(run):
 32 |     return [
 33 |         (r.disclosed_scope_id, r.reports_submitted_today, r.detection_fired)
 34 |         for r in run.daily_records
 35 |     ]
 36 |
 37 |
 38 | class ScaleTest(unittest.TestCase):
 39 |     def test_2000_agents_build_with_four_distinct_scope_populations(self) -> None:
 40 |         run = run_with_detection(seed=1, days=5, spec=SCALE_SPECS[2000])
 41 |         reg = run.simulation.registry
 42 |         self.assertEqual(len(run.simulation.agents), 2000)
 43 |         self.assertEqual(reg.campus_population(), 2000)
 44 |         self.assertEqual({s.population for s in reg.scopes.values()}, {2000, 500, 125,
25})
 45 |         self.assertEqual(
 46 |             {s.level for s in reg.scopes.values()},
 47 |             {ScopeLevel.CAMPUS, ScopeLevel.BUILDING, ScopeLevel.FLOOR, ScopeLevel.SUIT
E},
 48 |         )
 49 |
 50 |     def test_evidence_gate_matches_its_formula_at_the_largest_size(self) -> None:
```

```
        r = measure_size(2000, list(range(1, 13)))
        self.assertEqual(r.errors, 0)
        self.assertGreater(r.scope_evaluations_checked, 0)
        self.assertEqual(r.threshold_formula_violations, 0)
        self.assertEqual(r.pass_flag_violations, 0)
        self.assertEqual(r.selected_without_both_gates, 0)
        # ceil(sqrt(n)) at every level of the 2,000-agent structure
        self.assertEqual(
            r.thresholds_by_level,
            {
                "CAMPUS": max(5, math.isqrt(2000) + 1),   # 45
                "BUILDING": max(5, math.isqrt(500) + 1),  # 23
                "FLOOR": max(5, math.isqrt(125) + 1),     # 12
                "SUITE": 5,
            },
        )

    def test_privacy_gate_still_triggers_at_the_largest_size(self) -> None:
        r = measure_size(2000, list(range(1, 13)))
        self.assertGreater(r.roster_refusals, 0)
        self.assertGreater(r.n_established, 0)


class RealismOptInTest(unittest.TestCase):
    def test_default_config_is_the_clean_model(self) -> None:
        self.assertEqual(DEFAULT_SIMULATION_CONFIG.reporting_model, REPORTING_MODEL_CLEAN)

    def test_unknown_reporting_model_is_rejected(self) -> None:
        with self.assertRaises(ValueError):
            SimulationConfig(reporting_model="whatever")

    def test_simulate_and_report_refuses_the_realistic_model(self) -> None:
        cfg = replace(DEFAULT_SIMULATION_CONFIG, reporting_model=REPORTING_MODEL_REALISTIC)
        with self.assertRaises(ValueError):
            simulate_and_report(seed=1, days=3, config=cfg)

    def test_run_with_detection_dispatches_on_the_config(self) -> None:
        cfg = replace(DEFAULT_SIMULATION_CONFIG, reporting_model=REPORTING_MODEL_REALISTIC)
        clean = run_with_detection(seed=4, days=30)
        realistic = run_with_detection(seed=4, days=30, config=cfg)
        self.assertNotEqual(_sig(clean), _sig(realistic))

    def test_all_effects_off_reduces_to_the_clean_model_exactly(self) -> None:
        off = RealismConfig(heterogeneity=False, correlation=False, stigma=False)
        for seed in (1, 4, 9):
            self.assertEqual(
                _sig(run_with_detection(seed=seed, days=30)),
                _sig(run_realistic(seed=seed, days=30, realism=off)),
            )

    def test_the_epidemic_is_identical_under_both_models(self) -> None:
        clean = run_with_detection(seed=7, days=30)
```

```
        realistic = run_realistic(seed=7, days=30)
        self.assertEqual(
            [tuple(s.counts.values()) for s in clean.simulation.history],
            [tuple(s.counts.values()) for s in realistic.simulation.history],
        )


class RealismEffectsTest(unittest.TestCase):
    def test_heterogeneity_keeps_the_mean_but_widens_the_spread(self) -> None:
        run = run_realistic(
            seed=3, days=0,
            realism=RealismConfig(heterogeneity=True, correlation=False, stigma=False),
        )
        p = [a.reporting_probability for a in run.simulation.agents]
        self.assertAlmostEqual(sum(p) / len(p), 0.5, delta=0.08)
        self.assertLess(min(p), 0.3)   # the clean model never goes below 0.3
        self.assertGreater(max(p), 0.7)

    def test_correlation_raises_total_reports(self) -> None:
        only_corr = RealismConfig(heterogeneity=False, correlation=True, stigma=False)
        base = corr = 0
        for seed in range(1, 21):
            base += len(run_with_detection(seed=seed, days=30).reports)
            corr += len(run_realistic(seed=seed, days=30, realism=only_corr).reports)
        self.assertGreater(corr, base)

    def test_stigma_lowers_reports_and_never_changes_the_first_disclosure(self) -> Non
e:
        only_stigma = RealismConfig(heterogeneity=False, correlation=False, stigma=Tru
e)
        base = stig = 0
        for seed in range(1, 21):
            a = run_with_detection(seed=seed, days=30)
            b = run_realistic(seed=seed, days=30, realism=only_stigma)
            base += len(a.reports)
            stig += len(b.reports)
            # suppression only starts after a scope is named
            self.assertEqual(
                a.daily_records[-1].first_disclosed_day,
                b.daily_records[-1].first_disclosed_day,
            )
        self.assertLess(stig, base)


if __name__ == "__main__":
    unittest.main()
```

tests/test_simulation.py

```
"""Simulation core tests: state machine, contact model, population builder.

These do not touch the detection or disclosure engines -- they prove the
simulated world behaves before any Report is generated from it.
"""

from __future__ import annotations

import random
import unittest
from collections import Counter

from microcluster.models import ScopeLevel, ScopeRegistry

from simulation import contact, infection
from simulation.agent import Agent
from simulation.config import SimulationConfig
from simulation.contact import suite_ancestor_ids
from simulation.infection import STATE_ORDER, InfectionState, order_index
from simulation.population import StructureSpec, build_population
from simulation.simulator import Simulation

# Derived from the default spec, not hard-coded.
_SPEC = StructureSpec()
_TOTAL_AGENTS = (
    _SPEC.buildings
    * _SPEC.floors_per_building
    * _SPEC.suites_per_floor
    * _SPEC.agents_per_suite
)


class ZeroInfectedTest(unittest.TestCase):
    def test_population_with_no_seed_stays_fully_susceptible(self) -> None:
        sim = Simulation(seed=1, seed_infections=0)
        history = sim.run(30)

        total = len(sim.agents)
        for snapshot in history:
            self.assertEqual(snapshot.count(InfectionState.SUSCEPTIBLE), total)
            self.assertEqual(snapshot.count(InfectionState.INCUBATING), 0)
            self.assertEqual(snapshot.count(InfectionState.SYMPTOMATIC), 0)
            self.assertEqual(snapshot.count(InfectionState.RECOVERED), 0)


class CrossSuiteTransmissionTest(unittest.TestCase):
    """Contact is tiered (suite / floor / building), so an outbreak is no
    longer trapped in its starting suite -- but the floor/building rates
    are lower, so crossing a suite boundary should stay rarer than
    transmitting within one."""

    def test_cross_suite_transmission_occurs_but_is_rarer_than_within_suite(
```

```
        self,
    ) -> None:
        total_within = 0
        total_cross = 0
        for seed in range(1, 41):
            sim = Simulation(seed=seed, seed_infections=1)
            sim.run(30)
            total_within += sim.total_within_suite_infections
            total_cross += sim.total_cross_suite_infections

        self.assertGreater(
            total_cross, 0, "cross-suite transmission never occurred over 40 seeds"
        )
        self.assertLess(
            total_cross,
            total_within,
            f"cross-suite transmission ({total_cross}) was not rarer than "
            f"within-suite transmission ({total_within})",
        )


class WithinSuiteAttackRateTest(unittest.TestCase):
    """A seeded suite's final attack rate should be a real, seed-dependent
    outcome -- never a guaranteed 100%, and not the same number every
    time."""

    def _suite_attack_rate(self, seed: int, days: int = 30) -> float:
        sim = Simulation(seed=seed, seed_infections=1)
        seeded_suite = next(
            a.suite_id for a in sim.agents if a.state is not InfectionState.SUSCEPTIBLE
        )
        history = sim.run(days)
        last = history[-1]
        n = sum(1 for s in last.agents if s.suite_id == seeded_suite)
        infected = sum(
            1
            for s in last.agents
            if s.suite_id == seeded_suite and s.state is not InfectionState.SUSCEPTIBLE
        )
        return infected / n

    def test_attack_rate_is_below_100_percent_and_varies_across_seeds(self) -> None:
        rates = [self._suite_attack_rate(seed) for seed in range(1, 11)]

        self.assertTrue(
            all(rate < 1.0 for rate in rates),
            f"some seed reached a 100% within-suite attack rate: {rates}",
        )
        self.assertGreater(
            max(rates) - min(rates),
            0.1,
            f"within-suite attack rate did not vary meaningfully across seeds: {rate
s}",
        )

```

```
|
| class SecondFloorReachedTest(unittest.TestCase):
|     def test_infection_reaches_a_second_floor_at_least_sometimes(self) -> None:
|         reached_second_floor = False
|         for seed in range(1, 31):
|             sim = Simulation(seed=seed, seed_infections=1)
|             history = sim.run(30)
|
|             touched_floors = {
|                 suite_ancestor_ids(sim.registry, s.suite_id)[0]
|                 for s in history[-1].agents
|                 if s.state is not InfectionState.SUSCEPTIBLE
|             }
|             if len(touched_floors) > 1:
|                 reached_second_floor = True
|                 break
|
|         self.assertTrue(
|             reached_second_floor,
|             "no seed among the first 30 reached a second floor within 30 days",
|         )
|
|
| class CertainTransmissionTest(unittest.TestCase):
|     def test_probability_one_infects_the_whole_suite_in_a_single_step(self) -> None:
|         # With p = 1.0 a single infectious agent infects every susceptible
|         # suite-mate in one daily step, so the expected number of steps to
|         # a fully-infected suite is 1.
|         config = SimulationConfig(suite_transmission_probability=1.0)
|         sim = Simulation(seed=3, seed_infections=1, config=config)
|         seeded_suite = next(
|             a.suite_id for a in sim.agents if a.state is not InfectionState.SUSCEPTIBLE
|         )
|
|         history = sim.run(5)
|         day_one = history[1]
|         in_suite = [s for s in day_one.agents if s.suite_id == seeded_suite]
|
|         self.assertEqual(len(in_suite), _SPEC.agents_per_suite)
|         self.assertTrue(
|             all(s.state is not InfectionState.SUSCEPTIBLE for s in in_suite)
|         )
|
|
| class ForwardProgressionTest(unittest.TestCase):
|     def test_states_never_regress_and_never_skip(self) -> None:
|         config = SimulationConfig(suite_transmission_probability=1.0)
|         sim = Simulation(seed=5, seed_infections=1, config=config)
|         history = sim.run(30)
|
|         sequences: dict[str, list[InfectionState]] = {}
|         for snapshot in history:
|             for agent_snapshot in snapshot.agents:
|                 sequences.setdefault(agent_snapshot.id, []).append(agent_snapshot.stat
e)
```

```
        for agent_id, states in sequences.items():
            for previous, following in zip(states, states[1:]):
                delta = order_index(following) - order_index(previous)
                self.assertGreaterEqual(delta, 0, f"{agent_id} moved backwards")
                self.assertLessEqual(delta, 1, f"{agent_id} skipped a state")

        # Non-vacuous: the p=1.0 suite runs the full course within 30 days.
        self.assertGreater(history[-1].count(InfectionState.RECOVERED), 0)


class ConservationTest(unittest.TestCase):
    def test_agent_count_is_conserved_across_states_every_step(self) -> None:
        config = SimulationConfig(suite_transmission_probability=0.5)
        sim = Simulation(seed=2, seed_infections=2, config=config)
        history = sim.run(30)

        total = len(sim.agents)
        for snapshot in history:
            self.assertEqual(sum(snapshot.counts.values()), total)
            self.assertEqual(len(snapshot.agents), total)
            self.assertEqual(set(snapshot.counts), set(STATE_ORDER))


class RegistryPopulationTest(unittest.TestCase):
    def setUp(self) -> None:
        self.spec = StructureSpec()
        self.agents, self.registry = build_population(self.spec, random.Random(0))

    def test_declared_populations_equal_actual_agent_counts(self) -> None:
        for scope_id, scope in self.registry.scopes.items():
            actual = sum(
                1
                for agent in self.agents
                if self.registry.contains(scope_id, agent.suite_id)
            )
            self.assertEqual(scope.population, actual, f"scope {scope_id}")

    def test_populations_per_level_match_the_spec(self) -> None:
        expected_suite = self.spec.agents_per_suite
        expected_floor = self.spec.suites_per_floor * expected_suite
        expected_campus = _TOTAL_AGENTS

        per_level: dict[ScopeLevel, set[int]] = {}
        for scope in self.registry.scopes.values():
            per_level.setdefault(scope.level, set()).add(scope.population)

        self.assertEqual(per_level[ScopeLevel.SUITE], {expected_suite})
        self.assertEqual(per_level[ScopeLevel.FLOOR], {expected_floor})
        self.assertEqual(per_level[ScopeLevel.CAMPUS], {expected_campus})
        self.assertEqual(len(self.agents), expected_campus)
        # The concrete acceptance numbers from the task.
        self.assertEqual(expected_suite, 25)
        self.assertEqual(expected_floor, 75)
        self.assertEqual(expected_campus, 150)
```

```
    def test_registry_passes_scoperegistry_validation(self) -> None:
        # Rebuilding from the produced scopes must not raise.
        rebuilt = ScopeRegistry(dict(self.registry.scopes))
        self.assertEqual(rebuilt.campus_population(), len(self.agents))

    def test_a_suite_reaches_campus_through_the_registry(self) -> None:
        suite_id = next(
            s.id for s in self.registry.scopes.values() if s.level is ScopeLevel.SUITE
        )
        chain = self.registry.chain_to_root(suite_id)
        self.assertEqual(
            [s.level for s in chain],
            [
                ScopeLevel.SUITE,
                ScopeLevel.FLOOR,
                ScopeLevel.BUILDING,
                ScopeLevel.CAMPUS,
            ],
        )

    def test_suite_at_or_below_privacy_minimum_is_rejected(self) -> None:
        with self.assertRaises(ValueError):
            build_population(
                StructureSpec(agents_per_suite=20), random.Random(0)
            )

    def test_nonpositive_structure_is_rejected(self) -> None:
        with self.assertRaises(ValueError):
            build_population(StructureSpec(buildings=0), random.Random(0))


class DeterminismTest(unittest.TestCase):
    def test_same_seed_same_history(self) -> None:
        a = Simulation(seed=42, seed_infections=1).run(20)
        b = Simulation(seed=42, seed_infections=1).run(20)
        self.assertEqual(
            [s.counts for s in a],
            [s.counts for s in b],
        )


def _force_state(agent: Agent, target: InfectionState) -> None:
    while order_index(agent.state) < order_index(target):
        infection.set_state(agent, STATE_ORDER[order_index(agent.state) + 1], 0)


def _curve(seed: int, days: int) -> tuple[list[int], list[int]]:
    history = Simulation(seed=seed, seed_infections=1).run(days)
    active = [
        h.count(InfectionState.INCUBATING) + h.count(InfectionState.SYMPTOMATIC)
        for h in history
    ]
    cumulative = [
        sum(1 for a in h.agents if a.state is not InfectionState.SUSCEPTIBLE)
```

```
        for h in history
    ]
    return active, cumulative


class EpidemicCurvePlausibilityTest(unittest.TestCase):
    """Sept 7 milestone: across seeds the headless simulator produces
    believable outbreaks -- a rise, a peak, a decline -- and is neither
    always-zero nor always-everyone."""

    DAYS = 30
    SEEDS = tuple(range(1, 41))
    ESTABLISHED = 10  # cumulative infections that count as a real outbreak

    @classmethod
    def setUpClass(cls) -> None:
        cls.runs = [(s, *_curve(s, cls.DAYS)) for s in cls.SEEDS]
        cls.total = len(Simulation(seed=1).agents)

    def _established(self):
        return [(s, a, c) for s, a, c in self.runs if c[-1] >= self.ESTABLISHED]

    def test_no_seed_infects_the_whole_population(self) -> None:
        for seed, _active, cum in self.runs:
            self.assertLess(cum[-1], self.total, f"seed {seed} infected everyone")

    def test_some_seeds_fizzle_and_some_take_off(self) -> None:
        established = self._established()
        fizzled = [r for r in self.runs if r[2][-1] <= 3]
        self.assertGreater(len(established), 5, "almost no seed produced an outbreak")
        self.assertLess(
            len(established), len(self.runs),
            "every seed produced an outbreak; the branching process is degenerate",
        )
        self.assertGreater(len(fizzled), 0, "no seed ever fizzled from its index case")

    def test_a_real_outbreak_reaches_a_large_share_of_the_population(self) -> None:
        worst = max(cum[-1] for _s, _a, cum in self.runs)
        self.assertGreater(worst, self.total * 0.4)

    def test_cumulative_infections_never_decrease(self) -> None:
        for seed, _active, cum in self.runs:
            for earlier, later in zip(cum, cum[1:]):
                self.assertLessEqual(earlier, later, f"seed {seed} cumulative fell")

    def test_established_outbreaks_rise_off_day_zero_and_mostly_turn_over(self) -> Non
e:
        established = self._established()
        self.assertTrue(established)
        turned_over = 0
        for seed, active, _cum in established:
            peak = max(active)
            peak_day = active.index(peak)
            self.assertGreater(peak, active[0], f"seed {seed} never rose")
            self.assertGreaterEqual(peak_day, 2, f"seed {seed} peaked at day {peak_da
```

```
y}")
            if peak_day <= self.DAYS - 3 and active[-1] < peak:
                turned_over += 1
        self.assertGreaterEqual(
            turned_over, 0.6 * len(established),
            "most established outbreaks did not peak and decline inside the window",
        )

    def test_a_long_run_burns_out_to_zero_active(self) -> None:
        seed = self._established()[0][0]
        active, _cum = _curve(seed, 90)
        self.assertEqual(active[-1], 0, f"seed {seed} still active at day 90")


class InfectionStateMachineTest(unittest.TestCase):
    def _agent(self) -> Agent:
        return Agent(id="a", suite_id="s", reporting_probability=0.5)

    def test_infect_moves_to_incubating_and_draws_durations_in_range(self) -> None:
        cfg = SimulationConfig()
        rng = random.Random(0)
        for _ in range(50):
            agent = self._agent()
            infection.infect(agent, 0, rng, cfg)
            self.assertIs(agent.state, InfectionState.INCUBATING)
            self.assertGreaterEqual(agent.incubation_days, cfg.incubation_days_min)
            self.assertLessEqual(agent.incubation_days, cfg.incubation_days_max)
            self.assertGreaterEqual(agent.symptomatic_days, cfg.symptomatic_days_min)
            self.assertLessEqual(agent.symptomatic_days, cfg.symptomatic_days_max)

    def test_time_progression_respects_the_drawn_durations(self) -> None:
        cfg = SimulationConfig(
            incubation_days_min=2, incubation_days_max=2,
            symptomatic_days_min=3, symptomatic_days_max=3,
        )
        agent = self._agent()
        infection.infect(agent, 0, random.Random(1), cfg)
        infection.advance_time_based(agent, 1)
        self.assertIs(agent.state, InfectionState.INCUBATING)
        infection.advance_time_based(agent, 2)
        self.assertIs(agent.state, InfectionState.SYMPTOMATIC)
        infection.advance_time_based(agent, 4)
        self.assertIs(agent.state, InfectionState.SYMPTOMATIC)
        infection.advance_time_based(agent, 5)
        self.assertIs(agent.state, InfectionState.RECOVERED)

    def test_advance_makes_at_most_one_transition_per_call(self) -> None:
        cfg = SimulationConfig(
            incubation_days_min=1, incubation_days_max=1,
            symptomatic_days_min=1, symptomatic_days_max=1,
        )
        agent = self._agent()
        infection.infect(agent, 0, random.Random(1), cfg)
        infection.advance_time_based(agent, 100)
        self.assertIs(agent.state, InfectionState.SYMPTOMATIC)
```

```
379 |
380 |     def test_set_state_rejects_backward_and_skipping(self) -> None:
381 |         agent = self._agent()
382 |         infection.set_state(agent, InfectionState.INCUBATING, 0)
383 |         with self.assertRaises(ValueError):
384 |             infection.set_state(agent, InfectionState.SUSCEPTIBLE, 1)
385 |         with self.assertRaises(ValueError):
386 |             infection.set_state(agent, InfectionState.RECOVERED, 1)
387 |
388 |
389 | class ContactModelTest(unittest.TestCase):
390 |     def _floor(self, *, floors=1, suites=2, per_suite=25):
391 |         spec = StructureSpec(
392 |             buildings=1, floors_per_building=floors,
393 |             suites_per_floor=suites, agents_per_suite=per_suite,
394 |         )
395 |         agents, registry = build_population(spec, random.Random(0))
396 |         by_suite: dict[str, list[Agent]] = {}
397 |         for agent in agents:
398 |             by_suite.setdefault(agent.suite_id, []).append(agent)
399 |         floor_of: dict[str, str] = {}
400 |         building_of: dict[str, str] = {}
401 |         for suite_id in by_suite:
402 |             floor_id, building_id = suite_ancestor_ids(registry, suite_id)
403 |             floor_of[suite_id] = floor_id
404 |             building_of[suite_id] = building_id
405 |         return by_suite, floor_of, building_of
406 |
407 |     def test_all_susceptible_produces_no_infections(self) -> None:
408 |         by_suite, floor_of, building_of = self._floor()
409 |         cfg = SimulationConfig(suite_transmission_probability=1.0)
410 |         self.assertEqual(
411 |             contact.new_infections(by_suite, floor_of, building_of, random.Random(0), c
fg),
412 |             ([], []),
413 |         )
414 |
415 |     def test_zero_probabilities_never_infect(self) -> None:
416 |         by_suite, floor_of, building_of = self._floor()
417 |         _force_state(next(iter(by_suite.values()))[0], InfectionState.SYMPTOMATIC)
418 |         cfg = SimulationConfig(
419 |             suite_transmission_probability=0.0,
420 |             floor_transmission_probability=0.0,
421 |             building_transmission_probability=0.0,
422 |         )
423 |         self.assertEqual(
424 |             contact.new_infections(by_suite, floor_of, building_of, random.Random(0), c
fg),
425 |             ([], []),
426 |         )
427 |
428 |     def test_recovered_agents_do_not_transmit(self) -> None:
429 |         by_suite, floor_of, building_of = self._floor()
430 |         _force_state(next(iter(by_suite.values()))[0], InfectionState.RECOVERED)
431 |         cfg = SimulationConfig(
```

```
            suite_transmission_probability=1.0,
            floor_transmission_probability=1.0,
            building_transmission_probability=1.0,
        )
        self.assertEqual(
            contact.new_infections(by_suite, floor_of, building_of, random.Random(0), c
fg),
            ([], []),
        )

    def test_floor_contact_infects_the_neighbouring_suite_not_the_source(self) -> None:
        by_suite, floor_of, building_of = self._floor(suites=2)
        source, neighbour = list(by_suite)
        _force_state(by_suite[source][0], InfectionState.SYMPTOMATIC)
        cfg = SimulationConfig(
            suite_transmission_probability=0.0,
            floor_transmission_probability=1.0,
            building_transmission_probability=0.0,
        )
        within, cross = contact.new_infections(
            by_suite, floor_of, building_of, random.Random(0), cfg
        )
        self.assertEqual(within, [])
        self.assertEqual({a.id for a in cross}, {a.id for a in by_suite[neighbour]})

    def test_building_contact_reaches_another_floor(self) -> None:
        by_suite, floor_of, building_of = self._floor(floors=2, suites=1)
        source, other_floor = list(by_suite)
        _force_state(by_suite[source][0], InfectionState.SYMPTOMATIC)
        cfg = SimulationConfig(
            suite_transmission_probability=0.0,
            floor_transmission_probability=0.0,
            building_transmission_probability=1.0,
        )
        within, cross = contact.new_infections(
            by_suite, floor_of, building_of, random.Random(0), cfg
        )
        self.assertEqual(within, [])
        self.assertEqual({a.id for a in cross}, {a.id for a in by_suite[other_floor]})


class TimeSteppingTest(unittest.TestCase):
    def test_history_has_one_snapshot_per_day_including_day_zero(self) -> None:
        history = Simulation(seed=1, seed_infections=1).run(25)
        self.assertEqual(len(history), 26)
        self.assertEqual([h.day for h in history], list(range(26)))

    def test_seed_infections_are_present_at_day_zero(self) -> None:
        sim = Simulation(seed=1, seed_infections=3)
        self.assertEqual(sim.history[0].count(InfectionState.INCUBATING), 3)
        self.assertEqual(
            sim.history[0].count(InfectionState.SUSCEPTIBLE), len(sim.agents) - 3
        )

    def test_an_agent_infected_today_cannot_be_symptomatic_today(self) -> None:
```

```
        cfg = SimulationConfig(suite_transmission_probability=1.0)
        sim = Simulation(seed=2, seed_infections=1, config=cfg)
        day_zero = {s.id: s.state for s in sim.history[0].agents}
        day_one = sim.run(1)[1]
        became_infected = [
            s for s in day_one.agents
            if day_zero[s.id] is InfectionState.SUSCEPTIBLE
            and s.state is not InfectionState.SUSCEPTIBLE
        ]
        self.assertTrue(became_infected)
        self.assertTrue(
            all(s.state is InfectionState.INCUBATING for s in became_infected)
        )


class PopulationGenerationTest(unittest.TestCase):
    def setUp(self) -> None:
        self.spec = StructureSpec()
        self.agents, self.registry = build_population(self.spec, random.Random(7))

    def test_agent_ids_are_unique(self) -> None:
        ids = [a.id for a in self.agents]
        self.assertEqual(len(ids), len(set(ids)))

    def test_every_agent_belongs_to_one_declared_suite(self) -> None:
        suite_ids = {
            s.id for s in self.registry.scopes.values() if s.level is ScopeLevel.SUITE
        }
        for agent in self.agents:
            self.assertIn(agent.suite_id, suite_ids)

    def test_reporting_probability_is_within_the_config_range(self) -> None:
        cfg = SimulationConfig()
        for agent in self.agents:
            self.assertGreaterEqual(agent.reporting_probability, cfg.reporting_probabil
ity_min)
            self.assertLessEqual(agent.reporting_probability, cfg.reporting_probability
_max)

    def test_each_suite_holds_exactly_the_specified_headcount(self) -> None:
        by_suite = Counter(a.suite_id for a in self.agents)
        self.assertTrue(all(n == self.spec.agents_per_suite for n in by_suite.values
()))
        self.assertEqual(
            len(by_suite),
            self.spec.buildings * self.spec.floors_per_building * self.spec.suites_per_
floor,
        )

    def test_a_custom_structure_scales_every_level(self) -> None:
        spec = StructureSpec(
            buildings=2, floors_per_building=3, suites_per_floor=2, agents_per_suite=30
        )
        agents, registry = build_population(spec, random.Random(0))
        self.assertEqual(len(agents), 2 * 3 * 2 * 30)
```

```
        per_level: dict[ScopeLevel, set[int]] = {}
        for scope in registry.scopes.values():
            per_level.setdefault(scope.level, set()).add(scope.population)
        self.assertEqual(per_level[ScopeLevel.SUITE], {30})
        self.assertEqual(per_level[ScopeLevel.FLOOR], {60})
        self.assertEqual(per_level[ScopeLevel.BUILDING], {180})
        self.assertEqual(per_level[ScopeLevel.CAMPUS], {360})


if __name__ == "__main__":
    unittest.main()
```

scripts/__init__.py

(empty file)

scripts/precompute_results.py

```
  1 | """Run the three heavy analyses once and cache their outputs as JSON.
  2 |
  3 |     python -m scripts.precompute_results
  4 |
  5 | Writes results/disclosure_sweep.json, results/benchmark.json and
  6 | results/adversarial.json. Those files are committed and served by the API
  7 | so nothing heavy runs on request. Re-run this when the underlying policy
  8 | constants or the model change.
  9 | """
 10 |
 11 | from __future__ import annotations
 12 |
 13 | import json
 14 | import sys
 15 | import time
 16 | from pathlib import Path
 17 |
 18 | sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
 19 |
 20 | from adversarial.selftest import run_self_test  # noqa: E402
 21 | from simulation.benchmark import (  # noqa: E402
 22 |     DayRow,
 23 |     benchmark_cross_regime,
 24 |     benchmark_in_distribution,
 25 |     split_by_run,
 26 | )
 27 | from simulation.config import (  # noqa: E402
 28 |     BENCHMARK_DEFAULT_SEEDS,
 29 |     BENCHMARK_TEST_SUITE_TRANSMISSIONS,
 30 |     DEFAULT_EXPERIMENT_SEEDS,
 31 |     DEFAULT_RUN_DAYS,
 32 | )
 33 | from simulation.experiments import run_disclosure_sweeps  # noqa: E402
 34 |
 35 | RESULTS_DIR = Path(__file__).resolve().parent.parent / "results"
 36 |
 37 | ADVERSARIAL_SEEDS = 200
 38 |
 39 |
 40 | def _write(name: str, payload: dict) -> None:
 41 |     RESULTS_DIR.mkdir(exist_ok=True)
 42 |     path = RESULTS_DIR / f"{name}.json"
 43 |     path.write_text(json.dumps(payload, indent=2) + "\n")
 44 |     print(f"  wrote {path.relative_to(RESULTS_DIR.parent)}")
 45 |
 46 |
 47 | def build_disclosure_sweep() -> dict:
 48 |     results = run_disclosure_sweeps()
 49 |     sweeps: dict[str, dict] = {}
 50 |     for name, summaries in results.sweeps.items():
 51 |         rows = []
 52 |         for s in summaries:
```

```
            rows.append(
                {
                    "value": s.value,
                    "degenerate": s.degenerate,
                    "degenerate_reason": s.degenerate_reason,
                    "n_disclosed": s.n_disclosed,
                    "n_established": s.n_established,
                    "disc_delay": s.mean_disclosure_delay,
                    "fire_delay": s.mean_fire_delay,
                    "gap": s.mean_gap,
                    "inf_before_disclosure_unbiased": s.mean_infections_before_disclosu
re_unbiased,
                    "inf_at_disclosure_conditional": s.mean_infections_at_disclosure,
                    "finest_level": (
                        s.finest_disclosed_level.name if s.finest_disclosed_level else
None
                    ),
                }
            )
        sweeps[name] = {"rows": rows}
    return {
        "seeds": DEFAULT_EXPERIMENT_SEEDS,
        "days": DEFAULT_RUN_DAYS,
        "fizzle_rate": results.fizzle_rate,
        "false_alarm_rate": results.false_alarm_rate,
        "control_fire_delay": results.control_fire_delay,
        "primary_sweep": "STATISTICAL_GATE_FLOOR",
        "sweeps": sweeps,
    }


def _score_dict(score) -> dict:
    return {
        "precision": round(score.precision, 3),
        "recall": round(score.recall, 3),
        "delay": (
            None if score.mean_detection_delay is None else round(score.mean_detection_
delay, 2)
        ),
    }


def build_benchmark() -> dict:
    all_seeds = list(range(1, BENCHMARK_DEFAULT_SEEDS + 1))
    in_dist = benchmark_in_distribution(all_seeds)
    train_seeds, test_seeds = split_by_run(
        [DayRow(s, 0, 0.0, 0, 0, 0, False) for s in all_seeds]
    )
    train_ids = sorted({r.seed for r in train_seeds})
    test_ids = sorted({r.seed for r in test_seeds})
    cross = benchmark_cross_regime(train_ids, test_ids)

    cross_regime = []
    model_still_wins = True
    for res, tp in zip(cross, BENCHMARK_TEST_SUITE_TRANSMISSIONS):
```

```
        rules, model = res.scores[0], res.scores[1]
        cross_regime.append(
            {
                "label": f"suite p={tp:g}",
                "rules": _score_dict(rules),
                "model": _score_dict(model),
            }
        )
        if not (model.recall > rules.recall + 0.02):
            model_still_wins = False

    if model_still_wins:
        cross_conclusion = (
            "The model's small recall and latency edge survives the shift in "
            "transmission probability: it is not purely memorised. The edge is still "
            "small and still bought with lower precision, and it is still a model of "
            "this simulator."
        )
    else:
        cross_conclusion = (
            "The model's advantage does not hold once the transmission probability "
            "differs from training. Reported as a negative result: the classifier "
            "mostly learned this simulator."
        )

    return {
        "seeds": BENCHMARK_DEFAULT_SEEDS,
        "days": DEFAULT_RUN_DAYS,
        "in_distribution": {
            "rules": _score_dict(in_dist.scores[0]),
            "model": _score_dict(in_dist.scores[1]),
        },
        "conclusion": (
            "In distribution the learned model is marginally faster and catches a few "
            "more real outbreak-days, at a cost of about three points of precision on a
"
            "detector that already over-alarms. The authored threshold rules ship; the
"
            "model is kept only as this benchmark."
        ),
        "cross_regime": cross_regime,
        "cross_regime_conclusion": cross_conclusion,
    }


def build_adversarial() -> dict:
    s = run_self_test(n_seeds=ADVERSARIAL_SEEDS)
    return {
        "seeds": s.n_runs,
        "days": s.days,
        "same_scope_pairs": s.same_scope_pairs,
        "exact_one_person_pins": s.exact_one_person_pinned,
        "band_one_person_pins": s.band_one_person_pinned,
        "exact_direction_known": s.exact_direction_known,
        "band_direction_known": s.band_direction_known,
```

```
        "band_median_possible_deltas": (
            None
            if s.band_median_possible_values == float("inf")
            else round(s.band_median_possible_values)
        ),
        "wandering_on_mean": round(s.wandering_on_mean, 2),
        "wandering_off_mean": round(s.wandering_off_mean, 2),
        "wandering_on_max": s.wandering_on_max,
        "wandering_off_max": s.wandering_off_max,
        "residual": s.band_residual,
    }


def main() -> None:
    steps = (
        ("disclosure_sweep", build_disclosure_sweep),
        ("benchmark", build_benchmark),
        ("adversarial", build_adversarial),
    )
    for name, build in steps:
        start = time.time()
        print(f"{name} ...", flush=True)
        _write(name, build())
        print(f"  {time.time() - start:.0f}s")


if __name__ == "__main__":
    main()
```

scripts/stress_tests.py

```
"""Run the two limitation-closing stress tests and cache them as JSON.

    python -m scripts.stress_tests             # both (a few minutes)
    python -m scripts.stress_tests --scale     # scale validation only
    python -m scripts.stress_tests --realism   # reporting-realism only

Writes results/scale.json and results/reporting_realism.json. They are
committed for the docs to cite; the API does not serve them, and
``scripts.precompute_results`` (the three shipped panels) is unchanged.
"""

from __future__ import annotations

import argparse
import json
import sys
import time
from dataclasses import asdict
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from simulation.realism import (  # noqa: E402
    VARIANTS,
    format_sweep,
    run_realism_sweep,
    run_sensitivity_grid,
)
from simulation.scale import format_reports, run_scale_validation  # noqa: E402

RESULTS_DIR = Path(__file__).resolve().parent.parent / "results"
SEEDS = list(range(1, 501))


def _write(name: str, payload: dict) -> None:
    RESULTS_DIR.mkdir(exist_ok=True)
    path = RESULTS_DIR / f"{name}.json"
    path.write_text(json.dumps(payload, indent=2) + "\n")
    print(f"  wrote {path.relative_to(RESULTS_DIR.parent)}")


def build_scale() -> dict:
    reports = run_scale_validation(SEEDS)
    print(format_reports(reports))
    return {"seeds": len(SEEDS), "days": 30, "sizes": [asdict(r) for r in reports]}


def build_realism() -> dict:
    sweep = run_realism_sweep(SEEDS)
    print(format_sweep(sweep))
    grid = run_sensitivity_grid(SEEDS)
    return {"seeds": len(SEEDS), "days": 30, "variants": sweep, "grid": grid,
```

```
            "variant_names": list(VARIANTS)}


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--scale", action="store_true")
    parser.add_argument("--realism", action="store_true")
    args = parser.parse_args()
    both = not (args.scale or args.realism)
    if args.scale or both:
        t = time.time()
        _write("scale", build_scale())
        print(f"  {time.time() - t:.0f}s")
    if args.realism or both:
        t = time.time()
        _write("reporting_realism", build_realism())
        print(f"  {time.time() - t:.0f}s")


if __name__ == "__main__":
    main()
```

scripts/verify_epidemic_curves.py

```
"""Week 1 sanity check: do the simulator's epidemic curves look plausible?

    python -m scripts.verify_epidemic_curves

Runs the headless simulator across a range of seeds and prints, per seed,
the peak active count, the day it peaks, the final attack rate, and the
curve shape. Then a PASS/FAIL summary: not always-zero, not
always-everyone, most established outbreaks rise then turn over. This is a
milestone record, not a test -- the assertions live in
tests/test_simulation.py::EpidemicCurvePlausibilityTest.
"""

from __future__ import annotations

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from simulation.infection import InfectionState  # noqa: E402
from simulation.simulator import Simulation  # noqa: E402

SEEDS = range(1, 41)
DAYS = 30
ESTABLISHED = 10


def main() -> None:
    total = len(Simulation(seed=1).agents)
    established = fizzled = turned_over = 0
    worst_attack = 0.0

    print(f"{total} agents, {DAYS} days, seeds {SEEDS.start}..{SEEDS.stop - 1}")
    print(f"{'seed':>4} {'peak':>5} {'peak_day':>8} {'attack%':>8}  shape")
    for seed in SEEDS:
        history = Simulation(seed=seed, seed_infections=1).run(DAYS)
        active = [
            h.count(InfectionState.INCUBATING) + h.count(InfectionState.SYMPTOMATIC)
            for h in history
        ]
        cumulative = sum(
            1 for a in history[-1].agents if a.state is not InfectionState.SUSCEPTIBLE
        )
        peak = max(active)
        peak_day = active.index(peak)
        attack = cumulative / total * 100
        worst_attack = max(worst_attack, attack)

        shape = []
        if cumulative >= ESTABLISHED:
            established += 1
            if peak > active[0]:
```

```
                shape.append("rise")
            if peak_day <= DAYS - 3 and active[-1] < peak:
                shape.append("decline")
                turned_over += 1
        elif cumulative <= 3:
            fizzled += 1
            shape.append("fizzle")
        print(f"{seed:>4} {peak:>5} {peak_day:>8} {attack:>7.1f}  {'/'.join(shape)}")

    n = len(list(SEEDS))
    checks = [
        (established > 5, f"established outbreaks: {established}/{n} (> 5)"),
        (established < n, f"non-degenerate: {n - established}/{n} did not take off"),
        (fizzled > 0, f"index-case fizzle happens: {fizzled}/{n}"),
        (worst_attack > 40.0, f"a real outbreak occurs: worst attack {worst_attack:.0f}%
(> 40)"),
        (worst_attack < 100.0, f"never everyone: worst attack {worst_attack:.0f}% (< 10
0)"),
        (
            turned_over >= 0.6 * established,
            f"most established outbreaks peak then decline: {turned_over}/{establishe
d}",
        ),
    ]
    print()
    ok = True
    for passed, label in checks:
        print(f"  [{'PASS' if passed else 'FAIL'}] {label}")
        ok = ok and passed
    print()
    print("RESULT:", "believable outbreaks" if ok else "SOMETHING IS OFF")
    sys.exit(0 if ok else 1)


if __name__ == "__main__":
    main()
```

render.yaml

```
# Render.com Blueprint. Deploys the FastAPI app + static web/ as one free
# web service. No database, no build step for the frontend, no secrets.
#
# The precomputed results/*.json are committed, so nothing heavy runs on
# the host -- the runtime install is just FastAPI + Uvicorn.
#
# Free tier note: the instance sleeps after ~15 min idle; the next request
# cold-starts in ~50s. Hit the URL once a minute before demoing.

services:
  - type: web
    name: microcluster
    runtime: python
    plan: free
    buildCommand: pip install -r requirements-runtime.txt
    startCommand: uvicorn api.app:app --host 0.0.0.0 --port $PORT --workers 1
    autoDeploy: true
    healthCheckPath: /api/health
    envVars:
      - key: PYTHON_VERSION
        value: "3.12.7"
```

requirements.txt

```
# This machine runs Python 3.14. Use >= constraints only, never exact pins:
# exact pins have already caused binary-wheel resolution failures on this
# project. Let pip pick the newest compatible build for the interpreter.

pytest>=8.3

# The simulator's benchmark (simulation/benchmark.py) trains a small
# classifier on simulated ground truth and scores it against the
# hand-authored threshold rules. numpy is scikit-learn's own dependency
# and is also used directly for the feature matrices.
numpy>=2.1
scikit-learn>=1.5

# The interface layer (api/ + web/): a FastAPI service that runs the
# simulation and the engines server-side and serves the plain-JS floor-plan
# animation in web/. No frontend build step.
fastapi>=0.115
uvicorn>=0.30
httpx>=0.27   # FastAPI's TestClient (tests/test_api.py) needs it
```

requirements-runtime.txt

```
# Runtime-only dependencies for a deployed server.
#
# api.app imports NOTHING heavy -- no numpy, scipy, scikit-learn (verified).
# The precomputed results/*.json are committed, so the host never runs
# scripts/precompute_results. Use this file for hosting; use the full
# requirements.txt for development, tests and regenerating results.
#
# >= constraints only (see requirements.txt for why).

fastapi>=0.115
uvicorn>=0.30
```

Procfile

```
web: uvicorn api.app:app --host 0.0.0.0 --port ${PORT:-8000}
```

CLAUDE.md

```
# MicroCluster

**Thesis: it detects the signal without exposing the person who created it.**

MicroCluster is a privacy-preserving health-signal system for small shared
communities (a co-op, a dorm, an office floor, a shared studio building).
People submit anonymous symptom reports from a **fixed checklist**. The
system looks for statistically unusual activity and, when it finds some,
discloses it **only at a scope that is both statistically supported and
privacy-safe** — otherwise it discloses nothing.

This is not an outbreak-prediction tool and it does not diagnose anyone.
The engineering problem is narrow and specific:

> Extract a community-level signal from sparse, anonymous data without
> letting the signal become a privacy leak.

**Idea locked September 2. No concept evaluation before October 6.**

Everything below is a **decided rule**, not a design open for revision.

---

## What is in this repository right now

Only three things:

1. **Detection engine** (`microcluster/detection.py`) — decides whether
   anything unusual is happening.
2. **Disclosure engine** (`microcluster/disclosure.py`) — decides what, if
   anything, may be said out loud and at what scope.
3. **Tests** (`tests/`) — cover the scenarios listed at the bottom.

No frontend. No API. No replay mode. `microcluster/engine.py` is a thin
wire between the two engines and `microcluster/demo.py` prints the six
reference scenarios.

Detection and disclosure are **separate, independently testable modules**.
They share only plain aggregate data structures. The disclosure engine
never receives individual records (see rule 10).

**Being added now (see the SIMULATION section):** an interactive simulator
that generates anonymous reports and feeds them to the detection engine to
measure how late the detector catches an outbreak. It does **not** modify
`config.py`, `models.py`, `detection.py`, `disclosure.py`, `engine.py` or
their tests — those are the foundation and stay exactly as they are. The
simulator only produces `Report` objects for the existing engines to
consume.

---

## Decided rules
```

```
 53 |
 54 | ### 1. Fixed checklist, never free text
 55 |
 56 | Input is a **fixed checklist**. There is no natural-language interpretation
 57 | anywhere in the system.
 58 |
 59 | - Four categories: **Respiratory, Gastrointestinal, General, Other**, each
 60 |   with 3–5 named symptoms (`microcluster/models.py::CHECKLIST`).
 61 | - Coarse onset: `today` / `1-2 days` / `3-7 days` / `1+ week`.
 62 | - A location group: one node in the scope hierarchy.
 63 |
 64 | The specific symptom is recorded for the reporter's own clarity. It is
 65 | **never** used for matching.
 66 |
 67 | ### 2. Clustering at CATEGORY level only
 68 |
 69 | Reports are only ever grouped by **category**, never by individual symptom.
 70 | This is what makes *"we don't diagnose"* structurally true rather than a
 71 | promise — the code has no finer-grained medical concept to reason about.
 72 |
 73 | ### 3. Two independent detectors, either can fire
 74 |
 75 | - **Relative detector** — is this location unusual compared to its
 76 |   *contemporaneous peers* (sibling scopes, same detection window)?
 77 | - **Absolute detector** — is overall activity unusual compared to a
 78 |   configured background rate?
 79 |
 80 | Either one firing counts as detection. They are computed separately and
 81 | reported separately.
 82 |
 83 | ### 4. `BACKGROUND_RATE` is a named, configurable, illustrative constant
 84 |
 85 | `microcluster/config.py::BACKGROUND_RATE` is the approximate fraction of a
 86 | population expected to file a qualifying report within one detection
 87 | window under ordinary conditions.
 88 |
 89 | **It is an illustrative figure, not a clinical or epidemiological
 90 | estimate.** It lives in `config.py` with a comment saying exactly that.
 91 | It is never inlined into scoring code. Tune it from real baseline data
 92 | before trusting the absolute detector.
 93 |
 94 | ### 5. `DETECTION_WINDOW` is a named constant, default 72 hours
 95 |
 96 | `microcluster/config.py::DETECTION_WINDOW_HOURS = 72`. A report is
 97 | "current activity" if it was submitted within this many hours of the
 98 | evaluation time.
 99 |
100 | ### 6. No composite score
101 |
102 | The detection engine reports **three findings, plainly**, and never
103 | combines them into one number:
104 |
105 | - `relative_excess_ratio` — the driving location's per-capita rate divided
106 |   by its peers' pooled per-capita rate.
107 | - `count_within_window` — total qualifying reports in the detection window.
```

```
108 | - `count_sharing_category` — how many of those share a single category.
109 |
110 | A composite score would hide which of these actually moved, and would
111 | invite threshold-tuning that trades privacy for sensitivity invisibly.
112 |
113 | ### 7. Every scope has a DECLARED population `n`
114 |
115 | Each scope (campus / building / floor / suite) carries a population `n`
116 | **declared at group creation**. It is never the number of people who
117 | happened to report. All gate math uses the declared `n`.
118 |
119 | ### 8. Two separate gates, both must pass
120 |
121 | A scope may be disclosed only if it passes **both** of these, evaluated
122 | independently and reported separately:
123 |
124 | - **Statistical gate:**
125 |   `qualifying_reports >= max(STATISTICAL_GATE_FLOOR, ceil(sqrt(n)))`
126 |   (`STATISTICAL_GATE_FLOOR = 5`). This scales **up** with `n`: a claim
127 |   about a bigger population needs more evidence.
128 |
129 | - **Privacy gate:**
130 |   `n >= MIN_SCOPE_POPULATION` (default 20)
131 |   **and** `qualifying_reports / n <= MAX_REPORT_FRACTION` (configurable).
132 |   This scales **down** with `n`: small scopes are the dangerous ones.
133 |
134 | The two gates **pull in opposite directions by design**. They are never
135 | merged into a single number. Merging them would make the most identifying
136 | statement (a tiny, precise scope) the *cheapest* to make — which is
137 | backwards. Keeping them separate means a small scope must clear a bar that
138 | a large scope does not, and a large scope must clear a bar that a small
139 | scope does not.
140 |
141 | ### 9. Disclose the finest scope passing both gates
142 |
143 | Evaluate every candidate scope with the single rule in rule 8. Disclose
144 | the **finest** scope (deepest level, then smallest `n`) that passes both
145 | gates. If none passes, **disclose nothing**. There is no hardcoded tier
146 | list — `campus → building → floor → suite` falls out of applying one rule
147 | to a set of candidates.
148 |
149 | ### 10. Privacy evaluation runs AFTER detection, before presentation
150 |
151 | Order is fixed: detection → disclosure → presentation. The disclosure
152 | engine is handed **aggregate per-scope counts and declared populations**,
153 | never individual records. The presentation layer (not built yet) will be
154 | handed only the evaluation table from rule 12 and must render it as-is.
155 |
156 | ### 11. Anonymity over identity-based abuse prevention
157 |
158 | Reports are anonymous. That means abuse — someone spamming reports to
159 | manufacture a signal — is **possible, and we accept it** as the cost of
160 | anonymity.
161 |
162 | Light mitigation only: a per-session submission cap
```

(`SESSION_SUBMISSION_CAP`) and a per-day cap (`DAILY_SUBMISSION_CAP`), both named constants in `config.py`, enforced at intake (intake is not part of this repo yet). We deliberately choose these weak, anonymous mitigations over any identity-based control (accounts, device binding, verification), because identity is precisely what the system exists to protect.

12. Every candidate scope gets a complete evaluation record

`disclosure.evaluate(...)` returns a `ScopeEvaluation` for **every** candidate scope, including the rejected ones — not just the winner. Each record contains:

- scope id, label, level
- declared population `n`
- qualifying report count
- statistical threshold and PASS/FAIL
- privacy threshold(s) and PASS/FAIL (minimum population, report fraction)
- final `disclosure_eligible` (both gates) and `selected`
- a `disclosure_reason` string explaining the outcome, e.g.
  "Rejected: population n=18 below minimum privacy scope (20)"

The winning disclosure is **selected from this table**, never computed on its own path. A future UI renders these records directly and must never recompute a gate result itself.

Every threshold is a policy, not a law

`ceil(sqrt(n))`, `STATISTICAL_GATE_FLOOR`, `MIN_SCOPE_POPULATION`, `MAX_REPORT_FRACTION`, `BACKGROUND_RATE`, `ABSOLUTE_EXCESS_RATIO`, `RELATIVE_EXCESS_RATIO`, `DETECTION_WINDOW_HOURS` — these are **project-defined policies**. They are:

- **named** — all in `microcluster/config.py`,
- **configurable** — via `DetectionConfig` / `DisclosureConfig`,
- **documented as choices** — here and in `config.py` comments.

None of them is derived from a proven result. `sqrt(n)` is a reasonable shape for "evidence should scale with the size of the claim", not a theorem about this problem.

Detection hysteresis and disclosure scope stability

`detection.evaluate` and `disclosure.evaluate` are each still, in isolation, a pure single-instant computation. Called in a SEQUENCE (once a day over a live event), both accept optional state threaded from the previous call, and both use that state the same way: prefer continuity over reacting to a single noisy instant.

- **Detection hysteresis.** Once fired, `fired` is held `True` for at least `HYSTERESIS_MIN_FIRED_DAYS` consecutive evaluations even if that

```
  day's raw relative/absolute check alone says no — an ordinary wobble in
  which reports fall inside a rolling 72h window should not flip
  detection off and back on inside one ongoing event. It releases early
  regardless of the minimum if the window's qualifying count drops to
  `HYSTERESIS_RELEASE_THRESHOLD` or below: a drop that low means the event
  is genuinely over. State is `HysteresisState`, threaded explicitly
  (never global); a call with no state behaves exactly as if hysteresis
  did not exist.

- **Disclosure scope stability.** Once a scope has been disclosed, passing
  its id as `prior_disclosed_scope_id` makes `evaluate` prefer to keep
  disclosing that scope, or a coarser ancestor of it if the exact scope
  stops qualifying, over hopping to a different scope. It only switches
  away from that lineage when some other scope's qualifying count exceeds
  it by more than `SIBLING_SWITCH_MARGIN`. Each disclosure still passes
  both gates on its own; the point is that the SEQUENCE of disclosures
  leaks more than any one of them does — an observer who watches it
  wander learns about every group it ever names, not just one. A call
  with no prior id is unaffected.

Both constants live in `microcluster/config.py` next to every other
threshold, for the same reason: named, configurable, and not a proven
law. `simulation/pipeline.py` is the reference caller — it threads both
kinds of state day to day and records, in `DailyRecord`, the running
`first_fired_day` and `first_disclosed_day`. The gap between them is the
measurable cost of the privacy policy: how long the system knew something
before it was permitted to say where.

---

## Test scenarios (`tests/`)

- **Localized cluster** — relative detector fires, absolute does not; a
  suite-level disclosure.
- **Campus-wide rise** — absolute detector fires, relative does not; a
  campus-level disclosure.
- **Scattered noise** — neither detector fires; nothing disclosed.
- **Overclaim refusal** — 7 reports, campus `n=2000`: statistical gate
  fails (`7 < 45`); no campus-level disclosure.
- **Privacy refusal** — 5 reports, floor `n=18`: statistical gate passes
  (`5 >= 5`) but privacy gate fails (`18 < 20`); a coarser scope is
  disclosed instead.
- **Both gates across populations** — `n ∈ {12, 18, 40, 100, 500, 2000}`:
  the statistical threshold rises with `n` while the privacy minimum
  rejects the smallest scopes.
- **Roster refusal** — ~15 qualifying reports in a declared group of
  `n=25`: passes the statistical gate (`15 >= 5`) and the population
  minimum (`25 >= 20`), but refused because `15/25 = 0.60` exceeds
  `MAX_REPORT_FRACTION` (0.5). Naming the group would name its members.
- **Edge cases** — zero reports (nothing detected, nothing disclosed); a
  `Report` carrying a symptom outside the fixed checklist is rejected at
  construction.

Run them:
```

```
python -m unittest discover -s tests -v
python -m microcluster.demo      # prints the scenario evaluation tables
```

Relative detector: additive smoothing

The relative excess ratio is additively (Laplace) smoothed on both sides:

```
(reports_here + k) / (n_here + k)
-----------------------------------  >= RELATIVE_EXCESS_RATIO  ->  fires
(reports_control + k) / (n_control + k)
```

`k` is `config.RELATIVE_SMOOTHING` (default 1, one pseudo-report per pseudo-person). Without it, a quiet peer group pushes the control rate to almost zero and the ratio blows up into the hundreds or to infinity: a number that reads as precise but is really just division-by-noise. `k` pulls both rates toward a neutral prior, so the ratio stays finite and comparable across scopes. It is a policy constant like every other threshold here, not a derived value.

SIMULATION

The project is expanding into an **interactive simulator**. Everything already built stays exactly as it is — `config.py`, `models.py`, `detection.py`, `disclosure.py`, `engine.py` and their passing tests are the foundation and are **not** redesigned. The simulator is new code that sits *around* them.

What the simulator does

1. Simulates an illness spreading through a small shared environment — a dorm floor of **20 to 200 people**.
2. Generates **anonymous symptom reports** from the simulated people, from the same fixed checklist the real system uses.
3. Feeds those reports into the **existing detection engine**, unchanged.
4. Measures **how late** the detector catches the outbreak — the delay between the outbreak actually taking hold and the detector firing.

The thesis does not move: *it detects the signal without exposing the person who created it.* The project still does **not** claim to predict real outbreaks. It is an engineering problem with a measurement attached: extract a community-level signal from sparse anonymous data without letting the signal become a privacy leak, and quantify how well that works.

Three outputs by submission

1. **Live animation** of the outbreak spreading through the environment, with the detector firing shown against it.

2. **Delay curve** — how many extra infections each additional day of
   detection delay costs.
3. **Benchmark** — a classifier trained on the simulator's ground truth,
   scored against the hand-authored threshold rules in `config.py` /
   `disclosure.py`. The point is to see how the simple, legible policy
   rules compare to a fitted model on the same data.

The ground-truth firewall

The simulator knows everything: who is infected, when, who infects whom.
That knowledge is the simulator's alone. It is **never** passed to
`detection` or `disclosure`. Those engines receive exactly what they
receive in production — anonymous `Report` objects and declared scope
populations — and nothing else. This is the same boundary as rule 10,
extended: detection sees reports, disclosure sees aggregates, and neither
ever sees the simulation's ground truth. The benchmark classifier (output
3) is the *only* consumer of ground truth, and it is scored, not wired
into the disclosure path.

Constraints (unchanged, and binding on the simulator)

- **Fixed symptom checklist**, never free text. No natural-language
  interpretation anywhere, including in the report generator.
- **Clustering at category level only**, never individual-symptom
  matching.
- **Two detectors in parallel**: relative (versus contemporaneous peers)
  and absolute (versus the configured background rate).
- **Two disclosure gates, both must pass**: statistical (scales up with
  `n`) and privacy (scales down with `n`).
- **Every threshold is a named, configurable constant** with a comment
  explaining the choice — the simulator's own parameters included.
- **Privacy evaluation runs after detection, before presentation.** The
  animation and the delay curve render engine output; they do not
  recompute gates.
- **Every emitted report is at the same `ScopeLevel` — `SUITE`.**
  `detection.evaluate` raises `ValueError` on mixed-level input (its
  peer-population math only holds for sibling locations). Each simulated
  agent belongs to one suite; a report's `location_id` is always that
  suite. The coarser scopes (`FLOOR`, `BUILDING`, `CAMPUS`) exist only in
  the `ScopeRegistry` as the suite's ancestors, and are reached through
  the registry, never by emitting a report against them.

Simulation parameters are chosen, not measured

Transmission probability, incubation distribution, reporting propensity,
contact structure, background symptom rate — every one of these is a
chosen figure, picked to produce a legible demonstration, not measured
from real data. **Anything user-facing must say so**, in the same plain
language `config.py` uses for `BACKGROUND_RATE`. The simulator's constants
live in their own config module with the same naming and commenting
discipline as `microcluster/config.py`.

Timeline

- **Idea locked: September 2.** No concept evaluation before October 6.

- **Feature freeze: September 25.**
- **Submission: October 6.**

SECURITY POSTURE

- No accounts, no passwords, no sessions, no external API keys, no uploads, no SQL, no free text. Most standard web vulnerabilities are **structurally absent, not mitigated**. There is no auth to bypass, no injection sink, no file handler, no session to fixate.
- Input validation is **whitelist-based** via the fixed checklist (`models.py`). A `Report` whose category, onset, or symptom is not an exact member of the closed vocabulary is rejected at construction.
- The privacy strategy is **DATA MINIMIZATION, not encryption**. The system never holds identity, free text, precise time, or precise location, so there is very little to encrypt. You cannot leak what you did not collect.
- **CRITICAL for the future API layer:** detection and disclosure MUST run server side. Raw `Report` objects must NEVER be sent to the client. The API returns `ScopeEvaluation` records (and the detection findings) only. Doing gate evaluation or scope selection in the client would make the privacy gate bypassable with devtools, network inspection, or a patched bundle.
- Anonymity was chosen over identity-based abuse prevention. Per-session and per-day submission caps (`SESSION_SUBMISSION_CAP`, `DAILY_SUBMISSION_CAP`) are the only mitigation, and this is deliberate: identity is the thing the system exists to protect, so we will not introduce it to stop abuse.

UI CONSTRAINTS (binding when the frontend is built)

There is no frontend yet. When one is built, these are binding.

Prohibited: gradients, Lucide icons, pure white backgrounds, rainbow coloring, drop shadows, three-feature-card rows, emoji, glassmorphism, em dashes in copy, Inter / Geist / Space Grotesk, colored left stripes, fake testimonials, bento grids, terminal window mockups, "it's not X, it's Y" phrasing, checkmark bullets, three pricing tiers, soft corner radii, purple-and-black, radial orbs, dot grids, sparkle icons, animated arrows, neon colors, basic pastels.

Required: real working product demos (not mockups); skeleton loaders for async states; a Terms of Service page; and a privacy policy that states the ACTUAL mechanism (what is stored, what is never collected, how the thresholds work) rather than boilerplate.

Visual direction: flat, high contrast, documentary / clinical. Off-white or light gray background. Fonts limited to the system UI stack, IBM Plex, Source Sans, or a serif such as Charter.

Motion (amended after the Week-4 freeze): the rule used to be "no transitions, no animation," full stop. Changed because the shipped UI was

```
438 | correct but inert -- nothing moved until a judge clicked a button they
439 | might not click, which is worse than the gradients/glassmorphism/emoji
440 | this list exists to keep out. Motion is now permitted ONLY where it:
441 |
442 | - carries a state change already happening in the simulation (an agent's
443 |   colour changing because its infection state changed; a gate table row's
444 |   background flashing, once, because its PASS/FAIL changed; the refusal
445 |   box or status line settling in when what they say changes), or
446 | - gives ordinary interaction feedback (hover/focus background and border
447 |   on a control).
448 |
449 | Every transition or animation is capped at 400ms, plays once, and never
450 | loops -- no `infinite`, no continuously repeating `@keyframes`. The one
451 | purely decorative exception, and the only one permitted anywhere, is a
452 | single ~350ms monochrome ring on the instant a circle turns symptomatic
453 | during real playback. Every rule above must be inert under
454 | `prefers-reduced-motion: reduce`. "Hover animations" is removed from the
455 | prohibited list above for exactly this narrow case; it is not open to
456 | further motion beyond what is described here. Gradients, shadows,
457 | glassmorphism/blur, and anything that loops remain banned without
458 | exception, and `tests/test_motion_constraints.py` enforces that.
```